Alexei Bazavov, Zohreh Davoudi, Morten Hjorth-Jensen, Ryan LaRose, Dean Lee, and Alessandro Roggero

Quantum Computing for Many-body Physics

From non-relativistic quantum mechanics to quantum field theory

September 19, 2026

Springer

Contents

1 Introduction and Motivation, all . . . 1
- 1.1 The problem . . . 1
- 1.2 Feynman's idea and what became of it . . . 2
- 1.3 What a quantum computer is, and is not, for a many-body physicist . . . 3
- 1.4 The approach of this book . . . 4
- 1.5 A guide to the chapters . . . 5
- 1.6 How to read this book . . . 6

2 Fundamentals of Quantum Computation . . . 7
- 2.1 Quantum computing in a nutshell . . . 7
- 2.2 Linear algebra in Dirac notation . . . 9
  - 2.2.1 Vectors, inner and outer products . . . 9
  - 2.2.2 Hermitian, unitary and normal matrices . . . 10
  - 2.2.3 Computational basis states, projectors and superposition . . . 10
  - 2.2.4 The Pauli matrices . . . 11
  - 2.2.5 Tensor products . . . 12
  - 2.2.6 Spectral decomposition of Hermitian operators . . . 13
- 2.3 Qubits and the Bloch sphere . . . 14
- 2.4 Measurements . . . 16
  - 2.4.1 The projective measurement postulate . . . 16
  - 2.4.2 Measuring one qubit of a register and in other bases . . . 17
  - 2.4.3 Statistics of repeated measurements . . . 17
- 2.5 Density matrices . . . 18
  - 2.5.1 Pure and mixed states . . . 18
  - 2.5.2 Reduced density matrices and the partial trace . . . 20
  - 2.5.3 Decoherence . . . 20
- 2.6 Two qubits and entanglement . . . 21
  - 2.6.1 Bell states . . . 21
  - 2.6.2 Separable and entangled states . . . 22
  - 2.6.3 Reduced density matrices of entangled states . . . 23
  - 2.6.4 The Schmidt decomposition . . . 23
  - 2.6.5 Entropy and entanglement entropy . . . 24
- 2.7 Quantum gates . . . 26
  - 2.7.1 From Hamiltonians to gates . . . 26
  - 2.7.2 Single-qubit gates . . . 26
  - 2.7.3 Two-qubit gates . . . 27
  - 2.7.4 Universal gate sets . . . 29
- 2.8 Quantum circuits . . . 29

2.9 Two protocols: superdense coding and teleportation . . . 30
2.9.1 Superdense coding . . . 30
2.9.2 Quantum teleportation . . . 31
2.10 A first Hamiltonian: the two-level system . . . 32
2.11 Exercises . . . 34
Programs for this chapter . . . 35

3 Quantum Many-Body Physics Primer . . . 37
3.1 Introduction . . . 37
3.2 Second quantization . . . 38
3.2.1 Creation and annihilation operators . . . 38
3.2.2 Operators as matrices . . . 39
3.2.3 One- and two-body operators . . . 40
3.2.4 The particle-hole formalism and normal ordering . . . 41
3.2.5 The normal-ordered Hamiltonian . . . 42
3.2.6 A worked example: two modes . . . 42
3.2.7 Bosons and spins . . . 43
3.3 Four model Hamiltonians . . . 43
3.3.1 The Lipkin model . . . 44
3.3.2 The pairing model . . . 45
3.3.3 The Hubbard model . . . 46
3.3.4 The Heisenberg model . . . 46
3.4 Full configuration interaction . . . 48
3.4.1 The configuration-interaction expansion . . . 48
3.4.2 The structure of the matrix . . . 49
3.4.3 Iterative eigensolvers . . . 50
3.4.4 The exponential wall . . . 50
3.4.5 Truncated configuration interaction and size consistency . . . 51
3.5 Hartree–Fock theory . . . 52
3.5.1 The energy functional and the Hartree–Fock equations . . . 52
3.5.2 Thouless' theorem and the second-quantized form . . . 54
3.5.3 The Baker–Campbell–Hausdorff expansion and Trotter splitting . . . 54
3.5.4 Stability and the Lipkin model . . . 55
3.5.5 When the mean field does nothing: pairing and Hubbard . . . 57
3.6 Coupled-cluster theory . . . 58
3.6.1 The exponential ansatz . . . 58
3.6.2 The CCD equations . . . 59
3.6.3 Unitary coupled cluster . . . 61
3.6.4 Trotterized unitary coupled cluster . . . 62
3.7 Summary . . . 63
3.8 Exercises . . . 64
Programs for this chapter . . . 65

4 Mapping Many-Body Hamiltonians to Qubits . . . 67
4.1 Introduction . . . 67
4.2 Pauli strings and Pauli sums . . . 68
4.2.1 Pauli sums on a classical computer . . . 69
4.3 The Jordan–Wigner transformation . . . 69
4.3.1 Why the naive map fails . . . 69
4.3.2 The transformation . . . 70
4.3.3 Operator identities . . . 71
4.3.4 The cost for a generic Hamiltonian . . . 71

4.3.5 A Hamiltonian on two qubits . . . 73
4.4 Symmetries and encodings . . . 73
4.4.1 Symmetries in Pauli form . . . 73
4.4.2 The Lipkin model: four encodings . . . 74
4.4.3 The pairing model: one qubit per orbital or per level . . . 75
4.4.4 The Hubbard model: the order of the modes . . . 77
4.4.5 The Heisenberg model: nothing to do, and a warning . . . 78
4.4.6 Tapering a discrete symmetry . . . 78
4.4.7 Spin models and bosonic modes . . . 79
4.4.8 Counting qubits for real systems . . . 80
4.5 From Pauli strings to circuits . . . 80
4.5.1 The exponential of a Pauli string . . . 80
4.5.2 Trotter steps as circuits . . . 81
4.5.3 A first time evolution . . . 82
4.5.4 Preparing the reference state . . . 82
4.5.5 Fermionic rotations and mean-field states . . . 83
4.6 Measuring a Pauli sum . . . 84
4.7 A first variational circuit: unitary coupled cluster for the pairing model . . . 86
4.8 Summary . . . 87
4.9 Exercises . . . 88
Programs for this chapter . . . 89

5 Hamiltonian Simulation and Time Evolution, Alessandro++ . . . 91

6 Quantum Fourier Transform and Phase Estimation . . . 93
6.1 Introduction . . . 93
6.2 From the discrete Fourier transform to the quantum one . . . 94
6.2.1 The product form . . . 95
6.2.2 The circuit . . . 96
6.2.3 A worked example . . . 97
6.2.4 Cost, approximation and conventions . . . 98
6.3 Quantum phase estimation . . . 100
6.3.1 The bits, one at a time . . . 101
6.3.2 The exact and the inexact case . . . 102
6.3.3 Phase estimation with one control qubit . . . 104
6.4 From phases to energies . . . 105
6.4.1 The time step and the reading of the phase . . . 105
6.4.2 Building the controlled evolution . . . 106
6.5 Two small Hamiltonians, worked through . . . 107
6.5.1 One qubit: the tapered pairing Hamiltonian . . . 107
6.5.2 Two qubits: the two-mode Hamiltonian . . . 108
6.6 The model Hamiltonians on a quantum computer . . . 109
6.6.1 Phase estimation on the models . . . 109
6.6.2 The second error: Trotterization . . . 111
6.6.3 The state you start from . . . 113
6.7 An application: the linear-system algorithm of Harrow, Hassidim and Lloyd . . . 115
6.7.1 The problem and the spectral idea . . . 115
6.7.2 The circuit . . . 116
6.7.3 A two-by-two example . . . 117
6.7.4 A differential equation . . . 118
6.7.5 Resolvents, Green’s functions and linear response . . . 119
6.7.6 The fine print . . . 121

6.8 Summary . . . 122
6.9 Exercises . . . 123
Programs for this chapter . . . 124

7 Variational Quantum Algorithms for Many-Body Systems . . . 127
7.1 Introduction . . . 127
7.2 The variational principle on a circuit . . . 128
7.3 The ansatz . . . 130
7.3.1 One qubit is a sphere . . . 130
7.3.2 The hardware-efficient ansatz . . . 130
7.3.3 Ansätze that know the physics . . . 131
7.4 Measuring the energy . . . 132
7.4.1 Shot noise, and what it costs the loop . . . 133
7.4.2 Why measuring one qubit at a time does not work . . . 134
7.5 Gradients on a quantum computer . . . 136
7.5.1 The parameter-shift rule . . . 137
7.5.2 When two terms are not enough . . . 138
7.5.3 Optimizers . . . 138
7.5.4 Barren plateaus . . . 139
7.6 Two small Hamiltonians, worked through . . . 140
7.6.1 One qubit: the tapered pairing Hamiltonian . . . 140
7.6.2 Two qubits: the two-mode Hamiltonian . . . 141
7.7 The model Hamiltonians . . . 141
7.7.1 The hardware-efficient ansatz . . . 142
7.7.2 Ansätze that know the physics . . . 143
7.7.3 Phase estimation and VQE weighed against each other . . . 144
7.8 Letting the algorithm choose the circuit: ADAPT-VQE . . . 145
7.9 A second variational algorithm: QAOA . . . 148
7.9.1 From a combinatorial problem to a Hamiltonian . . . 148
7.9.2 The ansatz and where it comes from . . . 149
7.9.3 What can be computed by hand, and what the program finds . . . 150
7.9.4 Perspectives: variational circuits as machine-learning models . . . 152
7.10 Summary . . . 153
7.11 Exercises . . . 154
Programs for this chapter . . . 156

8 Advanced Eigenstate Algorithms: The Rodeo Algorithm and Filtering, Ryan and Dean . . . 157
8.1 Introduction . . . 157
8.2 Resolution refinement . . . 158
8.2.1 The idea . . . 158
8.2.2 The prolongation operator . . . 159
8.2.3 The refinement path . . . 160
8.2.4 A first example: the pairing model from two levels to four . . . 161
8.2.5 What sets the time: the gap, and the state one starts from . . . 162
8.3 The rodeo algorithm . . . 163
8.3.1 One cycle . . . 163
8.3.2 Many cycles and random times . . . 164
8.3.3 One qubit, worked through . . . 165
8.3.4 Eigenstate preparation and the convergence criterion . . . 166
8.4 Refinement and rodeo together: the pairing model end to end . . . 167
8.4.1 The coarse solver . . . 167

8.4.2 Two chains to the same state . . . 168
8.4.3 Filtering the refined state . . . 168
8.5 The pairing model three ways: rodeo, phase estimation and VQE . . . 171
8.5.1 Phase estimation as a filter . . . 171
8.5.2 The variational state . . . 172
8.6 The other three Hamiltonians . . . 173
8.7 Circuits, depth and the bottlenecks of quantum computing . . . 176
8.7.1 Building blocks . . . 176
8.7.2 The pipeline with product formulas . . . 178
8.7.3 The depth budget . . . 179
8.7.4 Shorter variational circuits . . . 180
8.7.5 The budget against the hardware . . . 180
8.8 Summary . . . 182
8.9 Exercises . . . 182
Programs for this chapter . . . 183

9 Quantum Error Correction and Fault Tolerance, Ryan . . . 185

10 A Case Study: Toward Simulating Lattice Gauge Theories, Zohreh . . . 187

11 Conclusion and Outlook, all . . . 189
11.1 What the book has shown . . . 189
11.2 Where the difficulty went . . . 190
11.3 The hardware . . . 191
11.4 Beyond the gate model . . . 191
11.5 Open problems . . . 192

A Quantum Computing Tools and Tutorials, all . . . 195

B Mathematical Background and Reference, all . . . 197

Index . . . 199

Acronyms

The following abbreviations are used throughout the book. Pauli strings such as XX or Z_0Z_1 and the names of gates and of program classes are not listed.

ADAPT-VQE	Adaptive derivative-assembled pseudo-Trotter variational quantum eigensolver
BCH	Baker–Campbell–Hausdorff expansion
BCS	Bardeen–Cooper–Schrieffer theory of superconductivity (pairing)
BFGS	Broyden–Fletcher–Goldfarb–Shanno quasi-Newton optimizer
CC	Coupled-cluster theory
CCD	Coupled cluster with double excitations
CCSD	Coupled cluster with single and double excitations
CI	Configuration interaction
CID	Configuration interaction with double excitations
CIS	Configuration interaction with single excitations
CISD	Configuration interaction with single and double excitations
CNOT	Controlled-NOT gate
CZ	Controlled-Z gate
DFT	Discrete Fourier transform
FCI	Full configuration interaction
FD	Finite differences
HEA	Hardware-efficient ansatz
HF	Hartree–Fock theory
HHL	Harrow–Hassidim–Lloyd algorithm for linear systems of equations
HVA	Hamiltonian-variational ansatz
MP2	Second-order Møller–Plesset perturbation theory
JW	Jordan–Wigner transformation
NMR	Nuclear magnetic resonance
QCCD	Quantum charge-coupled device (trapped-ion architecture)
QAOA	Quantum approximate optimization algorithm
QEC	Quantum error correction
QFT	Quantum Fourier transform
QML	Quantum machine learning
QPE	Quantum phase estimation
QUBO	Quadratic unconstrained binary optimization
RPA	Random-phase approximation
SPSA	Simultaneous perturbation stochastic approximation
TDHF	Time-dependent Hartree–Fock theory
UCC	Unitary coupled-cluster theory
UCCD	Unitary coupled cluster with double excitations

UCCSD	Unitary coupled cluster with single and double excitations
UHF	Unrestricted Hartree–Fock theory
VQE	Variational quantum eigensolver

Chapter 1
Introduction and Motivation, all

Abstract This chapter sets the stage for the book by discussing the motivation for using quantum computing in many-body physics. We explain the exponential growth of Hilbert space in interacting many-body systems and why classical simulation methods become infeasible as system size increases. The historical context of quantum simulation is presented, from Feynman's original idea to modern experiments with near-term quantum devices. We also highlight recent initiatives (such as TALENT quantum computing courses and new quantum hardware) that have renewed interest in this field. The chapter concludes with an overview of the book's structure and key goals.

1.1 The problem

Almost everything interesting in physics involves many interacting particles. The electrons of a molecule, the nucleons of a nucleus, the spins of a magnet, the quarks and gluons of a hadron, the atoms in an optical lattice: in every case the quantities one wants, a binding energy, a spectrum, a response function, a phase diagram, follow in principle from a Hamiltonian that can be written on one line and a Schrödinger equation whose solution is, in practice, out of reach. The reason is not that the equation is complicated. It is that its solution is large. A system of N fermions distributed over M single-particle states has a Hilbert space of dimension $\binom{M}{N}$, and a chain of n spins one of dimension 2^n; doubling the system squares the dimension, and no reorganization of the calculation removes that. Chapter 3 gives the numbers. Forty single-particle states at half filling is a space of 10^{11} dimensions, the limit of what an iterative eigensolver on a large parallel machine can handle; eighty states is 10^{23}, more than the number of bytes of storage on Earth; a 6×6 Hubbard cluster at half filling has 8×10^{19} states, and ^{16}O in a modest model space of forty orbitals per nucleon species has 6×10^{15} determinants. This is the *exponential wall*, and it stands between us and the exact solution of every problem in this book.

Many-body theory is the art of not solving the Schrödinger equation exactly. Mean-field theory replaces the interacting problem by a product state in a self-consistent potential and captures, at polynomial cost, the bulk of the energy and none of the correlations; perturbation theory and coupled-cluster theory build the correlations back in with a polynomial number of parameters, and are exact in a limit that is rarely reached; configuration interaction truncates the space and converges slowly; quantum Monte Carlo samples the space stochastically and, for fermions, pays for it with a sign problem that grows exponentially in the particle number or the inverse temperature; tensor networks compress the state and succeed exactly when the entanglement is small, which in one dimension is often and in more is not. Each of these has produced results of lasting value, and each has a regime in which it fails, and the regimes of

failure overlap: strongly correlated fermions in two or three dimensions, real-time dynamics far from equilibrium, and the spectra of systems with many competing scales are hard for all of them at once. The exponential wall is not an inconvenience of the current methods but a property of the problem, and the question this book asks is what a computer whose own state space grows exponentially can do about it.

1.2 Feynman's idea and what became of it

The idea is older than the machines. In 1981, at a conference on the physics of computation, Richard Feynman asked whether a classical computer could simulate quantum mechanics efficiently, answered that in general it could not, since the number of amplitudes to be tracked grows exponentially with the size of the system, and proposed the obvious remedy: build a computer out of quantum-mechanical elements, whose state space grows in the same way, and let one quantum system imitate another [**?**]. The proposal was a thought experiment for a decade. Deutsch formalized the notion of a universal quantum computer, Shor's factoring algorithm of 1994 showed that such a machine could do something classical computers apparently cannot, and in 1996 Lloyd showed that Feynman's simulator was universal in the sense that matters here: any local Hamiltonian can be simulated by a sequence of short evolutions under its pieces, the Trotter formula of Chapter 3, with a cost that grows polynomially in the size of the system and the length of the evolution [**?**]. Abrams and Lloyd then showed how to turn simulated dynamics into eigenvalues, by phase estimation [**?**], and Aspuru-Guzik and collaborators worked out in 2005 what it would cost to compute the ground-state energy of a molecule that way [**?**]. By then the algorithmic side of quantum simulation was, in outline, complete: map the fermions to qubits, prepare a trial state, evolve it under the Hamiltonian, read off the energy with a phase register. Chapters 4 to 6 of this book follow that outline.

The machines caught up slowly and then quickly. The first demonstrations, on two or three qubits of nuclear spins, trapped ions or superconducting circuits, computed the energy of the hydrogen molecule or simulated a two-site model, and their point was that the outline could be executed at all. Between 2014 and 2020 a second family of algorithms appeared, adapted to machines with tens of noisy qubits and no error correction: the variational quantum eigensolver of Peruzzo and collaborators [**?**], which replaces the deep coherent circuit of phase estimation by a shallow parametrized one and a classical optimizer, and its relatives, which are the subject of Chapter 7. Preskill named the era, noisy intermediate-scale quantum computing [**?**], and predicted with some accuracy what it would and would not deliver. For nuclear physics the entry point was the calculation of the deuteron binding energy on a cloud-accessible superconducting processor in 2018 [**?**], a two- and three-qubit calculation whose importance was not the number but the demonstration that the machinery of effective field theory, the Jordan–Wigner mapping and a variational circuit could be put together end to end. Since then the field has computed the spectra of small nuclei and lattice gauge theories, the dynamics of spin models on a hundred or more qubits at depths where classical simulation becomes laborious [**?**], and, most consequentially, has crossed the threshold at which a logical qubit encoded in many physical ones is better than its constituents [**?**, **?**]. That last result is the one that makes the fault-tolerant algorithms of Chapters 6 and 8 a matter of engineering rather than of principle, and it is the reason this book treats them on an equal footing with the near-term ones.

The community around these questions has organized itself accordingly. The long-range planning exercises of nuclear physics in the United States and Europe now list quantum information science among their priorities [**?**]; the TALENT schools (Training in Advanced Low Energy Nuclear Theory), from which parts of this book descend, have in recent years run

courses on quantum computing and quantum machine learning for nuclear physics; and university curricula that a decade ago offered quantum computing as an elective in the physics of information now offer it as a tool of many-body theory. This book is written for the graduate student or researcher who arrives from the many-body side, knows what a Slater determinant and a coupled-cluster amplitude are, and wants to know, concretely and with code, what a quantum computer does with them.

1.3 What a quantum computer is, and is not, for a many-body physicist

It is worth being precise at the outset about the kind of advantage that is on offer, because the subject attracts both overstatement and its reaction. A register of n qubits holds a state of 2^n complex amplitudes, and a circuit of polynomial depth transforms all of them at once. That is the source of every advantage in this book, and it is a source with three restrictions attached. First, the amplitudes cannot be read: a measurement returns n bits drawn from the distribution of the squared amplitudes and destroys the state, so that every quantity one wants must be arranged to appear as the statistics of a few bits over many repetitions, and Chapters 4 and 7 show what that costs. Second, the state must be prepared: an eigenstate of an interacting Hamiltonian is no more available on a quantum computer than on a classical one, and the algorithms that extract its energy exactly, phase estimation and its descendants, need a trial state whose overlap with it is not exponentially small. That overlap is the quantity on which the promise of exponential advantage for ground-state problems hinges, and it is not known to be large in general [**?**]. Third, the machines are noisy, and the depth of a circuit that can be executed before the noise overwhelms the signal is, on current hardware, of the order of a few hundred to a few thousand two-qubit gates, a budget that the algorithms of Chapter 6 exceed by orders of magnitude and the algorithms of Chapter 7 were designed to respect. Quantum error correction, Chapter 9, removes the third restriction at a large constant cost in qubits, and does nothing about the first two.

What survives these restrictions is real. Real-time dynamics is the clearest case: a classical computer must store the state, and a quantum computer need only propagate it, so that the evolution of an interacting system for a time t costs resources polynomial in t and in the size of the system, which no classical method achieves in general; this is Lloyd's theorem, and it is the content of Chapter 5 and the reason the lattice gauge theories of Chapter 10, whose real-time and finite-density physics is closed to lattice Monte Carlo by the sign problem, are the application with the strongest motivation. Eigenvalues are the second case, with the overlap caveat above: given a state that is a fair approximation to an eigenstate, phase estimation refines its energy to a precision exponential in the number of qubits spent on it, a guarantee no variational method has. And the variational algorithms, whose guarantees are weaker, are the ones that run today and the ones through which most of what is known empirically about these machines has been learned. The honest summary is that quantum computers move the difficulty of the many-body problem rather than remove it, from the size of the state to its preparation and measurement, and the book is organized around seeing exactly where it goes.

Table 1.1 The four model Hamiltonians followed through the book, the physics each stands for, the symmetry that makes small instances exactly solvable, and the size used in the calculations of Chapters 3–7.

model	stands for	symmetry used	size in the book
Lipkin	collective nuclear motion	quasispin J	$N = 4$, four qubits
pairing	superconductivity, nuclear pairing	seniority	$L = 4$, two pairs, four qubits
Hubbard	correlated electrons, cold atoms	particle number, S_z	four-site ring, eight qubits
Heisenberg	quantum magnetism, spin chains	S^z, total spin	four-site ring, four qubits

1.4 The approach of this book

Three decisions shape the book. The first is to follow a small number of model Hamiltonians all the way through. Chapter 3 introduces four of them, the Lipkin model, the pairing model, the Hubbard model and the Heisenberg model, each a caricature of a class of physical systems, each exactly solvable at small size by symmetry, and each the standard test case for some family of methods. They are then solved classically by exact diagonalization, Hartree–Fock and coupled cluster in Chapter 3, mapped to qubits in Chapter 4, time-evolved in Chapter 5, handed to phase estimation in Chapter 6 and to the variational algorithms in Chapter 7, and filtered in Chapter 8, at the same sizes and couplings throughout, so that the tables of different chapters can be laid side by side. Table 1.1 collects the four. The comparison between methods is more instructive than any one of them, and the thread of one problem through the whole book is what makes the comparison possible.

The second decision is that every number in the book is produced by a program the reader can run. The programs are written in Python on `numpy` and `matplotlib` alone, and they are not calls to a quantum-computing framework: the state-vector simulator, the gates, the circuits, the Pauli sums, the Jordan–Wigner mapping, the Trotter steps, the phase-estimation register and the variational loop are all written out, in a few hundred lines each, so that nothing is hidden and everything can be modified. Each chapter ends with a list of its programs and is accompanied by a Jupyter notebook, generated from the chapter text, that executes every listing and every program in order. The frameworks that run on real hardware, Qiskit, Cirq, PennyLane and their relatives, are introduced in Appendix A once the reader knows what they do, and the translation from the book's simulator to any of them is mechanical. A reader who works through the notebooks will have, at the end, a complete and understood toolkit; a reader who only reads will still find every claim backed by a table whose provenance is stated.

The third decision is a matter of tone. Quantum computing for many-body physics is a young field in which the gap between what has been proposed and what has been demonstrated is wide, and the book tries to keep the two apart. Where an algorithm has a theorem behind it, the theorem is stated and, when it is short, proved; where a method is heuristic, the book says so and shows, on the models, both where it works and where it fails, since the failures of variational circuits and of trial states are as instructive as their successes and considerably less advertised. Where the answer to a question is not known, whether the overlap problem is benign for nuclei, whether variational circuits can be trained at scale, whether any of this beats the best classical methods on a problem anyone cares about, the book says that too.

1.5 A guide to the chapters

The book has three parts, though they are not marked as such. Chapters 2 to 4 are foundations. Chapter 2 sets out the gate model of quantum computation in the language of linear algebra that a physicist already speaks: qubits, the Bloch sphere, Pauli matrices, tensor products, measurement, density matrices and decoherence, entanglement, the elementary gates and the circuits built from them, with superdense coding and teleportation as first examples of what entanglement buys and a variational calculation on a two-level system as a first glimpse of Chapter 7. Chapter 3 is the many-body side of the same foundations: second quantization, the four models, and the three classical methods, exact diagonalization, Hartree–Fock and coupled cluster, against which every quantum result in the book is measured, ending with the unitary coupled-cluster ansatz whose evaluation is exactly the thing a classical computer cannot do and a quantum computer can. Chapter 4 joins the two: the Jordan–Wigner transformation turns the second-quantized Hamiltonians into sums of Pauli strings, symmetries reduce the number of qubits and strings, and three circuits follow, the exponential of a Pauli string, the Trotter step and the measurement of an energy, which are the building blocks of everything after.

Chapters 5 to 8 are algorithms. Chapter 5 is Hamiltonian simulation: the Trotter–Suzuki decomposition of $e^{-i\hat{H}t}$ and its error, the higher-order formulas, the modern alternatives of qubitization and quantum signal processing, real-time dynamics of the lattice models, and the non-unitary evolutions, imaginary time in particular, that can be embedded in a unitary at the price of a probability of success. Chapter 6 is the quantum Fourier transform and phase estimation, the algorithm that turns simulated dynamics into eigenvalues, applied to the four models with a careful account of the two errors, the finite register and the Trotterized evolution, and of the trial state; the Harrow–Hassidim–Lloyd algorithm for linear systems is presented there as an application of the same register to the resolvent of a many-body Hamiltonian. Chapter 7 is the variational family: the variational quantum eigensolver, its ansätze, its measurement cost, its gradients and its failure modes, applied to the same four models and compared with phase estimation; the adaptive version that builds its own circuit; and the quantum approximate optimization algorithm, with a glance at quantum machine learning. Chapter 8 is the rodeo algorithm and its relatives, which use controlled evolutions and measurements to filter an eigenstate out of a trial state, and which sit between phase estimation and the variational methods in what they require and what they guarantee. Because every filter needs a trial state with a large overlap, the chapter begins with resolution refinement, which carries an eigenstate adiabatically from a coarse model space to a fine one; it runs the combination on all four models, and it ends with a gate-level depth budget that collects the bottlenecks met along the way.

Chapters 9 to 11 are about the machines and the frontier. Chapter 9 is quantum error correction: the error models, the repetition and Shor codes, syndrome measurement, the surface code and the threshold theorem, and the error-mitigation techniques that stand in for error correction on present hardware. Chapter 10 is a case study in the direction the field is heading, the simulation of a lattice gauge theory, the Schwinger model in one spatial dimension, from the Kogut–Susskind Hamiltonian through the mapping of matter and gauge fields to qubits to circuits for its dynamics and observables, with results from exact diagonalization and from quantum hardware and a discussion of what changes in more dimensions and with non-Abelian gauge groups. Chapter 11 closes the book with an assessment of where the hardware stands, of what the analog simulators, the annealers and the hybrid methods add to the gate-model picture, and of the open problems. Appendix A collects the software tools, and Appendix B the mathematical background. Table 1.2 gives the map in one place.

Table 1.2 The chapters, the question each answers, and the tool it introduces.

	chapter	question	tool
2	Fundamentals	what is a qubit, a gate, a measurement?	state-vector simulator, circuits
3	Many-body primer	what are we trying to compute?	second quantization, the four models, FCI, HF, CC
4	Mapping to qubits	how does a Hamiltonian reach the machine?	Jordan–Wigner, Pauli sums, symmetry encodings
5	Hamiltonian simulation	how is $e^{-i\hat{H}t}$ built?	Trotter–Suzuki, higher order, qubitization
6	QFT and phase estimation	how is an eigenvalue read out?	QFT, QPE, HHL
7	Variational algorithms	what runs on noisy hardware?	VQE, ADAPT-VQE, QAOA
8	Rodeo and filtering	how is an eigenstate isolated?	rodeo algorithm, spectral filters
9	Error correction	how are errors removed?	codes, syndromes, surface code, mitigation
10	Lattice gauge theories	where is this heading?	Schwinger model on qubits
11	Outlook	what is open?	hardware, analog simulators, perspectives

1.6 How to read this book

The prerequisites are a first course in quantum mechanics, linear algebra at the level of eigenvalue problems and tensor products, and some acquaintance with second quantization, all of which Chapters 2 and 3 and Appendix B review at the pace of a reminder rather than a first encounter. No prior knowledge of quantum computing is assumed. A reader who has it can begin at Chapter 3; a reader who comes from quantum information rather than many-body physics should read Chapter 3 with care, since the models and the classical methods it introduces are what every later chapter computes and compares against. The chapters are meant to be read in order through Chapter 7, after which Chapters 8, 9 and 10 can be taken in any order.

A few conventions hold throughout and are worth fixing now, because the literature does not agree on them. Qubit 0 is the leftmost factor of a tensor product and the most significant bit of a computational basis state, so that $|q_0 q_1 \cdots q_{n-1}\rangle$ is read as a binary number from the left; in the occupation-number encodings, $|1\rangle$ is an occupied mode. Circuits are read from left to right, operators from right to left. Rotations are $R_a(\theta) = e^{-i\theta\sigma_a/2}$, so that a full turn is $\theta = 2\pi$ and changes the sign of the state. Hamiltonians carry a hat, $\hat{H}$, and their qubit images are written as Pauli sums $\sum_\alpha h_\alpha P_\alpha$ with strings P_α of the matrices I, X, Y, Z. Energies are quoted in the units of the model, with $\hbar = 1$ throughout. The exercises at the end of each chapter range from short derivations to numerical projects that extend the chapter's programs, and the programs, which are listed at the end of each chapter and collected in the accompanying repository, are as much a part of the book as the text.

Chapter 2
Fundamentals of Quantum Computation

Abstract This chapter reviews the basic concepts of quantum computing needed for the rest of the book. We define qubits and quantum states using Dirac notation and the Bloch sphere representation. Elementary quantum gates (Pauli rotations, Hadamard, phase gate, CNOT) and universal gate sets are introduced, along with quantum circuits and circuit diagrams. Key principles such as superposition and entanglement are explained. Finally, we cover measurement postulates, decoherence, and simple quantum protocols (like teleportation and superdense coding). These protocols exemplify how quantum information processing differs fundamentally from classical computing.

2.1 Quantum computing in a nutshell

A quantum computer is a well-controlled, out-of-equilibrium quantum many-body system that we intend to use to perform a calculation. Such a system can be described at several levels, from the underlying physics, usually expressed through a Hamiltonian with its energies and time scales, up to the abstract representation used to formulate quantum algorithms, the so-called *gate-based quantum computer*. This chapter is about that abstract level: the states a quantum computer can be in, the operations we can perform on them, and what we can learn by measuring them. The chapters that follow build on these notions when they map many-body Hamiltonians onto qubits (Chapter 4), simulate their time evolution (Chapter 5) and extract eigenvalues from them (Chapters 6–8).

The gate-based quantum computer is defined as follows. We have a set of N two-level systems, called quantum bits or *qubits* (for instance the spin of an electron, or two selected levels of a superconducting circuit), each of which can be in the states $|0\rangle$ and $|1\rangle$. The most general state of the quantum computer has the form

$$|\Psi\rangle = \sum_{i_1 i_2 \cdots i_N} \Psi_{i_1 i_2 \cdots i_N} |i_1 i_2 \cdots i_N\rangle, \tag{2.1}$$

where the sum runs over all qubit values $i_a \in \{0,1\}$, and $|i_1 i_2 \cdots i_N\rangle$ is a shorthand for the tensor product $|i_1\rangle \otimes |i_2\rangle \otimes \cdots \otimes |i_N\rangle$, to be defined in Section 2.2.5. The tensor $\Psi_{i_1 i_2 \cdots i_N}$ can be thought of as a large vector containing 2^N complex numbers. The potential capabilities of quantum computers stem from the fact that this vector is exponentially large and, once $N \gtrsim 50$, cannot even be stored in a classical computer.

When one operates a quantum computer, one initializes it in a simple state $|\Psi^{(0)}\rangle$, usually $|\Psi^{(0)}\rangle = |000\cdots0\rangle$, and the state then evolves according to the Schrödinger equation, which transforms $|\Psi^{(n)}\rangle$ into $|\Psi^{(n+1)}\rangle = U^{(n)}|\Psi^{(n)}\rangle$, with the evolution operator

$$U^{(n)} = T \exp\left[-i \int_{t_n}^{t_{n+1}} dt\, H(t)\right], \tag{2.2}$$

where T is the time-ordering operator and $H(t)$ is the Hamiltonian of the system (we set $\hbar = 1$ throughout this book unless stated otherwise). In physics one starts from $H(t)$. In quantum computing one starts instead from the evolution operators $U^{(n)}$, assuming that somebody else has worked out how to engineer the appropriate Hamiltonians. The different evolution operators that one uses are called *gates* and fall into two main categories, depending on whether they act on a single qubit or on two qubits.

A single-qubit gate is a 2×2 unitary matrix U acting on one qubit, $U|i\rangle = \sum_j U_{ji}|j\rangle$. In terms of the full state $\Psi^{(n)}$, a single-qubit gate on qubit a acts as

$$\Psi^{(n+1)}_{i_1 i_2 \cdots i_N} = \sum_{i'_a} U_{i_a i'_a} \Psi^{(n)}_{i_1 \cdots i_{a-1} i'_a i_{a+1} \cdots i_N}, \tag{2.3}$$

and a two-qubit gate on qubits a and b is a 4×4 unitary matrix which transforms the state as

$$\Psi^{(n+1)}_{i_1 i_2 \cdots i_N} = \sum_{i'_a, i'_b} U_{i_a i_b, i'_a i'_b} \Psi^{(n)}_{i_1 \cdots i'_a \cdots i'_b \cdots i_N}. \tag{2.4}$$

Depending on the hardware, some gates are easier to implement than others, but a small set of them suffices to build any unitary operation on the 2^N-dimensional space, as we discuss in Section 2.7.4.

The last ingredient of the gate model is *measurement*. When qubit a is measured, it returns the value $\alpha \in \{0,1\}$ with probability

$$P_\alpha = \sum_{i_1 \cdots i_{a-1} i_{a+1} \cdots i_N} \left| \Psi^{(n)}_{i_1 \cdots i_{a-1} \alpha i_{a+1} \cdots i_N} \right|^2, \tag{2.5}$$

and the state of the computer becomes

$$\Psi^{(n)}_{i_1 \cdots i_N} \to \delta_{i_a, \alpha} \frac{1}{\sqrt{P_\alpha}} \Psi^{(n)}_{i_1 \cdots i_{a-1} \alpha i_{a+1} \cdots i_N}. \tag{2.6}$$

That is essentially it. Equations (2.1)–(2.6) describe completely what a gate-based quantum computer does, and the field of quantum algorithms consists of using these rules to perform useful computations. In a nutshell, a quantum computer allows one to perform a subset of linear algebra, namely matrix-vector multiplications with very special matrices (the gates, which act only on a few indices and belong to a fixed set of unitary matrices) on exponentially large vectors. The appeal of quantum computers clearly comes from the exponentially large size of these vectors.

There is however a strong downside: at the end of the calculation one does not hold the exponentially large vector of 2^N amplitudes, but only a much smaller amount of information, namely N bits drawn from the distribution $|\Psi_{i_1 \cdots i_N}|^2$. This is one of the two Achilles' heels of quantum computing, often referred to as the input/output bottleneck. A second caveat is that the largest part of the 2^N-dimensional Hilbert space will remain forever inaccessible, to quantum computers as well as to classical methods. This follows from a simple counting argument. Suppose the quantum circuit consists of D layers of gates (D being the depth of the circuit), that every layer is packed with as many gates as possible, and that each gate is parametrized by a few angles. The dimension of the manifold of states that such circuits can reach is then $\mathcal{O}(DN)$, which for any realistic depth is vastly smaller than 2^N. With $N = 100$ qubits the Hilbert space has dimension $2^{100} \approx 10^{30}$; even with a depth $D = 10^6$ the reachable manifold has twenty orders of magnitude fewer degrees of freedom. The question

is therefore whether the reachable subspace contains the *relevant* part of Hilbert space, and, conversely, whether the relevant part is amenable to classical simulation. Much of the discussion of variational algorithms in Chapter 7 revolves around this question; the so-called barren plateaus, for instance, have been traced back to the fact that most states in the $\mathcal{O}(DN)$-dimensional manifold are essentially featureless [**?**].

To make all of the above precise we need a modest amount of linear algebra, which we now review in the language of quantum mechanics. Readers who want a more thorough treatment, including norms, matrix factorizations and the numerical eigenvalue algorithms that we lean on throughout the book, will find it in Appendix **??**. Standard references for the material of this chapter are the books of Nielsen and Chuang [**?**], Scherer [**?**] and, for readers who prefer to learn by programming, Hundt [**?**].

2.2 Linear algebra in Dirac notation

2.2.1 Vectors, inner and outer products

Vectors, matrices and higher-order tensors are written in boldface, with lower-case letters for vectors and upper-case letters for matrices; see Section **??** of the appendix for the full set of conventions. A vector with n complex components is written as

$$\boldsymbol{x} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix}, \qquad \boldsymbol{x}^\dagger = \begin{bmatrix} x_0^* \; x_1^* \; \dots \; x_{n-1}^* \end{bmatrix}, \tag{2.7}$$

where the dagger denotes the Hermitian conjugate, the complex conjugate of the transpose. We start counting from zero, so that the first component x_0 multiplies the basis state $|0\rangle$ below and the index of a component agrees with the integer encoded by a bit string of qubits.

Throughout the book we use Dirac's *bra-ket* notation and replace the boldfaced vector by a ket and its Hermitian conjugate by a bra,

$$\boldsymbol{x} = |x\rangle, \qquad \boldsymbol{x}^\dagger = \langle x|. \tag{2.8}$$

The inner product of two vectors of the same length is

$$\langle y|x\rangle = \sum_{i=0}^{n-1} y_i^* x_i, \tag{2.9}$$

so that $\langle x|x\rangle = \sum_i |x_i|^2$ is real and non-negative, while in general

$$\langle x|y\rangle^* = \langle y|x\rangle. \tag{2.10}$$

As an example, for $|x\rangle = \begin{bmatrix} 1-i & 2+i \end{bmatrix}^T$ we find $\langle x|x\rangle = (1+i)(1-i)+(2-i)(2+i) = 7$, and the normalized vector is $|x\rangle/\sqrt{7}$. For $|x\rangle = \begin{bmatrix} -1 & 2i & 1 \end{bmatrix}^T$ and $|y\rangle = \begin{bmatrix} 1 & 0 & i \end{bmatrix}^T$ we get $\langle x|y\rangle = -1+i$ but $\langle y|x\rangle = -1-i$, as Eq. (2.10) demands. The program `dirac_algebra.py` reproduces these numbers.

The *outer product* of two vectors is the matrix

$$|x\rangle\langle y| = \begin{bmatrix} x_0 y_0^* & x_0 y_1^* & \dots & x_0 y_{n-1}^* \\ x_1 y_0^* & x_1 y_1^* & \dots & x_1 y_{n-1}^* \\ \vdots & \vdots & & \vdots \\ x_{n-1} y_0^* & x_{n-1} y_1^* & \dots & x_{n-1} y_{n-1}^* \end{bmatrix}, \tag{2.11}$$

with elements $x_i y_j^*$. Outer products are the building blocks of projection operators and of density matrices, and they play a central role in all of quantum mechanics.

2.2.2 Hermitian, unitary and normal matrices

A general $n \times n$ matrix $\boldsymbol{A}$ has elements a_{ij} with the row index i and column index j both starting from zero; matrix-vector and matrix-matrix products are $y_i = \sum_j a_{ij} x_j$ and $c_{ij} = \sum_k a_{ik} b_{kj}$, see Section **??** of the appendix. Three classes of matrices dominate quantum mechanics:

- *Hermitian* (self-adjoint) matrices, $\boldsymbol{A} = \boldsymbol{A}^\dagger$, that is $a_{ij} = a_{ji}^*$. Observables are represented by Hermitian operators; their eigenvalues are real and their eigenvectors can be chosen orthonormal.
- *Unitary* matrices, $\boldsymbol{U}\boldsymbol{U}^\dagger = \boldsymbol{U}^\dagger\boldsymbol{U} = \boldsymbol{I}$, so that $\boldsymbol{U}^{-1} = \boldsymbol{U}^\dagger$. The time evolution of a closed quantum system, and hence every gate, is unitary.
- *Normal* matrices, which commute with their adjoint, $[\boldsymbol{A}, \boldsymbol{A}^\dagger] = 0$. A matrix is normal if and only if it is diagonalizable by a unitary transformation. Hermitian and unitary matrices are both normal.

The relations $\boldsymbol{A} = \boldsymbol{A}^T$ (symmetric), $\boldsymbol{A} = (\boldsymbol{A}^T)^{-1}$ (real orthogonal) and $\boldsymbol{A} = \boldsymbol{A}^*$ (real) are the real-valued counterparts, and Table **??** of the appendix lists them together with the conditions on the matrix elements.

Unitary transformations are important because they preserve inner products. If $|\phi_i\rangle = \boldsymbol{U}|\psi_i\rangle$ for two states $|\psi_i\rangle$ and $|\psi_j\rangle$, then

$$\langle\phi_i|\phi_j\rangle = \langle\psi_i|\boldsymbol{U}^\dagger\boldsymbol{U}|\psi_j\rangle = \langle\psi_i|\psi_j\rangle, \tag{2.12}$$

so norms and orthogonality are preserved: unitary transformations are rotations in state space. Since the inverse of a unitary matrix is its Hermitian conjugate, every unitary transformation is reversible. This is why quantum mechanics is often described as a reversible theory with a probabilistic outcome; the irreversibility enters only through measurement, Section 2.4. Section **??** of the appendix contains further discussion.

2.2.3 Computational basis states, projectors and superposition

Consider a two-level system whose two states are represented by the state vectors $|\phi_0\rangle$ and $|\phi_1\rangle$. These could be selected degrees of freedom of a single particle (fermion or boson), or effective many-body degrees of freedom. In actual realizations of quantum computers one searches for candidate systems where two low-lying states can be isolated and controlled; in many-body physics, where the number of degrees of freedom explodes, one is similarly always looking for effective ways to reduce the dimensionality to something a given many-body method can handle. We relabel the two states as the orthonormal *computational basis*

$$|\phi_0\rangle = |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad |\phi_1\rangle = |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \tag{2.13}$$

with $\langle 0|1\rangle = 0$ and $\langle 0|0\rangle = \langle 1|1\rangle = 1$. With these two states we can write the identity operator as a sum of outer products, the *completeness relation*

$$\boldsymbol{I} = \sum_{i=0}^{1} |i\rangle\langle i| = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \tag{2.14}$$

and define the *projection operators*

$$\boldsymbol{P} = |0\rangle\langle 0| = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \qquad \boldsymbol{Q} = |1\rangle\langle 1| = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}. \tag{2.15}$$

They are idempotent, $\boldsymbol{P}^2 = \boldsymbol{P}$ and $\boldsymbol{Q}^2 = \boldsymbol{Q}$, they commute and satisfy $\boldsymbol{PQ} = 0$, and their determinants vanish, so they cannot be used as unitary transformations. They do however play a central role in defining measurements, and in the many-body chapters in defining effective model spaces.

The quantum mechanical counterpart of a classical bit, which takes only the values 0 or 1, is a qubit in a general *superposition* of the two computational basis states,

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}, \tag{2.16}$$

where the complex coefficients $\alpha = \langle 0|\psi\rangle$ and $\beta = \langle 1|\psi\rangle$ are the overlaps of the state with the basis states. Normalization requires

$$\langle\psi|\psi\rangle = |\alpha|^2\langle 0|0\rangle + |\beta|^2\langle 1|1\rangle = |\alpha|^2 + |\beta|^2 = 1. \tag{2.17}$$

Acting with the projection operators on $|\psi\rangle$ we obtain

$$\boldsymbol{P}|\psi\rangle = |0\rangle\langle 0|(\alpha|0\rangle + \beta|1\rangle) = \alpha|0\rangle, \qquad \boldsymbol{Q}|\psi\rangle = \beta|1\rangle, \tag{2.18}$$

that is, $\boldsymbol{P}$ projects out the $|0\rangle$ component of the state and $\boldsymbol{Q}$ the $|1\rangle$ component. The outer product of the state with itself,

$$\rho = |\psi\rangle\langle\psi| = \begin{bmatrix} \alpha\alpha^* & \alpha\beta^* \\ \beta\alpha^* & \beta\beta^* \end{bmatrix}, \tag{2.19}$$

is the *density matrix* of the state, with unit trace $\mathrm{Tr}\,\rho = |\alpha|^2 + |\beta|^2 = 1$. We return to density matrices in Section 2.5.

2.2.4 The Pauli matrices

The Pauli matrices

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \qquad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \tag{2.20}$$

play a role in quantum computing that is hard to overstate: they are the simplest gates, every single-qubit gate is generated by them, every operator on N qubits can be expanded in tensor products of them, and, after the mappings of Chapter 4, every many-body Hamiltonian becomes a weighted sum of such tensor products. In the physics literature they are often written $\sigma_x, \sigma_y, \sigma_z$; we use both notations, with X, Y, Z preferred whenever we talk about gates. The Pauli matrices are Hermitian and unitary at the same time, they are involutory,

$$X^2 = Y^2 = Z^2 = \boldsymbol{I}, \tag{2.21}$$

their determinants equal -1, and they obey the commutation and anticommutation relations

$$[X,Y] = 2iZ, \quad [Y,Z] = 2iX, \quad [Z,X] = 2iY, \qquad \{X,Y\} = \{Y,Z\} = \{Z,X\} = 0, \tag{2.22}$$

which can be summarized as $XY = iZ$ and cyclic permutations. Acting with X on the computational basis states gives

$$X|0\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = |1\rangle, \qquad X|1\rangle = |0\rangle, \tag{2.23}$$

a spin flip, which is the quantum analogue of the classical not gate. On a superposition, $X(\alpha|0\rangle + \beta|1\rangle) = \beta|0\rangle + \alpha|1\rangle$. The matrix Z leaves $|0\rangle$ unchanged and multiplies $|1\rangle$ by -1, a relative phase shift, and $Y = iXZ$ combines the two. The computational basis is the eigenbasis of Z with eigenvalues ± 1; the eigenbases of X and Y are the states $|\pm\rangle$ and $|\pm i\rangle$ introduced in Section 2.3.

Two further 2×2 matrices appear again and again. The *Hadamard* matrix

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \tag{2.24}$$

creates superpositions from the basis states,

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \qquad H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle), \tag{2.25}$$

and the *phase* matrix

$$S = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} \tag{2.26}$$

multiplies the $|1\rangle$ component by i. Note that $H = (X+Z)/\sqrt{2}$ is Hermitian and unitary, so $H^2 = \boldsymbol{I}$, while $S^2 = Z$. The identity $HZH = X$, which follows from Eq. (2.22), is the reason why a measurement in the eigenbasis of X can be performed by first applying a Hadamard gate and then measuring in the computational basis, see Section 2.4.2. We do not use a special symbol to distinguish the Hadamard gate from a Hamiltonian; the context, and in the many-body chapters the hat on $\widehat{H}$, makes the meaning clear.

2.2.5 Tensor products

Consider two vectors of length two, $|x\rangle = \begin{bmatrix} x_0 & x_1 \end{bmatrix}^T$ and $|y\rangle = \begin{bmatrix} y_0 & y_1 \end{bmatrix}^T$. Their *tensor product* (also called Kronecker product) is the vector of length four

$$|x\rangle \otimes |y\rangle = |xy\rangle = \begin{bmatrix} x_0 y_0 \\ x_0 y_1 \\ x_1 y_0 \\ x_1 y_1 \end{bmatrix}. \tag{2.27}$$

Applied to the computational basis states this gives the two-qubit basis

$$|00\rangle = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad |01\rangle = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad |10\rangle = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad |11\rangle = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \tag{2.28}$$

where, for instance, $|01\rangle = |0\rangle \otimes |1\rangle$. The bit string labelling a basis state, read as a binary number with the first qubit as the most significant bit, is the index of the single non-zero component: $|00\rangle = |0\rangle$, $|01\rangle = |1\rangle$, $|10\rangle = |2\rangle$, $|11\rangle = |3\rangle$ in the decimal labelling that is sometimes convenient. The construction extends to any number of qubits, $|000\rangle = |0\rangle \otimes |0\rangle \otimes |0\rangle$ being a vector of length eight with a one in the first position, and in general N qubits span a space of dimension 2^N. This is the origin of Eq. (2.1), and we take care in the programs of this chapter to store the amplitudes $\Psi_{i_1 \cdots i_N}$ in exactly this order.

The tensor product of two 2×2 matrices is the 4×4 matrix

$$\boldsymbol{A} \otimes \boldsymbol{B} = \begin{bmatrix} a_{00}\boldsymbol{B} & a_{01}\boldsymbol{B} \\ a_{10}\boldsymbol{B} & a_{11}\boldsymbol{B} \end{bmatrix} = \begin{bmatrix} a_{00}b_{00} & a_{00}b_{01} & a_{01}b_{00} & a_{01}b_{01} \\ a_{00}b_{10} & a_{00}b_{11} & a_{01}b_{10} & a_{01}b_{11} \\ a_{10}b_{00} & a_{10}b_{01} & a_{11}b_{00} & a_{11}b_{01} \\ a_{10}b_{10} & a_{10}b_{11} & a_{11}b_{10} & a_{11}b_{11} \end{bmatrix}, \tag{2.29}$$

and the essential rule for computing with tensor products is

$$(\boldsymbol{A} \otimes \boldsymbol{B})(|x\rangle \otimes |y\rangle) = \boldsymbol{A}|x\rangle \otimes \boldsymbol{B}|y\rangle, \qquad (\boldsymbol{A} \otimes \boldsymbol{B})(\boldsymbol{C} \otimes \boldsymbol{D}) = \boldsymbol{AC} \otimes \boldsymbol{BD}. \tag{2.30}$$

A single-qubit gate U acting on qubit a of an N-qubit register is the operator $\boldsymbol{I} \otimes \cdots \otimes U \otimes \cdots \otimes \boldsymbol{I}$ with U in position a, and Eq. (2.3) is nothing but this operator written out in components. Section **??** of the appendix collects further properties.

2.2.6 Spectral decomposition of Hermitian operators

Let $\boldsymbol{A}$ be a Hermitian operator on a Hilbert space of dimension n and let $|\psi\rangle$ be an eigenvector with eigenvalue λ, $\boldsymbol{A}|\psi\rangle = \lambda|\psi\rangle$. The equation $(\boldsymbol{A} - \lambda \boldsymbol{I})|\psi\rangle = 0$ has a non-trivial solution only if $|(|\boldsymbol{A} - \lambda \boldsymbol{I}) = 0$, the characteristic equation discussed in Section **??** of the appendix. For a Hermitian matrix the n eigenvalues λ_i are real and the eigenvectors $|i\rangle$, $i = 0, \ldots, n-1$, form an orthonormal basis. Expanding an arbitrary state in this eigenbasis, $|\psi\rangle = \sum_i \alpha_i |i\rangle$ with $\sum_i |\alpha_i|^2 = 1$, and acting with $\boldsymbol{A}$ gives

$$\boldsymbol{A}|\psi\rangle = \sum_{i=0}^{n-1} \alpha_i \lambda_i |i\rangle. \tag{2.31}$$

Since the projector $\boldsymbol{P}_j = |j\rangle\langle j|$ extracts the component $\boldsymbol{P}_j|\psi\rangle = \alpha_j|j\rangle$, we can rewrite this as $\boldsymbol{A}|\psi\rangle = \sum_i \lambda_i \boldsymbol{P}_i|\psi\rangle$, and because the state was arbitrary we conclude that

$$\boldsymbol{A} = \sum_{i=0}^{n-1} \lambda_i \boldsymbol{P}_i = \sum_{i=0}^{n-1} \lambda_i |i\rangle\langle i|. \tag{2.32}$$

This is the *spectral decomposition* of a Hermitian (more generally, of a normal) operator. The projectors are orthogonal, $\boldsymbol{P}_i \boldsymbol{P}_j = \delta_{ij} \boldsymbol{P}_i$, and complete, $\sum_i \boldsymbol{P}_i = \boldsymbol{I}$. Two consequences will be used constantly. First, any function of the operator is defined by acting on the eigenvalues,

$$f(\boldsymbol{A}) = \sum_{i=0}^{n-1} f(\lambda_i) |i\rangle\langle i|, \tag{2.33}$$

which is how we compute e^{-iHt} for a Hamiltonian H in Section 2.7.1 and $\rho \log \rho$ for a density matrix in Section 2.6.5. Second, the spectral decomposition of an observable exhaustively specifies a measurement of it, as we show in Section 2.4.

As a two-dimensional example, let $|\psi_a\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle$ and $|\psi_b\rangle = \beta_0|0\rangle + \beta_1|1\rangle$ be the eigenstates of a Hermitian 2×2 matrix with eigenvalues λ_a and λ_b. Then

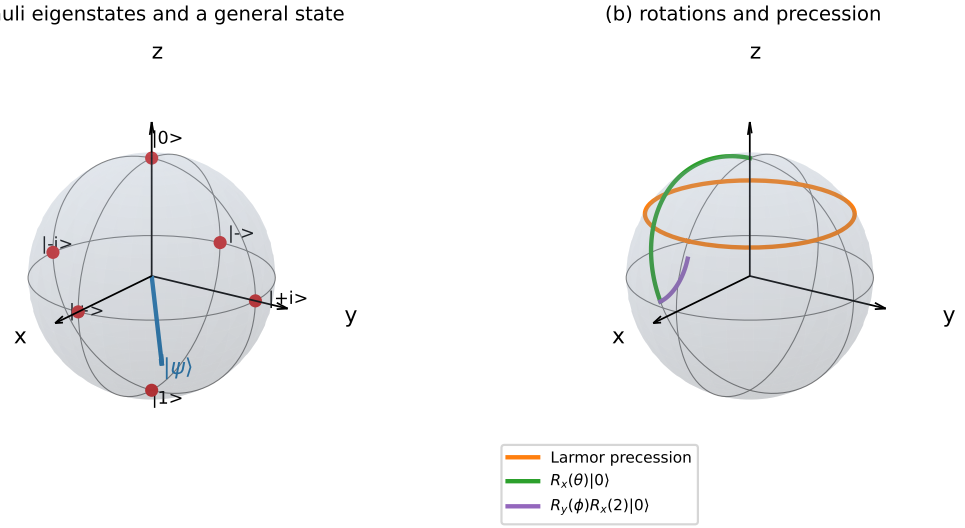


Fig. 2.1 The Bloch sphere. (a) The six eigenstates of the Pauli matrices sit at the ends of the three axes, and a general state $|\psi\rangle$ with $\theta = 2.0$ and $\phi = 0.7$ is drawn as a vector. (b) The trajectory traced by a spin precessing in a magnetic field along z (Larmor precession, Section 2.7.1), a rotation $R_x(\theta)$ about the x axis starting from $|0\rangle$, and a subsequent rotation $R_y(\phi)$ about the y axis; the two-angle family $R_y(\phi)R_x(\theta)|0\rangle$ is the variational ansatz of Section 2.10. Produced by `bloch_sphere.py`.

$$\boldsymbol{A} = \lambda_a \begin{bmatrix} |\alpha_0|^2 & \alpha_0\alpha_1^* \\ \alpha_1\alpha_0^* & |\alpha_1|^2 \end{bmatrix} + \lambda_b \begin{bmatrix} |\beta_0|^2 & \beta_0\beta_1^* \\ \beta_1\beta_0^* & |\beta_1|^2 \end{bmatrix}. \tag{2.34}$$

For the Pauli matrix Z the eigenstates are the computational basis states themselves and $Z = \boldsymbol{P} - \boldsymbol{Q}$ with the projectors of Eq. (2.15); for X they are the Hadamard-rotated states of Eq. (2.25) and $X = |+\rangle\langle+| - |-\rangle\langle-|$. The program `dirac_algebra.py` performs the decomposition numerically for a complex Hermitian matrix and verifies Eq. (2.33) for the exponential.

2.3 Qubits and the Bloch sphere

A normalized qubit state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ is specified by two complex numbers subject to one real constraint, $|\alpha|^2 + |\beta|^2 = 1$, leaving three real parameters. One more is unobservable: multiplying the state by a global phase $e^{i\gamma}$ changes no inner product, no probability and no expectation value, so $|\psi\rangle$ and $e^{i\gamma}|\psi\rangle$ describe the same physical state. We can therefore choose α real and non-negative and write

$$|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\phi}\sin\frac{\theta}{2}|1\rangle, \qquad 0 \le \theta \le \pi, \quad 0 \le \phi < 2\pi. \tag{2.35}$$

The two angles are the polar and azimuthal angles of a point on the unit sphere, the *Bloch sphere* shown in Fig. 2.1. The north pole $\theta = 0$ is $|0\rangle$, the south pole $\theta = \pi$ is $|1\rangle$, and every superposition with equal weights lies on the equator, where ϕ measures the relative phase between the two amplitudes. The half angle $\theta/2$ in Eq. (2.35) is what makes orthogonal states, such as $|0\rangle$ and $|1\rangle$, antipodal on the sphere rather than perpendicular.

The Cartesian coordinates of the point are the expectation values of the three Pauli matrices, the *Bloch vector*

Table 2.1 Eigenstates of the Pauli matrices, their eigenvalues, Bloch angles and Bloch vectors. Produced by `bloch_sphere.py`.

state	operator	eigenvalue	θ	ϕ	Bloch vector
$\|0\rangle$	Z	$+1$	0	0	$(0,0,1)$
$\|1\rangle$	Z	-1	π	0	$(0,0,-1)$
$\|+\rangle = (\|0\rangle + \|1\rangle)/\sqrt{2}$	X	$+1$	$\pi/2$	0	$(1,0,0)$
$\|-\rangle = (\|0\rangle - \|1\rangle)/\sqrt{2}$	X	-1	$\pi/2$	π	$(-1,0,0)$
$\|+i\rangle = (\|0\rangle + i\|1\rangle)/\sqrt{2}$	Y	$+1$	$\pi/2$	$\pi/2$	$(0,1,0)$
$\|-i\rangle = (\|0\rangle - i\|1\rangle)/\sqrt{2}$	Y	-1	$\pi/2$	$3\pi/2$	$(0,-1,0)$

$$\boldsymbol{r} = \begin{bmatrix} \langle\psi|X|\psi\rangle \\ \langle\psi|Y|\psi\rangle \\ \langle\psi|Z|\psi\rangle \end{bmatrix} = \begin{bmatrix} \sin\theta\cos\phi \\ \sin\theta\sin\phi \\ \cos\theta \end{bmatrix}, \tag{2.36}$$

as one verifies by inserting Eq. (2.35). For a pure state $|\boldsymbol{r}| = 1$. We shall see in Section 2.5 that mixed states fill the interior of the ball, and that the density matrix of a qubit can be written in terms of its Bloch vector alone. Table 2.1 lists the six eigenstates of the Pauli matrices together with their Bloch angles. The states $|\pm\rangle = (|0\rangle \pm |1\rangle)/\sqrt{2}$ are the X eigenstates on the $\pm x$ axis, the states $|\pm i\rangle = (|0\rangle \pm i|1\rangle)/\sqrt{2}$ are the Y eigenstates on the $\pm y$ axis, and $|0\rangle, |1\rangle$ are the Z eigenstates on the $\pm z$ axis. Every unitary operation on one qubit is a rotation of the Bloch sphere, as we show in Section 2.7.2; the Hadamard gate, for example, is a rotation by π about the axis $(\hat{x}+\hat{z})/\sqrt{2}$, which maps the z axis onto the x axis and thereby $|0\rangle$ onto $|+\rangle$.

The conversion between amplitudes and Bloch angles is a few lines of code. The following functions from `bloch_sphere.py` are used by several of the programs of this chapter.

```
import numpy as np
from gates import X, Y, Z

def bloch_state(theta, phi):
    """|psi> = cos(theta/2)|0> + e^{i phi} sin(theta/2)|1>."""
    return np.array([np.cos(theta / 2), np.exp(1j * phi) * np.sin(theta / 2)])

def bloch_angles(psi):
    """Inverse of bloch_state, removing the global phase first."""
    psi = np.asarray(psi, dtype=complex) / np.linalg.norm(psi)
    psi = psi * np.exp(-1j * np.angle(psi[0])) # make the |0> amplitude real
    theta = 2 * np.arccos(np.clip(psi[0].real, -1, 1))
    phi = np.angle(psi[1]) if abs(psi[1]) > 1e-12 else 0.0
    return theta, phi % (2 * np.pi)

def bloch_vector(psi):
    """r = (<X>, <Y>, <Z>) for a pure single-qubit state."""
    return np.array([np.vdot(psi, P @ psi).real for P in (X, Y, Z)])

psi = bloch_state(2.0, 0.7)
print(bloch_vector(psi), bloch_angles(np.exp(0.3j) * psi))
```

The last line checks that a global phase changes neither the Bloch vector nor the recovered angles.

2.4 Measurements

2.4.1 The projective measurement postulate

Armed with the spectral decomposition we can now state how quantum mechanics describes a measurement. The postulate is that mutually exclusive measurement outcomes correspond to orthogonal projection operators. Let the observable $\boldsymbol{A}$ have the spectral decomposition $\boldsymbol{A} = \sum_i \lambda_i \boldsymbol{P}_i$ of Eq. (2.32), with projectors that are idempotent, mutually orthogonal and sum to the identity, $\sum_i \boldsymbol{P}_i = \boldsymbol{I}$. A measurement of $\boldsymbol{A}$ on a system in the state $|\psi\rangle$ yields one of the eigenvalues λ_i, with probability

$$\mathrm{Prob}(\lambda_i) = \|\boldsymbol{P}_i|\psi\rangle\|^2 = \langle\psi|\boldsymbol{P}_i^\dagger \boldsymbol{P}_i|\psi\rangle = \langle\psi|\boldsymbol{P}_i|\psi\rangle, \tag{2.37}$$

which is the *Born rule*. Completeness of the projectors guarantees $\sum_i \mathrm{Prob}(\lambda_i) = \langle\psi|\psi\rangle = 1$. Immediately after the measurement the system is in the normalized state

$$|\psi'\rangle = \frac{\boldsymbol{P}_i|\psi\rangle}{\sqrt{\langle\psi|\boldsymbol{P}_i|\psi\rangle}}, \tag{2.38}$$

the *collapse* of the state onto the eigenspace of the observed eigenvalue; a second measurement of the same observable therefore returns the same value with certainty. The expectation value of the observable follows from the Born rule as

$$\langle\boldsymbol{A}\rangle = \sum_i \lambda_i \,\mathrm{Prob}(\lambda_i) = \sum_i \lambda_i \langle\psi|\boldsymbol{P}_i|\psi\rangle = \langle\psi|\boldsymbol{A}|\psi\rangle. \tag{2.39}$$

For a single qubit measured in the computational basis, the projectors are $\boldsymbol{P}_0 = |0\rangle\langle 0|$ and $\boldsymbol{P}_1 = |1\rangle\langle 1|$, and for the state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ we obtain

$$\mathrm{Prob}(0) = \langle\psi|\boldsymbol{P}_0|\psi\rangle = |\alpha|^2, \qquad \mathrm{Prob}(1) = \langle\psi|\boldsymbol{P}_1|\psi\rangle = |\beta|^2, \tag{2.40}$$

with post-measurement states

$$|\psi_0'\rangle = \frac{\alpha}{|\alpha|}|0\rangle, \qquad |\psi_1'\rangle = \frac{\beta}{|\beta|}|1\rangle, \tag{2.41}$$

which are the basis states up to an irrelevant global phase. The measured observable is $Z = \boldsymbol{P}_0 - \boldsymbol{P}_1$, with $\langle Z\rangle = |\alpha|^2 - |\beta|^2$, the z component of the Bloch vector. Since a measurement can only yield one of the two basis states, the amplitudes α and β are never observed directly: the only way to learn about them is to prepare the same state many times and to collect statistics, a point we return to in Section 2.4.3.

It is useful to rewrite the Born rule as a trace. Using the cyclic property $\mathrm{Tr}(\boldsymbol{ABC}) = \mathrm{Tr}(\boldsymbol{CAB})$ and $\mathrm{Tr}(\boldsymbol{A}|\psi\rangle\langle\psi|) = \langle\psi|\boldsymbol{A}|\psi\rangle$,

$$\mathrm{Prob}(\lambda_i) = \mathrm{Tr}\,[\boldsymbol{P}_i|\psi\rangle\langle\psi|] = \mathrm{Tr}\,[\boldsymbol{P}_i\rho]\,, \tag{2.42}$$

with $\rho = |\psi\rangle\langle\psi|$ the density matrix of Eq. (2.19). For the qubit,

$$\boldsymbol{P}_0|\psi\rangle\langle\psi| = \begin{bmatrix}1 & 0\\ 0 & 0\end{bmatrix}\begin{bmatrix}|\alpha|^2 & \alpha\beta^*\\ \alpha^*\beta & |\beta|^2\end{bmatrix} = \begin{bmatrix}|\alpha|^2 & \alpha\beta^*\\ 0 & 0\end{bmatrix}, \tag{2.43}$$

whose trace is $|\alpha|^2$, in agreement with Eq. (2.40). The trace form is the one that generalizes to mixed states in Section 2.5.

2.4.2 Measuring one qubit of a register and in other bases

For an N-qubit state, a measurement of qubit a alone in the computational basis corresponds to the projectors $\boldsymbol{P}_{\alpha}^{(a)} = \boldsymbol{I} \otimes \cdots \otimes |\alpha\rangle\langle\alpha| \otimes \cdots \otimes \boldsymbol{I}$, which act as the identity on all other qubits. Inserting them into the Born rule gives Eq. (2.5), the marginal probability obtained by summing $|\Psi|^2$ over the values of the unmeasured qubits, and the collapse rule Eq. (2.6), which sets to zero every amplitude incompatible with the outcome and renormalizes the rest. The unmeasured qubits are in general left in a state that depends on the outcome, the mechanism behind the correlations of entangled states discussed in Section 2.6.

A measurement in a basis other than the computational one is performed by rotating the basis first. To measure X, whose eigenstates are $|\pm\rangle$, we note that $H|\pm\rangle = |0\rangle, |1\rangle$ and therefore

$$\mathrm{Prob}(\pm) = |\langle \pm|\psi\rangle|^2 = |\langle 0,1|H|\psi\rangle|^2, \tag{2.44}$$

that is, we apply a Hadamard gate and then measure in the computational basis. This is the operational content of the identity $X = HZH$. In the same way Y is measured by applying $S^\dagger$ followed by H. Real quantum hardware only ever measures in the computational basis, and the expectation value of a Hamiltonian written as a sum of Pauli strings, as in Chapter 7, is obtained by measuring each string in its own rotated basis.

2.4.3 Statistics of repeated measurements

A single measurement returns one bit and destroys the state. To estimate $\mathrm{Prob}(0) = |\alpha|^2$ we prepare the state and measure it N_s times, in the jargon of the field N_s *shots*, and count the fraction $\hat{p}$ of outcomes 0. The count is binomially distributed, so the estimate has the standard deviation

$$\sigma_{\hat{p}} = \sqrt{\frac{p(1-p)}{N_s}}, \tag{2.45}$$

which decreases only as $1/\sqrt{N_s}$: to gain one more decimal digit one needs a hundred times more shots. Figure 2.2 shows five independent experiments on a qubit with $|\alpha|^2 = 0.3$. This *shot noise* is a fundamental cost of every quantum algorithm that ends with an expectation value, and it dominates the resource estimates of the variational methods in Chapter 7.

The state-vector simulator used throughout this chapter implements the measurement postulate directly. The class `StateVector` in `statevector.py` stores the 2^N amplitudes of Eq. (2.1) as a NumPy array; a gate on a subset of qubits is applied by reshaping the array into an N-index tensor and contracting the relevant indices, Eq. (2.3), and the marginal probabilities and collapse of Eqs. (2.5)–(2.6) become

```
class StateVector:
    def marginal(self, qubit):
        """P(qubit = 0), P(qubit = 1) by summing |Psi|^2 over the others."""
        p = self.probabilities().reshape((2,) * self.n)
        axes = tuple(a for a in range(self.n) if a != qubit)
        return p.sum(axis=axes)

    def measure(self, qubit, rng=None, outcome=None):
        """Draw an outcome, zero the incompatible amplitudes, renormalise."""
        p = self.marginal(qubit)
        if outcome is None:
            rng = np.random.default_rng() if rng is None else rng
            outcome = int(rng.random() < p[1])
        tensor = self.psi.reshape((2,) * self.n).copy()
```

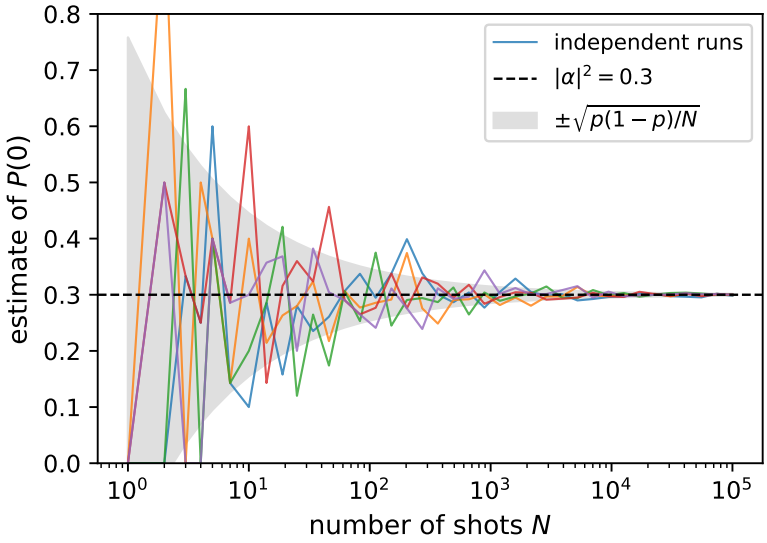


Fig. 2.2 Estimate of Prob(0) for the qubit $|\psi\rangle = \sqrt{0.3}|0\rangle + \sqrt{0.7}e^{0.4i}|1\rangle$ as a function of the number of shots, for five independent sequences of measurements. The shaded band is the one-standard-deviation interval of Eq. (2.45). Produced by `measurement.py`.

```
        keep = [slice(None)] * self.n
        keep[qubit] = 1 - outcome
        tensor[tuple(keep)] = 0.0
        self.psi = tensor.reshape(self.dim) / np.sqrt(p[outcome])
        return outcome, float(p[outcome])
```

Simulating a measurement classically is easy precisely because the simulator holds the full amplitude vector; the quantum computer does not, and this asymmetry is the reason why Eq. (2.5) is the only window we have on the state.

2.5 Density matrices

2.5.1 Pure and mixed states

A state vector describes a system whose preparation is completely known; such a state is called *pure*. Very often the preparation is not completely known: the system may be one member of an ensemble in which the state $|\psi_i\rangle$ occurs with probability p_i, it may be part of a larger system that we do not control, or it may have interacted with an environment that we do not observe. In all these cases the state of the system is itself uncertain and only probabilistic statements about the preparation can be made. This is what density matrices describe, and they describe pure states as a special case.

Consider the ensemble $\{p_i, |\psi_i\rangle\}$ and a measurement with projectors $\boldsymbol{P}_x$. The probability of outcome x is the average of the Born rule (2.42) over the ensemble,

$$p(x) = \sum_i p_i \,\mathrm{Tr}\,[\boldsymbol{P}_x |\psi_i\rangle\langle\psi_i|] = \mathrm{Tr}\,[\boldsymbol{P}_x \rho]\,, \tag{2.46}$$

where we have defined the *density matrix* or density operator

$$\rho = \sum_i p_i |\psi_i\rangle\langle\psi_i|, \qquad p_i \geq 0, \quad \sum_i p_i = 1. \tag{2.47}$$

Every measurable quantity depends on the ensemble only through ρ, so two ensembles with the same density matrix are physically indistinguishable. From the definition, ρ is Hermitian, positive semidefinite (all its eigenvalues are non-negative) and has unit trace,

$$\rho = \rho^\dagger, \qquad \lambda(\rho) \geq 0, \qquad \mathrm{Tr}(\rho) = 1, \tag{2.48}$$

and conversely every matrix with these three properties is a valid density matrix. By the spectral decomposition there is always an orthonormal basis $\{|b_i\rangle\}$ in which

$$\rho = \sum_i \lambda_i |b_i\rangle\langle b_i|, \qquad \sum_i \lambda_i = 1, \tag{2.49}$$

so that any density matrix can be viewed as an ensemble of orthogonal pure states with the eigenvalues as probabilities. A state is pure if and only if one eigenvalue equals one and the others vanish, which can be tested without diagonalizing:

$$\rho \text{ is pure} \iff \rho = |\psi\rangle\langle\psi| \iff \rho^2 = \rho \iff \mathrm{Tr}(\rho^2) = 1. \tag{2.50}$$

The quantity $\mathrm{Tr}(\rho^2)$ is called the *purity*; it is at most one, and its smallest value $1/d$ is attained by the *maximally mixed state* $\rho = \boldsymbol{I}/d$ of a d-dimensional system, which represents complete ignorance about the preparation. The expectation value of an observable and the post-measurement state after outcome x are

$$\langle \boldsymbol{A} \rangle = \mathrm{Tr}(\rho \boldsymbol{A}), \qquad \rho'_x = \frac{\boldsymbol{P}_x \rho \boldsymbol{P}_x}{\mathrm{Tr}[\boldsymbol{P}_x \rho]}, \tag{2.51}$$

and under a unitary transformation, in particular under the time evolution of a closed system with Hamiltonian H,

$$\rho \to U \rho U^\dagger, \qquad \rho(t) = e^{-iHt} \rho(0) e^{iHt}. \tag{2.52}$$

The trace and the eigenvalues, and therefore the purity, are invariant under unitary transformations: $\mathrm{Tr}(U\rho U^\dagger) = \mathrm{Tr}(U^\dagger U \rho) = 1$. Unitary evolution can never turn a mixed state into a pure one, nor the reverse.

For a single qubit, the four matrices $\boldsymbol{I}, X, Y, Z$ form a basis of the 2×2 Hermitian matrices, and the conditions (2.48) imply that every density matrix can be written as

$$\rho = \frac{1}{2}(\boldsymbol{I} + r_x X + r_y Y + r_z Z) = \frac{1}{2}\begin{bmatrix} 1 + r_z & r_x - i r_y \\ r_x + i r_y & 1 - r_z \end{bmatrix}, \qquad |\boldsymbol{r}| \leq 1, \tag{2.53}$$

where $\boldsymbol{r} = (\mathrm{Tr}(\rho X), \mathrm{Tr}(\rho Y), \mathrm{Tr}(\rho Z))$ is the Bloch vector of Eq. (2.36). The eigenvalues of Eq. (2.53) are $(1 \pm |\boldsymbol{r}|)/2$ and the purity is $(1 + |\boldsymbol{r}|^2)/2$, so pure states lie on the surface of the Bloch sphere, mixed states fill its interior, and the maximally mixed state $\rho = \boldsymbol{I}/2$ sits at the centre. An equal mixture of $|0\rangle$ and $|1\rangle$ and an equal mixture of $|+\rangle$ and $|-\rangle$ both give $\rho = \boldsymbol{I}/2$, a first illustration that different ensembles can be the same state. The superposition $(|0\rangle + |1\rangle)/\sqrt{2}$, on the other hand, has $\rho = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$: the difference between the superposition and the mixture is entirely in the off-diagonal elements, the *coherences*, and it becomes visible when we measure X rather than Z, since $\langle X \rangle = 1$ for the superposition and 0 for the mixture.

2.5.2 Reduced density matrices and the partial trace

The second, and for many-body physics the most important, source of mixed states is the description of a subsystem. Let a system be composed of two parts A and B with Hilbert space $\mathscr{H}_A \otimes \mathscr{H}_B$ and density matrix ρ_{AB}, and let $\boldsymbol{O}_A$ be an observable acting on A alone, that is $\boldsymbol{O}_A \otimes \boldsymbol{I}_B$ on the full space. Its expectation value can be written as $\mathrm{Tr}_A(\rho_A \boldsymbol{O}_A)$ in terms of the *reduced density matrix*

$$\rho_A = \mathrm{Tr}_B(\rho_{AB}) = \sum_b \left(\boldsymbol{I}_A \otimes \langle b|\right) \rho_{AB} \left(\boldsymbol{I}_A \otimes |b\rangle\right), \tag{2.54}$$

where the sum runs over an orthonormal basis of B. The operation Tr_B is the *partial trace* over B; on outer products it acts as

$$\mathrm{Tr}_B\big(|ab\rangle\langle cd|\big) = \langle d|b\rangle\, |a\rangle\langle c|, \tag{2.55}$$

which together with linearity determines it completely. In components, for two qubits with ρ_{AB} indexed by the pairs (a,b) and (c,d), $(\rho_A)_{ac} = \sum_b \rho_{(ab),(cb)}$. The reduced density matrix is the complete description of A for an observer who has no access to B, and it is in general mixed even when ρ_{AB} is pure. Section 2.6 turns this observation into the definition of entanglement.

The class `DensityMatrix` in `density_matrix.py` implements Eqs. (2.47)–(2.54). The partial trace is a reshape followed by traces over pairs of axes:

```
class DensityMatrix:
    def partial_trace(self, keep):
        """Trace out every qubit not in `keep`."""
        keep = [keep] if isinstance(keep, int) else sorted(keep)
        trace_out = [q for q in range(self.n) if q not in keep]
        t = self.rho.reshape((2,) * (2 * self.n))  # row indices, then column indices
        for k, q in enumerate(trace_out):
            axis_row = q - k                       # axes already removed shift left
            axis_col = axis_row + (self.n - k)
            t = np.trace(t, axis1=axis_row, axis2=axis_col)
        d = 2 ** len(keep)
        return DensityMatrix(t.reshape(d, d), len(keep))

    def von_neumann_entropy(self, base=2):
        lam = np.clip(np.linalg.eigvalsh(self.rho), 0, None)
        lam = lam[lam > 1e-15]
        return float(-np.sum(lam * np.log(lam)) / np.log(base))
```

We will use the entropy method in Section 2.6.5.

2.5.3 Decoherence

Density matrices also describe what happens when a qubit interacts with an environment that we do not observe: after the interaction the joint state of qubit and environment may be pure, but the qubit alone is described by the reduced density matrix, and information has leaked into correlations with the environment. This loss of coherence, *decoherence*, is the dominant obstacle for present-day quantum hardware and the reason for the error-correction methods of Chapter 9. The most general evolution of a subsystem that starts uncorrelated with its environment is a *quantum channel*, which can always be written in the Kraus form

$$\rho \to \sum_k E_k \rho E_k^\dagger, \qquad \sum_k E_k^\dagger E_k = \boldsymbol{I}, \tag{2.56}$$

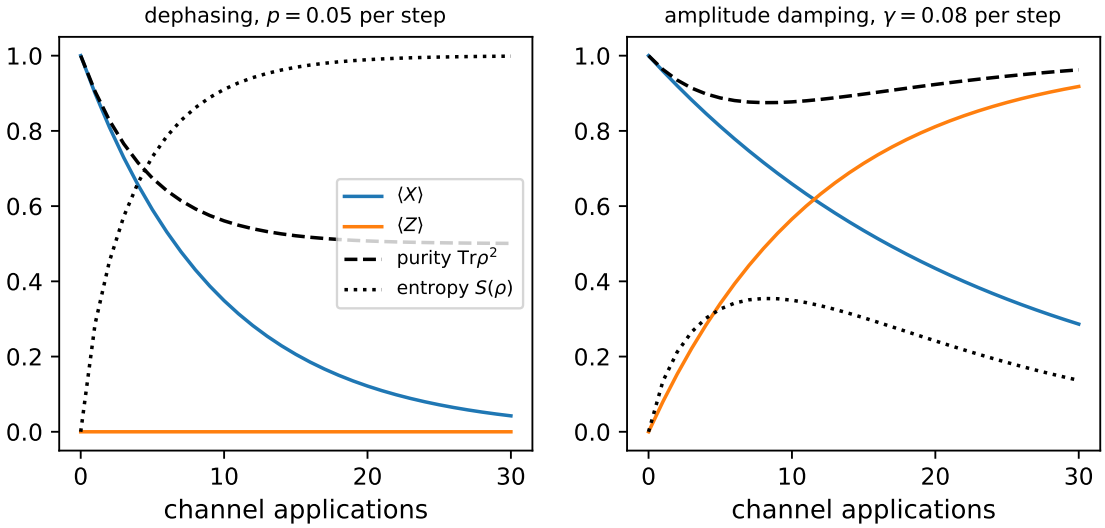


Fig. 2.3 Bloch-vector components, purity and von Neumann entropy of a qubit initially in $|+\rangle$ under repeated application of the dephasing channel (left) and the amplitude-damping channel (right). Dephasing drives the state to the maximally mixed state at the centre of the Bloch ball; amplitude damping drives it to $|0\rangle$, through a mixed intermediate state. Produced by `decoherence.py`.

a generalization of Eq. (2.52) that preserves the three properties (2.48) but not the purity. Two elementary channels illustrate the idea. In the *dephasing* channel a Z error hits the qubit with probability p,

$$\rho \to (1-p)\rho + pZ\rho Z = \begin{bmatrix} \rho_{00} & (1-2p)\rho_{01} \\ (1-2p)\rho_{10} & \rho_{11} \end{bmatrix}, \tag{2.57}$$

so the populations are untouched while the coherences shrink by the factor $1-2p$; after many applications the qubit ends in the diagonal mixture $\mathrm{diag}(\rho_{00}, \rho_{11})$, and the Bloch vector has collapsed onto the z axis. Physically this is what a fluctuating magnetic field along z does to a spin. In the *amplitude damping* channel the excited state $|1\rangle$ decays to $|0\rangle$ with probability γ, as in spontaneous emission, with Kraus operators

$$E_0 = \begin{bmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{bmatrix}, \qquad E_1 = \begin{bmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{bmatrix}, \tag{2.58}$$

which drives every state towards the pure state $|0\rangle$, so that the purity first drops and then recovers. Figure 2.3 shows both channels applied repeatedly to the state $|+\rangle$. On hardware, the time scales on which the coherences and the populations decay are called T_2 and T_1; they set the depth of the circuits that can be run before the output is dominated by noise.

2.6 Two qubits and entanglement

2.6.1 Bell states

We now consider a system composed of two subsystems A and B, each with computational basis states $|0\rangle$ and $|1\rangle$. The subsystems may be single particles, single qubits, or composite many-particle systems of a given symmetry; the results of this section are independent of

what they represent. The computational basis of the composite system consists of the four product states of Eq. (2.28), $|ab\rangle = |a\rangle_A \otimes |b\rangle_B$. From them we can build another orthonormal basis, the *Bell states*

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\0\\0\\1\end{bmatrix}, \qquad |\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\0\\0\\-1\end{bmatrix},$$
$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle) = \frac{1}{\sqrt{2}}\begin{bmatrix}0\\1\\1\\0\end{bmatrix}, \qquad |\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) = \frac{1}{\sqrt{2}}\begin{bmatrix}0\\1\\-1\\0\end{bmatrix}. \tag{2.59}$$

They are normalized, for instance $\langle\Phi^+|\Phi^+\rangle = \frac{1}{2}(\langle 00|00\rangle + \langle 00|11\rangle + \langle 11|00\rangle + \langle 11|11\rangle) = 1$, and mutually orthogonal, for instance $\langle\Phi^+|\Phi^-\rangle = \frac{1}{2}(1-0+0-1) = 0$; four orthonormal vectors in a four-dimensional space form a basis. The corresponding density matrices, in the ordering $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$, are

$$\rho_{\Phi^\pm} = \frac{1}{2}\begin{bmatrix}1 & 0 & 0 & \pm 1\\ 0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\\ \pm 1 & 0 & 0 & 1\end{bmatrix}, \qquad \rho_{\Psi^\pm} = \frac{1}{2}\begin{bmatrix}0 & 0 & 0 & 0\\ 0 & 1 & \pm 1 & 0\\ 0 & \pm 1 & 1 & 0\\ 0 & 0 & 0 & 0\end{bmatrix}. \tag{2.60}$$

Measuring one of the qubits of a Bell state determines the state of the other. To see this, let us measure qubit A of $|\Phi^+\rangle$ in the computational basis. The projectors on the two-qubit space are

$$\boldsymbol{P}_0 = |0\rangle\langle 0|_A \otimes \boldsymbol{I}_B = \begin{bmatrix}1 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\end{bmatrix}, \qquad \boldsymbol{P}_1 = |1\rangle\langle 1|_A \otimes \boldsymbol{I}_B = \begin{bmatrix}0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\\ 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\end{bmatrix}, \tag{2.61}$$

and the Born rule gives

$$\langle\Phi^+|\boldsymbol{P}_0|\Phi^+\rangle = \frac{1}{2}, \qquad \langle\Phi^+|\boldsymbol{P}_1|\Phi^+\rangle = \frac{1}{2}, \tag{2.62}$$

with the post-measurement states

$$|\Phi_0'\rangle = \sqrt{2}\,\boldsymbol{P}_0|\Phi^+\rangle = |00\rangle, \qquad |\Phi_1'\rangle = \sqrt{2}\,\boldsymbol{P}_1|\Phi^+\rangle = |11\rangle. \tag{2.63}$$

The outcome on A is completely random, but once it is known the state of B is fixed, even though the measurement took place locally on A. In the language of density matrices, $p(k) = \mathrm{Tr}(\boldsymbol{P}_k \rho_{\Phi^+}) = 1/2$ and $\rho^{(0)} = |00\rangle\langle 00|$, $\rho^{(1)} = |11\rangle\langle 11|$: the two qubits are perfectly correlated.

2.6.2 Separable and entangled states

A state like $|00\rangle = |0\rangle_A \otimes |0\rangle_B$ is a *product state*: each subsystem is in a definite pure state of its own, and a measurement on A tells us nothing we did not already know about B. The Bell state $|\Phi^+\rangle$ cannot be written in this form for any choice of single-qubit states, as one verifies by trying: $(\alpha|0\rangle + \beta|1\rangle) \otimes (\gamma|0\rangle + \delta|1\rangle)$ has the $|01\rangle$ component $\alpha\delta$ and the $|10\rangle$ component $\beta\gamma$, and both vanish only if $\alpha\delta = \beta\gamma = 0$, which is incompatible with non-zero $|00\rangle$ and $|11\rangle$ components $\alpha\gamma$ and $\beta\delta$. This leads to the definition: a pure bipartite state $|\psi\rangle_{AB}$ is *entangled* if it cannot be written as a product state $|\psi\rangle_A \otimes |\phi\rangle_B$, and *separable* otherwise. For mixed states the definition

is that ρ_{AB} is separable if it is a convex combination of product density matrices,

$$\rho_{AB} = \sum_i p_i \rho_A^{(i)} \otimes \rho_B^{(i)}, \tag{2.64}$$

and entangled otherwise.

A familiar example from atomic physics is the ground state of the helium atom with two electrons in the $1s$ orbital. Each single-electron state is the tensor product of the spatial orbital and a spinor, $|1s, s, m_s\rangle$, and the antisymmetric spin-singlet ground state is

$$\Psi(\boldsymbol{r}_1, \boldsymbol{r}_2; s_1, s_2) = \frac{1}{\sqrt{2}} \left[|1s, \tfrac{1}{2}, \tfrac{1}{2}\rangle_1 |1s, \tfrac{1}{2}, -\tfrac{1}{2}\rangle_2 - |1s, \tfrac{1}{2}, -\tfrac{1}{2}\rangle_1 |1s, \tfrac{1}{2}, \tfrac{1}{2}\rangle_2 \right], \tag{2.65}$$

which in the spin degrees of freedom is precisely the Bell state $|\Psi^-\rangle$: the spins of the two electrons are maximally entangled. The Slater determinants of Chapter 3 are entangled states of this kind, and much of the difficulty of the many-body problem can be phrased as the difficulty of representing highly entangled states with few parameters.

2.6.3 Reduced density matrices of entangled states

The partial trace makes the distinction between product and entangled states quantitative. Take Tr_B of ρ_{Φ^+} term by term with Eq. (2.55):

$$\begin{aligned} \mathrm{Tr}_B(|00\rangle\langle 00|) &= \langle 0|0\rangle\, |0\rangle\langle 0| = |0\rangle\langle 0|, & \mathrm{Tr}_B(|00\rangle\langle 11|) &= \langle 1|0\rangle\, |0\rangle\langle 1| = 0, \\ \mathrm{Tr}_B(|11\rangle\langle 00|) &= \langle 0|1\rangle\, |1\rangle\langle 0| = 0, & \mathrm{Tr}_B(|11\rangle\langle 11|) &= \langle 1|1\rangle\, |1\rangle\langle 1| = |1\rangle\langle 1|, \end{aligned} \tag{2.66}$$

so that

$$\rho_A = \mathrm{Tr}_B(\rho_{\Phi^+}) = \frac{1}{2}\left(|0\rangle\langle 0| + |1\rangle\langle 1|\right) = \frac{\boldsymbol{I}}{2}. \tag{2.67}$$

The same holds for all four Bell states: although the two-qubit state is pure, each qubit on its own is in the maximally mixed state, with purity $1/2$ and a vanishing Bloch vector. All the information in a Bell state resides in the correlations between the qubits and none in the qubits themselves. For a product state, on the other hand, the reduced density matrices are pure (Exercise 6). Purity of the reduced density matrix is therefore a test for entanglement of a pure bipartite state, and its entropy, introduced next, a measure of it.

2.6.4 The Schmidt decomposition

A systematic way to analyse a pure bipartite state is the *Schmidt decomposition*, which is an application of the singular value decomposition of Section **??** in the appendix. Any pure state can be expanded in product states of orthonormal bases $|i\rangle_A$ and $|j\rangle_B$,

$$|\psi\rangle = \sum_{ij} c_{ij} |i\rangle_A |j\rangle_B, \tag{2.68}$$

where the coefficients form a $d_A \times d_B$ matrix $\boldsymbol{C}$. Its singular value decomposition $\boldsymbol{C} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\dagger$, with unitary $\boldsymbol{U}$ and $\boldsymbol{V}$ and a diagonal $\boldsymbol{\Sigma}$ with non-negative entries σ_k, gives $c_{ij} = \sum_k u_{ik}\sigma_k v_{kj}^\dagger$ and therefore

$$|\psi\rangle = \sum_k \sigma_k \left(\sum_i u_{ik}|i\rangle_A\right) \otimes \left(\sum_j v^\dagger_{kj}|j\rangle_B\right) = \sum_{k=0}^{d-1} \sigma_k |k\rangle_A |k\rangle_B, \tag{2.69}$$

where the new bases $|k\rangle_A$ and $|k\rangle_B$ are orthonormal because $\boldsymbol{U}$ and $\boldsymbol{V}$ are unitary. The non-negative numbers σ_k are the *Schmidt coefficients*, they satisfy $\sum_k \sigma_k^2 = 1$, and their number d, the *Schmidt rank*, is at most $\min(d_A, d_B)$. A state is a product state if and only if its Schmidt rank is one, and it is entangled if the rank is larger than one. Moreover, the reduced density matrices follow at once from Eq. (2.69),

$$\rho_A = \sum_k \sigma_k^2 |k\rangle_A\langle k|_A, \qquad \rho_B = \sum_k \sigma_k^2 |k\rangle_B\langle k|_B, \tag{2.70}$$

so both have the same non-zero eigenvalues σ_k^2. For the Bell states $\sigma_0 = \sigma_1 = 1/\sqrt{2}$, the largest possible spread; in general the state $\frac{1}{\sqrt{d}}\sum_i |ii\rangle$ with equal coefficients is called *maximally entangled*. The program `entanglement.py` computes the Schmidt decomposition of arbitrary two-qubit states with `numpy.linalg.svd` and checks Eqs. (2.69) and (2.70).

2.6.5 Entropy and entanglement entropy

Entropy quantifies uncertainty. For a classical random variable with outcomes x of probability $p(x)$, the information content of a single outcome is $-\log_2 p(x)$: a rare event carries much information, a certain one none. The average information, the *Shannon entropy*, is

$$H(X) = -\sum_x p(x)\log_2 p(x), \tag{2.71}$$

measured in bits, with the convention $0\log 0 = 0$. For a binary variable with probabilities p and $1-p$ we get the binary entropy

$$H_2(p) = -p\log_2 p - (1-p)\log_2(1-p), \tag{2.72}$$

shown in the left panel of Fig. 2.4: $H_2(0) = H_2(1) = 0$ when the outcome is certain, and $H_2(1/2) = 1$ bit when our knowledge is no better than a coin toss. The function is continuous, non-negative, additive for independent variables, and maximal for the uniform distribution, which is what one asks of a measure of ignorance.

The quantum counterpart replaces the probability distribution by the density matrix. The *von Neumann entropy* is

$$S(\rho) = -\mathrm{Tr}(\rho\log_2\rho) = -\sum_i \lambda_i \log_2 \lambda_i, \tag{2.73}$$

where λ_i are the eigenvalues of ρ and the second form follows from the spectral decomposition (2.49) and Eq. (2.33): in the eigenbasis, ρ is the diagonal matrix $\boldsymbol{D} = \mathrm{diag}(\lambda_0, \ldots, \lambda_{n-1})$, the unitary that diagonalizes it drops out of the trace, and $-\mathrm{Tr}(\boldsymbol{D}\log_2\boldsymbol{D}) = -\sum_i \lambda_i \log_2 \lambda_i$ is the Shannon entropy of the eigenvalue distribution. A pure state has eigenvalues $\{1, 0, \ldots\}$ and $S = 0$; the maximally mixed state in d dimensions has $S = \log_2 d$, its largest possible value; and S is invariant under unitary transformations, so it does not change during the evolution of a closed system. One cannot simply use the classical definition with the diagonal elements of ρ in some basis, because the result would depend on the basis; the eigenvalues are the only basis-independent probabilities associated with ρ.

For a pure bipartite state the von Neumann entropy of the reduced density matrix,

$$S_A = S(\rho_A) = -\sum_k \sigma_k^2 \log_2 \sigma_k^2 = S(\rho_B), \tag{2.74}$$

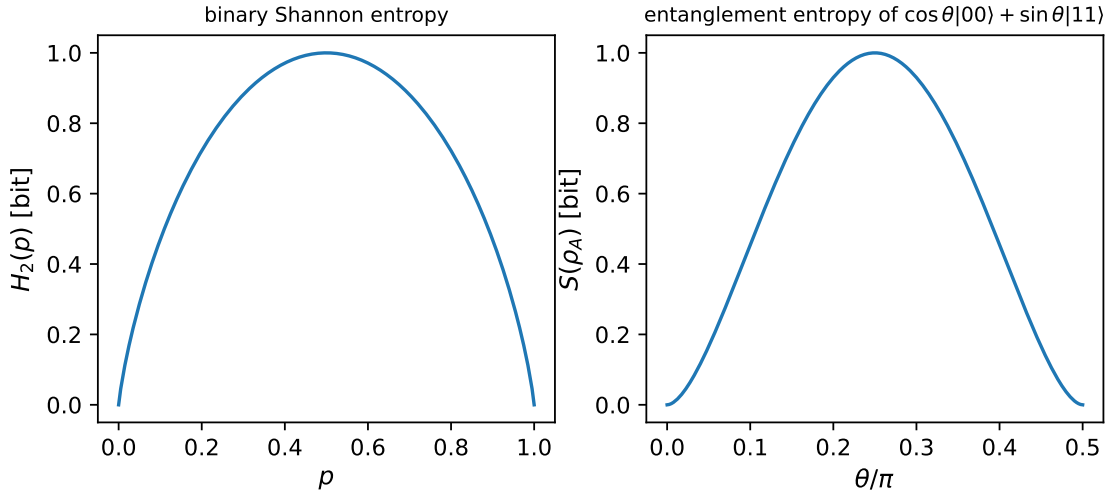


Fig. 2.4 Left: the binary Shannon entropy $H_2(p)$ of Eq. (2.72). Right: the entanglement entropy $S(\rho_A)$ of the two-qubit state $\cos\theta|00\rangle + \sin\theta|11\rangle$, which reaches one bit for the Bell state at $\theta = \pi/4$. Produced by `entanglement.py`.

Table 2.2 Entanglement entropy of $\cos\theta|00\rangle + \sin\theta|11\rangle$, whose Schmidt coefficients are $\cos\theta$ and $\sin\theta$. Produced by `entanglement.py`.

θ/π	0	0.05	0.10	0.15	0.20	0.25
$S(\rho_A)$ [bit]	0	0.1659	0.4545	0.7340	0.9300	1.0000

is the *entanglement entropy*. It vanishes for product states, it equals one bit for the Bell states, and it is symmetric between the two subsystems because both reduced density matrices have the same eigenvalues σ_k^2. The right panel of Fig. 2.4 shows S_A for the family $\cos\theta|00\rangle + \sin\theta|11\rangle$, which interpolates between the product state $|00\rangle$ at $\theta = 0$, the Bell state at $\theta = \pi/4$ and the product state $|11\rangle$ at $\theta = \pi/2$; Table 2.2 gives the numbers. Entanglement entropy is the central diagnostic for the classical simulability of many-body states: tensor-network methods, mentioned in Chapter 3, are efficient precisely for states whose entanglement entropy between any bipartition stays small.

The following excerpt from `entanglement.py` computes the reduced density matrix and entropy of the Bell states with the classes introduced above; the printed table reproduces Eq. (2.67).

```
import numpy as np
from density_matrix import DensityMatrix

bell = {"Phi+": [1, 0, 0, 1], "Phi−": [1, 0, 0, −1],
        "Psi+": [0, 1, 1, 0], "Psi−": [0, 1, −1, 0]}
for name, v in bell.items():
    rho_AB = DensityMatrix.from_state(np.array(v) / np.sqrt(2))
    rho_A = rho_AB.partial_trace(keep=0)
    print(name, rho_A.rho.real.round(3).tolist(),
          "purity", rho_A.purity(), "S =", rho_A.von_neumann_entropy())

def entanglement_entropy(psi):
    return DensityMatrix.from_state(psi).partial_trace(0).von_neumann_entropy()

for theta in np.linspace(0, np.pi / 2, 11):
    print(f"{theta/np.pi:5.2f} {entanglement_entropy([np.cos(theta), 0, 0, np.sin(theta)]):.4f}")
```

2.7 Quantum gates

2.7.1 From Hamiltonians to gates

Quantum gates are physical actions applied to the system that carries the qubits. Mathematically they are unitary matrices acting on the normalized vectors that represent the qubits; physically they are the time evolution generated by a Hamiltonian that the experimenter switches on for a controlled time. The time-dependent Schrödinger equation

$$i\frac{\partial}{\partial t}|\Psi(t)\rangle = H|\Psi(t)\rangle \tag{2.75}$$

has, for a time-independent Hamiltonian, the formal solution

$$|\Psi(t)\rangle = e^{-iH(t-t_0)}|\Psi(t_0)\rangle = U(t,t_0)|\Psi(t_0)\rangle, \tag{2.76}$$

and because H is Hermitian the time-evolution operator $U(t,t_0)$ is unitary, $U^\dagger = U^{-1}$. It satisfies $U(t_0,t_0) = \boldsymbol{I}$, $U(t,t')U(t',t'') = U(t,t'')$ and $U(t,t')^\dagger = U(t',t)$, so a sequence of evolutions under different Hamiltonians is a product of unitaries, which is what a sequence of gates is. When the Hamiltonian depends on time, Eq. (2.76) is replaced by the time-ordered exponential of Eq. (2.2); the interaction picture and the Dyson series that make this precise are the subject of Chapter 5, where we discuss how to implement e^{-iHt} for a many-body Hamiltonian as a circuit.

The simplest example is a spin-$1/2$ in a magnetic field, the working principle of nuclear magnetic resonance and, in essence, of every qubit platform. The magnetic moment $\boldsymbol{\mu} = \gamma \boldsymbol{S}$ couples to the field through $H = -\boldsymbol{\mu}\cdot\boldsymbol{B}$, and for a constant field along z this becomes

$$H_z = -\frac{\omega_L}{2}Z, \tag{2.77}$$

where ω_L is the Larmor frequency. The computational basis states $|0\rangle$ and $|1\rangle$ are the eigenstates of H_z with energies $\mp\omega_L/2$, and using Eq. (2.33), which for an operator $\boldsymbol{A}$ with eigenvector $|a\rangle$ reads $e^{\boldsymbol{A}}|a\rangle = \sum_n \frac{1}{n!}\boldsymbol{A}^n|a\rangle = e^a|a\rangle$, the time evolution of the state (2.35) is

$$|\psi(t)\rangle = e^{i\omega_L t Z/2}|\psi(0)\rangle = e^{i\omega_L t/2}\cos\frac{\theta}{2}|0\rangle + e^{-i\omega_L t/2}e^{i\phi}\sin\frac{\theta}{2}|1\rangle \simeq \cos\frac{\theta}{2}|0\rangle + e^{i(\phi-\omega_L t)}\sin\frac{\theta}{2}|1\rangle, \tag{2.78}$$

where in the last step we dropped a global phase. The polar angle is constant and the azimuthal angle decreases linearly in time: the Bloch vector precesses about the z axis at the Larmor frequency, as drawn in Fig. 2.1(b). Letting the field act for a time t therefore implements a rotation about z by the angle $\omega_L t$, the gate $R_z(-\omega_L t)$ defined below. A field along x or y, or an oscillating field resonant with ω_L, produces rotations about the other axes, and this is how single-qubit gates are realized in NMR and, with microwave or laser pulses, in superconducting and trapped-ion processors.

2.7.2 Single-qubit gates

A single-qubit gate is a 2×2 unitary matrix. Up to a global phase, which we may always discard, it is an element of the group $\mathrm{SU}(2)$ and can be parametrized by three angles as

$$U(\theta,\phi,\lambda)=\begin{bmatrix}\cos\frac{\theta}{2} & -e^{i\lambda}\sin\frac{\theta}{2}\\ e^{i\phi}\sin\frac{\theta}{2} & e^{i(\phi+\lambda)}\cos\frac{\theta}{2}\end{bmatrix}. \tag{2.79}$$

Acting on $|0\rangle$ it produces the Bloch state (2.35) with angles θ and ϕ, so every point on the Bloch sphere can be reached from $|0\rangle$ with a single gate. The Pauli matrices form a basis of the Lie algebra $\mathfrak{su}(2)$, and exponentiating them yields the *rotation gates*. For any matrix with $\boldsymbol{A}^2=\boldsymbol{I}$ the power series of the exponential splits into even and odd terms,

$$e^{-i\alpha\boldsymbol{A}}=\sum_{n=0}^{\infty}\frac{(-i\alpha)^n}{n!}\boldsymbol{A}^n=\boldsymbol{I}\cos\alpha-i\boldsymbol{A}\sin\alpha, \tag{2.80}$$

which applied to the Pauli matrices with $\alpha=\theta/2$ gives

$$\begin{aligned}R_x(\theta)=e^{-i\frac{\theta}{2}X}&=\begin{bmatrix}\cos\frac{\theta}{2} & -i\sin\frac{\theta}{2}\\ -i\sin\frac{\theta}{2} & \cos\frac{\theta}{2}\end{bmatrix},\qquad R_y(\theta)=e^{-i\frac{\theta}{2}Y}=\begin{bmatrix}\cos\frac{\theta}{2} & -\sin\frac{\theta}{2}\\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2}\end{bmatrix},\\ R_z(\theta)=e^{-i\frac{\theta}{2}Z}&=\begin{bmatrix}e^{-i\theta/2} & 0\\ 0 & e^{i\theta/2}\end{bmatrix}.\end{aligned} \tag{2.81}$$

The gate $R_\sigma(\theta)$ rotates the Bloch vector by the angle θ about the axis $\sigma=x,y,z$, and more generally $R_{\hat{n}}(\theta)=e^{-i\theta\hat{n}\cdot\boldsymbol{\sigma}/2}$ rotates it about the unit vector $\hat{n}$; the factor $1/2$ in the exponent is the same half angle that appears in the Bloch parametrization. Rotations by 2π give $-\boldsymbol{I}$ rather than $\boldsymbol{I}$, which is harmless for a single qubit but matters for controlled gates. Any single-qubit gate can be decomposed into three rotations,

$$R_z(\phi)R_y(\theta)R_z(\lambda)=e^{-i(\phi+\lambda)/2}\begin{bmatrix}\cos\frac{\theta}{2} & -e^{i\lambda}\sin\frac{\theta}{2}\\ e^{i\phi}\sin\frac{\theta}{2} & e^{i(\phi+\lambda)}\cos\frac{\theta}{2}\end{bmatrix}=e^{-i(\phi+\lambda)/2}U(\theta,\phi,\lambda), \tag{2.82}$$

as one verifies by multiplying the three matrices of Eq. (2.81). This is the Euler-angle decomposition of a rotation, and it is what a compiler uses to express an arbitrary single-qubit operation in terms of the rotations that the hardware provides. Table 2.3 collects the gates we use most, with their matrices and their interpretation as rotations. The Pauli gates are rotations by π about the coordinate axes (up to a global phase, $X=iR_x(\pi)$ and so on); the phase gate S and the $\pi/8$ gate T are rotations about z by $\pi/2$ and $\pi/4$, with $T^2=S$ and $S^2=Z$; the Hadamard gate is the rotation by π about the axis $(\hat{x}+\hat{z})/\sqrt{2}$, which exchanges the x and z axes and therefore satisfies $HXH=Z$ and $HZH=X$. All the identities in the table are verified numerically by `gates.py`.

2.7.3 Two-qubit gates

A two-qubit gate is a 4×4 unitary matrix, an element of $\mathrm{SU}(4)$ up to a phase. Tensor products $U_1\otimes U_2$ of single-qubit gates are two-qubit gates, but they cannot create entanglement: they map product states onto product states, by Eq. (2.30). The gates that do create entanglement are the *controlled gates*. Given a single-qubit gate U, the controlled-U gate acts on a control qubit $|x\rangle$ and a target qubit $|y\rangle$: it applies U to the target if the control is in the state $|1\rangle$ and does nothing if the control is in $|0\rangle$, while the control itself is never changed. Written as a sum over the two cases,

$$CU=|0\rangle\langle0|\otimes\boldsymbol{I}+|1\rangle\langle1|\otimes U=\begin{bmatrix}1&0&0&0\\0&1&0&0\\0&0&u_{00}&u_{01}\\0&0&u_{10}&u_{11}\end{bmatrix}. \tag{2.83}$$

Table 2.3 Elementary quantum gates. Single-qubit gates act on the basis $\{|0\rangle,|1\rangle\}$, two-qubit gates on $\{|00\rangle,|01\rangle,|10\rangle,|11\rangle\}$ with the first qubit as control. The $P(\lambda)$ family contains $Z = P(\pi)$, $S = P(\pi/2)$ and $T = P(\pi/4)$. Verified by `gates.py`.

gate	symbol	matrix	Bloch-sphere action
Pauli X (not)	X	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$	rotation by π about x
Pauli Y	Y	$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$	rotation by π about y
Pauli Z	Z	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$	rotation by π about z
Hadamard	H	$\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$	rotation by π about $(\hat{x}+\hat{z})/\sqrt{2}$
phase	S	$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$	rotation by $\pi/2$ about z
$\pi/8$	T	$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$	rotation by $\pi/4$ about z
phase shift	$P(\lambda)$	$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\lambda} \end{bmatrix}$	rotation by λ about z
rotations	$R_x, R_y, R_z(\theta)$	Eq. (2.81)	rotation by θ about x, y, z
general	$U(\theta,\phi,\lambda)$	Eq. (2.79)	$R_z(\phi)R_y(\theta)R_z(\lambda)$
controlled-not	CNOT, CX	$\begin{bmatrix} 1&0&0&0 \\ 0&1&0&0 \\ 0&0&0&1 \\ 0&0&1&0 \end{bmatrix}$	flips target if control is $\|1\rangle$
controlled-Z	CZ	$\mathrm{diag}(1,1,1,-1)$	phase -1 on $\|11\rangle$
controlled-U	CU	$\|0\rangle\langle 0\| \otimes \boldsymbol{I} + \|1\rangle\langle 1\| \otimes U$	U on target if control is $\|1\rangle$
swap	SWAP	$\begin{bmatrix} 1&0&0&0 \\ 0&0&1&0 \\ 0&1&0&0 \\ 0&0&0&1 \end{bmatrix}$	$\|xy\rangle \to \|yx\rangle$

The most important controlled gate is the controlled-not or *CNOT* gate, CX, obtained with $U = X$,

$$\mathrm{CNOT}|00\rangle = |00\rangle, \quad \mathrm{CNOT}|01\rangle = |01\rangle, \quad \mathrm{CNOT}|10\rangle = |11\rangle, \quad \mathrm{CNOT}|11\rangle = |10\rangle, \tag{2.84}$$

that is, $\mathrm{CNOT}|xy\rangle = |x, y\oplus x\rangle$ with $\oplus$ the addition modulo two. On a superposition of the control it does something no classical gate can do: applied to $(|0\rangle + |1\rangle)/\sqrt{2} \otimes |0\rangle$ it produces the Bell state $|\Phi^+\rangle$, because the sum over the two values of the control is carried through to the target. The controlled-Z gate $\mathrm{CZ} = \mathrm{diag}(1,1,1,-1)$ is symmetric in control and target and related to the CNOT by conjugation with Hadamard gates on the target, $\mathrm{CNOT} = (\boldsymbol{I}\otimes H)\,\mathrm{CZ}\,(\boldsymbol{I}\otimes H)$, a direct consequence of $HZH = X$. Finally the *SWAP* gate exchanges the two qubits, $\mathrm{SWAP}|xy\rangle = |yx\rangle$; it is not a controlled gate, but it can be built from three CNOT gates with alternating control and target, a circuit identity shown in Fig. 2.5 and verified in `circuit.py`. The three-qubit Toffoli gate, a CNOT with two controls, completes the standard repertoire; it is universal for classical reversible computation.

2.7.4 Universal gate sets

A set of gates is *universal* if every unitary operation on N qubits can be approximated to arbitrary precision by a circuit built from that set. Two facts make universality tractable. First, any unitary on N qubits can be decomposed exactly into single-qubit gates and CNOT gates, so the pair {arbitrary single-qubit rotations, CNOT} is universal; this is the standard choice when compiling for hardware whose native operations are parametrized rotations and one entangling gate. Second, the single-qubit rotations can themselves be approximated by a finite set: the gates H and T generate a dense subset of $\mathrm{SU}(2)$, so {H, T, CNOT} is universal, and the Solovay–Kitaev theorem guarantees that an accuracy ε costs only $\mathcal{O}(\log^c(1/\varepsilon))$ gates with a small constant c. The discrete set matters because the fault-tolerant constructions of Chapter 9 can implement a finite set of gates on encoded qubits, and T is typically by far the most expensive of them. The gates H, S and CNOT alone generate the *Clifford group*, which is not universal: circuits made of Clifford gates can be simulated efficiently on a classical computer (the Gottesman–Knill theorem), a reminder that entanglement by itself is not the source of quantum computational power.

2.8 Quantum circuits

Quantum circuits are diagrammatic representations of quantum algorithms; they are read a little like music, with one line per qubit. The horizontal direction is time, running from left to right. Each qubit is a horizontal line, a *quantum wire*, labelled on the left with its initial state, usually $|0\rangle$. Gates are boxes placed on the wires, and are applied in chronological order from left to right, so that the circuit A followed by B corresponds to the operator product BA, in the opposite order from the diagram. A controlled gate is drawn as a dot on the control wire connected by a vertical line to the gate on the target wire, and the CNOT target is drawn as $\oplus$. Measurements are drawn as a meter symbol and turn the quantum wire into a classical one, drawn doubled. Figure 2.5 shows the circuit that prepares $|\Phi^+\rangle$ from $|00\rangle$, the other three Bell states being obtained by an additional X on the second qubit and/or Z on the first, and the decomposition of the SWAP gate into three CNOT gates.

Let us follow the Bell circuit step by step, since the same pattern returns in every algorithm of this book. The Hadamard gate on the first qubit produces $\frac{1}{\sqrt{2}}(|0\rangle+|1\rangle)\otimes|0\rangle = \frac{1}{\sqrt{2}}(|00\rangle+|10\rangle)$, and the CNOT then flips the second qubit in the second term only, $\frac{1}{\sqrt{2}}(|00\rangle+|11\rangle) = |\Phi^+\rangle$. Two gates and no interaction with anything else have produced a maximally entangled state; the inverse circuit, a CNOT followed by a Hadamard, maps the four Bell states back onto the four computational basis states and is therefore a *Bell measurement* when followed by

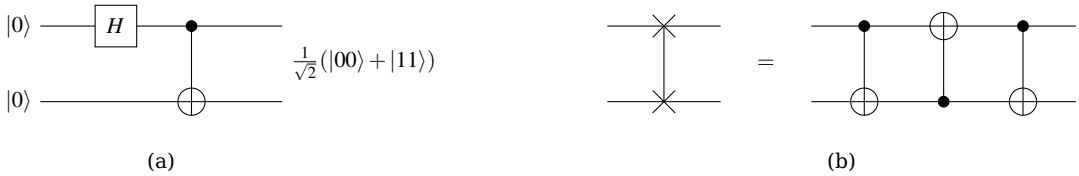


Fig. 2.5 (a) The circuit that prepares the Bell state $|\Phi^+\rangle$: a Hadamard gate on the first qubit followed by a CNOT with the first qubit as control. (b) The SWAP gate as three CNOT gates with alternating control and target. Both identities are verified in `circuit.py`.

a measurement in the computational basis. This is the tool we need for the protocols of Section 2.9.

Describing circuits in code is as natural as drawing them. The class `Circuit` in `circuit.py` is a list of `Gate` objects, each holding a matrix, its target qubits and optional control qubits; a circuit is executed by applying the gates in order to a `StateVector`, and its full $2^N \times 2^N$ unitary is obtained by running it on every basis state. Chapters 5 and 7 build their Trotter and variational circuits with the same class.

```
from circuit import Circuit
import gates as G

bell = Circuit(2).h(0).cnot(0, 1)
print(bell.draw())          # the ASCII version of the Bell circuit
print(bell.run())           # (0.7071)|00> + (0.7071)|11>

swap = Circuit(2).cnot(0, 1).cnot(1, 0).cnot(0, 1)
assert np.allclose(swap.unitary(), G.SWAP)
print(bell, "depth", bell.depth)
```

Running the two lines above prints

```
q0: |0>--[H]--*---
q1: |0>-------(+)--
(+0.7071+0.0000j)|00> + (+0.7071+0.0000j)|11>
```

The *depth* of a circuit, the number of layers of gates that act on disjoint qubits and can be executed in parallel, is the relevant measure of its running time on hardware and enters the counting argument of Section 2.1.

2.9 Two protocols: superdense coding and teleportation

Entanglement is a resource. Two protocols, both using a single Bell pair shared between two parties conventionally called Alice and Bob, show what the resource buys and how the rules of Sections 2.4 and 2.7 combine into something a classical channel cannot do.

2.9.1 Superdense coding

Alice wants to send Bob two classical bits $b_1 b_2$ but may transmit only one qubit. If they share the Bell state $|\Phi^+\rangle$, with Alice holding the first qubit, she can do it **[?]**. She applies to her qubit one of the four Pauli operations

$$I|\Phi^+\rangle = |\Phi^+\rangle, \quad X|\Phi^+\rangle = |\Psi^+\rangle, \quad Z|\Phi^+\rangle = |\Phi^-\rangle, \quad ZX|\Phi^+\rangle = |\Psi^-\rangle, \tag{2.85}$$

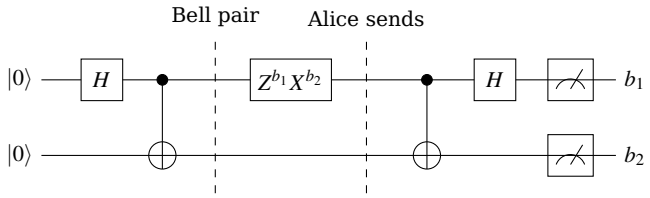


Fig. 2.6 Superdense coding. A Bell pair is shared, Alice encodes two bits by one of four Pauli operations on her qubit and sends it, and Bob decodes with the inverse Bell circuit and two measurements.

for the messages 00, 01, 10 and 11 respectively, and sends her qubit to Bob. Since the four Bell states are orthogonal, Bob can distinguish them perfectly: he applies the inverse of the Bell circuit, a CNOT followed by a Hadamard on the first qubit, which maps $|\Phi^+\rangle, |\Psi^+\rangle, |\Phi^-\rangle, |\Psi^-\rangle$ onto $|00\rangle, |01\rangle, |10\rangle, |11\rangle$, and measures both qubits in the computational basis to read $b_1 b_2$. A single qubit thus carries two bits of classical information, twice what a qubit could carry without shared entanglement; the entanglement was consumed in the process. The circuit is shown in Fig. 2.6, and `protocols.py` runs it for all four messages.

2.9.2 Quantum teleportation

Teleportation is the converse task: Alice holds a qubit in an unknown state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ and wants Bob to end up with it, but she may only send classical bits [**?**]. Since she cannot measure α and β, and cannot copy the state either (the no-cloning theorem, Exercise 10), she again uses a shared Bell pair. Number the qubits so that Alice holds qubit 0 in the state $|\psi\rangle$ and qubit 1, while Bob holds qubit 2; qubits 1 and 2 are in $|\Phi^+\rangle$. The initial three-qubit state is

$$|\psi\rangle \otimes |\Phi^+\rangle = \frac{1}{\sqrt{2}}\left[\alpha|0\rangle(|00\rangle + |11\rangle) + \beta|1\rangle(|00\rangle + |11\rangle)\right]. \tag{2.86}$$

Alice applies a CNOT from qubit 0 to qubit 1 and then a Hadamard to qubit 0, the inverse Bell circuit. The CNOT gives $\frac{1}{\sqrt{2}}[\alpha|0\rangle(|00\rangle + |11\rangle) + \beta|1\rangle(|10\rangle + |01\rangle)]$, and the Hadamard turns $|0\rangle$ into $(|0\rangle + |1\rangle)/\sqrt{2}$ and $|1\rangle$ into $(|0\rangle - |1\rangle)/\sqrt{2}$. Collecting the terms according to the values $m_0 m_1$ of Alice's two qubits,

$$\begin{aligned} |\Psi\rangle = \frac{1}{2}\Big[&|00\rangle \otimes (\alpha|0\rangle + \beta|1\rangle) + |01\rangle \otimes (\alpha|1\rangle + \beta|0\rangle) \\ &+ |10\rangle \otimes (\alpha|0\rangle - \beta|1\rangle) + |11\rangle \otimes (\alpha|1\rangle - \beta|0\rangle)\Big]. \end{aligned} \tag{2.87}$$

Alice now measures her two qubits and obtains each of the four outcomes with probability $1/4$; by Eq. (2.6) Bob's qubit collapses onto the corresponding state in Eq. (2.87), which is $|\psi\rangle$ up to a known Pauli operation. Alice sends the two bits $m_0 m_1$ to Bob over a classical channel, and Bob applies the correction

$$X^{m_1} Z^{m_0} \quad \text{to his qubit, after which it is in the state } |\psi\rangle. \tag{2.88}$$

Note the order: Z acts after X, so that for $m_0 m_1 = 11$ the state $\alpha|1\rangle - \beta|0\rangle$ first becomes $\alpha|0\rangle - \beta|1\rangle$ and then $\alpha|0\rangle + \beta|1\rangle$. The state has been transferred without any quantum system travelling from Alice to Bob, and without Alice learning anything about α and β; her outcomes are uniformly random. Nothing travels faster than light, since Bob's qubit is in the maximally mixed state $\boldsymbol{I}/2$ until the two classical bits arrive, and the original state is destroyed on Alice's side, in agreement with no-cloning. Figure 2.7 shows the circuit, and `protocols.py` verifies

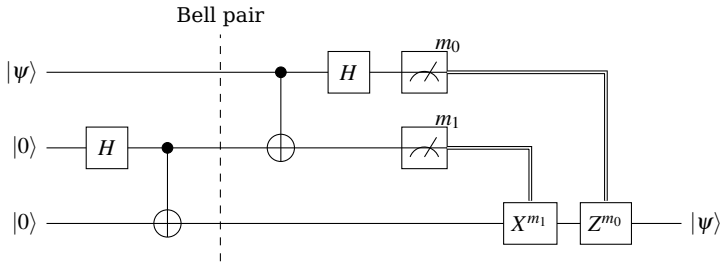


Fig. 2.7 Quantum teleportation. Alice (upper two wires) performs a Bell measurement on her unknown qubit and her half of a Bell pair, sends the two outcomes to Bob over a classical channel (double lines), and Bob (lower wire) applies the Pauli correction of Eq. (2.88).

Eq. (2.88) for two hundred random input states by comparing Bob's reduced density matrix with $|\psi\rangle\langle\psi|$. The same circuit, with the classical corrections replaced by controlled gates, is used in Chapter 9 to move logical qubits around.

The implementation follows the circuit line by line; the measurement outcomes are drawn by the simulator and the corrections are ordinary Python conditionals, exactly as a control program on real hardware would do it.

```
from statevector import StateVector
from density_matrix import DensityMatrix

def teleport(psi, rng=None):
    sv = StateVector.product(psi, [1, 0], [1, 0]) # qubit 0: |psi>, qubits 1,2: |00>
    sv.h(1).cnot(1, 2)                            # Bell pair between Alice (1) and Bob (2)
    sv.cnot(0, 1).h(0)                            # Alice's Bell measurement, basis change
    m0, _ = sv.measure(0, rng=rng)
    m1, _ = sv.measure(1, rng=rng)
    if m1: sv.x(2)                                # classical corrections
    if m0: sv.z(2)
    return DensityMatrix.from_state(sv).partial_trace(2), (m0, m1)
```

2.10 A first Hamiltonian: the two-level system

We close the chapter with the simplest quantum many-body problem there is, a system with two states, because it already contains the ingredients of the variational algorithms of Chapter 7: a Hamiltonian written in terms of Pauli operators, a parametrized circuit that prepares trial states, and an energy that is minimized over the parameters. Consider the real symmetric Hamiltonian

$$H = H_0 + \lambda H_I, \qquad H_0 = \begin{bmatrix} E_1 & 0 \\ 0 & E_2 \end{bmatrix}, \qquad H_I = \begin{bmatrix} V_{11} & V_{12} \\ V_{21} & V_{22} \end{bmatrix}, \tag{2.89}$$

where H_0 plays the role of a non-interacting one-body part whose eigenstates $|0\rangle$ and $|1\rangle$, with energies E_1 and E_2, define the computational basis, H_I an interaction with $V_{12} = V_{21}$ by hermiticity, and $\lambda \in [0,1]$ a coupling strength interpolating between the non-interacting and the fully interacting system. Since $\boldsymbol{I}$ and Z span the diagonal 2×2 matrices and X the symmetric off-diagonal ones, we can rewrite both parts in the Pauli basis,

$$H_0 = E_+ \boldsymbol{I} + E_- Z, \qquad H_I = V_+ \boldsymbol{I} + V_- Z + V_o X, \tag{2.90}$$

with $E_\pm = (E_1 \pm E_2)/2$, $V_\pm = (V_{11} \pm V_{22})/2$ and $V_o = V_{12}$. This rewriting is the one-qubit version of the mappings of Chapter 4: every Hamiltonian on N qubits is a linear combination of the 4^N

Table 2.4 Eigenvalues of the two-level Hamiltonian (2.89) and the weight of $|0\rangle$ in the ground state, as functions of the coupling λ. Produced by `two_level_system.py`.

λ	E_0	E_1	$\lvert\langle 0\rvert E_0\rangle\rvert^2$
0	0.0000	4.0000	1.0000
0.3	0.8984	3.1016	0.9993
0.6	1.7668	2.2332	0.9287
2/3	1.8667	2.1333	0.5000
0.7	1.8280	2.1720	0.2094
1	0.9802	3.0198	0.0097

tensor products of Pauli matrices, and its expectation value is the corresponding combination of expectation values of Pauli strings, each of which can be measured as in Section 2.4.2.

We choose $E_1 = 0$, $E_2 = 4$, $V_{11} = -V_{22} = 3$ and $V_{12} = V_{21} = 0.2$. The left panel of Fig. 2.8 shows the two eigenvalues as functions of λ. The diagonal elements $E_1 + \lambda V_{11}$ and $E_2 + \lambda V_{22}$ cross at $\lambda = 2/3$, but the small coupling V_o turns the crossing into an *avoided crossing* and interchanges the character of the eigenvectors: for $\lambda \leq 2/3$ the ground state is dominated by $|0\rangle$, with a weight above 90%, while at $\lambda = 1$ its $|0\rangle$ component has dropped to 1%, as Table 2.4 shows. The ground-state energy at $\lambda = 1$ is $E_0 = 0.9802$.

Diagonalizing a 2×2 matrix is of course trivial, but let us pretend we could only prepare states with gates and measure Pauli operators, which is the situation on a quantum computer. From Section 2.7.2 we know that every point on the Bloch sphere can be reached from $|0\rangle$ by two rotations, and we take as trial state

$$|\psi(\theta,\phi)\rangle = R_y(\phi)R_x(\theta)|0\rangle, \tag{2.91}$$

a circuit with two parametrized gates. Its energy,

$$E(\theta,\phi) = \langle\psi(\theta,\phi)|(E_+ + \lambda V_+)\boldsymbol{I} + (E_- + \lambda V_-)Z + \lambda V_o X|\psi(\theta,\phi)\rangle, \tag{2.92}$$

requires only the expectation values of Z and X in the trial state, which we would obtain from measurements in the computational basis with and without a preceding Hadamard gate, and by the variational principle it is bounded from below by the exact ground-state energy. The right panel of Fig. 2.8 shows $E(\theta,\phi)$ at $\lambda = 1$ on a grid of angles in steps of ten degrees. The single guess $\theta = \pi/2$, $\phi = 0.2\pi$ gives $E = 2.0000$, not impressive; the best point of the coarse grid, $\theta = 170^\circ$, $\phi = 10^\circ$, gives $E = 0.9960$, within 0.016 of the exact value, and a one-degree grid reduces the error to 1.5×10^{-5}. Scanning a grid is clearly not the way to proceed for more than a handful of parameters; computing the gradient of $E(\theta,\phi)$ with respect to the angles and following it downhill is, and this leads to the variational quantum eigensolver of Chapter 7, where the same Hamiltonian, extended to two qubits, serves as the first test case.

The class `TwoLevelSystem` in `two_level_system.py` holds the Hamiltonian in both forms and the trial state:

```
import numpy as np
import gates as G

class TwoLevelSystem:
    def __init__(self, E1=0.0, E2=4.0, V11=3.0, V22=-3.0, V12=0.2):
        self.E_plus, self.E_minus = (E1 + E2) / 2, (E1 - E2) / 2
        self.V_plus, self.V_minus, self.V_o = (V11 + V22) / 2, (V11 - V22) / 2, V12

    def hamiltonian(self, lam):
        return ((self.E_plus + lam * self.V_plus) * G.I
                + (self.E_minus + lam * self.V_minus) * G.Z
```

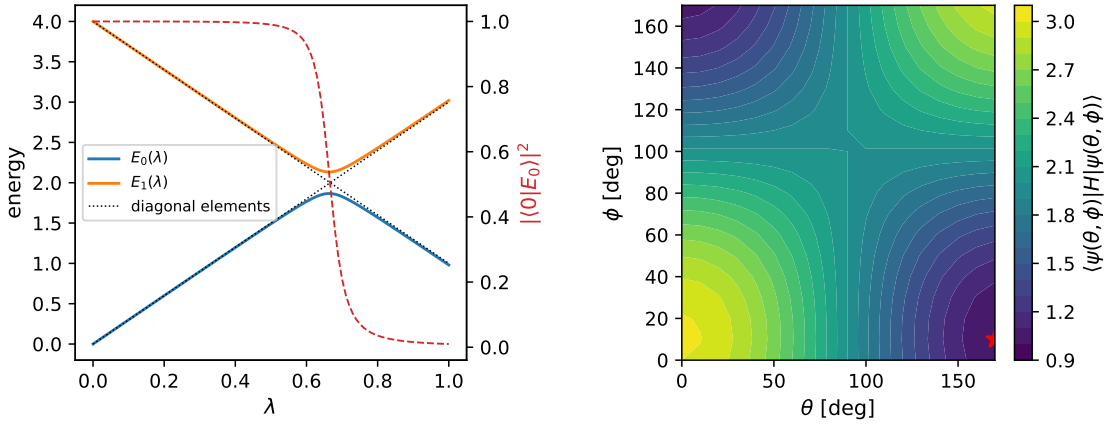


Fig. 2.8 Left: eigenvalues of the two-level Hamiltonian (2.89) as functions of λ (solid), the diagonal matrix elements (dotted) and the weight of $|0\rangle$ in the ground state (dashed, right axis). Right: the energy of the trial state $R_y(\phi)R_x(\theta)|0\rangle$ at $\lambda = 1$; the star marks the lowest point of the ten-degree grid. Produced by `two_level_system.py`.

```
            + lam * self.V_o * G.X)

    def ansatz(self, theta, phi):
        return G.Ry(phi) @ G.Rx(theta) @ np.array([1, 0], dtype=complex)

    def energy(self, theta, phi, lam):
        psi = self.ansatz(theta, phi)
        return float(np.real(np.vdot(psi, self.hamiltonian(lam) @ psi)))

tls = TwoLevelSystem()
E0 = np.linalg.eigvalsh(tls.hamiltonian(1.0))[0]
angles = np.deg2rad(np.arange(0, 180, 10))
grid = np.array([[tls.energy(th, ph, 1.0) for ph in angles] for th in angles])
print(f"exact E0 = {E0:.4f}, best grid point E = {grid.min():.4f}")
```

2.11 Exercises

1. **Commutator identities.** Prove that for operators $\widehat{A}$, $\widehat{B}$ and $\widehat{C}$
   a. $[\widehat{A}+\widehat{B},\widehat{C}] = [\widehat{A},\widehat{C}]+[\widehat{B},\widehat{C}]$;
   b. $[\widehat{A},\widehat{B}\widehat{C}] = [\widehat{A},\widehat{B}]\widehat{C}+\widehat{B}[\widehat{A},\widehat{C}]$;
   c. $[\widehat{A},[\widehat{B},\widehat{C}]]+[\widehat{B},[\widehat{C},\widehat{A}]]+[\widehat{C},[\widehat{A},\widehat{B}]] = 0$ (the Jacobi identity).
2. **Pauli matrices.** Derive the commutation rules $[\sigma_i,\sigma_j]$ for $i,j = x,y,z$ and show that $XX = YY = ZZ = \boldsymbol{I}$. Which of the Pauli matrices has the computational basis as its eigenbasis, and what are the eigenvalues? Show that $\sigma_i\sigma_j = \delta_{ij}\boldsymbol{I} + i\varepsilon_{ijk}\sigma_k$ and use it to prove that $HZH = X$ with the Hadamard matrix $H = (X+Z)/\sqrt{2}$.
3. **Shared eigenvectors.** Prove that two Hermitian operators $\widehat{A}$ and $\widehat{B}$ that commute share a basis of eigenstates. Why does this matter for the simultaneous measurement of two observables?
4. **One-qubit gates in code.** Write a function that sets up the one-qubit computational basis and applies the Pauli matrices, the Hadamard gate and the phase gate to the basis states.

Verify Eqs. (2.23) and (2.25) and check that all five matrices are unitary. Then determine the Bloch angles of the resulting states with the functions of Section 2.3.

5. **Traces and exponentials.** Prove that the trace is cyclic, $\mathrm{Tr}(\boldsymbol{ABC}) = \mathrm{Tr}(\boldsymbol{CAB}) = \mathrm{Tr}(\boldsymbol{BCA})$. Let $\boldsymbol{A}$ satisfy $\boldsymbol{A}^2 = \boldsymbol{I}$ and let α be real; show from the power series that $e^{i\alpha \boldsymbol{A}} = \boldsymbol{I}\cos\alpha + i\boldsymbol{A}\sin\alpha$. Does this apply to the Pauli matrices? Use it to verify the rotation matrices of Eq. (2.81) and the decomposition (2.82).
6. **Reduced density matrices.** For each of the four Bell states find the reduced density matrix of each qubit by computing the partial trace by hand, and compare with `entanglement.py`. Then suppose the composite system is in the product state $|a\rangle \otimes |b\rangle$ of pure states of A and B; show that the reduced density matrix of A is the pure state $|a\rangle\langle a|$, and similarly for B.
7. **Bloch angles of Pauli eigenstates.** Determine the Bloch angles θ and ϕ of the eigenstates of each Pauli matrix and compare with Table 2.1. Show that two states are orthogonal if and only if their Bloch vectors are antipodal, and that $|\langle\phi|\psi\rangle|^2 = (1 + \boldsymbol{r}_\phi \cdot \boldsymbol{r}_\psi)/2$.
8. **Mixed states of a qubit.** Show that Eq. (2.53) has eigenvalues $(1 \pm |\boldsymbol{r}|)/2$, so that positivity requires $|\boldsymbol{r}| \le 1$, and that the purity is $(1 + |\boldsymbol{r}|^2)/2$. Compute the von Neumann entropy as a function of $|\boldsymbol{r}|$ and plot it. Which ensembles of pure states produce the state with $\boldsymbol{r} = (0, 0, 1/2)$?
9. **Entanglement entropy.** Write the state $\cos\theta|00\rangle + \sin\theta|11\rangle$ in Schmidt form and derive the analytic expression for its entanglement entropy. Extend the computation to the state $\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle - |11\rangle)$; is it entangled? Answer both by hand and with the Schmidt decomposition of `entanglement.py`.
10. **No cloning.** Suppose a unitary U on two qubits copies an arbitrary state, $U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle$. Apply it to $|0\rangle$, $|1\rangle$ and $|+\rangle$ and derive a contradiction with linearity. Explain why the teleportation protocol does not violate this theorem.
11. **Circuits.** Using the `Circuit` class, verify that (a) $\mathrm{CNOT} = (\boldsymbol{I}\otimes H)\,\mathrm{CZ}\,(\boldsymbol{I}\otimes H)$, (b) a CNOT with control and target exchanged equals $(H\otimes H)\,\mathrm{CNOT}\,(H\otimes H)$, and (c) the circuit of Fig. 2.5(a) followed by X on the second qubit and Z on the first produces $|\Psi^-\rangle$. Draw each circuit.
12. **Superdense coding and teleportation.** Run `protocols.py` and modify the superdense-coding function so that the shared state is $|\Psi^-\rangle$ instead of $|\Phi^+\rangle$; which encoding operations must Alice now use? Then modify the teleportation function so that Bob's correction is omitted, and compute Bob's reduced density matrix averaged over Alice's outcomes. Interpret the result.
13. **The two-level system.** Show that with $E_\pm = (E_1 \pm E_2)/2$ the non-interacting part can be written as $H_0 = E_+\boldsymbol{I} + E_- Z$, and that with $V_\pm = (V_{11} \pm V_{22})/2$ and $V_o = V_{12} = V_{21}$ the interaction becomes $H_I = V_+\boldsymbol{I} + V_- Z + V_o X$. Explain how $\langle X\rangle$ is measured on a device that only measures in the computational basis, using $X = HZH$. Finally compute the gradient of $E(\theta,\phi)$ in Eq. (2.92) analytically and use gradient descent to find the minimum; compare with the grid search of Fig. 2.8.

Programs for this chapter

Every number and figure in this chapter comes from the programs in `BookPrograms/chapter2/` and from the shared modules in `BookPrograms/common/` that this chapter introduces and the later chapters reuse. All of them are imported by the notebook `Notebooks/chapter2/chapter2.ipynb`, and all run on `numpy` and `matplotlib` alone.

`common/gates.py` The gate matrices of Table 2.3, the rotation gates and $U(\theta,\phi,\lambda)$, the controlled-U construction, and helpers for commutators, tensor products and unitarity checks; run as a script it verifies every gate identity quoted in the text. Instantaneous.

`common/statevector.py` The class `StateVector`: the 2^N amplitudes of Eq. (2.1), gates on arbitrary subsets of qubits by tensor contraction, marginal probabilities, projective measurement with collapse, sampling of shots, expectation values. Instantaneous.

`common/circuit.py` The classes `Gate` and `Circuit`: gate lists with controls, execution on a `StateVector`, the full unitary, depth, inverse and an ASCII drawing; run as a script it checks the Bell circuits and the SWAP identity of Fig. 2.5. Instantaneous.

`common/density_matrix.py` The class `DensityMatrix`: ensembles, Bloch vector, purity, Born rule and post-measurement state, unitary evolution, partial trace and von Neumann entropy, and the Shannon entropy. Instantaneous.

`common/bookplot.py` Figure directory and matplotlib conventions shared by all chapters.

`dirac_algebra.py` The worked examples of Section 2.2: inner and outer products, projectors, Pauli algebra, tensor products, spectral decomposition of a Hermitian matrix. Instantaneous.

`bloch_sphere.py` Bloch angles and Bloch vectors, Table 2.1 and Fig. 2.1 with the Larmor precession of Eq. (2.78). A few seconds.

`measurement.py` The Born rule, post-measurement states, the X-basis measurement, marginals in a three-qubit register and the shot-noise study of Fig. 2.2. A few seconds.

`decoherence.py` The dephasing and amplitude-damping channels in Kraus form and Fig. 2.3. A second.

`entanglement.py` Bell states, measurement correlations, reduced density matrices, the Schmidt decomposition through the singular value decomposition, Table 2.2 and Fig. 2.4. A second.

`protocols.py` Superdense coding for all four messages and teleportation of random states, both on the state-vector simulator, with the circuit of Fig. 2.7. A second.

`two_level_system.py` The two-level Hamiltonian in matrix and Pauli form, Table 2.4, the grid search over the trial state (2.91) and Fig. 2.8. A few seconds.

Chapter 3
Quantum Many-Body Physics Primer

Abstract Here we introduce the physical systems and Hamiltonians relevant to many-body physics. Typical models are presented such as spin chains (for example the Heisenberg and Ising models), the Lipkin and Pairing models and other models, illustrating the complexity of interacting particles in various many-body contexts (condensed matter physics, atomic physics and subnuclear physics). We review second quantization formalism, with creation and annihilation operators for fermions and bosons, and discuss Fock space. The challenges of solving many-body problems (computing ground-state energies or simulating dynamics) on classical computers are highlighted. We mention classical methods like exact diagonalization, unitary and standard Coupled Cluster theory and tensor-network approaches to contrast with quantum methods.

3.1 Introduction

The systems we want to study on a quantum computer are collections of interacting particles: electrons in atoms, molecules and solids, nucleons in nuclei, cold atoms in traps, spins on a lattice. What they have in common is that the number of degrees of freedom grows linearly with the number of particles while the dimension of the Hilbert space grows exponentially, and this is both the reason why such systems are hard for classical computers and the reason why quantum computers, whose state space also grows exponentially with the number of qubits, are a natural tool for them. This chapter collects the many-body physics that the rest of the book relies on. It is a primer rather than a treatise: every concept is introduced in the minimal form in which it is needed later, and every one of them is illustrated on the same four model Hamiltonians, so that the reader can follow the numbers from a formula to a program to a table.

Section 3.2 introduces second quantization, the language in which many-body Hamiltonians are written and in which the mapping to qubits of Chapter 4 is formulated. The models themselves are presented in Section 3.3: the Lipkin model, the pairing model, the Hubbard model and the Heisenberg model. Each of them can be solved exactly for small sizes, each has a symmetry that reduces the problem, and each is used again and again in the later chapters as a test case for quantum algorithms. Section 3.4 discusses exact diagonalization, or full configuration interaction, the method against which everything else is measured, and explains why it cannot scale. Section 3.5 introduces the mean-field, or Hartree–Fock, approximation, together with two mathematical tools that reappear throughout the book: the Baker–Campbell–Hausdorff expansion and the Trotter–Suzuki splitting of exponentials of non-commuting operators. Section 3.6 presents coupled-cluster theory, the most successful classical wave-function method for correlated fermions, and its unitary variant, which cannot

be evaluated efficiently on a classical computer but is the natural ansatz for the variational quantum algorithms of Chapter 7. The chapter closes with exercises and with the list of the programs that produce every number quoted in it.

Throughout the chapter we use the notation established in Chapter 2: $\hbar = 1$, $|0\rangle$ and $|1\rangle$ for the two states of a two-level system, and the Pauli matrices X, Y and Z. Single-particle states are labelled by p,q,r,s when they can be any state, by i,j,k,l when they are occupied in a reference state and by a,b,c,d when they are unoccupied. Matrices are written in bold when the distinction from an operator matters.

3.2 Second quantization

3.2.1 Creation and annihilation operators

Consider a set of M orthonormal single-particle states $|\phi_p\rangle$, $p = 0,1,\ldots,M-1$, which could be the eigenstates of a one-body Hamiltonian, a set of lattice sites with spin, or any other convenient basis. A system of N identical fermions occupying N of these states is described, in first quantization, by a Slater determinant

$$\Phi(x_1,\ldots,x_N) = \frac{1}{\sqrt{N!}} \begin{vmatrix} \phi_{p_1}(x_1) & \phi_{p_1}(x_2) & \cdots & \phi_{p_1}(x_N) \\ \phi_{p_2}(x_1) & \phi_{p_2}(x_2) & \cdots & \phi_{p_2}(x_N) \\ \vdots & \vdots & & \vdots \\ \phi_{p_N}(x_1) & \phi_{p_N}(x_2) & \cdots & \phi_{p_N}(x_N) \end{vmatrix}, \tag{3.1}$$

which changes sign when two particles are exchanged and vanishes when two of the indices p_k coincide, so that the Pauli principle is built in. Working with determinants directly is clumsy: the number of particles is fixed, the antisymmetrization has to be carried along in every calculation, and matrix elements of operators between determinants require the bookkeeping of the Slater–Condon rules. Second quantization replaces all of this by an algebra of operators.

We introduce, for every single-particle state p, a creation operator $a_p^\dagger$ and its Hermitian conjugate, the annihilation operator a_p, and define the Slater determinant of Eq. (3.1) as the state obtained by creating the particles one after the other on the vacuum $|0\rangle$, the state with no particles,

$$|\Phi\rangle = |p_1 p_2 \cdots p_N\rangle = a_{p_1}^\dagger a_{p_2}^\dagger \cdots a_{p_N}^\dagger |0\rangle, \qquad a_p|0\rangle = 0 \text{ for all } p. \tag{3.2}$$

The antisymmetry of the determinant is then equivalent to the requirement that the creation operators anticommute among themselves, and the orthonormality of the single-particle basis to the requirement that creation and annihilation operators anticommute up to a Kronecker delta,

$$\{a_p, a_q^\dagger\} = a_p a_q^\dagger + a_q^\dagger a_p = \delta_{pq}, \qquad \{a_p, a_q\} = \{a_p^\dagger, a_q^\dagger\} = 0. \tag{3.3}$$

Everything else follows from Eq. (3.3). Setting $p = q$ in the second relation gives $(a_p^\dagger)^2 = 0$: a state cannot be occupied twice, which is the Pauli principle. The operator

$$\hat{n}_p = a_p^\dagger a_p \tag{3.4}$$

satisfies $\hat{n}_p^2 = a_p^\dagger a_p a_p^\dagger a_p = a_p^\dagger (1 - a_p^\dagger a_p) a_p = \hat{n}_p$, so its eigenvalues are 0 and 1: it counts whether state p is occupied, and $\hat{N} = \sum_p \hat{n}_p$ counts the particles. The space spanned by all states of the form of Eq. (3.2), for all particle numbers from 0 to M, is the *Fock space*. Its dimension is 2^M, and it is the direct sum of the N-particle spaces of dimension $\binom{M}{N}$.

Because the eigenvalues of every $\hat{n}_p$ are 0 or 1, a basis state of Fock space is completely specified by a string of M bits, one per single-particle state,

$$|n_0 n_1 \cdots n_{M-1}\rangle, \qquad n_p \in \{0,1\}, \tag{3.5}$$

the *occupation-number representation*. This is the representation in which second quantization and quantum computing meet: a register of M qubits has exactly the basis states of Eq. (3.5), and in Chapter 4 the qubit $|1\rangle$ will mean "occupied". In this chapter we already adopt the corresponding conventions: single-particle state p is the p-th bit from the left, and the ordered product of creation operators in Eq. (3.2) is taken with increasing indices, $p_1 < p_2 < \cdots < p_N$, so that the state with bits n_p is

$$|n_0 n_1 \cdots n_{M-1}\rangle = \left(a_0^\dagger\right)^{n_0} \left(a_1^\dagger\right)^{n_1} \cdots \left(a_{M-1}^\dagger\right)^{n_{M-1}} |0\rangle. \tag{3.6}$$

The action of $a_p^\dagger$ on such a state is then fixed by anticommuting $a_p^\dagger$ past the operators to its left,

$$\begin{aligned} a_p^\dagger |n_0 \cdots n_p \cdots n_{M-1}\rangle &= \delta_{n_p,0}\,(-1)^{\sum_{q<p} n_q}\, |n_0 \cdots 1_p \cdots n_{M-1}\rangle, \\ a_p |n_0 \cdots n_p \cdots n_{M-1}\rangle &= \delta_{n_p,1}\,(-1)^{\sum_{q<p} n_q}\, |n_0 \cdots 0_p \cdots n_{M-1}\rangle. \end{aligned} \tag{3.7}$$

The sign counts the occupied states with an index smaller than p, that is the number of operators that $a_p^\dagger$ has to jump over to reach its canonical position. This *parity* is the whole content of fermionic statistics in the occupation-number representation, and it is the origin of the strings of Z operators in the Jordan–Wigner transformation of Chapter 4. As an example, with $M = 16$ and the state $|3\,6\,10\,13\rangle$, that is the bit string 0001001000100100, the program `second_quantization.py` gives

$$\begin{aligned} a_4^\dagger |3\,6\,10\,13\rangle &= -|0001101000100100\rangle, \qquad a_6^\dagger |3\,6\,10\,13\rangle = 0, \\ a_{12}^\dagger |3\,6\,10\,13\rangle &= -|0001001000101100\rangle, \end{aligned}$$

where the first sign comes from the single occupied state 3 to the left of 4, and the last from the three states $3, 6, 10$ to the left of 12. The rule is a few lines of code:

```
def apply_creation(bits, p):
    """a^dagger_p on the bit string `bits`; returns (sign, new bits) or (0, None)."""
    if bits[p] == "1":
        return 0, None
    sign = (-1) ** bits[:p].count("1")
    return sign, bits[:p] + "1" + bits[p + 1:]
```

3.2.2 Operators as matrices

Since Fock space has dimension 2^M, every operator on it is a $2^M \times 2^M$ matrix, and for small M it is instructive, and for the programs of this book practical, to construct these matrices explicitly. For a single state, $M = 1$, the Fock space is spanned by $|0\rangle$ and $|1\rangle$ and

$$a = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} = |0\rangle\langle 1|, \qquad a^\dagger = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} = |1\rangle\langle 0|, \qquad \hat{n} = a^\dagger a = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}. \tag{3.8}$$

In the language of Chapter 2, $a^\dagger = (X - iY)/2$ and $a = (X + iY)/2$ are the raising and lowering operators of a qubit, and $\hat{n} = (I - Z)/2$. For M states the basis is the tensor product of M such

two-dimensional spaces, and the naive guess $a_p = I\otimes\cdots\otimes a\otimes\cdots\otimes I$ fails: operators on different factors of a tensor product commute, whereas Eq. (3.3) demands that they anticommute. The sign rule of Eq. (3.7) shows what is missing. The factor $(-1)^{\sum_{q<p} n_q}$ is the eigenvalue of $\prod_{q<p}(1-2\hat{n}_q) = \prod_{q<p} Z_q$ on the state, so that

$$a_p = \underbrace{Z\otimes Z\otimes\cdots\otimes Z}_{p \text{ factors}}\otimes a\otimes \underbrace{I\otimes\cdots\otimes I}_{M-p-1 \text{ factors}}, \qquad a_p^\dagger = Z^{\otimes p}\otimes a^\dagger\otimes I^{\otimes(M-p-1)}. \tag{3.9}$$

Because Z anticommutes with a and $a^\dagger$ on the same factor and commutes with everything else, two operators a_p and a_q with $p<q$ pick up exactly one relative sign, from the Z on factor p of a_q against the a on factor p of a_p, and this is what turns the commutation of tensor factors into anticommutation. The program `fockspace.py` builds these matrices and checks all M^2 anticommutators numerically; for $M=4$ the largest violation is zero to machine precision. Equation (3.9) is the Jordan–Wigner transformation of Chapter 4 written as a matrix, and the string of Z's is what will be called the parity string there. We have not chosen it; it is forced on us by the anticommutation relations.

3.2.3 One- and two-body operators

A Hamiltonian for identical particles contains a one-body part, the kinetic energy and an external potential, and a two-body interaction. In second quantization

$$\hat{H} = \sum_{pq} h_{pq}\, a_p^\dagger a_q + \frac{1}{4}\sum_{pqrs}\langle pq|v|rs\rangle\, a_p^\dagger a_q^\dagger a_s a_r, \tag{3.10}$$

with the one-body matrix elements $h_{pq} = \langle p|\hat{h}|q\rangle$ and the *antisymmetrized* two-body matrix elements

$$\langle pq|v|rs\rangle = \langle pq|\hat{v}|rs\rangle - \langle pq|\hat{v}|sr\rangle, \qquad \langle pq|v|rs\rangle = -\langle qp|v|rs\rangle = -\langle pq|v|sr\rangle = \langle qp|v|sr\rangle. \tag{3.11}$$

The factor $1/4$ compensates for the fact that each physically distinct pair of states appears four times in the unrestricted sum. Note the order of the annihilation operators, $a_s a_r$, opposite to the order of the bra indices; it is what makes the expectation value in a two-particle state come out with a positive sign. The proof that Eq. (3.10) reproduces the first-quantized Hamiltonian $\sum_i \hat{h}(x_i) + \sum_{i<j}\hat{v}(x_i,x_j)$ on Slater determinants is standard and we do not repeat it; what matters for us is that once the matrix elements h_{pq} and $\langle pq|v|rs\rangle$ are tabulated, Eq. (3.10) defines the Hamiltonian on all of Fock space at once, for every particle number, and that every model in this book is of this form with a specific, sparse choice of matrix elements.

The matrix elements of $\hat{H}$ between two occupation-number states follow from the anticommutators. A one-body operator $a_p^\dagger a_q$ moves one particle from q to p, so it connects states that differ in at most one occupation; a two-body operator connects states that differ in at most two. These are the *Slater–Condon rules*: for a state $|\Phi\rangle$ with occupied set $\{i\}$,

$$\langle\Phi|\hat{H}|\Phi\rangle = \sum_i h_{ii} + \frac{1}{2}\sum_{ij}\langle ij|v|ij\rangle, \tag{3.12}$$

$$\langle\Phi_i^a|\hat{H}|\Phi\rangle = h_{ai} + \sum_j \langle aj|v|ij\rangle, \tag{3.13}$$

$$\langle\Phi_{ij}^{ab}|\hat{H}|\Phi\rangle = \langle ab|v|ij\rangle, \tag{3.14}$$

and zero for states differing in three or more occupations, where $|\Phi_i^a\rangle = a_a^\dagger a_i|\Phi\rangle$ and $|\Phi_{ij}^{ab}\rangle = a_a^\dagger a_b^\dagger a_j a_i|\Phi\rangle$ are the singly and doubly excited states. The program `second_quantization.py` checks these rules against the explicit $2^M \times 2^M$ matrices for a random Hamiltonian on four states, for instance $\langle 01|\hat{H}|01\rangle = h_{00} + h_{11} + \langle 01|v|01\rangle = 4.038893$. The rules imply that the Hamiltonian matrix in a basis of determinants is sparse, with at most $1 + N(M-N) + \binom{N}{2}\binom{M-N}{2}$ non-zero elements per row, and this sparsity is what makes exact diagonalization feasible at all (Section 3.4).

3.2.4 The particle-hole formalism and normal ordering

For a system with many particles the vacuum $|0\rangle$ is a poor starting point; the natural reference is a determinant $|\Phi_0\rangle$ in which the N lowest single-particle states are occupied, the *Fermi sea*. Relative to $|\Phi_0\rangle$ an excitation is described by moving a particle from an occupied state i to an unoccupied state a, that is by creating a *hole* in i and a *particle* in a. The operators that create excitations are therefore $a_a^\dagger$ (a particle above the Fermi level) and a_i (a hole below it), and the operators that annihilate them are a_a and $a_i^\dagger$. The reference state is annihilated by all of the latter,

$$a_a|\Phi_0\rangle = 0 \quad (a > F), \qquad a_i^\dagger|\Phi_0\rangle = 0 \quad (i \le F), \tag{3.15}$$

which is the same structure as $a_p|0\rangle = 0$ with the roles of some operators interchanged. One says that $|\Phi_0\rangle$ is the Fermi vacuum.

A product of creation and annihilation operators is in *normal order* with respect to $|\Phi_0\rangle$ when all operators that annihilate $|\Phi_0\rangle$ stand to the right of all operators that do not. The normal-ordered form of a product $ABC\cdots$, written $N[ABC\cdots]$ or $\{ABC\cdots\}$, is obtained by reordering the operators as if they anticommuted freely, with the sign of the permutation, and its defining property is that its expectation value in the reference state vanishes, $\langle\Phi_0|N[\cdots]|\Phi_0\rangle = 0$. The difference between a product and its normal-ordered form is a *contraction*,

$$\overline{AB} = AB - N[AB] = \langle\Phi_0|AB|\Phi_0\rangle, \tag{3.16}$$

which is a number. With respect to the Fermi vacuum the only non-zero elementary contractions are

$$\overline{a_a a_b^\dagger} = \delta_{ab} \ (a,b > F), \qquad \overline{a_i^\dagger a_j} = \delta_{ij} \ (i,j \le F), \tag{3.17}$$

while contractions of two creation or two annihilation operators, and those in the opposite order, vanish. *Wick's theorem* states that any product of creation and annihilation operators equals its normal-ordered form plus the sum over all possible ways of contracting pairs of operators, each term being the normal-ordered product of the remaining operators times the contractions, with a sign $(-1)^k$ where k is the number of crossings of the contraction lines. Taking the expectation value in $|\Phi_0\rangle$ kills all terms that still contain a normal-ordered product, so that

$$\langle\Phi_0|A_1 A_2 \cdots A_{2n}|\Phi_0\rangle = \sum_{\text{full pairings}} (-1)^k \overline{A_{i_1} A_{j_1}} \cdots \overline{A_{i_n} A_{j_n}}. \tag{3.18}$$

The number of full pairings of $2n$ operators is $(2n-1)!! = 1, 3, 15, 105$ for $n = 1, 2, 3, 4$, and most of them vanish by Eq. (3.17); what the theorem buys is that one never has to anticommute operators through each other by hand. The program `second_quantization.py` evaluates a few vacuum expectation values both ways, for instance $\langle 0|a_1 a_0 a_1^\dagger a_0^\dagger|0\rangle = -1$ from the crossing of the two contraction lines, and confirms that the matrix products agree with the pairing count.

3.2.5 The normal-ordered Hamiltonian

Applying Wick's theorem to the Hamiltonian of Eq. (3.10) with respect to the reference determinant gives the central formula of this section,

$$\hat{H} = E_{\mathrm{ref}} + \sum_{pq} f_{pq}\, N[a_p^\dagger a_q] + \frac{1}{4}\sum_{pqrs} \langle pq|v|rs\rangle\, N[a_p^\dagger a_q^\dagger a_s a_r] = E_{\mathrm{ref}} + \hat{F}_N + \hat{V}_N, \tag{3.19}$$

with the reference energy and the *Fock matrix*

$$E_{\mathrm{ref}} = \sum_{i\le F} h_{ii} + \frac{1}{2}\sum_{ij\le F}\langle ij|v|ij\rangle, \qquad f_{pq} = h_{pq} + \sum_{i\le F}\langle pi|v|qi\rangle. \tag{3.20}$$

The reference energy is the expectation value of $\hat{H}$ in $|\Phi_0\rangle$, Eq. (3.12); the Fock matrix contains the one-body part plus the mean field that the occupied states exert on a particle in state q; and the normal-ordered two-body term $\hat{V}_N$ contains what is left, the genuine two-particle correlations. Equation (3.19) is exact, it is the form of the Hamiltonian from which Hartree–Fock theory (Section 3.5), perturbation theory and coupled-cluster theory (Section 3.6) all start, and the Slater–Condon rules read off from it are particularly transparent: the matrix elements of $\hat{H}$ between the reference and its single and double excitations are

$$\langle \Phi_i^a|\hat{H}|\Phi_0\rangle = f_{ai}, \qquad \langle \Phi_{ij}^{ab}|\hat{H}|\Phi_0\rangle = \langle ab|v|ij\rangle. \tag{3.21}$$

The first of these will vanish at the Hartree–Fock solution (Brillouin's theorem); the second is the driving term of perturbation theory and of the coupled-cluster equations. With the matrices h and v tabulated as `numpy` arrays, E_{ref} and f are one line each,

```
import numpy as np
from models import PairingModel
h, v, N = PairingModel(4, g=1.0).h, PairingModel(4, g=1.0).v, 4
occ = range(N)                          # the reference: states 0 ... N−1
E_ref = sum(h[i, i] for i in occ) + 0.5 * sum(v[i, j, i, j] for i in occ for j in occ)
f = h + np.einsum("piqi->pq", v[:, :N, :, :N]) # f_pq = h_pq + sum_i <pi|v|qi>
print(E_ref, np.diag(f))                # 1.0 and the Fock energies xi(p−1) − g/2, xi(p−1)
```

and `second_quantization.py` verifies Eq. (3.21) against the explicit matrices.

3.2.6 A worked example: two modes

The smallest Hamiltonian with all the ingredients of Eq. (3.10) has two single-particle states with energies ε_0 and ε_1, a hopping amplitude t between them, and an interaction U that acts when both are occupied,

$$\hat{H} = \varepsilon_0\hat{n}_0 + \varepsilon_1\hat{n}_1 + t\left(a_0^\dagger a_1 + a_1^\dagger a_0\right) + U\hat{n}_0\hat{n}_1, \tag{3.22}$$

that is $h_{00}=\varepsilon_0$, $h_{11}=\varepsilon_1$, $h_{01}=h_{10}=t$ and the single antisymmetrized element $\langle 01|v|01\rangle = U$. Its Fock space has four states, $|00\rangle$, $|01\rangle$, $|10\rangle$ and $|11\rangle$, and since $\hat{H}$ conserves the particle number the 4×4 matrix is block diagonal,

$$\hat{H} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & \varepsilon_1 & t & 0 \\ 0 & t & \varepsilon_0 & 0 \\ 0 & 0 & 0 & \varepsilon_0+\varepsilon_1+U \end{pmatrix}, \tag{3.23}$$

with the vacuum, the one-particle block, and the doubly occupied state on the diagonal. The one-particle block is the two-level system of Chapter 2, with eigenvalues $\frac{\varepsilon_0+\varepsilon_1}{2} \pm \sqrt{\left(\frac{\varepsilon_0-\varepsilon_1}{2}\right)^2 + t^2}$; the interaction is felt only by the two-particle state, where there is nothing for it to mix with. For $\varepsilon_0 = 1$, $\varepsilon_1 = 2$, $t = 0.3$ and $U = 0.5$ the program `second_quantization.py` builds the matrix from Eq. (3.9) and finds the eigenvalues 0, 0.9169, 2.0831 and 3.5. Note the ordering of the basis: $|01\rangle$, with state 0 empty and state 1 occupied, has the energy ε_1, in keeping with the convention that the leftmost bit is state 0. This Hamiltonian returns in Chapter 4 as the first example of a mapping to two qubits, where Eq. (3.23) is what the qubit Hamiltonian must reproduce.

3.2.7 Bosons and spins

Bosonic creation and annihilation operators satisfy commutation rather than anticommutation relations, $[b_p, b_q^\dagger] = \delta_{pq}$, $[b_p, b_q] = 0$, and a mode can hold any number of particles; the bosonic Hamiltonian has the same form as Eq. (3.10) with symmetrized instead of antisymmetrized matrix elements. Bosons do not appear in the models of this chapter, but two hybrids do. A *hard-core boson* is a boson whose mode can hold at most one particle: on a single mode it is identical to a fermion, Eq. (3.8), while operators on different modes commute instead of anticommuting. A spin-1/2 is the same object, with $\hat{S}^+ = |\uparrow\rangle\langle\downarrow|$ as the creation operator and $\hat{S}^z + \frac{1}{2}$ as the number operator, which is why the Heisenberg model of Section 3.3.4 can be written in terms of Pauli matrices without any sign rule. And the pair operators of the pairing model, $\hat{P}_p^+ = a_{p+}^\dagger a_{p-}^\dagger$, are hard-core bosons as well: two fermionic operators anticommuted past two others pick up two signs, so $\hat{P}_p^\pm$ commutes with $\hat{P}_q^\pm$ for $p \neq q$, while $(\hat{P}_p^+)^2 = 0$. Whenever the physics is carried by such composite objects, the parity string of Eq. (3.9) is not needed, and the qubit encodings of Chapter 4 become correspondingly cheaper. A qubit, in this language, is a hard-core boson.

3.3 Four model Hamiltonians

Every method of this book is illustrated on the same four models. All four are of the form of Eq. (3.10); what differs between them is which single-particle states there are and which matrix elements are allowed to be non-zero. The restrictions bring symmetries, and the symmetries are what make the models tractable and, in Chapter 4, what allows them to be put on a small number of qubits. The program `models.py` constructs h and $\langle pq|v|rs\rangle$ for each of them, so that one and the same machinery diagonalizes all four; the numbers quoted below come from it and from `fci.py`.

Table 3.1 The Lipkin model with $N=\Omega=4$, $\varepsilon=2$: eigenvalues of the $J=2$ block and the weight of the reference state $|2,-2\rangle$ in the ground state. The last column is the lowest eigenvalue in the full 70-dimensional four-particle space. Produced by `fci.py`.

V	W	eigenvalues of the $J=2$ block	$\lvert\langle 2,-2\rvert\Psi_0\rangle\rvert^2$	E_0 (70×70)
$-1/3$	$-1/4$	$-4.21288,\ -2.98607,\ -0.91914,\ 1.48607,\ 4.13201$	0.93576	-4.21288
$-4/3$	-1	$-7.75122,\ -7.47214,\ -1.55581,\ 1.47214,\ 5.30704$	0.41303	-7.75122

3.3.1 The Lipkin model

The Lipkin–Meshkov–Glick model describes N fermions distributed over two levels, each Ω-fold degenerate, separated by an energy ε. A single-particle state is labelled by $p=1,\dots,\Omega$ and by $\sigma=\pm 1$ for the upper and lower level, and the Hamiltonian is

$$\hat{H}=\frac{\varepsilon}{2}\sum_{p\sigma}\sigma\, a^\dagger_{p\sigma}a_{p\sigma}+\frac{V}{2}\sum_{pp'\sigma}a^\dagger_{p\sigma}a^\dagger_{p'\sigma}a_{p'-\sigma}a_{p-\sigma}+\frac{W}{2}\sum_{pp'\sigma}a^\dagger_{p\sigma}a^\dagger_{p'-\sigma}a_{p'\sigma}a_{p-\sigma}. \tag{3.24}$$

The V term moves two particles from one level to the other; the W term moves one up and one down. Neither changes the label p of the particles it acts on, and this is the symmetry of the model. Defining the *quasispin* operators

$$\hat{J}_z=\frac{1}{2}\sum_{p\sigma}\sigma\, a^\dagger_{p\sigma}a_{p\sigma},\qquad \hat{J}_+=\sum_p a^\dagger_{p+}a_{p-},\qquad \hat{J}_-=\hat{J}_+^\dagger, \tag{3.25}$$

one verifies from the anticommutators that they obey the angular-momentum algebra $[\hat{J}_+,\hat{J}_-]=2\hat{J}_z$, $[\hat{J}_z,\hat{J}_\pm]=\pm\hat{J}_\pm$, and that the Hamiltonian is

$$\hat{H}=\varepsilon\hat{J}_z+\frac{V}{2}\left(\hat{J}_+^2+\hat{J}_-^2\right)+\frac{W}{2}\left(\hat{J}_+\hat{J}_-+\hat{J}_-\hat{J}_+-\hat{N}\right). \tag{3.26}$$

Since $\hat{H}$ commutes with $\hat{J}^2$, it is block diagonal in the basis $|J,J_z\rangle$, and the ground state lies in the block of maximal quasispin $J=N/2$, of dimension $N+1$. In that block the matrix elements follow from the ladder relations $\hat{J}_\pm|J,J_z\rangle=\sqrt{J(J+1)-J_z(J_z\pm1)}\,|J,J_z\pm1\rangle$, and the V term connects J_z to $J_z\pm2$ only, so the block splits further into even and odd J_z. For $N=4$, $J=2$, the matrix in the basis $|2,-2\rangle,\dots,|2,2\rangle$ is

$$\hat{H}=\begin{pmatrix} -2\varepsilon+2W & 0 & \sqrt{6}V & 0 & 0\\ 0 & -\varepsilon+3W & 0 & 3V & 0\\ \sqrt{6}V & 0 & 4W & 0 & \sqrt{6}V\\ 0 & 3V & 0 & \varepsilon+3W & 0\\ 0 & 0 & \sqrt{6}V & 0 & 2\varepsilon+2W \end{pmatrix}, \tag{3.27}$$

where the W term contributes $\frac{W}{2}[2J(J+1)-2J_z^2-N]$ on the diagonal. For $\varepsilon=2$, $V=-1/3$ and $W=-1/4$ the 5×5 matrix has the eigenvalues -4.21288, -2.98607, -0.91914, 1.48607 and 4.13201, and the ground state has weight 0.936 on the reference $|2,-2\rangle$, all four particles in the lower level. Diagonalizing the full $\binom{8}{4}=70$-dimensional four-particle space of Eq. (3.24) gives the same lowest eigenvalue to all digits, which is a check of both the quasispin algebra and the general machinery. Table 3.1 lists a second, more strongly coupled case where the reference state has lost half of its weight. In the mean-field language of Section 3.5 the dimensionless coupling $\chi=|V|(N-1)/\varepsilon$ decides whether the reference is a good starting point or not, and the Lipkin model is the standard example of a mean-field phase transition.

Table 3.2 The pairing model with $L = 4$ levels, two pairs and $\xi = 1$: the reference energy $E_{\mathrm{ref}} = 2 - g$, the lowest eigenvalue in the seniority-zero space and in the full four-particle space, and the correlation energy. Produced by `fci.py`.

g	E_{ref}	E_0 (6×6)	E_0 (70×70)	E_{corr}
-1.0	3.0	2.77987014	2.77987014	-0.22012986
-0.5	2.5	2.43688426	2.43688426	-0.06311574
0.0	2.0	2.00000000	2.00000000	0.00000000
0.5	1.5	1.41677428	1.41677428	-0.08322572
1.0	1.0	0.63554847	0.63554847	-0.36445153

3.3.2 The pairing model

The pairing model describes L doubly degenerate levels $p = 1,\dots,L$ with energies $\xi(p-1)$, each holding a pair of time-reversed states $(p+)$ and $(p-)$, and an interaction that scatters a pair from one level to another,

$$\hat{H} = \xi \sum_{p\sigma} (p-1)\, a^\dagger_{p\sigma} a_{p\sigma} - \frac{g}{2} \sum_{pq} a^\dagger_{p+} a^\dagger_{p-} a_{q-} a_{q+}. \tag{3.28}$$

It is the simplest model of nuclear superfluidity and of the BCS mechanism of superconductivity. Its symmetry is expressed by the pair operators

$$\hat{P}^+_p = a^\dagger_{p+} a^\dagger_{p-}, \qquad \hat{P}^-_p = a_{p-} a_{p+}, \qquad \hat{N}_p = a^\dagger_{p+} a_{p+} + a^\dagger_{p-} a_{p-}, \tag{3.29}$$

in terms of which $\hat{H} = \xi \sum_p (p-1)\hat{N}_p - \frac{g}{2}\sum_{pq} \hat{P}^+_p \hat{P}^-_q$. The interaction moves pairs as units, so the number of unpaired particles, the *seniority*, is conserved, and the ground state of an even system has seniority zero: every level is either empty or doubly occupied. In the seniority-zero space a state is specified by the set of occupied levels, so with n pairs in L levels the dimension is $\binom{L}{n}$ rather than $\binom{2L}{2n}$. The matrix of $\hat{H}$ in this basis has the diagonal elements $2\xi \sum_{p\in\mathrm{occ}} (p-1) - gn/2$, where the $-gn/2$ comes from the $p = q$ terms, and the off-diagonal element $-g/2$ between any two configurations that differ by moving one pair. For $L = 4$ and two pairs the seniority-zero matrix is 6×6, against 70×70 for the full four-particle space, and Table 3.2 shows that the two agree. The table also defines the *correlation energy* $E_{\mathrm{corr}} = E_0 - E_{\mathrm{ref}}$, the difference between the exact ground-state energy and the energy of the reference determinant with the two lowest levels filled, which is the quantity the approximate methods of Sections 3.5 and 3.6 try to capture.

The correlation energy is negative for either sign of g, as it must be for a variational ground state, and it is not additive in the number of pairs: at $g = 1$ it is -0.182 per pair for $L = 4$, -0.233 for $L = 6$ and -0.278 for $L = 8$, a first sign that the pairs are not independent objects. We shall also use a variant of the model with an additional *particle-hole* term,

$$\hat{V}_{\mathrm{ph}} = -\frac{f}{2} \sum_{pqr} \left(a^\dagger_{p+} a^\dagger_{p-} a_{q-} a_{r+} + \mathrm{h.c.} \right), \tag{3.30}$$

which breaks a pair and therefore does not conserve seniority; with it the full space is needed. Its effect is small for small f but its consequences for the qubit encoding, in Chapter 4, are drastic.

3.3.3 The Hubbard model

The Hubbard model describes electrons hopping between the sites of a lattice, with an energy cost U when two electrons of opposite spin occupy the same site,

$$\hat{H} = -t \sum_{\langle ij\rangle\sigma} \left(a^\dagger_{i\sigma} a_{j\sigma} + a^\dagger_{j\sigma} a_{i\sigma} \right) + U \sum_i \hat{n}_{i\uparrow}\hat{n}_{i\downarrow}, \tag{3.31}$$

where $\langle ij\rangle$ runs over nearest-neighbour bonds. It is the minimal model of correlated electrons in a solid, and despite its simplicity it is unsolved in two dimensions. In the form of Eq. (3.10) the one-body matrix has the entries $-t$ on the bonds and the two-body tensor has the single type of element $\langle i\uparrow i\downarrow|v|i\uparrow i\downarrow\rangle = U$. The model conserves the number of electrons of each spin, so with $N_\uparrow = N_\downarrow = N_s/2$, half filling, the relevant space has dimension $\binom{N_s}{N_s/2}^2$, which is 36 for a ring of four sites. Table 3.3 lists the ground-state energy and the double occupancy $\langle \sum_i \hat{n}_{i\uparrow}\hat{n}_{i\downarrow}\rangle$ of the four-site ring as U/t grows. At $U = 0$ the electrons are free and the energy is that of the four lowest plane-wave states, $-4t$; as U increases the double occupancy is suppressed and the energy approaches zero from below as $-4t^2/U$ per bond. In this limit the electrons are localized one per site and only their spins remain as degrees of freedom: second-order perturbation theory in t/U turns the Hubbard model into the antiferromagnetic Heisenberg model with $J = 4t^2/U$, which is the subject of the next section and the reason why the last column of the table approaches the Heisenberg ground-state energy $-2J$ of the four-site ring (with a constant offset $-N_sJ/4$ absorbed in the definition of the spin model).

The derivation is short enough to give. At $t = 0$ and half filling the ground states are the 2^{N_s} configurations with exactly one electron per site, all degenerate at energy zero, and the first excited states, with one doubly occupied and one empty site, lie at energy U. The hopping term connects the two sets, and second-order perturbation theory in t/U gives an effective Hamiltonian in the degenerate subspace, $\hat{H}_{\text{eff}} = -\hat{T}\hat{Q}\hat{T}/U$, where $\hat{Q}$ projects onto the states with one double occupancy. A hop from site j to a neighbouring site i and back is possible only if the two electrons have opposite spins, and it either returns them to their sites or exchanges them. Working out the four spin configurations of a bond one finds

$$\hat{H}_{\text{eff}} = \frac{4t^2}{U} \sum_{\langle ij\rangle} \left(\hat{\boldsymbol{S}}_i \cdot \hat{\boldsymbol{S}}_j - \frac{1}{4} \right) = J \sum_{\langle ij\rangle} \hat{\boldsymbol{S}}_i \cdot \hat{\boldsymbol{S}}_j - \frac{N_s J}{4}, \qquad J = \frac{4t^2}{U}, \tag{3.32}$$

the antiferromagnetic Heisenberg model: the electrons gain kinetic energy by virtual hopping only when neighbouring spins are antiparallel. Away from half filling the same expansion produces in addition a constrained hopping of the holes, the t–J model, which is one of the standard models of high-temperature superconductivity. The last column of Table 3.3 shows the convergence to Eq. (3.32): by $U/t = 128$ the Hubbard energy in units of J is within 0.06% of the Heisenberg value.

3.3.4 The Heisenberg model

The Heisenberg model is a model of interacting spins-1/2 on a lattice,

$$\hat{H} = J \sum_{\langle ij\rangle} \hat{\boldsymbol{S}}_i \cdot \hat{\boldsymbol{S}}_j = J \sum_{\langle ij\rangle} \left[\hat{S}^z_i \hat{S}^z_j + \frac{1}{2} \left(\hat{S}^+_i \hat{S}^-_j + \hat{S}^-_i \hat{S}^+_j \right) \right], \tag{3.33}$$

Table 3.3 The half-filled four-site Hubbard ring: ground-state energy per site, double occupancy and the energy in units of $J = 4t^2/U$, which approaches the Heisenberg value -3 (that is $-2J - N_s J/4$) as $U/t \to \infty$. Produced by `fci.py`.

U/t	E_0/t	$E_0/(N_s t)$	$\langle \sum_i \hat{n}_{i\uparrow}\hat{n}_{i\downarrow}\rangle$	E_0/J
2	−2.82842712	−0.707107	0.453082	−1.41421
4	−2.10274848	−0.525687	0.287325	−2.10275
8	−1.32023496	−0.330059	0.129992	−2.64047
16	−0.72286432	−0.180716	0.042033	−2.89146
32	−0.37141042	−0.092853	0.011387	−2.97128
64	−0.18704455	−0.046761	0.002908	−2.99271
128	−0.09369285	−0.023423	0.000731	−2.99817

Table 3.4 The antiferromagnetic Heisenberg ring: dimension of the $S^z = 0$ block, ground-state energy, energy per site and its deviation from the Bethe-ansatz limit $\frac{1}{4} - \ln 2 = -0.44314718$. Produced by `fci.py`.

N_s	dimension	E_0/J	$E_0/(N_s J)$	deviation
4	6	−2.00000000	−0.50000000	−0.056853
6	20	−2.80277564	−0.46712927	−0.023982
8	70	−3.65109341	−0.45638668	−0.013239
10	252	−4.51544635	−0.45154464	−0.008397
12	924	−5.38739092	−0.44894924	−0.005802

Table 3.5 The four models at a glance: degrees of freedom, the symmetry that reduces the problem, and the dimension of the symmetry-adapted space against that of the full space for the sizes used in this chapter.

model	degrees of freedom	conserved quantity	reduced space	full space
Lipkin	N fermions, 2Ω states	quasispin $\hat{J}^2$	$N+1$ (5)	$\binom{2\Omega}{N}$ (70)
pairing	n pairs, $2L$ states	seniority	$\binom{L}{n}$ (6)	$\binom{2L}{2n}$ (70)
Hubbard	N_s sites, $2N_s$ states	$N_\uparrow$, $N_\downarrow$	$\binom{N_s}{N_s/2}^2$ (36)	$\binom{2N_s}{N_s}$ (70)
Heisenberg	N_s spins	total S^z	$\binom{N_s}{N_s/2}$ (6)	2^{N_s} (16)

with $J > 0$ for an antiferromagnet. It is not a fermionic model: the spins on different sites commute, and $\hat{\boldsymbol{S}}_i = \frac{1}{2}(X_i, Y_i, Z_i)$ in terms of the Pauli matrices of site i. In the language of Chapter 2 the Heisenberg Hamiltonian is already a sum of products of Pauli matrices on a register of N_s qubits,

$$\hat{H} = \frac{J}{4} \sum_{\langle ij \rangle} (X_i X_j + Y_i Y_j + Z_i Z_j)\,, \tag{3.34}$$

which is why it is the easiest of the four models to put on a quantum computer, and why it appears in Chapter 4 as the reference against which the fermionic mappings are compared. Its symmetry is the conservation of the total $\hat{S}^z = \frac{1}{2}\sum_i Z_i$: the flip-flop term $\hat{S}_i^+ \hat{S}_j^-$ exchanges an up and a down spin but does not change their number, so the Hamiltonian is block diagonal in the number of up spins and the ground state of an even ring lies in the $S^z = 0$ block of dimension $\binom{N_s}{N_s/2}$. Table 3.4 lists the ground-state energy per site of rings of increasing length, which converges to the Bethe-ansatz value $E_0/(N_s J) \to \frac{1}{4} - \ln 2 = -0.443147$ of the infinite chain with corrections of order $1/N_s^2$. For the four-site ring $E_0 = -2J$, with nearest-neighbour correlation $\langle \hat{\boldsymbol{S}}_i \cdot \hat{\boldsymbol{S}}_{i+1} \rangle = -1/2$ and next-nearest $+1/4$, the antiferromagnetic alternation of a resonating-valence-bond state.

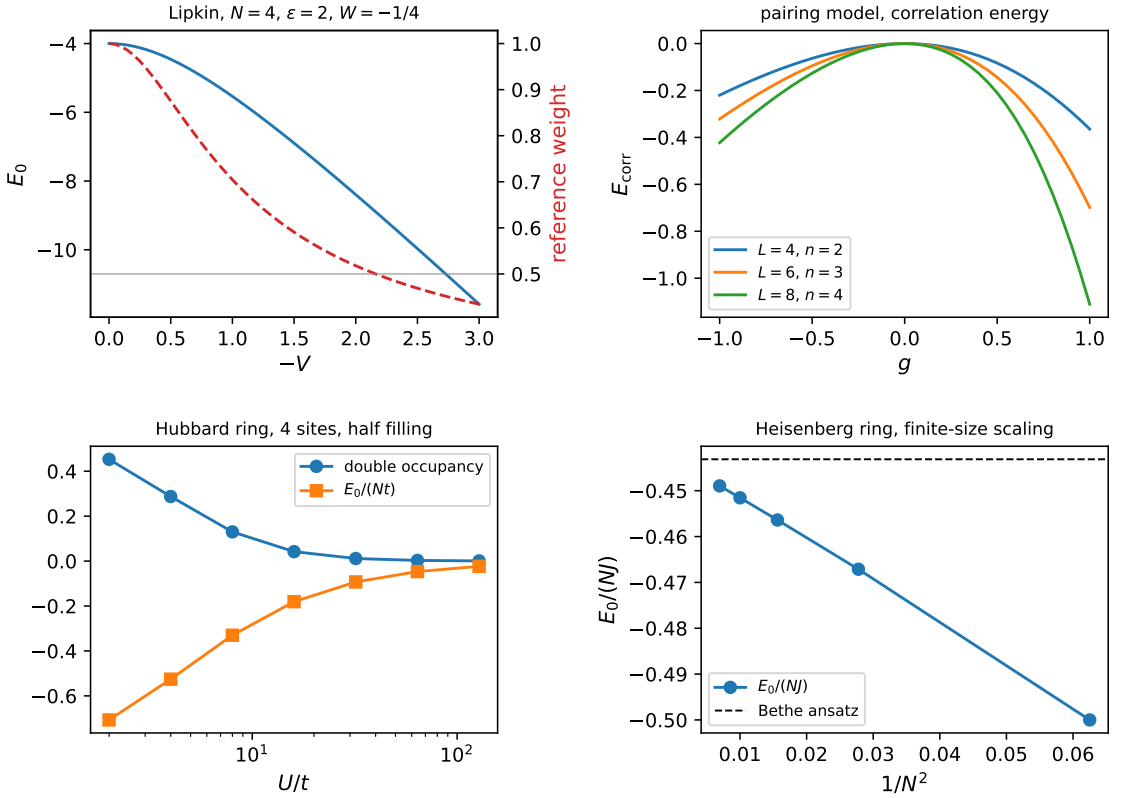


Fig. 3.1 The four models under exact diagonalization. Top left: the Lipkin ground-state energy and the weight of the reference state as the coupling $-V$ grows ($N=4$, $\varepsilon=2$, $W=-1/4$); the weight passes $1/2$ near $-V=2.15$. Top right: the correlation energy of the pairing model for $L=4,6,8$ levels at half filling. Bottom left: energy per site and double occupancy of the four-site Hubbard ring against U/t. Bottom right: the energy per site of the Heisenberg ring against $1/N_s^2$, converging to the Bethe-ansatz value. Produced by `fci.py`.

Table 3.5 summarizes the four models, and Fig. 3.1 collects their exact solutions as the coupling is varied. Two remarks apply to all of them. First, every model is exactly solvable at the sizes used here only because of its symmetry; without the quasispin, seniority or S^z block structure the full spaces would already be a factor ten larger, and the reductions grow exponentially with the system size. Second, the same symmetries will reduce the number of qubits needed in Chapter 4, and by the same factors. Identifying the symmetry is the first step in every case, classical or quantum.

3.4 Full configuration interaction

3.4.1 The configuration-interaction expansion

The Slater determinants built from M single-particle states form a complete basis of the N-particle space, so any N-particle state can be written as

$$|\Psi\rangle = C_0|\Phi_0\rangle + \sum_{ia} C_i^a |\Phi_i^a\rangle + \sum_{i<j,a<b} C_{ij}^{ab} |\Phi_{ij}^{ab}\rangle + \cdots , \tag{3.35}$$

organized by the number of excitations from the reference. Finding the coefficients that minimize $\langle\Psi|\hat{H}|\Psi\rangle/\langle\Psi|\Psi\rangle$ is the Rayleigh–Ritz variational problem, and its solution is the eigenvalue equation

$$\sum_{\beta}\langle\Phi_\alpha|\hat{H}|\Phi_\beta\rangle C_\beta = E\,C_\alpha, \tag{3.36}$$

that is the diagonalization of the Hamiltonian matrix in the determinant basis. When all determinants are kept this is *full configuration interaction* (FCI), which is nothing but exact diagonalization in the given single-particle basis: the lowest eigenvalue is the exact ground-state energy within that basis, and the other eigenvalues are the excited states. The method is exact, it is variational, its error is controlled only by the single-particle basis, and it provides all eigenstates at once. Its one defect is that it does not scale, and the purpose of this section is to make that statement quantitative.

The eigenvalues of the truncated problem obey a useful ordering principle. If $\hat{H}$ is diagonalized in a subspace and the subspace is then enlarged by one determinant, the new eigenvalues interlace the old ones: $E_k^{(d+1)} \le E_k^{(d)} \le E_{k+1}^{(d+1)}$. Every eigenvalue can therefore only go down as the space grows, and every truncated-CI energy is an upper bound on the corresponding exact eigenvalue. This is the Hylleraas–Undheim–MacDonald theorem, and it is what gives the tables of this chapter their monotone structure.

3.4.2 The structure of the matrix

By the Slater–Condon rules of Eqs. (3.12)–(3.14) the matrix $\langle\Phi_\alpha|\hat{H}|\Phi_\beta\rangle$ is sparse: each row has non-zero entries only for determinants that differ from $|\Phi_\alpha\rangle$ by at most two occupations. Furthermore, when $\hat{H}$ commutes with an operator such as $\hat{S}^z$, $\hat{J}^2$ or the seniority, the matrix is block diagonal in the eigenvalue of that operator, and only the block containing the ground state needs to be built. The program `fockspace.py` implements the general case: it represents determinants as integers whose bits are the occupation numbers of Eq. (3.5), applies strings of creation and annihilation operators with the sign rule of Eq. (3.7), and assembles the matrix column by column from the tabulated h_{pq} and $\langle pq|v|rs\rangle$,

```
class SlaterBasis:
    """All determinants with n_particles of n_modes states, as bit strings (integers)."""

    def apply(self, ops, state):
        """Apply [('+', p), ('−', q), ...]  right to left;  returns (sign, state) or (0, None)."""
        sign = 1
        for kind, p in reversed(ops):
            bit = self.bit(p)
            if (kind == '+') == bool(state & bit):
                return 0, None                  # Pauli principle
            sign *= −1 if bin(state & (bit − 1)).count("1") % 2 else 1
            state ^= bit
        return sign, state

    def hamiltonian_matrix(self, h, v=None):
        H = np.zeros((self.dim, self.dim))
        for col, state in enumerate(self.states):
            for p, q in itertools.product(self.modes, repeat=2):
                if h[p, q] != 0:
                    sign, new = self.apply([('+', p), ('−', q)], state)
                    if sign:
                        H[self.index[new], col] += sign * h[p, q]
            # ...  and the same loop over (p, q, r, s) with v[p, q, r, s] / 4
        return H
```

Table 3.6 Convergence of the Lanczos iteration for the $S^z = 0$ block of the 14-site antiferromagnetic Heisenberg ring (dimension 3432, exact ground-state energy $-6.26354953J$) from a random start vector: the lowest Ritz value after m steps and its error. Produced by `fci.py`.

m	lowest Ritz value	error
5	-4.94003212	1.3
10	-6.13896624	1.2×10^{-1}
20	-6.26335838	1.9×10^{-4}
40	-6.26354953	$< 10^{-8}$

Here `self.bit(p)` is the integer with a single one at the position of state p, counted from the left so that the leftmost state is the most significant bit, in keeping with the tensor-product convention of Chapter 2, and the parity of `state & (bit - 1)` counts the occupied states to the right of p in the integer, that is to the left of p in the bit string of Eq. (3.5). The cost is dominated by the M^4 two-body loop for each of the $\binom{M}{N}$ columns; for the 70-dimensional spaces of this chapter it is negligible. The class `FockSpace` in the same file builds instead the full $2^M \times 2^M$ matrices of Eq. (3.9) and is used whenever an operator has to be applied as a matrix, for instance the exponentials of Section 3.6.

3.4.3 Iterative eigensolvers

The matrices of Section 3.4.2 are sparse and only a few of their lowest eigenvalues are wanted, and for both reasons one does not diagonalize them completely. The method of choice is the Lanczos iteration: starting from a random vector $|v_0\rangle$, one builds the Krylov space spanned by $|v_0\rangle, \hat{H}|v_0\rangle, \ldots, \hat{H}^{m-1}|v_0\rangle$, orthonormalized step by step, in which $\hat{H}$ is represented by an $m \times m$ tridiagonal matrix with diagonal elements $\alpha_k = \langle v_k|\hat{H}|v_k\rangle$ and off-diagonal elements $\beta_k = \|(\hat{H} - \alpha_k)|v_k\rangle - \beta_{k-1}|v_{k-1}\rangle\|$. The eigenvalues of the tridiagonal matrix, the Ritz values, converge to the extreme eigenvalues of $\hat{H}$ long before m reaches the dimension of the space, because the power iteration implicit in the Krylov space amplifies the components of $|v_0\rangle$ along the extreme eigenvectors, and the only operation on the large matrix is the product $\hat{H}|v\rangle$, which for a sparse matrix costs as much as its number of non-zero elements. Table 3.6 shows the convergence for the 14-site Heisenberg ring, whose $S^z = 0$ block has dimension 3432: twenty steps give four digits, forty give machine precision. The function `lanczos` in `fci.py` is twenty lines, with full reorthogonalization of each new vector against all previous ones to suppress the loss of orthogonality that plagues the plain iteration in finite-precision arithmetic. Production codes for nuclear and condensed-matter exact diagonalization are elaborate versions of exactly this loop, with the matrix never stored but its action on a vector recomputed from the bit strings each time, and they reach dimensions of 10^{10} to 10^{11}. The quantum phase estimation algorithm of Chapter 6 and the filtering methods of Chapter 8 are the quantum analogues of this idea: they too extract eigenvalues by applying functions of $\hat{H}$ to a starting state, and they too converge faster the larger the overlap of that state with the target eigenvector.

3.4.4 The exponential wall

The dimension of the N-particle space, $\binom{M}{N}$, grows exponentially with the size of the system. At half filling $\binom{M}{M/2} \approx 2^M \sqrt{2/(\pi M)}$, so that doubling the number of single-particle states squares

Table 3.7 The dimension of the half-filled N-particle space, exactly and in the asymptotic form $2^M/\sqrt{M}$ up to a constant. Produced by `fci.py`.

M	$\binom{M}{M/2}$	$2^M/\sqrt{M}$
10	2.5×10^{2}	3.2×10^{2}
20	1.8×10^{5}	2.3×10^{5}
40	1.4×10^{11}	1.7×10^{11}
80	1.1×10^{23}	1.4×10^{23}
160	9.2×10^{46}	1.2×10^{47}

the dimension. Table 3.7 gives the numbers: with $M=40$ states the space has 10^{11} dimensions, which is the limit of what can be diagonalized with iterative eigensolvers and massive parallelism; with $M=80$ it has 10^{23}, more than the number of bytes of storage on Earth; with $M=160$ it has 10^{47}. A nucleus such as ^{16}O in a model space of 40 orbitals for each kind of nucleon has $\binom{40}{8}^2\approx6\times10^{15}$ determinants. For lattice models the numbers are the same: a 6×6 Hubbard cluster at half filling has $\binom{36}{18}^2\approx8\times10^{19}$ states. This is the *exponential wall*, and it is not a matter of better algorithms: the sparsity of the matrix reduces the cost per iteration, and the symmetries reduce the dimension by polynomial factors, but the exponential remains.

Classical many-body theory has developed three responses. The first is to keep only a part of the expansion of Eq. (3.35), which is the subject of the next subsection and leads, through its failure, to the coupled-cluster method of Section 3.6. The second is to give up the wave function in favour of a stochastic sampling of it, quantum Monte Carlo, which is exact for bosons and for some fermionic problems but suffers in general from the sign problem, itself a consequence of the antisymmetry of Eq. (3.7). The third is to compress the coefficient tensor of Eq. (3.35) into a network of small tensors, of which the matrix-product states of the density-matrix renormalization group are the prime example; these are exact when the entanglement across any cut of the system is bounded, which is the case for ground states of one-dimensional gapped systems, and they are the classical method of choice for the Heisenberg chain of Section 3.3.4. The quantum computer offers a fourth response: a register of M qubits stores the 2^M amplitudes of Eq. (3.35) natively, and the remaining question, which occupies the rest of this book, is how to prepare and interrogate the state one wants.

3.4.5 Truncated configuration interaction and size consistency

The obvious way to escape the wall is to truncate Eq. (3.35) at a fixed excitation level: CIS keeps the reference and the single excitations, CID the reference and the doubles, CISD singles and doubles, and so on. Table 3.8 shows what this gives for the pairing model with $L=4$ and two pairs, where the dimensions of the successive spaces are 1, 17, 53, 69 and 70. Singles do nothing at all, because the pairing interaction cannot move a single particle out of a pair: CIS returns the reference energy. Doubles capture most of the correlation energy at weak coupling but a decreasing fraction as g grows, and CISD equals CID for the same reason that CIS is inert. The remaining error is the contribution of the quadruple excitations, which for this model means both pairs moved at once.

The deeper problem with truncated CI is not the size of the error but the way it scales with the system. Consider two copies A and B of the same system, infinitely far apart so that they do not interact. The exact ground state is the product $|\Psi_A\rangle|\Psi_B\rangle$ and the exact energy is E_A+E_B. A method is called *size consistent* if it reproduces this additivity. CID does not: the product of the CID wave functions of A and B contains terms in which both A and B are doubly excited,

Table 3.8 Truncated CI for the pairing model with $L = 4$ levels, two pairs and $\xi = 1$. The dimensions of the spaces are 1 (reference), 17 (CIS), 53 (CID), 69 (CISD) and 70 (FCI). Produced by `fci.py`.

g	E_{ref}	CIS	CID	CISD	FCI
0.25	1.75	1.75000000	1.73050700	1.73050700	1.73045985
0.50	1.50	1.50000000	1.41759645	1.41759645	1.41677428
1.00	1.00	1.00000000	0.64890655	0.64890655	0.63554847
2.00	0.00	0.00000000	−1.35208670	−1.35208670	−1.48965216

Table 3.9 Size consistency: two non-interacting copies of the two-level pairing model with one pair each ($\xi = 1$). FCI of the combined eight-state system reproduces twice the energy of one copy; CID does not. Produced by `fci.py`.

g	$2E_A$	FCI	CID	CID error
0.25	−0.26556444	−0.26556444	−0.26550480	6.0×10^{-5}
0.50	−0.56155281	−0.56155281	−0.56066017	8.9×10^{-4}
1.00	−1.23606798	−1.23606798	−1.22474487	1.1×10^{-2}

that is quadruple excitations of the combined system, and these are precisely the terms that CID of the combined system drops. Table 3.9 makes this explicit for two non-interacting copies of a two-level pairing system with one pair each: FCI of the combined system reproduces $2E_A$ to machine precision, while CID misses it by an amount that grows with g. For m copies the missing fraction of the correlation energy grows with m, so that truncated CI becomes useless for extended systems, however small the error is for one unit. The cure, in Section 3.6, is to build the quadruples from products of doubles instead of dropping them.

3.5 Hartree–Fock theory

3.5.1 The energy functional and the Hartree–Fock equations

Instead of an exponentially large superposition of determinants one can keep exactly one, and ask which one is best. The energy of a single determinant $|\Phi\rangle$ built from orthonormal orbitals $|\psi_i\rangle = \sum_\alpha C_{\alpha i}|\phi_\alpha\rangle$, $i = 1,\dots,N$, follows from Eq. (3.12),

$$E[\Phi] = \sum_i \langle \psi_i|\hat{h}|\psi_i\rangle + \frac{1}{2}\sum_{ij}\langle \psi_i\psi_j|v|\psi_i\psi_j\rangle = \sum_{\alpha\beta} h_{\alpha\beta}\rho_{\beta\alpha} + \frac{1}{2}\sum_{\alpha\beta\gamma\delta}\langle \alpha\gamma|v|\beta\delta\rangle\rho_{\beta\alpha}\rho_{\delta\gamma}, \tag{3.37}$$

where the second form uses the one-body *density matrix*

$$\rho_{\beta\alpha} = \sum_{i=1}^{N} C_{\beta i}C^*_{\alpha i}, \qquad \boldsymbol{\rho} = \boldsymbol{C}_{\mathrm{occ}}\boldsymbol{C}^\dagger_{\mathrm{occ}}, \qquad \boldsymbol{\rho}^2 = \boldsymbol{\rho}, \quad \mathrm{Tr}\,\boldsymbol{\rho} = N, \tag{3.38}$$

the projector onto the occupied orbitals. The orbitals are to be chosen such that $E[\Phi]$ is minimal subject to their orthonormality, $\sum_\alpha C^*_{\alpha i}C_{\alpha j} = \delta_{ij}$. Introducing one Lagrange multiplier ε_i per orbital and varying the functional

$$F[C] = E[\Phi] - \sum_i \varepsilon_i \sum_\alpha C^*_{\alpha i}C_{\alpha i} \tag{3.39}$$

with respect to $C^*_{\alpha i}$, treating C and C^* as independent, the one-body term gives $\sum_\beta h_{\alpha\beta} C_{\beta i}$ directly, and in the two-body term $C^*_{\alpha i}$ appears in both the first and the second factor; the two contributions are equal by the antisymmetry of Eq. (3.11) under the simultaneous interchange of both pairs of indices, and together they cancel the factor $\frac{1}{2}$. The result is the *Hartree–Fock equations*

$$\sum_\beta f_{\alpha\beta} C_{\beta i} = \varepsilon_i C_{\alpha i}, \qquad f_{\alpha\beta} = h_{\alpha\beta} + \sum_{\gamma\delta} \rho_{\delta\gamma} \langle \alpha\gamma | v | \beta\delta \rangle. \tag{3.40}$$

This is an eigenvalue problem for the Fock matrix of Eq. (3.20), now written in the fixed basis $|\phi_\alpha\rangle$ and with the occupied orbitals expressed through $\boldsymbol{\rho}$; the one complication is that $\boldsymbol{f}$ depends on its own eigenvectors through $\boldsymbol{\rho}$. The standard solution is the *self-consistent field* iteration: guess the occupied orbitals, build $\boldsymbol{\rho}$ and $\boldsymbol{f}$, diagonalize $\boldsymbol{f}$, occupy its N lowest eigenvectors, and repeat until the eigenvalues stop changing. No matrix elements are recomputed during the iteration; each step is a contraction and a diagonalization. The class `HartreeFock` in `hartreefock.py` does exactly this,

```
class HartreeFock:
    def density_matrix(self, C):
        occ = C[:, :self.N]
        return occ @ occ.conj().T

    def fock_matrix(self, rho):
        return self.h + np.einsum("dg,agbd->ab", rho, self.v)

    def energy(self, rho):
        return float(np.einsum("ab,ba->", self.h, rho)
                     + 0.5 * np.einsum("agbd,ba,dg->", self.v, rho, rho))

    def solve(self, C0=None, tol=1e-10, maxiter=500):
        C = np.eye(self.n) if C0 is None else C0
        eps_old = np.zeros(self.n)
        for it in range(1, maxiter + 1):
            rho = self.density_matrix(C)
            eps, C = np.linalg.eigh(self.fock_matrix(rho))
            if np.abs(eps - eps_old).mean() < tol:
                break
            eps_old = eps
        return C, eps, self.energy(self.density_matrix(C)), it
```

and the Hartree–Fock energy is $E[\Phi]$ evaluated at the converged density matrix. Note that $E_{\mathrm{HF}} \neq \sum_i \varepsilon_i$: the sum of the eigenvalues counts the interaction twice, and $E_{\mathrm{HF}} = \sum_i \varepsilon_i - \frac{1}{2}\sum_{ij}\langle ij|v|ij\rangle$.

Two properties of the solution follow immediately. In the basis of the Hartree–Fock orbitals the Fock matrix is diagonal, so in particular $f_{ai} = 0$ between occupied and unoccupied orbitals. By Eq. (3.21) this means that

$$\langle \Phi_i^a | \hat{H} | \Phi_{\mathrm{HF}} \rangle = f_{ai} = 0, \tag{3.41}$$

Brillouin's theorem: the Hartree–Fock determinant does not interact with its single excitations, which is why the leading correction to it comes from the doubles, and why CIS on top of a Hartree–Fock reference does nothing for the ground state. And the normal-ordered Hamiltonian of Eq. (3.19) with respect to $|\Phi_{\mathrm{HF}}\rangle$ has a diagonal one-body part, $\hat{F}_N = \sum_p \varepsilon_p N[a_p^\dagger a_p]$, so that all the remaining physics is in $\hat{V}_N$. This is the starting point of every post-Hartree–Fock method.

3.5.2 Thouless' theorem and the second-quantized form

The derivation above varied the orbitals. It is illuminating, and necessary for what follows, to vary the determinant itself. Any determinant $|\Phi'\rangle$ that is not orthogonal to $|\Phi_0\rangle$ can be written as

$$|\Phi'\rangle = \exp\Big(\sum_{ia} t_{ai}\, a_a^\dagger a_i\Big)|\Phi_0\rangle = e^{\hat{T}_1}|\Phi_0\rangle, \tag{3.42}$$

with intermediate normalization $\langle\Phi_0|\Phi'\rangle = 1$. This is *Thouless' theorem*. Its content is that the exponential of a one-body particle-hole operator maps determinants onto determinants: the operator $\hat{T}_1$ commutes with itself term by term (all $a_a^\dagger a_i$ commute, since $a \neq i$ always), so $e^{\hat{T}_1} = \prod_i (1 + \sum_a t_{ai} a_a^\dagger a_i)$, and each factor replaces the occupied orbital i by the rotated orbital $|\psi_i\rangle + \sum_a t_{ai}|\psi_a\rangle$. The program `hartreefock.py` checks this numerically: the state $e^{\hat{T}_1}|\Phi_0\rangle$ computed as a matrix exponential in Fock space coincides with the product state of the rotated orbitals to 10^{-17}, while the linearization $(1+\hat{T}_1)|\Phi_0\rangle$ differs from it by 0.27 in norm, the doubly excited cross terms $t_{ai}t_{bj}$.

Thouless' theorem turns the variation of the determinant into a variation of the amplitudes t_{ai}. The energy changes to first order by $\sum_{ia} t_{ai}\langle\Phi_i^a|\hat{H}|\Phi_0\rangle + \text{c.c.}$, so the stationarity condition is again Brillouin's theorem, Eq. (3.41), now derived without any reference to orbitals. The second-order variation decides whether the stationary point is a minimum: expanding to second order in t and using Wick's theorem one finds

$$\delta^2 E = \frac{1}{2}\begin{pmatrix} \boldsymbol{t}^* & \boldsymbol{t} \end{pmatrix}\begin{pmatrix} \boldsymbol{A} & \boldsymbol{B} \\ \boldsymbol{B}^* & \boldsymbol{A}^* \end{pmatrix}\begin{pmatrix} \boldsymbol{t} \\ \boldsymbol{t}^* \end{pmatrix}, \qquad A_{ai,bj} = (\varepsilon_a - \varepsilon_i)\delta_{ab}\delta_{ij} + \langle aj|v|ib\rangle, \quad B_{ai,bj} = \langle ab|v|ij\rangle, \tag{3.43}$$

so that the Hartree–Fock solution is a local minimum if and only if the *stability matrix* $\boldsymbol{M}$ of Eq. (3.43) has no negative eigenvalue. The same matrix is the matrix of the random-phase approximation, and a negative eigenvalue of $\boldsymbol{M}$ is equivalent to an imaginary RPA frequency: an unstable mean field and an unstable small-amplitude oscillation about it are the same statement.

3.5.3 The Baker–Campbell–Hausdorff expansion and Trotter splitting

Exponentials of operators, as in Eq. (3.42), are the recurring theme of this book: the time-evolution operator $e^{-i\hat{H}t}$, the coupled-cluster wave operator $e^{\hat{T}}$, the unitary rotations $e^{\hat{T}-\hat{T}^\dagger}$ of Section 3.6.3, and every quantum gate. Two facts about them are needed everywhere. The first is that for non-commuting X and Y the product $e^X e^Y$ is not e^{X+Y}, but it is still a single exponential,

$$e^X e^Y = e^Z, \qquad Z = X + Y + \frac{1}{2}[X,Y] + \frac{1}{12}\big([X,[X,Y]] + [Y,[Y,X]]\big) - \frac{1}{24}[Y,[X,[X,Y]]] + \cdots, \tag{3.44}$$

the *Baker–Campbell–Hausdorff* (BCH) expansion, whose terms are all nested commutators. The series terminates when $[X,Y]$ commutes with both X and Y, which is the case for Thouless' theorem, where the generators commute outright, and for the harmonic oscillator, where $[a,a^\dagger] = 1$. In general it is an asymptotic series, but for small operators it converges fast: for the 2×2 matrices $X = 0.1\,a^\dagger$ and $Y = 0.1\,a$ the program `trotter.py` finds $\|e^X e^Y - e^{Z_n}\| = 7.1\times10^{-3}$, 2.4×10^{-4}, 1.2×10^{-5} and 4.8×10^{-7} when Z_n is truncated after the first, second, third and fourth order, each order gaining a factor of order $\|X\|$.

The second fact is the converse: an exponential of a sum can be approximated by a product of exponentials, provided the exponent is divided into small enough pieces. From Eq. (3.44),

Table 3.10 Trotter splitting of $e^{-i(X+Z)t}$ at $t=1$: the spectral norm of the difference between the exact exponential and the first-order product of Eq. (3.45) or the second-order product of Eq. (3.46) with n steps, and the ratio of successive errors. Produced by `trotter.py`.

n	first order	ratio	second order	ratio
1	7.99×10^{-1}	—	3.14×10^{-1}	—
2	3.62×10^{-1}	2.21	7.11×10^{-2}	4.41
4	1.76×10^{-1}	2.06	1.73×10^{-2}	4.12
8	8.75×10^{-2}	2.01	4.29×10^{-3}	4.03
16	4.37×10^{-2}	2.00	1.07×10^{-3}	4.01
32	2.18×10^{-2}	2.00	2.67×10^{-4}	4.00
64	1.09×10^{-2}	2.00	6.68×10^{-5}	4.00

$e^{X/n}e^{Y/n}=e^{(X+Y)/n+[X,Y]/(2n^2)+\cdots}$, and raising to the n-th power,

$$e^{X+Y}=\lim_{n\to\infty}\left(e^{X/n}e^{Y/n}\right)^n, \qquad \left\|e^{X+Y}-\left(e^{X/n}e^{Y/n}\right)^n\right\|\le\frac{1}{2n}\|[X,Y]\|\,e^{\|X\|+\|Y\|}, \tag{3.45}$$

the *Lie–Trotter* formula. The error is of order $1/n$ and proportional to the commutator; it vanishes when the pieces commute. The symmetric or *Strang* splitting

$$e^{X+Y}\approx\left(e^{X/2n}e^{Y/n}e^{X/2n}\right)^n \tag{3.46}$$

cancels the $[X,Y]$ term and has an error of order $1/n^2$; higher-order Suzuki products cancel more terms at the price of more factors. Applied to $e^{-i\hat H t}$ with $\hat H=\sum_\alpha \hat H_\alpha$ the formula reads

$$e^{-i\hat H t}\approx\left(\prod_\alpha e^{-i\hat H_\alpha t/n}\right)^n, \tag{3.47}$$

and it is the basis of Hamiltonian simulation on a quantum computer, Chapter 5: if each $\hat H_\alpha$ acts on a few qubits, each factor is a small gate, and the number of gates is the number of terms times the number of steps. Table 3.10 shows the two splittings for the simplest nontrivial case, a single qubit with $\hat H=X+Z$, where $\|[X,Z]\|=2$; the errors fall exactly as $1/n$ and $1/n^2$ once n is moderately large. Figure 3.2 shows the same for the four-site Heisenberg ring split into the two bonds (01), (23) and the two bonds (12), (30), each group consisting of commuting two-qubit operators, so that every factor in Eq. (3.47) is a two-qubit gate; the asymptotic slopes on the log-log plot are -1.00 and -2.00.

The two facts are used in opposite directions in this book. BCH tells us that a product of exponentials, such as a sequence of gates or a Trotterized coupled-cluster ansatz, is again an exponential of something, which is what makes the Trotterized ansatz of Section 3.6.4 legitimate. Trotter tells us that an exponential of something, such as $e^{-i\hat H t}$, can be built from elementary gates. The price is the same in both directions: the commutators, and a number of steps that grows with the accuracy required.

3.5.4 Stability and the Lipkin model

The Lipkin model with $W=0$ is the standard illustration of the stability analysis because everything is in closed form. The mean-field ansatz rotates every doublet by the same angle, $|\psi_p\rangle=\cos\frac{\alpha}{2}|p-\rangle+\sin\frac{\alpha}{2}|p+\rangle$, a Thouless rotation with $t_{ai}=\tan(\alpha/2)\delta_{pq}$, and the energy per particle is

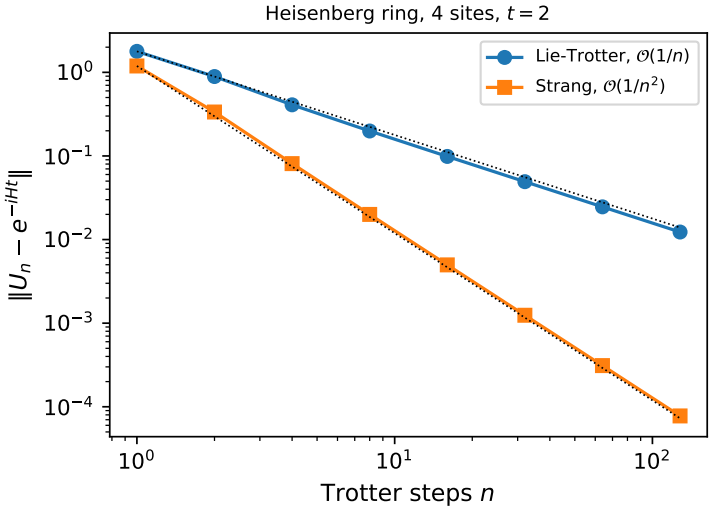


Fig. 3.2 Trotter splitting of the time-evolution operator of the four-site Heisenberg ring, $t = 2$, into even and odd bonds. The dotted lines are the $1/n$ and $1/n^2$ references through the first point. Produced by `trotter.py`.

Table 3.11 Hartree–Fock for the Lipkin model with $N = 4$, $\varepsilon = 1$, $W = 0$: the energy of the spherical solution, the energy of the self-consistent solution reached from a symmetry-broken start, the lowest eigenvalue of the stability matrix of the spherical solution, and the exact energy. Produced by `hartreefock.py`.

χ	$E_{\text{spherical}}$	E_{HF}	$\lambda_{\min}$	E_{exact}	solution
0.25	−2.000000	−2.000000	0.75	−2.020726	spherical
0.50	−2.000000	−2.000000	0.50	−2.081666	spherical
0.90	−2.000000	−2.000000	0.10	−2.253886	spherical
1.00	−2.000000	−2.000000	0.00	−2.309401	critical
1.50	−2.000000	−2.166667	−0.50	−2.645751	deformed
2.00	−2.000000	−2.500000	−1.00	−3.055050	deformed
2.50	−2.000000	−2.900000	−1.50	−3.511885	deformed

$$\frac{E(\alpha)}{N} = -\frac{\varepsilon}{2}\cos\alpha - \frac{\varepsilon\chi}{4}\sin^2\alpha, \qquad \chi = \frac{|V|(N-1)}{\varepsilon}. \tag{3.48}$$

Its derivative $\frac{\varepsilon}{2}\sin\alpha(1-\chi\cos\alpha)$ has the *spherical* stationary point $\alpha = 0$ for every χ and, for $\chi \geq 1$, a second, *deformed* one at $\cos\alpha = 1/\chi$, with energy $-\frac{\varepsilon}{4}(\chi + 1/\chi)$ per particle. The second derivative at $\alpha = 0$ is $\frac{\varepsilon}{2}(1-\chi)$: at $\chi = 1$ the spherical solution turns from a minimum into a maximum, at exactly the point where the deformed solution appears. The stability matrix of Eq. (3.43) reproduces this without any parametrization. Since the interaction changes J_z by two units it has no particle-hole matrix elements, so $\boldsymbol{A}$ is ε times the unit matrix, while $\boldsymbol{B}$ has entries $\pm V$ with largest eigenvalue $|V|(N-1)$, and the lowest eigenvalue of $\boldsymbol{M}$ is $\lambda_{\min} = \varepsilon(1-\chi)$.

Table 3.11 and Fig. 3.3 show the self-consistent solution of `hartreefock.py`, which knows nothing about α: started from the level-ordered basis it stays spherical for every χ, satisfying Brillouin's theorem exactly; started from a randomly rotated basis it converges to the deformed solution when $\chi > 1$, with an energy that matches $-\frac{N\varepsilon}{4}(\chi + 1/\chi)$ to six digits, and to the spherical one otherwise. Two lessons follow. A solution can satisfy the Hartree–Fock equations, obey Brillouin's theorem and still be a saddle point, which is why the stability matrix is needed. And the deformed solution lowers the energy by breaking a symmetry, the parity of J_z, that the exact ground state does not break; the gain, 0.17 at $\chi = 1.5$ and 0.5 at $\chi = 2$, is real but the state is qualitatively wrong, and the remaining error, 0.48 and 0.56, is

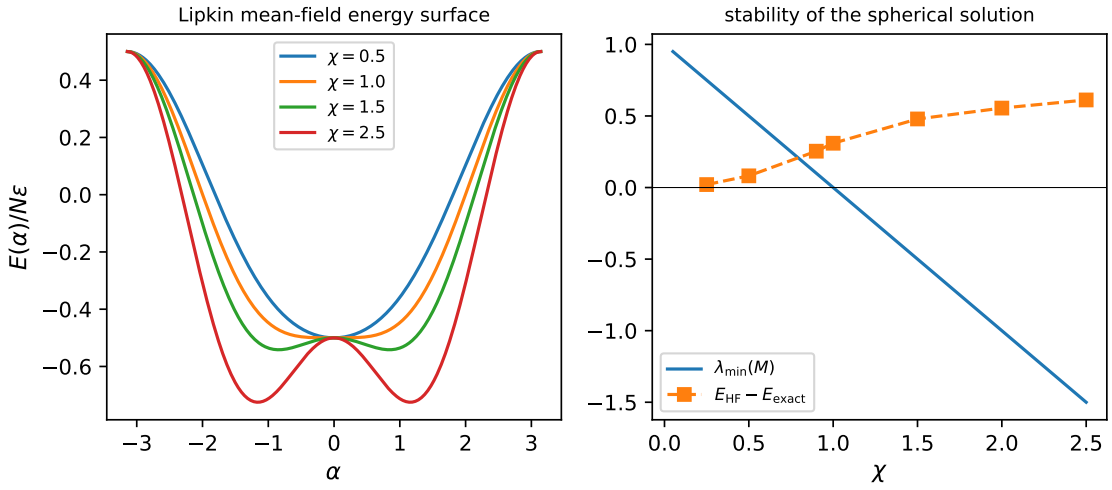


Fig. 3.3 The Lipkin model at the mean-field level ($N = 4$, $\varepsilon = 1$, $W = 0$). Left: the energy surface of Eq. (3.48) for several couplings; the minimum moves away from $\alpha = 0$ once $\chi > 1$. Right: the lowest eigenvalue of the stability matrix of the spherical solution, $\lambda_{\min} = \varepsilon(1-\chi)$, which crosses zero at $\chi = 1$, and the error $E_{\mathrm{HF}} - E_{\mathrm{exact}}$ of the self-consistent solution, which keeps growing even after the deformed solution takes over. Produced by `hartreefock.py`.

Table 3.12 Hartree–Fock for the pairing model with $L = 4$, two pairs and $\xi = 1$: the mean field returns the reference energy. Produced by `hartreefock.py`.

g	E_{ref}	E_{HF}	E_{exact}	iterations	max $\lvert f_{ai}\rvert$
0.25	1.75	1.75000000	1.73045985	2	0
0.50	1.50	1.50000000	1.41677428	2	0
1.00	1.00	1.00000000	0.63554847	2	0

larger than the gain. Restoring the symmetry by projection, or going beyond the mean field with the methods of Section 3.6, is unavoidable. In Chapter 7 the Lipkin model will show the same transition in a variational quantum calculation, where the symmetry-broken ansatz is a rotation of every qubit by the same angle.

3.5.5 When the mean field does nothing: pairing and Hubbard

For the pairing model the Hartree–Fock method is exactly useless. Table 3.12 shows that the self-consistent energy equals the reference energy $2 - g$ for every g, that the iteration converges in two steps, and that the Brillouin elements f_{ai} vanish identically from the start. The reason is the same as for CIS in Table 3.8: the interaction cannot move a single particle out of a pair, so there are no single excitations for the mean field to mix in, and the reference determinant is already the best determinant. All of the correlation energy of Table 3.2 is beyond the mean field. This is not a defect of the model but its point: pairing correlations are the paradigm of a many-body effect that no single determinant captures, which is why the BCS theory of superconductivity had to give up the fixed particle number, and why the pairing model is the test case of choice for the correlated methods of Section 3.6 and for the quantum algorithms of Chapter 7.

The Hubbard model shows the other face of the mean field, symmetry breaking as in the Lipkin model. With the constraint that spin-up and spin-down electrons occupy the same spa-

Table 3.13 The half-filled four-site Hubbard ring at the mean-field level: the paramagnetic energy $-4t+U$, the unrestricted Hartree–Fock energy, the exact energy, and the staggered magnetization $\frac{1}{N_s}\sum_i(-1)^i\langle\hat{n}_{i\uparrow}-\hat{n}_{i\downarrow}\rangle$ of the unrestricted solution. Produced by `hartreefock.py`.

U/t	E_{para}	E_{UHF}	E_{exact}	m_{stagger}
1	-3.000000	-3.285509	-3.340848	0.2853
2	-2.000000	-2.661001	-2.828427	0.3279
4	0.000000	-1.763298	-2.102748	0.4081
8	4.000000	-0.968871	-1.320235	0.4708

tial orbitals, the plane waves of the non-interacting ring are self-consistent and the paramagnetic energy of the half-filled four-site ring is $E=-4t+U$, the kinetic energy plus the double occupancy of a uniform density, $1/4$ per site. Allowing the two spin species to occupy different orbitals, the *unrestricted* Hartree–Fock solution develops a staggered magnetization, up spins preferring the even sites and down spins the odd ones, which reduces the double occupancy at the cost of kinetic energy. Table 3.13 shows the two energies against the exact ones: the unrestricted solution is much better, and the staggered magnetization grows with U, but the exact ground state of the finite ring is a singlet with no staggered magnetization at all. Once again the mean field trades a symmetry for energy, and once again the correlated methods have to repair it.

3.6 Coupled-cluster theory

3.6.1 The exponential ansatz

The failure of truncated CI in Section 3.4.5 had a precise cause: the product of two double excitations, one on each subsystem, is a quadruple excitation that a linear truncation drops. Coupled-cluster theory keeps such products by writing the wave function as an exponential,

$$|\Psi_{\text{CC}}\rangle = e^{\hat{T}}|\Phi_0\rangle, \qquad \hat{T}=\hat{T}_1+\hat{T}_2+\cdots, \qquad \hat{T}_1=\sum_{ia} t_i^a\, a_a^\dagger a_i, \quad \hat{T}_2=\frac{1}{4}\sum_{ijab} t_{ij}^{ab}\, a_a^\dagger a_b^\dagger a_j a_i, \tag{3.49}$$

where the *cluster operator* $\hat{T}$ is a sum of excitation operators with unknown *amplitudes* t. Expanding the exponential, $e^{\hat{T}_2}=1+\hat{T}_2+\frac{1}{2}\hat{T}_2^2+\cdots$, one sees that even with $\hat{T}=\hat{T}_2$ the wave function contains quadruple excitations with coefficients $t_{ij}^{ab}t_{kl}^{cd}$, sextuples with products of three, and so on: all of the terms that CID drops are present, with coefficients determined by the doubles. This is exactly the structure that size consistency demands, since for two non-interacting systems $e^{\hat{T}_A+\hat{T}_B}=e^{\hat{T}_A}e^{\hat{T}_B}$ and the wave function factorizes. Truncating $\hat{T}$ at doubles gives CCD, at singles and doubles CCSD, and adding a perturbative estimate of the triples gives CCSD(T), the workhorse of quantum chemistry. All excitation operators commute with each other, since they only contain $a_a^\dagger$ and a_i and the anticommutators of these vanish, so the exponential is unambiguous and $e^{\hat{T}}$ is easy to manipulate.

The amplitudes are not determined variationally. Instead one inserts the ansatz into the Schrödinger equation and multiplies from the left by $e^{-\hat{T}}$,

$$e^{-\hat{T}}\hat{H}e^{\hat{T}}|\Phi_0\rangle=\bar{H}|\Phi_0\rangle=E|\Phi_0\rangle. \tag{3.50}$$

The *similarity-transformed* Hamiltonian $\bar{H}$ is not Hermitian but it has the same spectrum as $\hat{H}$, and the BCH expansion of Eq. (3.44) gives it as a sum of nested commutators,

$$\bar{H} = \hat{H} + [\hat{H},\hat{T}] + \frac{1}{2}[[\hat{H},\hat{T}],\hat{T}] + \frac{1}{6}[[[\hat{H},\hat{T}],\hat{T}],\hat{T}] + \frac{1}{24}[[[[\hat{H},\hat{T}],\hat{T}],\hat{T}],\hat{T}]. \tag{3.51}$$

Here the series *terminates*: each commutator with $\hat{T}$ removes one of the operators of $\hat{H}$ by a contraction, since $\hat{T}$ commutes with itself and only its contractions with $\hat{H}$ survive, and a two-body Hamiltonian has four operators. This is what makes the method exact in principle and tractable in practice. Projecting Eq. (3.50) onto the reference and onto the excited determinants gives the energy and the amplitude equations,

$$E = \langle \Phi_0 | \bar{H} | \Phi_0 \rangle, \qquad 0 = \langle \Phi_{ij}^{ab} | \bar{H} | \Phi_0 \rangle, \tag{3.52}$$

as many equations as amplitudes. For $\hat{T} = \hat{T}_2$ the energy is

$$E_{\mathrm{CCD}} = E_{\mathrm{ref}} + \frac{1}{4}\sum_{ijab} \langle ij|v|ab\rangle\, t_{ij}^{ab}, \tag{3.53}$$

which involves only the doubles amplitudes and the two-body matrix elements: all the higher excitations enter the energy through the amplitude equations, not directly.

3.6.2 The CCD equations

Evaluating $\langle \Phi_{ij}^{ab} | \bar{H} | \Phi_0 \rangle$ with Wick's theorem, with $\hat{H}$ in the normal-ordered form of Eq. (3.19) and a Hartree–Fock or otherwise diagonal-dominated Fock matrix, gives the CCD amplitude equations

$$\begin{aligned} 0 = {} & \langle ab|v|ij\rangle + P(ab)\sum_c f_{bc}\, t_{ij}^{ac} - P(ij)\sum_k f_{kj}\, t_{ik}^{ab} + \frac{1}{2}\sum_{cd}\langle ab|v|cd\rangle t_{ij}^{cd} + \frac{1}{2}\sum_{kl}\langle kl|v|ij\rangle t_{kl}^{ab} \\ & + P(ij)P(ab)\sum_{kc}\langle kb|v|cj\rangle t_{ik}^{ac} + \frac{1}{4}\sum_{klcd}\langle kl|v|cd\rangle t_{ij}^{cd} t_{kl}^{ab} + \frac{1}{2}P(ij)P(ab)\sum_{klcd}\langle kl|v|cd\rangle t_{ik}^{ac} t_{jl}^{bd} \\ & - \frac{1}{2}P(ij)\sum_{klcd}\langle kl|v|cd\rangle t_{jl}^{cd} t_{ik}^{ab} - \frac{1}{2}P(ab)\sum_{klcd}\langle kl|v|cd\rangle t_{kl}^{bd} t_{ij}^{ac}, \end{aligned} \tag{3.54}$$

where $P(ij)X_{ij} = X_{ij} - X_{ji}$ antisymmetrizes. The terms linear in the amplitudes are the driving term, the one-body terms, the particle-particle and hole-hole ladders and the ring term; the four quadratic terms come from $\frac{1}{2}[[\hat{H},\hat{T}],\hat{T}]$. The equations are solved by iteration: the diagonal parts of the one-body terms are moved to the left, giving the denominator $D_{ij}^{ab} = f_{ii} + f_{jj} - f_{aa} - f_{bb}$, the remaining terms are evaluated with the current amplitudes and the new amplitudes are the result divided by D. Starting from $t = 0$ the first iteration gives $t_{ij}^{ab} = \langle ab|v|ij\rangle / D_{ij}^{ab}$, which inserted into Eq. (3.53) is the second-order perturbation energy,

$$E^{(2)} = \frac{1}{4}\sum_{ijab}\frac{|\langle ab|v|ij\rangle|^2}{f_{ii} + f_{jj} - f_{aa} - f_{bb}}, \tag{3.55}$$

the Møller–Plesset (MP2) energy; the converged amplitudes sum the ladder, ring and quadratic diagrams to all orders. For the pairing model the structure of Eq. (3.54) simplifies considerably. The only non-zero elements $\langle ab|v|ij\rangle$ are those with (a,b) and (i,j) both time-reversed partners, so the only amplitudes driven by the interaction are the pair amplitudes $t_{i\bar{i}}^{a\bar{a}}$, one per pair of levels, four in all for $L = 4$ and two pairs; the ring term vanishes for the same reason; and the MP2 energy of Eq. (3.55) reduces to a sum over pairs of levels,

Table 3.14 Coupled-cluster doubles for the pairing model with $L = 4$, two pairs and $\xi = 1$ (reference energy $2 - g$): the second-order (MP2) energy of the first iteration, the converged CCD energy, the exact energy and the number of iterations to 10^{-12}. Produced by `coupledcluster.py`.

g	MP2	CCD	FCI	CCD − FCI	iterations
0.25	1.73320261	1.73045621	1.73045985	-3.6×10^{-6}	13
0.50	1.43760684	1.41663766	1.41677428	-1.4×10^{-4}	18
1.00	0.78095238	0.63044275	0.63554847	-5.1×10^{-3}	26
1.50	0.05974026	−0.38416526	−0.34819051	-3.6×10^{-2}	30
2.00	−0.70833333	−1.60959440	−1.48965216	-1.2×10^{-1}	31

$$E^{(2)} = -\frac{g^2}{4} \sum_{i \le n} \sum_{a > n} \frac{1}{2\xi(a-i)+g}, \tag{3.56}$$

since the occupied levels have $f_{ii} = \xi(i-1) - g/2$ and the empty ones $f_{aa} = \xi(a-1)$. For $L = 4$, two pairs and $g = 1$ this is $-\frac{1}{4}(\frac{1}{5} + \frac{1}{3} + \frac{1}{7} + \frac{1}{5}) = -0.219048$, which added to $E_{\text{ref}} = 1$ gives the 0.78095238 of Table 3.14. Nothing in the program exploits this: the general equations are solved in the full spin-orbital basis, and the seniority structure emerges from the matrix elements. With `numpy` the equations are nine contractions, and `coupledcluster.py` writes them exactly in the form of Eq. (3.54),

```
def residual(self, t):
    v, P = self, lambda X: X - X.transpose(1, 0, 2, 3) - X.transpose(0, 1, 3, 2) + X.transpose(1, 0, 3, 2)
    R = np.einsum("abij->ijab", v.vvoo).copy()                  # <ab|v|ij>
    X = np.einsum("bc,ijac->ijab", v.fv, t); R += X - X.transpose(0, 1, 3, 2)
    X = np.einsum("kj,ikab->ijab", v.fo, t); R -= X - X.transpose(1, 0, 2, 3)
    R += 0.5 * np.einsum("abcd,ijcd->ijab", v.vvvv, t)          # pp ladder
    R += 0.5 * np.einsum("klij,klab->ijab", v.oooo, t)          # hh ladder
    R += P(np.einsum("kbcj,ikac->ijab", v.ovvo, t))             # ring
    R += 0.25 * np.einsum("klcd,ijcd,klab->ijab", v.oovv, t, t)
    R += 0.5 * P(np.einsum("klcd,ikac,jlbd->ijab", v.oovv, t, t))
    X = np.einsum("klcd,jlcd,ikab->ijab", v.oovv, t, t); R -= 0.5 * (X - X.transpose(1, 0, 2, 3))
    X = np.einsum("klcd,klbd,ijac->ijab", v.oovv, t, t); R -= 0.5 * (X - X.transpose(0, 1, 3, 2))
    return R                                                    # t_new = R / D
```

where `fo` and `fv` are the off-diagonal parts of the occupied and virtual blocks of the Fock matrix, zero for a canonical Hartree–Fock basis.

Table 3.14 shows the result for the pairing model. At weak coupling CCD is essentially exact, three orders of magnitude better than CID at $g = 0.25$ (compare Table 3.8), because the quadruples that CID drops are well approximated by products of doubles. As g grows the approximation deteriorates, and the CCD energy falls *below* the exact one: the method is not variational, and nothing prevents the non-Hermitian $\bar{H}$ from overshooting. At $g = 2$ the error is 0.12, comparable to CID's 0.14 but with the opposite sign. The strength of the method is at the same time its limitation: the coupled equations converge more slowly as g grows, and beyond the range of the table they may not converge at all. Size extensivity, however, holds at every g: for the two non-interacting copies of Table 3.9 CCD reproduces $2E_A$ exactly, since the amplitude equations of the combined system decouple into those of the two parts (Exercise 6).

3.6.3 Unitary coupled cluster

The non-variational character of coupled-cluster theory can be removed by making the wave operator unitary,

$$|\Psi_{\mathrm{UCC}}\rangle = e^{\hat{T}-\hat{T}^\dagger}|\Phi_0\rangle, \qquad E_{\mathrm{UCC}}(t) = \langle\Phi_0|e^{-(\hat{T}-\hat{T}^\dagger)}\hat{H}e^{\hat{T}-\hat{T}^\dagger}|\Phi_0\rangle, \tag{3.57}$$

the *unitary coupled-cluster* ansatz. Since $\hat{T}-\hat{T}^\dagger$ is anti-Hermitian its exponential is unitary, the state is normalized, $E_{\mathrm{UCC}}(t)$ is an expectation value and therefore an upper bound on the exact ground-state energy, and the amplitudes can be found by minimizing it. The price is that the BCH expansion of the similarity-transformed Hamiltonian no longer terminates: $\hat{T}^\dagger$ contains $a_i^\dagger a_a$, which does contract with $\hat{T}$, so every order of the commutator series contributes, and on a classical computer the energy of Eq. (3.57) can only be evaluated approximately, or exactly at the cost of the full Hilbert space. On a quantum computer the situation is reversed: a unitary operator is exactly what a circuit implements, the expectation value of $\hat{H}$ is what a measurement returns, and the ansatz of Eq. (3.57) becomes the natural trial state of the variational quantum eigensolver of Chapter 7. This section computes it classically, on the small pairing model where the full space is available, to establish what the quantum algorithm should reproduce.

The program `coupledcluster.py` builds the generators $\hat{\kappa}_k = \hat{A}_k - \hat{A}_k^\dagger$ as matrices in the four-particle sector of Fock space, forms $|\Psi(t)\rangle = \exp(\sum_k t_k \hat{\kappa}_k)|\Phi_0\rangle$ with the matrix exponential of `matrixfunctions.py`, and minimizes $\langle\Psi(t)|\hat{H}|\Psi(t)\rangle$ with the BFGS routine of `optimize.py`,

```
class UnitaryCC:
    def __init__(self, H, generators, reference, sector):
        idx = np.asarray(sector)                    # the N-particle block
        self.H = H[np.ix_(idx, idx)]
        self.kappa = [A[np.ix_(idx, idx)] - A[np.ix_(idx, idx)].T for A in generators]
        self.ref = reference[idx]

    def state(self, t):
        return expm_antihermitian(sum(tk * k for tk, k in zip(t, self.kappa))) @ self.ref

    def energy(self, t):
        psi = self.state(t)
        return float(np.real(np.vdot(psi, self.H @ psi)))

    def optimise(self, t0=None):
        t0 = np.zeros(len(self.kappa)) if t0 is None else t0
        return minimize(self.energy, t0, tol=1e-13, gtol=1e-9)
```

Which generators to include is a choice. For the pairing model the natural ones are the four pair excitations $\hat{P}_a^+\hat{P}_i^-$ that move a pair from an occupied level $i \in \{1,2\}$ to an empty level $a \in \{3,4\}$; the general doubles $a_a^\dagger a_b^\dagger a_j a_i$ number 36 and singles and doubles together 52. Table 3.15 shows that at $g = 1$ all three choices give the same energy, 0.6369869810, to ten digits: the extra generators are redundant because the interaction conserves seniority and the exact ground state lives in the seniority-zero space that the four pair excitations already span. Choosing generators that respect the symmetry of the problem is the classical analogue of choosing a compact qubit encoding, and Chapter 4 will make the connection explicit.

The lower part of Table 3.15 and the left panel of Fig. 3.4 compare UCCD with CCD. The unitary energy is above the exact one at every coupling, as it must be, and its error is three to four times smaller than that of CCD and grows more slowly. Both methods share the same limitation, that a single reference determinant dressed with double excitations cannot represent a state in which the reference has lost most of its weight, but the unitary ansatz fails more gracefully. That the remaining error is not zero even though the four generators

Table 3.15 Unitary coupled cluster for the pairing model with $L = 4$, two pairs and $\xi = 1$. Top: the energy at $g = 1$ for three choices of generators. Bottom: UCCD with the four pair excitations against CCD and the exact energy; UCCD is variational and lies above, CCD below. Produced by `coupledcluster.py`.

generators	amplitudes	E_{UCC} $(g = 1)$
pair excitations	4	0.6369869810
all doubles	36	0.6369869810
singles and doubles	52	0.6369869810

g	CCD	UCCD	FCI	CCD − FCI	UCCD − FCI
0.25	1.73045621	1.73046012	1.73045985	-3.6×10^{-6}	2.8×10^{-7}
0.50	1.41663766	1.41679562	1.41677428	-1.4×10^{-4}	2.1×10^{-5}
1.00	0.63044275	0.63698698	0.63554847	-5.1×10^{-3}	1.4×10^{-3}
1.50	−0.38416526	−0.33675149	−0.34819051	-3.6×10^{-2}	1.1×10^{-2}
2.00	−1.60959440	−1.45347633	−1.48965216	-1.2×10^{-1}	3.6×10^{-2}

span the whole seniority-zero space is worth a moment's thought: the exact ground state is some unit vector in the six-dimensional space, and $e^{\sum t_k \hat{\kappa}_k}|\Phi_0\rangle$ with four parameters sweeps out a four-dimensional surface in it, which does not contain every vector. A second exponential factor, or products of exponentials with independent amplitudes, would close the gap; this is the idea behind the adaptive and disentangled ansätze of Chapter 7.

3.6.4 Trotterized unitary coupled cluster

A quantum computer cannot apply $\exp(\sum_k t_k \hat{\kappa}_k)$ directly; it applies gates, and by Section 3.5.3 the natural gate sequence is the Trotter product

$$|\Psi_n(t)\rangle = \left(\prod_k e^{t_k \hat{\kappa}_k / n} \right)^n |\Phi_0\rangle, \tag{3.58}$$

in which each factor is the exponential of a single excitation operator, which Chapter 4 will turn into a short circuit. The factors do not commute, since $\hat{\kappa}_k$ contains both $\hat{A}_k$ and $\hat{A}_k^\dagger$, so Eq. (3.58) differs from Eq. (3.57) by the Trotter error, and one may ask how large n has to be. Table 3.16 answers the question in two ways. With the amplitudes frozen at their optimal values from Eq. (3.57), the state $|\Psi_n\rangle$ approaches $|\Psi\rangle$ as $1/n$ and the energy as $1/n^2$, the latter because the energy is stationary at the optimum, so that a first-order error in the state produces a second-order error in the energy; both are already small at $n = 1$. But the amplitudes need not be frozen. If they are re-optimized for each n, the energy is 0.6369869810 for $n = 1, 2, 4, 8$ alike: the one-step Trotter product with adjusted amplitudes is exactly as good a variational ansatz as the full exponential. The reason is again BCH, Eq. (3.44): the product of exponentials is a single exponential of a modified generator that lies in the same algebra, and the minimization simply finds the modified amplitudes. For the variational quantum eigensolver this is the important result: the Trotter number is not a convergence parameter but part of the definition of the ansatz, and $n = 1$ is usually the right choice.

Table 3.16 The Trotterized UCCD state of Eq. (3.58) for the pairing model at $g = 1$ with the amplitudes frozen at the optimum of the full exponential: the distance to the full-exponential state, the energy error, and the ratios of successive entries, which approach 2 and 4. With re-optimized amplitudes the energy is 0.6369869810 for every n. Produced by `coupledcluster.py`.

n	$\|\Psi_n - \Psi\|$	ratio	$E_n - E$	ratio
1	3.56×10^{-3}	—	6.78×10^{-5}	—
2	1.14×10^{-3}	3.13	6.38×10^{-6}	10.63
4	4.56×10^{-4}	2.50	9.37×10^{-7}	6.81
8	2.11×10^{-4}	2.16	1.93×10^{-7}	4.85
16	1.03×10^{-4}	2.04	4.57×10^{-8}	4.23
32	5.14×10^{-5}	2.01	1.13×10^{-8}	4.06
64	2.57×10^{-5}	2.00	2.81×10^{-9}	4.01

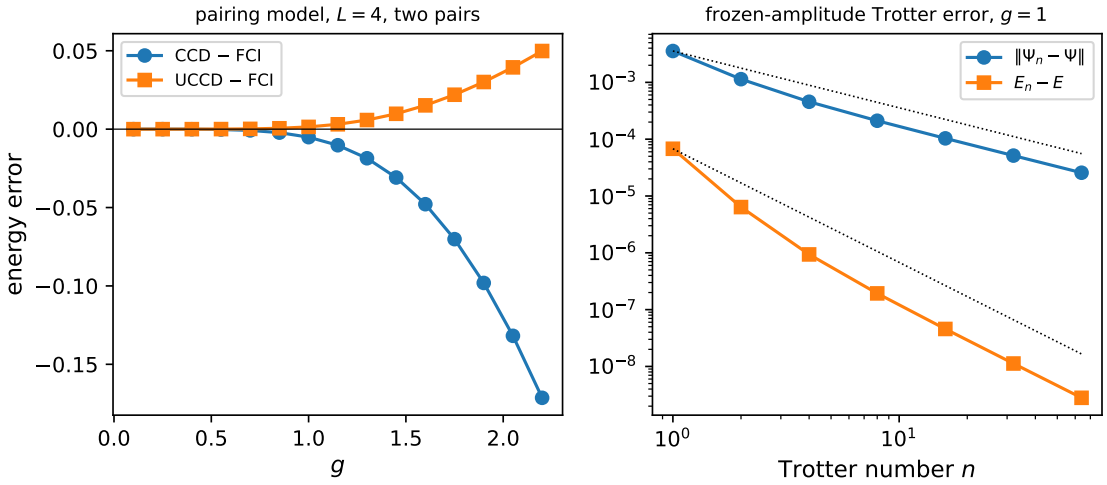


Fig. 3.4 Left: the error of CCD and UCCD for the pairing model with $L = 4$ and two pairs as a function of g; CCD undershoots, UCCD is an upper bound. Right: the Trotter error of the UCCD state with frozen amplitudes at $g = 1$, in the norm of the state and in the energy, with $1/n$ and $1/n^2$ reference lines. Produced by `coupledcluster.py`.

3.7 Summary

The chapter has moved from a language, second quantization, through four models, to three methods. The language reduces every many-fermion problem to an algebra of operators satisfying Eq. (3.3), to states that are bit strings, and to Hamiltonians of the form of Eq. (3.10) specified by two tables of matrix elements; the sign rule of Eq. (3.7) is the only place where the fermionic statistics enters, and it will reappear as the parity string of Chapter 4. The models, Lipkin, pairing, Hubbard and Heisenberg, are each defined by a restriction of those tables and each protected by a symmetry that makes small instances exactly solvable. The methods are exact diagonalization, which is exact and exponentially expensive; Hartree–Fock, which is cheap, variational, and either does nothing (pairing) or breaks a symmetry (Lipkin, Hubbard); and coupled cluster, which captures the correlation energy with a polynomial number of parameters, is size extensive, and in its standard form is not variational. The unitary variant repairs the last defect at a cost that only a quantum computer can pay, and the Trotterized form of Eq. (3.58) is, term by term, a sequence of exponentials of few-body operators. The next chapter turns each of those operators into a product of Pauli matrices, and each exponential into a circuit.

3.8 Exercises

1. **The anticommutator algebra.** Using only Eq. (3.3), show that (a) $\hat{n}_p^2 = \hat{n}_p$; (b) $[\hat{n}_p, a_q^\dagger] = \delta_{pq} a_q^\dagger$ and $[\hat{n}_p, a_q] = -\delta_{pq} a_q$, so that $a_q^\dagger$ raises the occupation of q by one; (c) the one-body operator $\hat{F} = \sum_{pq} f_{pq} a_p^\dagger a_q$ commutes with $\hat{N}$; (d) the quasispin operators of Eq. (3.25) satisfy $[\hat{J}_+, \hat{J}_-] = 2\hat{J}_z$.
2. **Signs.** Write the state $|0\,2\,3\rangle$ of a four-state system as a bit string and apply, in turn, $a_1^\dagger$, a_2, $a_1 a_1^\dagger$ and $a_1^\dagger a_1$, keeping track of the signs of Eq. (3.7). Check your result with `apply_creation` and with the matrices of `FockSpace`. Then show, from Eq. (3.9), that $a_p a_q = -a_q a_p$ for $p < q$ by locating the single tensor factor on which the two matrices anticommute.
3. **Wick's theorem.** (a) Evaluate $\langle 0 | a_s a_r a_p^\dagger a_q^\dagger | 0 \rangle$ by anticommuting the operators through each other, and again by listing the three full pairings; the result is $\delta_{rp}\delta_{sq} - \delta_{sp}\delta_{rq}$. (b) Show that a two-body operator has vanishing matrix elements between determinants that differ in more than two occupations, and count the non-zero elements per row of the FCI matrix for the pairing model with $L = 4$ and two pairs in the full 70-dimensional space. (c) Derive the reference energy and the Fock matrix of Eq. (3.20) by normal ordering $\hat{H}$ with respect to $|\Phi_0\rangle$.
4. **The pairing model in the seniority-zero space.** Write the matrix of Eq. (3.28) in the basis of the six pair configurations of $L = 4$, $n = 2$, and diagonalize it for $g = 1$ by hand or with `numpy`. Compare with Table 3.2. Then add the particle-hole term of Eq. (3.30) with $f = 0.5$, diagonalize in the full space with `PairingModel(4, g=1, f=0.5)` and show that the ground state acquires seniority-two components. What is the largest matrix element between the seniority-zero block and the rest?
5. **Lipkin mean field.** (a) Derive Eq. (3.48) by evaluating $\langle \Phi(\alpha) | \hat{H} | \Phi(\alpha) \rangle$ for the rotated determinant, using the quasispin form of Eq. (3.26) with $W = 0$ and the fact that the rotated state is a coherent state of $\hat{J}$. (b) Find the two stationary points and the energy of the deformed one. (c) Verify with `hartreefock.py` that the self-consistent iteration reproduces the deformed energy for $\chi = 1.5$ and 2 and that the lowest eigenvalue of the stability matrix is $\varepsilon(1-\chi)$. (d) Explain why the *exact* ground state has a definite parity of J_z and the deformed mean-field state does not.
6. **Size consistency and extensivity.** For the two non-interacting copies of Table 3.9: (a) write the CID wave function of one copy, $|\Phi_0\rangle + c|\Phi_\mathrm{D}\rangle$, and show that the product of two such states contains a quadruple excitation with coefficient c^2; (b) show that the CCD amplitude equations of the combined system separate into those of the parts, so that $E_\mathrm{CCD}(A+B) = E_\mathrm{CCD}(A) + E_\mathrm{CCD}(B)$; (c) run `CCD` on `NonInteractingCopies(2, g)` from `fci.py` and confirm (b) numerically; (d) is UCCD size extensive?
7. **Trotter errors.** (a) Derive the first-order error bound of Eq. (3.45) from Eq. (3.44). (b) Show that the Strang splitting of Eq. (3.46) has no term of order $1/n$. (c) For the Heisenberg ring of Fig. 3.2, split the Hamiltonian instead into the XX, YY and ZZ parts and compare the errors with the even/odd bond splitting at the same n. Which commutator norm decides?
8. **UCC by hand.** For the pairing model with $L = 2$ and one pair, the seniority-zero space is two-dimensional. (a) Write $\hat{H}$ as a 2×2 matrix and the single pair-excitation generator $\hat{\kappa} = \hat{P}_2^+ \hat{P}_1^- - \mathrm{h.c.}$ in the same basis. (b) Show that $e^{\theta\hat{\kappa}}|\Phi_0\rangle = \cos\theta\,|\Phi_0\rangle + \sin\theta\,|\Phi_\mathrm{D}\rangle$ and that minimizing the energy over θ gives the exact ground state. (c) Why does the same argument fail for $L = 4$ and two pairs?

Programs for this chapter

Every number and figure in this chapter comes from the programs in `BookPrograms/chapter3/` and from the shared modules in `BookPrograms/common/` that this chapter introduces and Chapters 4 and 7 reuse. All of them are imported by the notebook `Notebooks/chapter3/chapter3.ipynb`, and all run on `numpy` and `matplotlib` alone.

`common/fockspace.py` The class `FockSpace`: the $2^M \times 2^M$ matrices of Eq. (3.9), number operators, one- and two-body operators from tabulated matrix elements, the particle-number sectors, and a check of all anticommutators; and the class `SlaterBasis`: determinants as bit strings, the sign rule of Eq. (3.7), excitation levels, and the Hamiltonian matrix of Section 3.4.2. Instantaneous.

`common/models.py` The classes `LipkinModel`, `PairingModel`, `HubbardModel` and `HeisenbergModel`: h_{pq} and $\langle pq|v|rs\rangle$ for each model in a spin-orbital basis, the quasispin, seniority-zero and S^z blocks, and exact diagonalization in a particle-number sector; run as a script it reproduces the reference numbers of Section 3.3. Instantaneous.

`common/matrixfunctions.py` Matrix exponentials of Hermitian and anti-Hermitian matrices by diagonalization, and of general matrices by scaling and squaring. Instantaneous.

`common/optimize.py` A BFGS minimizer with backtracking line search and numerical gradients, used for the unitary coupled-cluster amplitudes here and for the variational algorithms of Chapter 7. Instantaneous.

`second_quantization.py` The worked examples of Section 3.2: bit strings and signs, anticommutators as matrices, Wick's theorem against brute force, the Slater–Condon rules, the reference energy, the Fock matrix and the Brillouin elements of Eq. (3.21), and the two-mode Hamiltonian of Chapter 4. Instantaneous.

`fci.py` Exact diagonalization of the four models, Tables 3.1–3.4 and 3.7, the Lanczos iteration of Table 3.6, truncated CI and the size-consistency test, Tables 3.8 and 3.9, and Fig. 3.1. A few seconds, dominated by the fourteen-site Heisenberg ring.

`hartreefock.py` The class `HartreeFock` of Section 3.5.1 with the stability matrix of Eq. (3.43) and the Thouless state of Eq. (3.42); the Lipkin, pairing and Hubbard results of Tables 3.11–3.13 and Fig. 3.3. Seconds.

`trotter.py` The BCH series order by order, the Lie–Trotter and Strang splittings of Table 3.10 and Fig. 3.2. Instantaneous.

`coupledcluster.py` The class `CCD` with the amplitude equations of Eq. (3.54) and the class `UnitaryCC` with the exact and Trotterized exponentials; Tables 3.14–3.16 and Fig. 3.4. About ten seconds.

Chapter 4
Mapping Many-Body Hamiltonians to Qubits

Abstract This chapter describes how to translate many-body quantum problems into a form suitable for quantum hardware. We explain fermion-to-qubit encoding techniques such as the Jordan–Wigner and Bravyi–Kitaev transformations, which map fermionic creation and annihilation operators to Pauli operators on qubits. The mapping of spin Hamiltonians and the choice of computational basis states (for example in an occupation-number basis) are also discussed. We examine how symmetries and conserved quantities (particle number, spin, etc.) can reduce resource requirements. We illustrate these mappings with explicit model Hamiltonians and showing how many-body Hamiltonians can be implemented on quantum circuits.

4.1 Introduction

Chapter 2 described a quantum computer as a register of qubits acted on by unitary gates and read out by measurements in the computational basis; Chapter 3 described a many-body system as a Fock space acted on by creation and annihilation operators. To run a many-body calculation on a quantum computer the two descriptions have to be connected, and this chapter is about that connection. Three things must be translated. The *states*: an occupation-number state $|n_0 n_1 \cdots n_{M-1}\rangle$ of M fermionic modes looks exactly like a computational basis state of M qubits, and the obvious identification, one qubit per mode with $|1\rangle$ meaning occupied, is the one we shall use. The *operators*: the creation and annihilation operators anticommute, whereas operators on different qubits commute, and the resolution of this conflict is the Jordan–Wigner transformation, which we have in fact already met as the string of Z matrices in Eq. (3.9). And the *Hamiltonian*: once every $a_p^\dagger a_q$ and $a_p^\dagger a_q^\dagger a_s a_r$ is a product of Pauli matrices, the Hamiltonian is a sum of such products, a *Pauli sum*, and it is in this form that every quantum algorithm of the following chapters consumes it. Time evolution (Chapter 5) exponentiates the Pauli sum term by term; phase estimation (Chapter 6) applies the resulting circuit under control of an ancilla; the variational algorithms (Chapter 7) measure the Pauli sum term by term.

The translation is exact and, for the operators, unique up to conventions. What is not unique is its cost. The number of qubits, the number of Pauli strings in the Hamiltonian and the number of qubits each string touches, its *weight*, are the three quantities that decide whether a calculation fits on a given device, and all three depend on choices: which single-particle basis, which ordering of the modes, and above all which symmetry of the problem has been used to shrink the register before the mapping is applied. The models of Chapter 3 were chosen because each has such a symmetry, and the second half of this chapter shows what each symmetry is worth on a qubit register. The last sections turn the Pauli sums into circuits: the exponential of a single Pauli string, the Trotter step of a Hamiltonian, the preparation

of a reference determinant, the measurement of an energy from a finite number of shots, and finally the unitary coupled-cluster circuit for the pairing model, which reproduces the numbers of Section 3.6.3 with four rotation angles on four qubits and is the first variational quantum calculation of the book.

4.2 Pauli strings and Pauli sums

A *Pauli string* on n qubits is a tensor product of one Pauli matrix or identity per qubit,

$$P = \sigma_0 \otimes \sigma_1 \otimes \cdots \otimes \sigma_{n-1}, \qquad \sigma_q \in \{I, X, Y, Z\}, \tag{4.1}$$

which we write as a word such as $ZZXI$, or as $Z_0 Z_1 X_2$ when the identities are left out, with qubit 0 the leftmost factor as in Chapters 2 and 3. The *weight* of a string is the number of factors different from I. There are 4^n strings, they are Hermitian and unitary, they square to the identity, and the product of two strings is again a string up to a phase ± 1 or $\pm i$, obtained factor by factor from the single-qubit rule $\sigma_i \sigma_j = \delta_{ij} I + i \varepsilon_{ijk} \sigma_k$. Two strings either commute or anticommute: they anticommute if and only if they differ, both being non-identity, on an odd number of qubits. Finally the strings are orthogonal with respect to the trace inner product, $\mathrm{Tr}(PQ) = 2^n \delta_{PQ}$, and since there are 4^n of them they form a basis of the $2^n \times 2^n$ matrices. Every Hermitian operator on the register is therefore a *Pauli sum*

$$\hat{H} = \sum_{\alpha} h_\alpha P_\alpha, \qquad h_\alpha = \frac{1}{2^n} \mathrm{Tr}\left(P_\alpha \hat{H}\right) \in \mathbb{R}, \tag{4.2}$$

with real coefficients. Equation (4.2) is a constructive statement: given any matrix, 4^n traces produce its Pauli decomposition, and the class `PauliSum` in `pauli.py` does this, more cleverly than by 4^n full traces, by contracting one qubit at a time. It stores a Pauli sum as a dictionary from strings to coefficients and implements the algebra,

```
from pauli import PauliSum

X0 = PauliSum.single(2, 0, "X"); Y0 = PauliSum.single(2, 0, "Y"); Z1 = PauliSum.single(2, 1, "Z")
print(X0 * Y0)                  # +(0.0000+1.0000j) ZI
print(X0.commutator(Z1))        # 0
H = PauliSum({"ZZ": 0.5, "XX": 0.25, "YY": 0.25, "IZ": -0.3})
print(H.matrix().shape, H.n_strings(), H.max_weight(), H.weights())
print(PauliSum.from_matrix(H.matrix()).terms == H.terms)
```

For a random Hermitian 8×8 matrix the decomposition has all 64 strings, one of weight zero, nine of weight one, and twenty-seven each of weights two and three, which is the generic situation and the one to be avoided. A structured Hamiltonian has far fewer strings, and the point of this chapter is to show how many, and how to keep the number small.

Why this form? Because the Pauli strings are exactly the operators a quantum computer can handle natively. The exponential of a single string, $e^{-i\theta P}$, is a short circuit (Section 4.5); the expectation value of a single string is obtained from measurements in the computational basis after a layer of single-qubit rotations (Section 4.6); and both procedures cost a number of gates proportional to the weight of the string. A Pauli sum with K strings of weight at most w can thus be time-evolved with $\mathcal{O}(Kw)$ gates per Trotter step and its energy can be measured with at most K distinct measurement settings. These are the cost functions against which the mappings below are judged.

Table 4.1 Two bounds read off from a Pauli sum: $\sum_\alpha |h_\alpha|$ against the exact spectral norm, and the commutator sum $\sum_{\alpha<\beta} |h_\alpha h_\beta| \, \|[P_\alpha, P_\beta]\|$ that controls the first-order Trotter error, for the Hamiltonians of Section 4.4 at $\xi = g = J = t = 1$, $U = 4$. Produced by `mapped_models.py`.

Hamiltonian	$\sum_\alpha \lvert h_\alpha \rvert$	$\lVert \hat{H} \rVert$	commutator bound
Lipkin, one qubit per doublet	6.000	4.213	8.583
pairing $L = 4$, one qubit per level	13.500	10.672	19.500
pairing $L = 4$, one qubit per orbital	14.500	10.672	19.500
Hubbard ring, spin-major ordering	24.000	16.000	72.000
Heisenberg ring	3.000	2.000	3.000

4.2.1 Pauli sums on a classical computer

A Pauli sum is also a convenient representation for classical computation, and the programs of this and the following chapters use it as such. The product $\hat{H}|\Psi\rangle$ never requires the $2^n \times 2^n$ matrix: each string acts on the tensor of amplitudes $\Psi_{i_0 i_1 \cdots i_{n-1}}$ one factor at a time, a 2×2 matrix applied along one index, so that the cost of the product is $\mathcal{O}(K w 2^n)$ for K strings of weight w rather than $\mathcal{O}(4^n)$, and the memory is that of the state alone. The method `apply` of `PauliSum` does this, and combined with the Lanczos iteration of Section 3.4.3 it diagonalizes the eight-qubit Hubbard Hamiltonian of Section 4.4.4, 29 strings on 256 amplitudes, in thirty steps and a few hundredths of a second, reproducing the -2.10274848 of Table 3.3. This is exactly what a classical exact-diagonalization code does with bit strings instead of Pauli strings, and it is what every simulator of quantum circuits does with gates; the number of qubits it can reach is limited by the 2^n amplitudes, about thirty on a laptop and fifty on a supercomputer, which is the exponential wall of Section 3.4.4 seen from the qubit side.

Two numbers that will be needed in Chapter 5 are also read off from the Pauli sum. The triangle inequality gives $\|\hat{H}\| \le \sum_\alpha |h_\alpha|$, an upper bound on the largest eigenvalue in absolute value which fixes the time scale of the dynamics, and the first-order Trotter error of Eq. (3.45) is bounded by the sum of the commutators of the pieces, $\sum_{\alpha<\beta} |h_\alpha h_\beta| \, \|[P_\alpha, P_\beta]\|$, which for Pauli strings is simply twice the sum of $|h_\alpha h_\beta|$ over the pairs of strings that anticommute, since $[P_\alpha, P_\beta]$ is either zero or $2P_\alpha P_\beta$ with $\|P_\alpha P_\beta\| = 1$. Table 4.1 lists both for the Hamiltonians of Section 4.4. The norm bound overestimates the norm by a factor of one and a half or so, and the commutator bound, which decides how many Trotter steps a given accuracy requires, is an order of magnitude larger for the Hubbard ring than for the four-qubit models: the Hubbard hopping strings anticommute with many of their neighbours, the Heisenberg strings with few.

4.3 The Jordan–Wigner transformation

4.3.1 Why the naive map fails

A single fermionic mode is a two-level system, and Eq. (3.8) identified its operators with those of a qubit: $a^\dagger = |1\rangle\langle 0| = \sigma^+$ and $a = |0\rangle\langle 1| = \sigma^-$ with

$$\sigma^\pm = \frac{X \mp iY}{2}, \qquad \hat{n} = \sigma^+ \sigma^- = \frac{I - Z}{2}. \tag{4.3}$$

Note the sign: with $|1\rangle$ occupied and the matrix convention of Chapter 2, $Z|1\rangle = -|1\rangle$, the raising operator that creates an occupation is $(X - iY)/2$. For M modes one is tempted to

write $a_p = I \otimes \cdots \otimes \sigma^- \otimes \cdots \otimes I$, that is σ_p^- on qubit p. This reproduces $\{a_p, a_p^\dagger\} = I$ and $a_p^2 = 0$ on every single qubit, but operators on different qubits commute, so $\{a_0, a_1\} = 2\sigma_0^-\sigma_1^-$ instead of zero; `jw_basics.py` prints its Pauli form, $\frac{1}{2}(XX + iXY + iYX - YY)$, and its spectral norm, 2. A Hamiltonian built from such operators would describe hard-core bosons (Section 3.2.7), not fermions, and would get every sign of every exchange wrong.

4.3.2 The transformation

The correct answer was derived in Section 3.2.2. The sign rule of Eq. (3.7) requires a_p to multiply the state by $(-1)^{\sum_{q<p} n_q}$, and this is the eigenvalue of $\prod_{q<p} Z_q$; inserting it turns Eq. (3.9) into

$$a_p = Z_0 Z_1 \cdots Z_{p-1}\, \sigma_p^- = Z_0 Z_1 \cdots Z_{p-1}\, \frac{X_p + iY_p}{2}, \qquad a_p^\dagger = Z_0 Z_1 \cdots Z_{p-1}\, \frac{X_p - iY_p}{2}, \tag{4.4}$$

the *Jordan–Wigner transformation*. The product of Z's is the *parity string*: it measures the parity of the number of occupied modes to the left of p, and it is what a fermionic operator has to know about the rest of the register. That Eq. (4.4) satisfies the anticommutation relations of Eq. (3.3) can be checked symbolically: for $p < q$, $a_p a_q$ and $a_q a_p$ differ by the order of σ_p^- and the Z_p in the parity string of a_q, and $Z\sigma^- = -\sigma^- Z$ supplies the sign, while all other factors commute. The program `jordanwigner.py` performs this check for all pairs of modes, and also verifies that the Pauli sums of Eq. (4.4) equal, as matrices, the operators of `FockSpace` from Chapter 3, which is the statement that the two chapters use one and the same convention. The class is short,

```
class JordanWigner:
    def __init__(self, n_modes):
        self.n = n_modes

    def _zstring(self, p):                  # Z_0 ... Z_{p-1}
        return PauliSum({"".join(["Z"] * p + ["I"] * (self.n - p)): 1.0}, self.n)

    def sigma_plus(self, p):                # (X_p - i Y_p)/2 = |1><0| on qubit p
        return PauliSum.single(self.n, p, "X", 0.5) + PauliSum.single(self.n, p, "Y", -0.5j)

    def create(self, p):
        return self._zstring(p) * self.sigma_plus(p)

    def annihilate(self, p):
        return self.create(p).dagger()

    def number(self, p):                    # (I - Z_p)/2
        return 0.5 * PauliSum.identity(self.n) - PauliSum.single(self.n, p, "Z", 0.5)

    def map_hamiltonian(self, h, v):        # sum h_pq a+_p a_q + 1/4 sum v_pqrs a+_p a+_q a_s a_r
        ...
```

and `map_hamiltonian` does nothing more than multiply out the products of Eq. (3.10) as Pauli sums, letting the dictionary collect the strings. Two remarks on conventions. First, the ordering of the modes is a choice, and a different ordering gives a different, unitarily equivalent set of strings; Section 4.4.4 shows that the choice matters. Second, the literature is divided on whether $|0\rangle$ or $|1\rangle$ means occupied and hence on the sign of Y in Eq. (4.4); the physics is the same, but Pauli forms copied from different sources cannot be combined without checking. We use $|1\rangle$ for occupied throughout.

4.3.3 Operator identities

Every term of a many-body Hamiltonian is a product of two or four of the operators of Eq. (4.4), and a handful of identities cover all of them. The *number operator* has no parity string, since $Z^2 = I$,

$$\hat{n}_p = a_p^\dagger a_p = \frac{I - Z_p}{2}, \qquad \sum_p \varepsilon_p \hat{n}_p = \frac{1}{2}\sum_p \varepsilon_p \,(I - Z_p)\,, \tag{4.5}$$

so a non-interacting Hamiltonian is diagonal on the register and needs no circuit to find its spectrum, only to prepare its eigenstates. The *hopping* term between modes $p < q$ carries the parity string of the modes in between,

$$a_p^\dagger a_q + a_q^\dagger a_p = \frac{1}{2}\left(X_p Z_{p+1}\cdots Z_{q-1} X_q + Y_p Z_{p+1}\cdots Z_{q-1} Y_q\right), \tag{4.6}$$

two strings of weight $q - p + 1$: a hop has to know how many particles it jumps over. The *pair-creation* term differs from it by one sign,

$$a_p^\dagger a_q^\dagger + a_q a_p = \frac{1}{2}\left(X_p Z_{p+1}\cdots Z_{q-1} X_q - Y_p Z_{p+1}\cdots Z_{q-1} Y_q\right), \tag{4.7}$$

and that one sign is the whole difference, on a quantum computer, between the hopping term of the Hubbard model and the pair-transfer term of the pairing model. Finally a *two-body* term with four distinct indices produces eight strings of weight four with coefficients $\pm\frac{1}{8}$; for four consecutive modes, where the intermediate parity strings are empty,

$$\begin{aligned} a_0^\dagger a_1^\dagger a_3 a_2 + \text{h.c.} = \frac{1}{8}\big(&XXXX + XYXY + XYYX + YXXY + YXYX + YYYY \\ &- XXYY - YYXX\big), \end{aligned} \tag{4.8}$$

where $XYXY$ means $X_0 Y_1 X_2 Y_3$. All four identities are printed and asserted by `jw_basics.py`, which also confirms that the eight-string form is what the pair-transfer term $\hat{P}_p^+ \hat{P}_q^- + \text{h.c.}$ of the pairing model becomes when the modes are ordered $(p+, p-, q+, q-)$. For a general two-body Hamiltonian the number of strings grows as $\mathcal{O}(M^4)$ and the longest of them span the whole register.

4.3.4 The cost for a generic Hamiltonian

Table 4.2 gives the count for a Hamiltonian of the form of Eq. (3.10) with random one- and two-body matrix elements on M modes: 6, 19, 55, 136 and 292 strings for $M = 2, \ldots, 6$, a small fraction of the 4^M strings available and consistent with $\mathcal{O}(M^4)$, but with a largest weight equal to M, because the parity strings of the hopping terms between the first and the last mode reach across everything. For a molecule with $M = 100$ spin-orbitals this means of order 10^7 strings, some of weight 100, which is why the mapping of quantum chemistry to qubits is a research field of its own. The program `jw_basics.py` checks, for $M \le 4$, that the Pauli sum equals the Fock-space matrix of Chapter 3, so that the counts are counts of an exact decomposition.

The Jordan–Wigner transformation is exact and the easiest to derive, but it is not the only fermion-to-qubit mapping, and the alternatives attack the weight. All of them store, on each qubit, some parity of a subset of the occupation numbers, and differ in which subsets. The Jordan–Wigner mapping stores n_j itself, so that the occupation is local, $\hat{n}_p = (I - Z_p)/2$, but the parity needed by a_p has to be collected from all qubits to its left. The *parity mapping*

Table 4.2 Pauli strings in the Jordan–Wigner image of a generic one- plus two-body Hamiltonian on M modes. The count grows as $\mathcal{O}(M^4)$ but the largest weight is M. Produced by `jw_basics.py`.

M	4^M	Pauli strings	largest weight
2	16	6	2
3	64	19	3
4	256	55	4
5	1024	136	5
6	4096	292	6

Table 4.3 Three fermion-to-qubit mappings. Top: the weight of $a_p^\dagger$ for every mode p on $M=4$ and $M=8$ qubits. Bottom: number of strings and largest weight of the pairing Hamiltonian with $L=4$ and of the four-site Hubbard ring in the spin-major ordering, both on eight qubits. Produced by `jw_basics.py`.

mapping	weights of $a_p^\dagger$, $M=4$	weights of $a_p^\dagger$, $M=8$
Jordan–Wigner	$1,2,3,4$	$1,2,3,4,5,6,7,8$
parity	$4,4,3,2$	$8,8,7,6,5,4,3,2$
Bravyi–Kitaev	$3,3,3,3$	$4,4,4,4,4,4,4,4$
mapping	**pairing, $L=4$ (strings, weight)**	**Hubbard ring (strings, weight)**
Jordan–Wigner	61, 4	29, 4
parity	61, 6	29, 5
Bravyi–Kitaev	61, 6	29, 5

stores on qubit j the cumulative parity $n_0 \oplus \cdots \oplus n_j$; then the parity needed by a_p sits on the single qubit $p-1$, but the occupation becomes $\hat{n}_p = (I - Z_{p-1}Z_p)/2$, and creating a particle in mode p changes the cumulative parity of every qubit $j \geq p$, so the ladder operators carry a string $X_{p+1}\cdots X_{M-1}$ of *updates* instead of a string of parities. Nothing is gained: the strings have simply moved from one end of the register to the other, as the second row of each block of Table 4.3 shows. The *Bravyi–Kitaev mapping* stores partial parities arranged in a binary tree, qubit j holding the parity of a set of modes that is a single mode for even j, a pair for $j \equiv 1 \pmod 4$, a block of four for $j \equiv 3 \pmod 8$, and so on, so that both the parity that a_p has to read and the set of qubits it has to update involve $\mathcal{O}(\log M)$ qubits. For $M=8$ every $a_p^\dagger$ has weight four in the Bravyi–Kitaev mapping, against weights up to eight in the other two, and the advantage grows with M. The class `BravyiKitaev` in `jordanwigner.py` implements the encoding as a permutation of the Fock basis, $b = \beta n \bmod 2$ with the recursively defined matrix β, followed by the decomposition of Eq. (4.2); it is exact but limited to eight modes. The lower half of Table 4.3 carries a warning, however. For the structured Hamiltonians of this book, whose operators act on adjacent modes when the modes are ordered well, the Jordan–Wigner strings are already short, and the Bravyi–Kitaev mapping makes them *longer*, weight six instead of four for the pairing model, because the tree structure ignores the locality of the problem. The asymptotic advantage of the cleverer mappings is real for generic Hamiltonians with $\mathcal{O}(M^4)$ long-range terms, such as those of quantum chemistry; for the small, structured models here the Jordan–Wigner transformation with a good ordering of the modes is the right choice, and that is what the next section does model by model.

4.3.5 A Hamiltonian on two qubits

The two-mode Hamiltonian of Eq. (3.22) is the smallest complete example. With Eqs. (4.5) and (4.6), and $\hat{n}_0\hat{n}_1 = (I - Z_0)(I - Z_1)/4$,

$$\hat{H} = \left(\frac{\varepsilon_0 + \varepsilon_1}{2} + \frac{U}{4}\right) I - \frac{2\varepsilon_0 + U}{4} Z_0 - \frac{2\varepsilon_1 + U}{4} Z_1 + \frac{U}{4} Z_0 Z_1 + \frac{t}{2} \left(X_0 X_1 + Y_0 Y_1\right), \tag{4.9}$$

six Pauli strings, none of weight greater than two, which is the $M = 2$ entry of Table 4.2. Its matrix in the basis $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ is Eq. (3.23), block diagonal because the strings Z_0, Z_1, $Z_0 Z_1$ are diagonal and $X_0 X_1 + Y_0 Y_1$ flips the two qubits in opposite directions, conserving the number of ones. For $\varepsilon_0 = 1$, $\varepsilon_1 = 2$, $t = 0.3$ and $U = 0.5$ the coefficients are 1.625, -0.625, -1.125, 0.125 and 0.15, and the one-particle block has the eigenvalues 0.9169 and 2.0831 of Section 3.2.6. Everything a quantum computer will be asked to do with a Hamiltonian in this book can be practised on Eq. (4.9): a Trotter step is five rotations, the energy is three measurement settings, and the exact ground state in the one-particle sector is reached from $|10\rangle$ by a single two-qubit rotation.

4.4 Symmetries and encodings

The generic mapping of Section 4.3 uses one qubit per single-particle mode and lets the Hamiltonian act on the whole Fock space, all particle numbers included. That is wasteful for the same reason that full diagonalization is wasteful in Chapter 3: a symmetry of the Hamiltonian restricts the physics to a sector, and if the sector can be parametrized by fewer bits, fewer qubits are needed and the operators on them are shorter. Every model of Section 3.3 is a case in point, and we take them in turn. The recipe is always the same. Identify the conserved quantity; choose a basis of the relevant sector whose states are labelled by bit strings; write the Hamiltonian in that basis in terms of operators that act on single bits; and map those operators onto Pauli matrices. When the operators are fermionic this is the Jordan–Wigner transformation restricted to a sector; when they are hard-core bosonic or spin operators, as in Section 3.2.7, the parity strings disappear.

One convention has to be fixed once and for all, because the models of Chapter 3 were written with two different ones. We take $|0\rangle$ to be the "lower" or "empty" state of every qubit and $|1\rangle$ the "upper" or "occupied" one, as in Eq. (4.4). Then an operator that raises a qubit from $|0\rangle$ to $|1\rangle$ is $\sigma^+ = (X - iY)/2$, one that counts the occupation is $(I - Z)/2$, and a spin-1/2 whose "up" state is $|1\rangle$ has $\hat{S}^z = -Z/2$. With the opposite convention, $|0\rangle$ up, every Y and every Z in this section changes sign, which is harmless as long as it is done consistently and fatal otherwise. The program `mapped_models.py` uses the convention above throughout and verifies every Pauli form below against the exact spectra of Chapter 3.

4.4.1 Symmetries in Pauli form

A conserved quantity is an operator that commutes with the Hamiltonian, and on the register both are Pauli sums, so the check is a commutator of Pauli sums, which `PauliSum` evaluates symbolically. The particle number is $\hat{N} = \sum_p (I - Z_p)/2$, the total spin projection of the Hubbard model is $\hat{S}^z = \frac{1}{2}\sum_i (\hat{n}_{i\uparrow} - \hat{n}_{i\downarrow})$, the number parity is the single string $Z_0 Z_1 \cdots Z_{M-1}$, and the seniority of the pairing model is $\sum_p (\hat{n}_{p+} - \hat{n}_{p-})^2$, the number of unpaired particles. The program `mapped_models.py` finds $[\hat{H}, \hat{N}] = [\hat{H}, \hat{S}^z] = [\hat{H}, Z_0 \cdots Z_7] = 0$ for the Hubbard ring,

$[\hat{H},\hat{N}] = [\hat{H},\text{seniority}] = 0$ for the pairing model, and, once the particle-hole term is added, a commutator with the seniority of 432 strings: the symmetry is gone, and with it the compact encoding of Section 4.4.3. The structure of the strings tells the same story. A string commutes with $Z_0 \cdots Z_{M-1}$ if and only if it contains an even number of X's and Y's, so a number-parity-conserving Hamiltonian has only such strings, which is visible in every Pauli form of this chapter; a string conserves $\hat{N}$ itself only in combination with its partner, XX with YY, which is why the hopping and pair terms of Eqs. (4.6) and (4.7) always come in pairs. On the quantum computer a symmetry is used in two ways: before the mapping, by choosing the basis of a sector, as in the encodings that follow; and after it, by removing qubits from the Pauli sum, as in Section 4.4.6. In either case the first step is the commutator.

4.4.2 The Lipkin model: four encodings

The Lipkin Hamiltonian of Eq. (3.24) can be put on qubits in at least four ways, and the comparison is instructive.

One qubit per spin-orbital. The 2Ω modes (p,σ) are mapped by Eq. (4.4); nothing about the model is used. For $N = \Omega = 4$ this needs eight qubits, and the Hamiltonian has 61 strings of weight up to four. The eigenvalues of the four-particle sector reproduce Table 3.1.

One qubit per doublet. At half filling, $N = \Omega$, each doublet $\{(p,-),(p,+)\}$ holds exactly one particle in every state that the Hamiltonian can reach from the reference, so one bit per doublet suffices: $|0\rangle_p$ for the particle in the lower level and $|1\rangle_p$ for the upper. The quasispin of a single doublet, $j_z^{(p)} = \frac{1}{2}\sum_\sigma \sigma\, a^\dagger_{p\sigma} a_{p\sigma}$ and $j_+^{(p)} = a^\dagger_{p+} a_{p-}$, is a spin-1/2 on that bit,

$$j_z^{(p)} \longrightarrow -\frac{Z_p}{2}, \qquad j_\pm^{(p)} \longrightarrow \frac{X_p \mp iY_p}{2}, \tag{4.10}$$

with no parity strings, because the operators $a^\dagger_{p+}a_{p-}$ of different doublets commute: each contains two fermionic operators and the two signs cancel. Inserting $\hat{J}_z = \sum_p j_z^{(p)}$ and $\hat{J}_\pm = \sum_p j_\pm^{(p)}$ into Eq. (3.26) gives, for $N = 4$, $\varepsilon = 2$, $V = -1/3$, $W = -1/4$,

$$\hat{H} = -\sum_p Z_p - 0.2917 \sum_{p<q} X_pX_q + 0.0417 \sum_{p<q} Y_pY_q, \tag{4.11}$$

sixteen strings of weight at most two on four qubits. The coefficients follow from $j_+^{(p)}j_+^{(q)} + j_-^{(p)}j_-^{(q)} = \frac{1}{2}(X_pX_q - Y_pY_q)$ and $j_+^{(p)}j_-^{(q)} + j_-^{(p)}j_+^{(q)} = \frac{1}{2}(X_pX_q + Y_pY_q)$: each pair of doublets receives $\frac{V}{2}(X_pX_q - Y_pY_q)$ from the $\hat{J}_\pm^2$ terms and $\frac{W}{2}(X_pX_q + Y_pY_q)$ from the $\hat{J}_+\hat{J}_- + \hat{J}_-\hat{J}_+$ terms, so that the XX coefficient is $(V+W)/2 = -0.2917$ and the YY coefficient $(W-V)/2 = 0.0417$, while the $p = q$ terms of $\hat{J}_+\hat{J}_- + \hat{J}_-\hat{J}_+$ give the constant N that the $-\hat{N}$ in Eq. (3.26) removes. The 16-dimensional register contains all quasispin sectors $J = 0, 1, 2$ at once; the $J = 2$ block appears as five of its eigenvalues, and the ground state is the lowest of all, so no projection is needed.

One qubit per basis state. The $2J+1$ states $|J,J_z\rangle$ of one quasispin block can be encoded *one-hot*: state m is the register with a single $|1\rangle$ at position m. A matrix element $H_{mm'}$ becomes $\sigma_m^+\sigma_{m'}^-$, which on the one-hot subspace is $\frac{1}{2}(X_mX_{m'} + Y_mY_{m'})$ for $m \neq m'$ and $(I - Z_m)/2$ on the diagonal, with no parity strings since the one-hot states are not fermionic. The 5×5 matrix of Eq. (3.27) becomes twelve strings of weight at most two on five qubits, but one qubit more than the doublet encoding, one circuit per J block, and a register in which almost all basis states are unphysical.

Binary encoding. The same $2J+1$ states can be stored in the $2^k \geq 2J+1$ basis states of $k = \lceil \log_2(2J+1) \rceil$ qubits, the unused states being assigned an energy above the spectrum.

Table 4.4 The Lipkin model with $N=\Omega=4$, $\varepsilon=2$, $V=-1/3$, $W=-1/4$ in four qubit encodings. All four give the ground-state energy -4.21288 of Table 3.1. Produced by `mapped_models.py`.

encoding	qubits	Pauli strings	largest weight
one per spin-orbital (Jordan–Wigner)	8	61	4
one per doublet (quasispin)	4	16	2
one-hot $\lvert J,J_z\rangle$, $J=2$	5	12	2
binary, $J=2$ block	3	16	3

The Pauli form then comes from Eq. (4.2) and has no structure to speak of: for $J=2$, sixteen strings of weight up to three on three qubits. This is the analogue of the compact pairing encoding below, and it is the most economical in qubits, but its operators are the least local and the circuits for them the most expensive.

Table 4.4 summarizes. The doublet encoding is the natural one: it uses the symmetry of the model without discarding any sector, its Hamiltonian is a sum of two-qubit terms, and its reference state, all particles in the lower level, is the register $|0000\rangle$. It is the encoding used in Chapter 7, where the symmetry-broken mean field of Section 3.5.4 reappears as the product state in which every qubit is rotated by the same angle.

4.4.3 The pairing model: one qubit per orbital or per level

One qubit per spin-orbital. Order the spin-orbitals so that the two partners of a level are adjacent, $(1+,1-,2+,2-,\dots)$, which keeps the parity strings inside a level empty, and apply Eq. (4.4) to Eq. (3.28). The one-body term and the diagonal part of the interaction, $-\frac{g}{2}\hat P_p^+\hat P_p^- = -\frac{g}{2}\hat n_{p+}\hat n_{p-}$, are diagonal,

$$\hat H_0+\hat V_{\mathrm{diag}}=\sum_p\left\{\frac{\xi\,(p-1)}{2}\left[(I-Z_{p+})+(I-Z_{p-})\right]-\frac{g}{8}\,(I-Z_{p+})\,(I-Z_{p-})\right\}, \tag{4.12}$$

and the pair transfers between different levels are the eight-string operators of Eq. (4.8). For $L=2$, $\xi=1$ and $g=1$ the result is fifteen strings on four qubits,

$$\begin{aligned}\hat H=\;&\tfrac{3}{4}\,IIII-\tfrac{3}{8}\,(IIZI+IIIZ)+\tfrac{1}{8}\,(ZIII+IZII)-\tfrac{1}{8}\,(ZZII+IIZZ)\\&-\tfrac{1}{16}\big(XXXX+XYXY+XYYX+YXXY+YXYX+YYYY-XXYY-YYXX\big),\end{aligned} \tag{4.13}$$

seven diagonal strings and eight of weight four, produced by `mapped_models.py` from the tabulated matrix elements, so that it is an exact decomposition rather than a hand calculation to be trusted. Figure 4.1 shows the distribution of weights for $L=1,2,3$. Only the weights 0, 1, 2 and 4 occur: one identity, $2L$ single Z's from the one-body term and the diagonal interaction, L strings $Z_{p+}Z_{p-}$ from the same diagonal interaction, and eight weight-four strings for each of the $L(L-1)/2$ pairs of levels, in total $4L(L-1)+3L+1$ strings, that is 4, 15, 34. The gap at weight three is structural: the pair transfer touches four spin-orbitals, the one-body part one or two, and nothing touches three. And the largest weight stays at four however large L becomes, because the operators come in pairs acting on adjacent modes $2p$ and $2p+1$, whose parity strings differ by a single Z and cancel.

One qubit per level. Section 3.3.2 showed that the interaction conserves the seniority and that the ground state of an even system has seniority zero: every level is either empty or holds a pair. In that space a level is one bit, and the pair operators of Eq. (3.29) act on it as a spin-1/2,

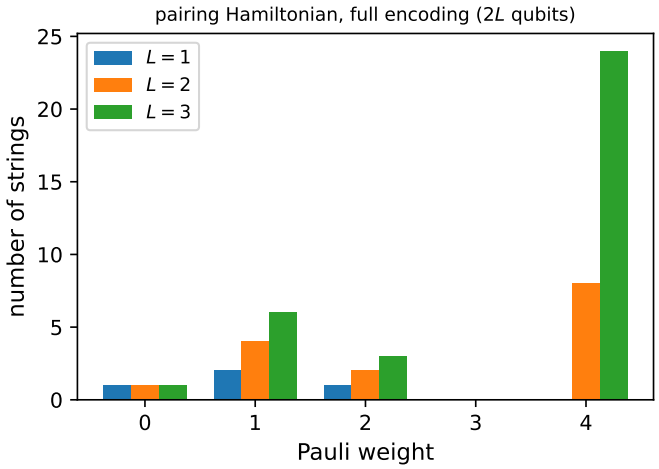


Fig. 4.1 Pauli weights of the pairing Hamiltonian ($\xi = 1$, $g = 1$) in the one-qubit-per-orbital encoding for $L = 1,2,3$ levels. Produced by `jw_basics.py`.

$$\hat{S}_p^+ = \hat{P}_p^+, \qquad \hat{S}_p^- = \hat{P}_p^-, \qquad \hat{S}_p^z = \frac{1}{2}\left(\hat{N}_p - 1\right), \qquad [\hat{S}_p^z, \hat{S}_p^\pm] = \pm \hat{S}_p^\pm, \quad [\hat{S}_p^+, \hat{S}_p^-] = 2\hat{S}_p^z, \tag{4.14}$$

with operators of different levels commuting. Each level carries its own $su(2)$, the same algebra that organized the Lipkin model, and the map to Pauli matrices is Eq. (4.3) without any parity string,

$$\hat{S}_p^+ = \frac{X_p - iY_p}{2}, \qquad \hat{S}_p^- = \frac{X_p + iY_p}{2}, \qquad \hat{S}_p^z = -\frac{Z_p}{2}, \qquad \hat{N}_p = I - Z_p, \tag{4.15}$$

with $|0\rangle_p$ an empty level and $|1\rangle_p$ a full one. The $p = q$ part of the interaction is $-\frac{g}{2}\sum_p \hat{S}_p^+ \hat{S}_p^- = -\frac{g}{4}\sum_p (I - Z_p)$ and merges with the one-body term, $\xi(p-1)\hat{N}_p = \xi(p-1)(I - Z_p)$; the $p \neq q$ part uses $\hat{S}_p^+ \hat{S}_q^- + \hat{S}_q^+ \hat{S}_p^- = \frac{1}{2}(X_pX_q + Y_pY_q)$, the imaginary parts cancelling between the two orderings. The result is

$$\hat{H} = \sum_{p=1}^{L} \left[\xi(p-1) - \frac{g}{4}\right](I - Z_p) - \frac{g}{4}\sum_{p<q}(X_pX_q + Y_pY_q), \tag{4.16}$$

$L^2 + 1$ strings of weight at most two on L qubits. For two levels with $\xi = 1$ and $g = 1$,

$$\hat{H}^{(L=2)} = \tfrac{1}{2} II + \tfrac{1}{4} ZI - \tfrac{3}{4} IZ - \tfrac{1}{4}(XX + YY), \tag{4.17}$$

five strings on two qubits against the fifteen strings on four qubits of Eq. (4.13). Equation (4.16) also identifies what kind of spin model the pairing Hamiltonian is: an XY model with couplings of infinite range in an inhomogeneous field, the Heisenberg model of Eq. (3.34) with the ZZ term removed and every site coupled to every other. The compact form is a restriction, not an approximation. Its spectrum in the sector with n ones is exactly the seniority-zero spectrum of the full Hamiltonian with $2n$ particles, and `mapped_models.py` checks this for $L = 2$ and 3 and every n, eigenvalue by eigenvalue, and checks as well that each of these eigenvalues appears in the spectrum of the full encoding.

What the particle-hole term costs. The compact encoding exists because the seniority is conserved, and the particle-hole term of Eq. (3.30) destroys exactly that. Once $f \neq 0$ the Hamiltonian connects the seniority-zero states to states with a broken pair, which have no representative on an L-qubit register, so one has to go back to $2L$ qubits, and the price is shown in Table 4.5 and Fig. 4.2. For $L = 3$ the Hamiltonian goes from 34 strings of weight at

Table 4.5 The cost of the pairing model on a quantum computer. The compact encoding halves the number of qubits and caps the weight at two but requires seniority to be conserved; the particle-hole term of Eq. (3.30) breaks that conservation and leaves only the full encoding, at a sharply higher count. Produced by `mapped_models.py`.

Hamiltonian	encoding	qubits	Pauli strings	largest weight
pairing, $L=2$	one per level	2	5	2
pairing, $L=2$	one per orbital	4	15	4
pairing + p-h, $L=2$	one per orbital	4	27	4
pairing, $L=3$	one per level	3	10	2
pairing, $L=3$	one per orbital	6	34	4
pairing + p-h, $L=3$	one per orbital	6	118	6

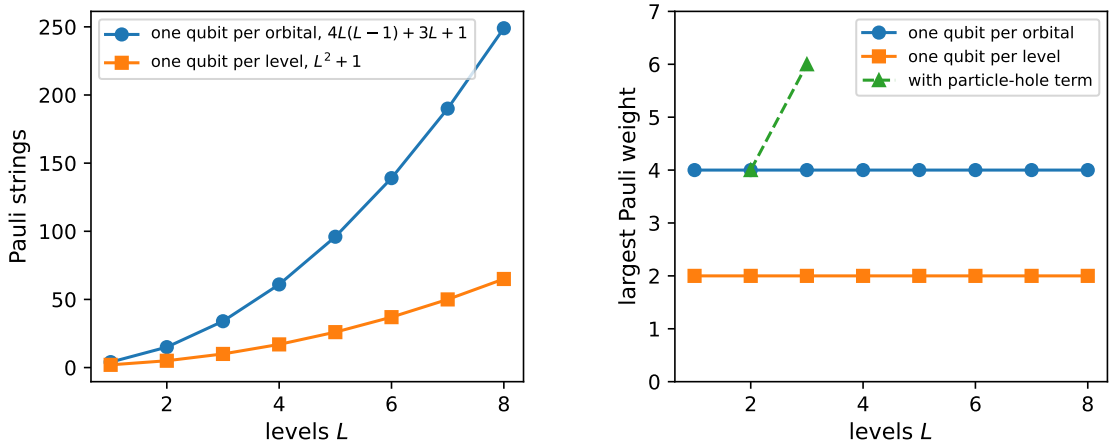


Fig. 4.2 The two encodings of the pairing model against the number of levels: the number of Pauli strings, $4L(L-1)+3L+1$ for one qubit per orbital and L^2+1 for one qubit per level (left), and the largest weight, four and two, with the values for the Hamiltonian including the particle-hole term (right). Produced by `mapped_models.py`.

most four to 118 strings of weight up to six on the same six qubits, while the encoding that would have needed three qubits and ten strings is no longer legitimate at all. Restricting to it would not be an approximation but a projection onto a subspace that the Hamiltonian does not leave invariant, with energies wrong even qualitatively. Every one of these numbers is a direct cost on hardware: strings are measurement settings in a variational calculation and weights are entangling gates in a Trotter step. The moral is that the classical structure of a problem, its symmetries and conserved quantities, is worth more on a small quantum device than any amount of additional circuit depth, and that identifying it is the first thing to do.

4.4.4 The Hubbard model: the order of the modes

The Hubbard model of Eq. (3.31) has no symmetry that reduces the register below one qubit per site and spin, $2N_s$ qubits, but it illustrates the one remaining freedom of the Jordan–Wigner transformation, the order in which the modes are numbered. The interaction $U\hat{n}_{i\uparrow}\hat{n}_{i\downarrow}$ is diagonal and has no parity strings whatever the order. The hopping terms are the two strings of Eq. (4.6), whose weight is the distance between the two modes in the chosen order plus one. Two natural orders for the four-site ring are the *site-major* order $(0\uparrow, 0\downarrow, 1\uparrow, 1\downarrow, \ldots)$

Table 4.6 The half-filled four-site Hubbard ring on eight qubits in two orderings of the modes: the number of Pauli strings, the largest weight, and the weights of the sixteen hopping strings. Produced by `mapped_models.py`.

ordering	Pauli strings	largest weight	hopping strings by weight
site-major $(i\uparrow, i\downarrow, \ldots)$	29	7	12 of weight 3, 4 of weight 7
spin-major (all $\uparrow$, then all $\downarrow$)	29	4	12 of weight 2, 4 of weight 4

and the *spin-major* order $(0\uparrow, 1\uparrow, 2\uparrow, 3\uparrow, 0\downarrow, \ldots)$. In the first, a hop between neighbouring sites jumps over the partner spin-orbital and has weight three, and the bond that closes the ring, between sites 3 and 0, has weight seven. In the second, the same hop has weight two and the closing bond weight four. Table 4.6 gives the counts: the number of strings, 29, is the same, but the longest string is seven in one case and four in the other, and since the cost of a Trotter step is dominated by the longest strings, the spin-major order is the better one. For a two-dimensional lattice no order makes all bonds short, and the weight of the worst hop grows with the linear size of the lattice; this is the case in which the Bravyi–Kitaev mapping, or one of the local encodings that add auxiliary qubits to cancel the parity strings, pays off. The ground-state energy of the eight-qubit Hamiltonian in the four-particle sector is -2.10274848 at $U = 4t$, the value of Table 3.3.

4.4.5 The Heisenberg model: nothing to do, and a warning

The Heisenberg model is the easiest of all, because Eq. (3.34) is already a sum of Pauli strings of weight two, $\frac{J}{4}(X_iX_j + Y_iY_j + Z_iZ_j)$ per bond, twelve strings for the four-site ring: there is no transformation to perform and no parity string to pay for. It is instructive nevertheless to run the Jordan–Wigner transformation backwards. Writing $\hat{S}_i^+ \to a_i^\dagger$ and $\hat{S}_i^z \to \hat{n}_i - \frac{1}{2}$ turns the spin chain into spinless fermions with nearest-neighbour hopping and interaction; for an open chain the parity strings between neighbours are empty and the fermionic and spin Hamiltonians have identical Pauli forms. For the ring the bond $(N_s - 1, 0)$ is not a neighbour in the ordering, and its fermionic hopping term carries the parity string $Z_1 \cdots Z_{N_s-2}$ of every site in between: `mapped_models.py` finds that the fermionic ring with periodic boundary conditions has the ground-state energy $-1.618J$ in the two-particle sector instead of the $-2J$ of the spin ring. The parity string twists the boundary condition, periodic for the spins into antiperiodic for the fermions when the particle number is even, and the spectra differ. This is not a paradox but a reminder: the Jordan–Wigner transformation is exact, and it is the fermionic ring with the naive boundary term that is the wrong model. For the spin problem one simply uses Eq. (3.34).

4.4.6 Tapering a discrete symmetry

The encodings above use continuous symmetries, particle number and seniority, by restricting the basis before the mapping. A discrete symmetry can be used after the mapping, to remove a qubit from a Pauli sum that already exists. Suppose a Pauli string Q commutes with every string of $\hat{H}$; the parity of the particle number, $Q = Z_0 Z_1 \cdots Z_{M-1}$, is the standard example, since every term of a number-conserving Hamiltonian contains an even number of X's and Y's. Then $\hat{H}$ is block diagonal in the eigenvalues ± 1 of Q, and a Clifford circuit U that maps Q onto a single-qubit operator, $UQU^\dagger = Z_j$, maps $\hat{H}$ onto a Pauli sum in which qubit j appears

Table 4.7 A bosonic mode truncated to d levels in the binary encoding on $\log_2 d$ qubits: the number of Pauli strings of the ladder operator b, the number operator and the position operator. Produced by `jw_basics.py`.

d	qubits	strings in b	strings in $b^\dagger b$	strings in $(b+b^\dagger)/\sqrt{2}$
2	1	2	2	1
4	2	8	3	4
8	3	24	4	12
16	4	64	5	32

only through Z_j. In the sector of interest Z_j is a number, ± 1, so it can be replaced by that number and qubit j dropped. For the two-level pairing Hamiltonian of Eq. (4.17) the symmetry is $Q = Z_0 Z_1$ and the circuit is a single CNOT, which maps $Z_1 \to Z_0 Z_1$ and hence $Z_0 Z_1 \to Z_1$, $X_0 X_1 \to X_0$ and $Y_0 Y_1 \to -X_0 Z_1$. In the one-pair sector $Q = -1$, and `mapped_models.py` finds

$$\hat{H}^{(L=2)} \longrightarrow \frac{1}{2} I + Z - \frac{1}{2} X, \qquad E_\pm = \frac{1}{2} \pm \frac{\sqrt{5}}{2} = -0.618034,\ 1.618034, \tag{4.18}$$

a single-qubit Hamiltonian whose spectrum is the seniority-zero spectrum with one pair. The two-level pairing model with one pair is a one-qubit problem, and its ground state is a single rotation $R_y(\theta)|0\rangle$ with $\tan\theta = -1/2$. The procedure, known as qubit tapering, removes one qubit per independent Z_2 symmetry, typically two or three for a molecular Hamiltonian, and is routinely applied before a variational calculation.

4.4.7 Spin models and bosonic modes

Two kinds of degrees of freedom remain that are neither fermions nor hard-core bosons. *Spin models* other than the Heisenberg model are, like it, already Pauli sums: the transverse-field Ising model $\hat{H} = -J\sum_{\langle ij\rangle} Z_i Z_j - h\sum_i X_i$, the XXZ chain with different couplings for the $XX+YY$ and the ZZ terms, and the XY model that the pairing Hamiltonian turned out to be. For all of them the only choice is the computational basis, that is which spin direction the Z eigenstates represent, and the only cost is the number of bonds. This is why spin chains are the first systems on which every new quantum device is tested. *Bosonic modes*, such as the phonons of a lattice, the photons of a cavity or the gauge fields of Chapter 10, have an infinite-dimensional Hilbert space that has to be truncated to d levels and then encoded. The unary or one-hot encoding uses d qubits with a single $|1\rangle$ marking the level, as for the Lipkin states in Section 4.4.2, and gives operators of weight two; the binary encoding stores the level number in $\log_2 d$ qubits and gives operators whose Pauli form follows from Eq. (4.2). Table 4.7 shows what the truncated ladder operator b, the number operator $\hat{n} = b^\dagger b$ and the position $\hat{x} = (b+b^\dagger)/\sqrt{2}$ cost in the binary encoding: the number operator is diagonal and needs $\log_2 d + 1$ strings, while b needs $d \log_2 d$ strings of weight up to $\log_2 d$. For four levels on two qubits $\hat{x} = 0.9659\,IX + 0.5\,XX + 0.5\,YY - 0.2588\,ZX$. The choice between the encodings is the choice between qubits and weight that runs through this whole chapter, and Chapter 10 returns to it for the gauge fields of a lattice theory.

Table 4.8 The four models at the sizes of Chapter 3 on a qubit register, in the encoding that uses the symmetry of each model.

model	encoding	qubits	Pauli strings	largest weight
Lipkin, $N = \Omega = 4$	one per doublet	4	16	2
pairing, $L = 4$	one per level	4	17	2
pairing, $L = 4$	one per orbital	8	61	4
Hubbard ring, $N_s = 4$	one per orbital, spin-major	8	29	4
Heisenberg ring, $N_s = 4$	one per spin	4	12	2

4.4.8 Counting qubits for real systems

The models of this chapter fit on four to eight qubits, and it is worth asking what the same accounting gives for the systems one would like to study. In the one-qubit-per-orbital encoding the register is the single-particle basis, whatever the number of particles. A small molecule in a minimal basis, such as water with seven spatial orbitals, needs 14 qubits, minus two for the tapering of the number parities of each spin; a transition-metal complex in a chemically meaningful basis needs hundreds, of which an active space of the strongly correlated orbitals, typically 20 to 50 spin-orbitals, is what one would put on the device, the remaining orbitals being treated at the mean-field level exactly as in Section 3.5. A nucleus in the sd shell has 12 orbitals per nucleon species, 24 qubits, and in the pf shell 40; the interactions are the general two-body form of Table 4.2, with $\mathcal{O}(M^4)$ strings, tens of thousands for the pf shell. Lattice models scale with the number of sites: a 4×4 Hubbard cluster is 32 qubits, a 10×10 Heisenberg lattice 100, and for them the count of strings is linear in the size while the weight is set by the ordering, as in Section 4.4.4. Compared with the classical exponential wall of Table 3.7 the qubit count is modest, M instead of $\binom{M}{N}$, and this linear scaling is the whole promise of the subject. What is not modest is the number of gates: a single Trotter step of a pf-shell Hamiltonian is millions of Pauli rotations, and a variational energy is tens of thousands of measurement settings. The remaining chapters are, in one way or another, about bringing those numbers down.

Table 4.8 collects the encodings that the rest of the book uses. Three of the four models fit on four qubits with two-qubit strings only; the pairing model with $L = 4$ and two pairs, the example of every method in Chapter 3, is a four-qubit calculation with seventeen strings, and the two-level version is a genuine two-qubit calculation that can be run on any hardware. With the particle-hole term switched on it is an eight-qubit calculation, which is a rather different proposition.

4.5 From Pauli strings to circuits

4.5.1 The exponential of a Pauli string

Every algorithm of the following chapters needs the unitary $e^{-i\theta P}$ for a Pauli string P and a real angle θ: the Trotter steps of Chapter 5 are products of such factors with $\theta = h_\alpha \Delta t$, and the unitary coupled-cluster circuits of Chapter 7 are products of such factors with variational angles. For a single qubit and $P = Z$ the operator is the rotation $R_z(2\theta)$ of Chapter 2. For a string of weight w the construction rests on two facts. First, the single-qubit Clifford gates rotate the Pauli matrices into each other, $HXH = Z$ and $(SH)Y(SH)^\dagger = Z$, so that any string can be turned into a string of Z's by a layer of H and $S^\dagger H$ gates, to be undone afterwards.

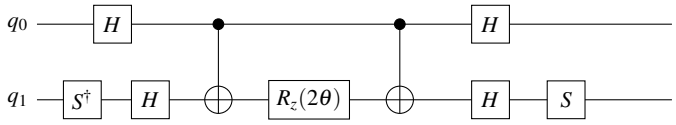


Fig. 4.3 The circuit of Eq. (4.19) for $e^{-i\theta X_0 Y_1}$: a basis change on each qubit of the support, a CNOT that collects the parity on the last qubit, the rotation, and the inverse of the first two steps. For a string of weight w the CNOT chain has $w-1$ gates on each side. Verified in `pauli_circuits.py`.

Second, a chain of CNOT gates along the support of the string collects the parity of the w qubits onto the last one: after $\mathrm{CNOT}_{q_1 q_2} \cdots \mathrm{CNOT}_{q_{w-1} q_w}$ the last qubit holds $z_{q_1} \oplus \cdots \oplus z_{q_w}$, which is the eigenvalue of $Z_{q_1} \cdots Z_{q_w}$ up to sign, and $R_z(2\theta)$ on that qubit multiplies each basis state by $e^{\mp i\theta}$ according to it. Undoing the chain and the basis change gives

$$e^{-i\theta P} = B^\dagger C^\dagger R_z^{(q_w)}(2\theta)\, C B, \tag{4.19}$$

with B the basis change and C the CNOT chain, a circuit with $2(w-1)$ CNOT gates, one rotation and at most $4w$ single-qubit gates, of depth $2w+1$ or so. Figure 4.3 shows it for $P = X_0 Y_1$. The function `pauli_rotation` in `pauli_circuits.py` appends this sequence to a `Circuit` of Chapter 2 for any string,

```
def pauli_rotation(circ, label, theta):
    """Append gates implementing exp(−i theta P) for the Pauli string `label`."""
    support = [q for q, c in enumerate(label) if c != "I"]
    for q in support:                                  # rotate X, Y into Z
        if label[q] == "X": circ.h(q)
        elif label[q] == "Y": circ.add(Gate("S^", G.S.conj().T, q)); circ.h(q)
    for a, b in zip(support[:−1], support[1:]):        # parity onto the last qubit
        circ.cnot(a, b)
    circ.rz(2 * theta, support[−1])
    for a, b in reversed(list(zip(support[:−1], support[1:]))):
        circ.cnot(a, b)
    for q in reversed(support):                        # undo the basis change
        if label[q] == "X": circ.h(q)
        elif label[q] == "Y": circ.h(q); circ.add(Gate("S", G.S, q))
    return circ
```

and checks that the unitary of the circuit equals the matrix exponential of $-i\theta P$ for ZZ, XX, YY, $XZZY$ and $XYXY$; the gate counts are 3, 7, 11, 13 and 19 gates with 2, 2, 2, 6 and 6 CNOTs. The weight of a string is thus, up to a factor two, the number of entangling gates its exponential costs, which is why the tables of Section 4.4 list it.

4.5.2 Trotter steps as circuits

With Eq. (4.19) the Trotter formula of Eq. (3.47) becomes a circuit: one Pauli rotation with $\theta_\alpha = h_\alpha \Delta t$ per string of the Hamiltonian, in some fixed order, repeated n times, with the identity string contributing only a global phase. The function `trotter_step` in `pauli_circuits.py` builds it for any `PauliSum`, in the first-order form and in the symmetric form of Eq. (3.46). Table 4.9 gives the cost of one first-order step for the two-level pairing model in the compact encoding and for the four-site Heisenberg ring: 4 and 24 CNOT gates respectively, and depths 14 and 55. The Heisenberg count can be reduced by a factor of three by exponentiating each bond's $XX+YY+ZZ$ as a single two-qubit gate rather than three separate rotations, which is one of the optimizations Chapter 5 discusses. The splitting error behaves as in Section 3.5.3: for the pairing Hamiltonian at $t = 1$ the distance between the

Table 4.9 One first-order Trotter step as a circuit of Pauli rotations: the number of strings, gates, CNOT gates and the circuit depth. Produced by `pauli_circuits.py`.

Hamiltonian	strings	gates	CNOTs	depth
pairing, $L=2$, one qubit per level, Eq. (4.17)	5	20	4	14
Heisenberg ring, $N_s=4$, Eq. (3.34)	12	84	24	55

n-step circuit and the exact exponential is 0.243, 0.106, 0.051, 0.025 and 0.013 for $n=1,2,4,8,16$ in first order and 0.056, 0.012, 0.0030, 0.00075 and 0.00019 in second order, the expected $1/n$ and $1/n^2$.

4.5.3 A first time evolution

The Trotter circuit can be run as it stands. Prepare the two-level pairing model in the reference state $|10\rangle$, the pair in the lower level, apply the step circuit of Table 4.9 repeatedly, and record after each step the probability $P(t)=|\langle 01|\Psi(t)\rangle|^2$ that the pair has been transferred to the upper level. The exact answer is the Rabi oscillation of a two-level system, $P(t)=\frac{H_{12}^2}{\omega^2}\sin^2\omega t$ with $\omega=\sqrt{(H_{22}-H_{11})^2/4+H_{12}^2}=\sqrt{5}/2$ for $\xi=g=1$, of amplitude 0.2 and period $2\pi/\sqrt{5}=2.81$. Figure 4.4 shows the circuit results: with $\Delta t=0.5$ the first-order circuit overshoots the maxima by 0.03 and the second-order one undershoots them by 0.012, while $\Delta t=0.1$ in first order is accurate to 10^{-3} at the cost of 320 CNOT gates for the full time span. The inset shows a second example that is not a two-level system: the four-site Heisenberg ring started in the Néel state $|0101\rangle$, which is not an eigenstate, and the staggered magnetization $m_s(t)=\frac{1}{4}\sum_i(-1)^i\langle Z_i\rangle/2$ that it develops as the spins exchange; the second-order circuit with $\Delta t=0.25$, 46 CNOT gates per step, follows the exact curve to 4×10^{-3}. This is Hamiltonian simulation in its simplest form, and Chapter 5 is about doing it better, with higher-order formulas, with error bounds that use the commutators of the strings, and with methods that are not products of exponentials at all.

4.5.4 Preparing the reference state

The reference determinant of Chapter 3, the N lowest modes occupied, is a computational basis state in every encoding of Section 4.4: $|1\cdots10\cdots0\rangle$ with N ones in the orbital encodings, $|1\cdots10\cdots0\rangle$ with n ones in the compact pairing encoding, and $|0\cdots0\rangle$ in the doublet encoding of the Lipkin model, where $|0\rangle$ is the lower level. Preparing it costs one X gate per occupied qubit and nothing else, which is the first reason the occupation-number identification of the states is so convenient. The second is that the symmetries are visible: the number of ones is the particle number, or the number of pairs, and a circuit that conserves it, as every $e^{-i\theta P}$ with P built from $X_pX_q+Y_pY_q$ does, stays in the right sector automatically. Chapter 7 will exploit this to build ansätze that cannot leave the physical subspace. A Hartree–Fock determinant in a rotated basis is also a product state, by Thouless' theorem, but no longer a basis state; preparing it requires the rotation $e^{\hat{T}_1}$ of Eq. (3.42) as a circuit, which is a product of the hopping-type rotations of Eq. (4.6) and costs $\mathcal{O}(N(M-N))$ of them. For the models of this book the reference basis is already the Hartree–Fock basis, in the sense of Section 3.5.5, and the X gates suffice.

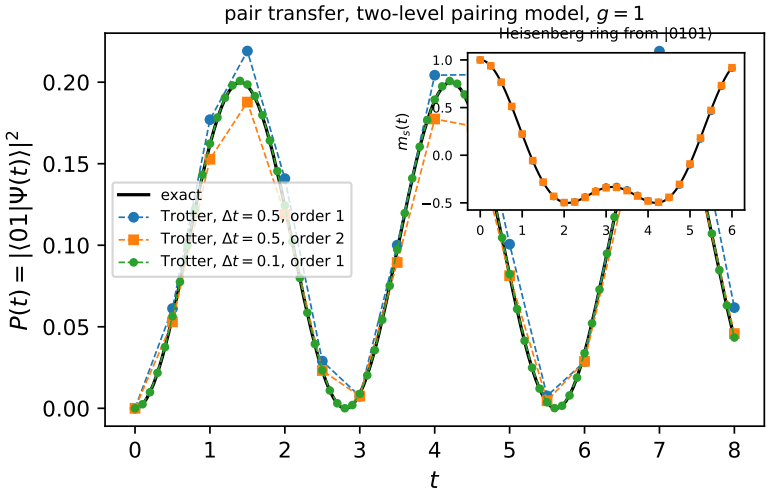


Fig. 4.4 Pair-transfer probability in the two-level pairing model ($\xi = 1$, $g = 1$) from the reference state $|10\rangle$: the exact result and three Trotter circuits, first order with $\Delta t = 0.5$ and 0.1 and second order with $\Delta t = 0.5$. Inset: the staggered magnetization of the four-site Heisenberg ring started in $|0101\rangle$, exact and from first- and second-order circuits with $\Delta t = 0.25$. Produced by `pauli_circuits.py`.

4.5.5 Fermionic rotations and mean-field states

The single excitation $a_q^\dagger a_p$, the building block of Thouless' theorem and of the singles part of coupled cluster, has the anti-Hermitian combination

$$a_q^\dagger a_p - a_p^\dagger a_q = \frac{i}{2}\left(Y_p Z_{p+1}\cdots Z_{q-1} X_q - X_p Z_{p+1}\cdots Z_{q-1} Y_q\right), \qquad p < q, \tag{4.20}$$

from Eq. (4.4), two strings that commute, so that its exponential is again a product of two Pauli rotations, with the parity string in between: four CNOT gates for adjacent modes and $4(q-p)$ in general. Acting on a state with mode p occupied and q empty it produces $\cos\theta\,|1_p 0_q\rangle + \sin\theta\,|0_p 1_q\rangle$ with the sign of the parity in between, the fermionic analogue of Eq. (4.24). The function `single_excitation` in `pauli_circuits.py` builds it and checks it against the matrix exponential; applied to the two-mode Hamiltonian of Eq. (4.9) from $|10\rangle$, a single such rotation reaches the one-particle ground state 0.916905 at $\theta = -0.2702$, that is $\tan 2\theta = 2t/(\varepsilon_0 - \varepsilon_1)$.

A product of such rotations is the circuit form of Thouless' theorem, and it prepares any Slater determinant from a computational basis state. The Lipkin model shows this most simply. In the doublet encoding the mean-field state of Section 3.5.4, every doublet rotated by the same angle α, is the product state $\prod_p R_y(\alpha)_p |0000\rangle$, one single-qubit gate per qubit and no entanglement at all. Minimizing the expectation value of Eq. (4.11) in this state over α reproduces the Hartree–Fock energies of Table 3.11 to six digits, Table 4.10, with $\alpha = 0$ below $\chi = 1$ and $\cos\alpha = 1/\chi$ above, 0.8411 at $\chi = 1.5$ and $\pi/3$ at $\chi = 2$. A mean field on a quantum computer is a layer of single-qubit rotations; everything the mean field misses, the difference between the last two columns of the table, requires entangling gates, and the variational algorithms of Chapter 7 are about adding them economically.

Table 4.10 The Lipkin mean field as a product state on four qubits ($N = \Omega = 4$, $\varepsilon = 1$, $W = 0$): the optimal common rotation angle, the energy of the product state, the self-consistent Hartree–Fock energy of Table 3.11 and the exact energy. Produced by `pauli_circuits.py`.

χ	α	E(product state)	E_{HF}	E_{exact}
0.5	0	−2.000000	−2.000000	−2.081666
1.0	0	−2.000000	−2.000000	−2.309401
1.5	0.8411	−2.166667	−2.166667	−2.645751
2.0	1.0472	−2.500000	−2.500000	−3.055050
2.5	1.1593	−2.900000	−2.900000	−3.511885

4.6 Measuring a Pauli sum

The last ingredient is the readout. A quantum computer measures in the computational basis, which gives the eigenvalue ± 1 of every Z_q at once; the expectation value of a string of Z's in a state $|\Psi\rangle$ is therefore the average of $(-1)^{z_{q_1}\oplus\cdots\oplus z_{q_w}}$ over repeated preparations and measurements, the *shots*. A string containing X or Y is measured the same way after the basis change B of Eq. (4.19), H on the X qubits and $S^\dagger H$ on the Y qubits, applied to the state just before the measurement. The energy of a Pauli sum is then

$$\langle\Psi|\hat{H}|\Psi\rangle = \sum_\alpha h_\alpha \langle\Psi|P_\alpha|\Psi\rangle \approx \sum_\alpha h_\alpha \frac{1}{S}\sum_{s=1}^{S}(-1)^{\pi_\alpha(s)}, \tag{4.21}$$

with $\pi_\alpha(s)$ the parity of shot s on the support of P_α. Each term has a statistical error of order $1/\sqrt{S}$, so that an energy accurate to ε needs of order $(\sum_\alpha |h_\alpha|)^2/\varepsilon^2$ shots in total, and this is the dominant cost of every variational algorithm. Not every string needs its own circuit. Strings that commute qubit by qubit, meaning that wherever both are not the identity they carry the same Pauli matrix, can be read off from the same basis rotation and the same shots, so the strings are first partitioned into such groups, and the number of groups, not the number of strings, is the number of measurement settings. The two-level pairing Hamiltonian of Eq. (4.17) has three: $\{II, ZI, IZ\}$, which need no rotation, $\{XX\}$ and $\{YY\}$. Table 4.12 gives the number of settings for the models of Section 4.4, obtained by a greedy grouping in `pauli.py`: all of the symmetry-adapted encodings need only three, the XX, YY and ZZ types of strings, whereas the one-qubit-per-orbital pairing Hamiltonian with its eight-string pair transfers needs seventeen. Better groupings exist, using strings that commute without commuting qubit by qubit and entangling measurements to read them out, but the qubit-wise grouping is what is routinely done, and the table makes the point once more that the symmetry-adapted encodings are cheaper in every respect. The function `energy_from_shots` in `pauli_circuits.py` does all of this on the simulator of Chapter 2, whose `sample` method draws bit strings from the exact probabilities,

```
def measure_group(state, group, shots, rng):
    """<P> for every string P of a qubit-wise commuting group, from one circuit."""
    basis = ["I"] * state.n
    for k in group:
        for q, c in enumerate(k):
            if c != "I": basis[q] = c
    rotated = state.copy()
    for q, c in enumerate(basis):
        if c == "X": rotated.h(q)
        elif c == "Y": rotated.apply(G.S.conj().T, q); rotated.h(q)
    counts = rotated.sample(shots, rng)                # {bit string: multiplicity}
    estimates = {}
```

Table 4.11 Selected strings of the four-level pairing Hamiltonian in the compact encoding ($\xi = g = 1$, 17 strings) in its exact ground state: coefficient, expectation value and variance. With shots allocated in proportion to $|h_\alpha|\sigma_\alpha$ an accuracy of 10^{-3} needs 2.1×10^7 shots in total; the naive bound is 1.8×10^8. Produced by `pauli_circuits.py`.

string	h_α	$\langle P_\alpha \rangle$	$1 - \langle P_\alpha \rangle^2$
IIII	5.0000	1.0000	0
$IIIZ$	−2.7500	0.9764	0.0467
$ZIII$	0.2500	−0.7883	0.3786
XXII	−0.2500	0.5014	0.7486
$YYII$	−0.2500	0.5014	0.7486
XIIX	−0.2500	0.2055	0.9578

Table 4.12 Measurement settings: the number of groups of qubit-wise commuting strings for the Hamiltonians of Section 4.4, by greedy grouping. Produced by `mapped_models.py`.

Hamiltonian	qubits	strings	settings
Lipkin, one qubit per doublet	4	16	3
pairing $L = 4$, one qubit per level	4	17	3
pairing $L = 4$, one qubit per orbital	8	61	17
Hubbard ring, spin-major ordering	8	29	5
Heisenberg ring	4	12	3

```
    for k in group:
        support = [q for q, c in enumerate(k) if c != "I"]
        parity = lambda bits: sum(int(bits[q]) for q in support) % 2
        estimates[k] = sum(m * (1 - 2 * parity(b)) for b, m in counts.items()) / shots
    return estimates
```

Table 4.13 shows the estimate of the ground-state energy of the two-level pairing model, -0.618034, from S shots per setting: the root-mean-square error over twenty repetitions falls as $1/\sqrt{S}$, with $\sqrt{S} \times \mathrm{rms} \approx 0.55$, so that 10^5 shots per setting, 3×10^5 in all, are needed for three decimal places. On hardware the same numbers are affected in addition by readout errors and by the imperfections of the state-preparation circuit, which are the subject of Chapter 9.

The $1/\sqrt{S}$ law has a prefactor, and the prefactor can be managed. A string P measured in a state with $\langle P \rangle = m$ returns ± 1 with variance $1 - m^2$, so its contribution to the variance of the energy estimate is $h_\alpha^2 (1 - m_\alpha^2)/S_\alpha$ when S_α shots are spent on it. Minimizing the total variance at fixed total number of shots gives the allocation $S_\alpha \propto |h_\alpha|\sigma_\alpha$ with $\sigma_\alpha = \sqrt{1 - m_\alpha^2}$, and then the number of shots needed for an accuracy ε is $(\sum_\alpha |h_\alpha|\sigma_\alpha)^2/\varepsilon^2$ rather than the naive $(\sum_\alpha |h_\alpha|)^2/\varepsilon^2$. Table 4.11 shows the ingredients for the ground state of the four-level pairing model in the compact encoding: the identity and the large diagonal strings, which carry most of the energy, have almost no variance, because the state is close to an eigenstate of Z_p, while the small XX and YY strings fluctuate strongly. With the optimal allocation 2×10^7 shots give $\varepsilon = 10^{-3}$, against 2×10^8 from the naive bound. In practice the m_α are not known in advance, and a variational calculation estimates them from a first round of measurements before allocating the rest; the correlated readout of the strings within a group, which the estimate above ignores, changes the numbers but not the strategy.

Table 4.13 The ground-state energy of the two-level pairing model ($\xi = 1$, $g = 1$, exact value -0.61803399) estimated from S shots per measurement setting with three settings: one estimate, the root-mean-square error over twenty repetitions, and the latter scaled by $\sqrt{S}$. Produced by `pauli_circuits.py`.

S	one estimate	rms error	rms error $\times\sqrt{S}$
10^2	-0.63000	6.5×10^{-2}	0.65
10^3	-0.61500	1.6×10^{-2}	0.52
10^4	-0.60780	5.5×10^{-3}	0.55
10^5	-0.61936	1.7×10^{-3}	0.54

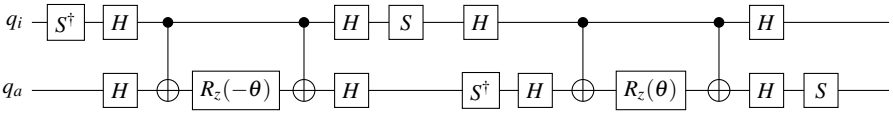


Fig. 4.5 The pair-excitation gate of Eq. (4.23), $\exp[\theta(\hat{S}_a^+\hat{S}_i^- - \text{h.c.})]$ in the one-qubit-per-level encoding: the rotation $e^{i(\theta/2)Y_iX_a}$ followed by $e^{-i(\theta/2)X_iY_a}$, each built as in Fig. 4.3. Four CNOT gates and one parameter. Verified in `pauli_circuits.py`.

4.7 A first variational circuit: unitary coupled cluster for the pairing model

Everything is now in place to redo the unitary coupled-cluster calculation of Section 3.6.3 as a quantum circuit. In the compact encoding of Section 4.4.3 a pair excitation from level i to level a is $\hat{S}_a^+\hat{S}_i^-$, and its anti-Hermitian combination is, by Eq. (4.15),

$$\hat{S}_a^+\hat{S}_i^- - \hat{S}_i^+\hat{S}_a^- = \frac{i}{2}\,(Y_iX_a - X_iY_a)\,, \tag{4.22}$$

two Pauli strings that commute with each other, so that the exponential factorizes exactly,

$$e^{\theta(\hat{S}_a^+\hat{S}_i^- - \text{h.c.})} = e^{-i(-\theta/2)Y_iX_a}\,e^{-i(\theta/2)X_iY_a}, \tag{4.23}$$

two Pauli rotations of Eq. (4.19), four CNOT gates in all, shown in Fig. 4.5. Acting on a register in which level i is full and level a empty, the gate produces

$$e^{\theta(\hat{S}_a^+\hat{S}_i^- - \text{h.c.})}|1_i0_a\rangle = \cos\theta\,|1_i0_a\rangle + \sin\theta\,|0_i1_a\rangle, \tag{4.24}$$

a rotation in the two-dimensional space of the pair, and it leaves the number of ones unchanged. For the two-level model with one pair this is already the complete ansatz: $|\Psi(\theta)\rangle = \cos\theta|10\rangle + \sin\theta|01\rangle$, the energy is $E(\theta) = H_{11}\cos^2\theta + H_{22}\sin^2\theta + H_{12}\sin2\theta$ with the 2×2 seniority-zero matrix of Section 3.3.2, that is $-\frac{1}{2}\cos^2\theta + \frac{3}{2}\sin^2\theta - \frac{1}{2}\sin2\theta$ for $\xi = g = 1$, and its minimum, at $\theta = 0.2318$, is the exact ground state $-0.618034 = \frac{1}{2} - \frac{\sqrt{5}}{2}$, as the right panel of Fig. 4.6 shows. The program `pauli_circuits.py` checks Eq. (4.24) against the circuit for every θ on a grid.

For $L = 4$ and two pairs the ansatz of Section 3.6.3 has the four generators $\hat{S}_a^+\hat{S}_i^-$ with $i \in \{1,2\}$ and $a \in \{3,4\}$, and its one-step Trotterized form, Eq. (3.58) with $n = 1$, is the circuit

$$|\Psi(\theta)\rangle = \prod_{k=1}^{4} e^{\theta_k(\hat{S}_{a_k}^+\hat{S}_{i_k}^- - \text{h.c.})}\,X_1X_2|0000\rangle, \tag{4.25}$$

four pair-excitation gates after the two X gates that prepare $|1100\rangle$: 74 gates, 16 CNOTs, depth 38, four parameters. The program evaluates the energy of the circuit state with the

Table 4.14 The unitary coupled-cluster circuit of Eq. (4.25) for the pairing model with $L = 4$ and two pairs ($\xi = 1$): the optimized energy of the four-parameter circuit, the UCCD energy of Table 3.15, and the exact energy. Produced by `pauli_circuits.py`.

g	E_{ref}	circuit UCCD, Chapter 3		FCI
0.25	1.75	1.73046012	1.73046012	1.73045985
0.50	1.50	1.41679562	1.41679562	1.41677428
1.00	1.00	0.63698698	0.63698698	0.63554847
1.50	0.50	−0.33675149	−0.33675149	−0.34819051
2.00	0.00	−1.45347633	−1.45347633	−1.48965216

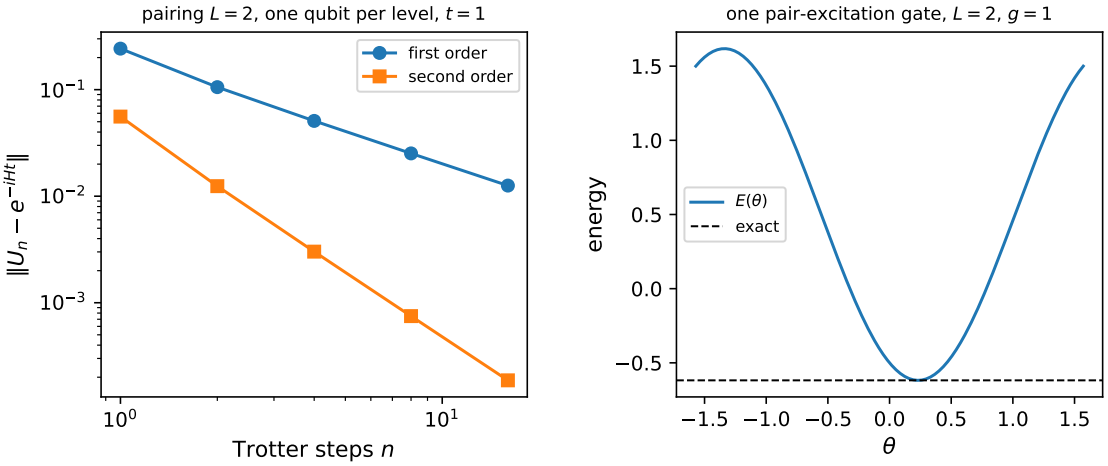


Fig. 4.6 Left: the splitting error of the n-step Trotter circuit for the two-level pairing Hamiltonian at $t = 1$, first and second order. Right: the energy of the state of Eq. (4.24) as a function of the angle of a single pair-excitation gate, for the two-level model with one pair; the minimum is the exact ground-state energy. Produced by `pauli_circuits.py`.

seventeen-string Hamiltonian of Eq. (4.16) and minimizes it with the BFGS routine of Chapter 3. Table 4.14 shows the result against the UCCD energies of Table 3.15: they agree to all eight digits, at every coupling. This is the statement, made in Section 3.6.4 and now verified on a circuit, that the one-step Trotter product with re-optimized amplitudes is exactly as good an ansatz as the full exponential, and it is the first result of this book obtained by a quantum algorithm, albeit on a simulator and with the energy evaluated exactly rather than from shots. Replacing the exact expectation value by Eq. (4.21) and the simulator by hardware is the variational quantum eigensolver of Chapter 7.

4.8 Summary

A many-body Hamiltonian reaches a quantum computer as a Pauli sum, and the Jordan–Wigner transformation of Eq. (4.4) is the exact recipe that produces it from the second-quantized form of Chapter 3: occupations become $(I - Z)/2$, hops and pair transfers become $XX \pm YY$ with a parity string of Z's in between, and a general two-body term becomes eight strings of weight four. The cost of the result is measured by three numbers, qubits, strings and weight, and Sections 4.3.4 and 4.4 showed how each of them can be reduced: the weight by the ordering of the modes or by a different mapping, the qubits and the strings by restrict-

ing to a symmetry sector before the mapping is applied, which turns the fermionic operators into spins or hard-core bosons and removes the parity strings altogether. The Lipkin model on one qubit per doublet, the pairing model on one qubit per level and the Heisenberg model as it stands are all sums of two-qubit strings on four qubits at the sizes of this book. From the Pauli sum three circuits follow mechanically: the rotation $e^{-i\theta P}$ of Eq. (4.19), whose entangling cost is the weight of the string; the Trotter step, one rotation per string; and the measurement of the energy, one setting per group of qubit-wise commuting strings and $1/\sqrt{S}$ statistics per setting. The pair-excitation gate of Eq. (4.23) put these together into the first variational circuit, which reproduces the classical unitary coupled-cluster energies. The next chapters take the same Pauli sums and circuits in three directions: Chapter 5 develops the Trotter circuit into an accurate time evolution, Chapter 6 uses that time evolution to read off eigenvalues, and Chapter 7 develops the variational circuit into the algorithms that run on today's hardware.

4.9 Exercises

1. **Pauli algebra.** (a) Show that two Pauli strings commute if and only if they anticommute on an even number of qubits. (b) Show that $\mathrm{Tr}(PQ) = 2^n \delta_{PQ}$ and derive Eq. (4.2). (c) Decompose the 4×4 matrix of Eq. (3.23) by hand using the trace formula and compare with Eq. (4.9). (d) Which pairs of strings of Eq. (4.9) commute qubit by qubit, and how many measurement settings does the Hamiltonian need?
2. **The Jordan–Wigner transformation by hand.** (a) Verify $\hat{n}_p = (I - Z_p)/2$ and $\{a_p, a_p^\dagger\} = I$ from Eq. (4.4). (b) Show that a_0 and a_1 anticommute and locate the factor where the parity string does the work. (c) Derive the hopping identity of Eq. (4.6) for $q = p+1$. (d) Derive the pair identity of Eq. (4.7) for the same pair and explain the sign difference. (e) Derive Eq. (4.8) for the modes $0,1,2,3$, or at least the coefficient of XXXX, and check with `jw_basics.py`.
3. **Parity and Bravyi–Kitaev.** In the parity mapping qubit j stores $\bigoplus_{q\le j} n_q$. (a) Write the state $|1011\rangle$ of four modes in the parity encoding. (b) Show that $\hat{n}_p = (I - Z_{p-1}Z_p)/2$ for $p > 0$. (c) Show that creating a particle in mode p requires flipping all qubits $j \ge p$, and write $a_p^\dagger$ as a Pauli sum. (d) For which operators is the parity mapping cheaper than Jordan–Wigner, and for which more expensive? The class `ParityMapping` in `jordanwigner.py` can be used to check the answers.
4. **Quasispin and the qubit encodings of the pairing model.** (a) Verify the algebra of Eq. (4.14) from the fermionic anticommutators. (b) Derive Eq. (4.16) from Eq. (3.28) and Eq. (4.15). (c) Show that its spectrum in the sector with n ones coincides with that of the seniority-zero matrix of Section 3.3.2. (d) Explain, without computing anything, why the compact encoding cannot be used once the particle-hole term of Eq. (3.30) is present. (e) Count the strings of the compact Hamiltonian and show that the count is $L^2 + 1$.
5. **The Lipkin model on qubits.** (a) Derive Eq. (4.11) from Eq. (3.26) and Eq. (4.10). (b) Show that the register state in which every qubit is rotated by the same angle, $\prod_p R_y(\alpha)_p |0000\rangle$, is the mean-field state of Section 3.5.4, and evaluate its energy as a function of α from the Pauli form. (c) Why does the doublet encoding contain the $J = 0$ and $J = 1$ sectors as well, and why is this harmless for a ground-state calculation?
6. **Orderings.** For the six-site Hubbard ring, determine the largest weight of the hopping strings in the site-major and spin-major orderings, and find an ordering for the 2×2 Hubbard plaquette with periodic boundary conditions that minimizes the largest weight. Verify with `HubbardModel` and `JordanWigner`.

7. **Pauli rotations.** (a) Show that $HXH = Z$ and $(SH)Y(SH)^\dagger = Z$, and that $\mathrm{CNOT}_{01}\,(I \otimes Z)\,\mathrm{CNOT}_{01} = Z \otimes Z$. (b) Use these to prove Eq. (4.19) for $P = Z_0 Z_1$ and for $P = X_0 Y_1$. (c) Build the circuit for $e^{-i\theta XYZ}$ with `pauli_rotation` and compare its unitary with the matrix exponential. (d) Show that a Trotter step of the Heisenberg bond $XX + YY + ZZ$ can be implemented with three CNOT gates instead of six.
8. **Symmetries and tapering.** (a) Show that a Pauli string commutes with $Z_0 Z_1 \cdots Z_{M-1}$ if and only if it contains an even number of X and Y factors, and that every string of a number-conserving Hamiltonian has this property. (b) For the compact pairing Hamiltonian with $L = 3$ find a Clifford circuit that maps $Z_0 Z_1 Z_2$ onto Z_2, apply it, fix the eigenvalue for one pair and write the resulting two-qubit Hamiltonian; check its spectrum against the seniority-zero matrix. (c) Why can the total particle number, as opposed to its parity, not be tapered in the same way?
9. **Shots and variances.** (a) Derive the optimal allocation $S_\alpha \propto |h_\alpha|\sigma_\alpha$ by minimizing $\sum_\alpha h_\alpha^2 \sigma_\alpha^2 / S_\alpha$ at fixed $\sum_\alpha S_\alpha$. (b) Reproduce Table 4.11 and extend it to all seventeen strings. (c) The strings within a group are measured from the same shots; show that the variance of the group's contribution involves the covariances $\langle P_\alpha P_\beta \rangle - \langle P_\alpha \rangle \langle P_\beta \rangle$ and evaluate them for the group $\{ZIII, IZII, IIZI, IIIZ, IIII\}$ in the ground state. Do they help or hurt?
10. **A two-qubit variational calculation.** Take the two-level pairing Hamiltonian of Eq. (4.17). (a) Derive the energy $E(\theta)$ of the state of Eq. (4.24) and find the optimal angle analytically. (b) Estimate $E(\theta)$ from $S = 1000$ shots per setting with `energy_from_shots` for twenty angles and locate the minimum; how does the noise affect the estimate of the angle? (c) Repeat with the Hamiltonian including the particle-hole term in the four-qubit encoding, using a single pair-excitation gate and, in addition, the two single-excitation rotations $e^{\theta'(a_a^\dagger a_i - \mathrm{h.c.})}$; is the ground state reached?

Programs for this chapter

Every number and figure in this chapter comes from the programs in `BookPrograms/chapter4/` and from the two shared modules in `BookPrograms/common/` that this chapter introduces, on top of those of Chapters 2 and 3. All of them are imported by the notebook `Notebooks/chapter4/chapter4.ipynb`, and all run on `numpy` and `matplotlib` alone.

`common/pauli.py` The class `PauliSum` of Section 4.2: Pauli strings as words, sums as dictionaries, products, commutators, the decomposition of a matrix by Eq. (4.2), weights, qubit-wise commuting groups, expectation values, the matrix-free product of Section 4.2.1 and the norm and commutator bounds. Instantaneous.

`common/jordanwigner.py` The class `JordanWigner` of Eq. (4.4) with `map_hamiltonian` for the general Hamiltonian of Eq. (3.10), and the classes `ParityMapping` and `BravyiKitaev`; run as a script it checks all three against the anticommutators and the first against the Fock-space matrices of Chapter 3. Instantaneous.

`jw_basics.py` The failure of the naive map, the identities of Eqs. (4.5)–(4.8), Tables 4.2, 4.3 and 4.7, the two-qubit Hamiltonian of Eq. (4.9) and Fig. 4.1. About one second.

`mapped_models.py` The four encodings of the Lipkin model, Table 4.4; the pairing model in both encodings, Eqs. (4.13), (4.17), Table 4.5 and Fig. 4.2; the Hubbard ring in two orderings, Table 4.6; the Heisenberg ring and its fermionic image; the symmetry commutators of Section 4.4.1, the matrix-free Lanczos run and the bounds of Table 4.1, the measurement settings of Table 4.12 and the tapering of Eq. (4.18). Every spectrum is checked against Chapter 3. About one second.

`pauli_circuits.py` The Pauli rotation of Eq. (4.19), Trotter steps as circuits, Table 4.9 and the time evolutions of Fig. 4.4, energy estimation from shots, the shot allocation of

Table 4.11, and Table 4.13, the single-excitation gate of Eq. (4.20) and the Lipkin product states of Table 4.10, the pair-excitation gate of Fig. 4.5, the unitary coupled-cluster circuit of Eq. (4.25) and Table 4.14, and Fig. 4.6. About ten seconds.

Chapter 5
Hamiltonian Simulation and Time Evolution, Alessandro++

Abstract This chapter presents methods for simulating the time evolution of quantum many-body systems on a quantum computer. We introduce the concept of Hamiltonian evolution e^{-iHt} and describe the Trotter–Suzuki decomposition to approximate it by short time steps. Higher-order Trotter formulas and modern techniques (such as quantum signal processing) are mentioned to improve accuracy. Examples of circuit implementations for real-time dynamics in lattice models are provided. We discuss the simulation of both unitary dynamics (real time) and non-unitary evolution (e.g. imaginary-time projection) and their relevance for computing physical observables.

Chapter 6
Quantum Fourier Transform and Phase Estimation

Abstract This chapter focuses on two fundamental quantum algorithms for spectral analysis. We derive the Quantum Fourier Transform (QFT) circuit and discuss its resource scaling. Building on the QFT, we introduce the Quantum Phase Estimation (QPE) algorithm, which extracts eigenvalues of unitary operators (e.g. energy eigenvalues via time evolution). Variants of QPE (iterative, Bayesian) are covered, along with discussions of precision and error, of the choice of the evolution time, and of the Trotter error inside the controlled evolution. The algorithm is applied to the ground-state energies of the model Hamiltonians of the previous chapters, and the roles of the trial state and of the overlap with the target eigenstate are examined. As an application of phase estimation beyond energies we present the Harrow–Hassidim–Lloyd algorithm for linear systems of equations, with examples from a discretized differential equation and from the resolvent of a many-body Hamiltonian, and discuss its assumptions.

6.1 Introduction

Every method of Chapter 3 was classical, and several of them ran into the exponential wall of Section 3.4.4: the number of Slater determinants that can be built from M single-particle states grows as $\binom{M}{N}$, and no amount of cleverness in the algorithm changes that. Chapter 4 moved the wall rather than removing it, by storing a state of M modes in M qubits instead of in 2^M complex numbers, and it left every model of the book as a Pauli sum $\hat{H} = \sum_\alpha h_\alpha P_\alpha$ together with the circuits that exponentiate it term by term. The question is then what can actually be computed with such a register, and this chapter answers it for the one quantity we have wanted throughout: an eigenvalue of a Hamiltonian.

The algorithm is *quantum phase estimation*. Its input is a unitary and one of its eigenstates; its output is the eigenphase, written out in binary in a register of qubits, to a precision that improves *exponentially* with the size of that register. Its engine is the *quantum Fourier transform*, which is the subject of the first part of the chapter and which is worth the space: the QFT is the transformation that converts phase information into computational-basis information, and essentially every quantum algorithm with an exponential speed-up uses it. Nothing new is needed from the formalism. Chapter 2 set out the gate model, with states $|x_1 x_2 \cdots x_n\rangle$ in the computational basis, unitaries acting on one or two qubits at a time and measurements given by projections, and observed that quantum computing is a restricted form of linear algebra: matrix-vector multiplications with special unitary matrices on exponentially large vectors. This chapter is that observation carried through to an algorithm.

The chapter runs in four parts. The first, Sections 6.2–6.2.4, is the quantum Fourier transform: the definition, the product form derived twice, the circuit, the gate count, the approx-

imate transform and the bit-ordering convention that causes more confusion than any other detail of the subject. The second, Sections 6.3 and 6.4, is phase estimation: the algorithm, why the inverse QFT is the right final step, the exact and the inexact case, the precision, the one-control-qubit variants, and the bridge from the eigenphases of $e^{-i\hat{H}t}$ to energies. The third, Sections 6.5 and 6.6, runs the algorithm on the Hamiltonians of Chapter 4, from the one-qubit pairing Hamiltonian of Eq. (4.18) to the eight-qubit Hubbard ring, and confronts the two errors that a real calculation must balance, the finite register and the Trotterization of the evolution operator, as well as the question of what happens when the input is not an eigenstate. The fourth, Section 6.7, is an application of phase estimation to something other than an energy: the algorithm of Harrow, Hassidim and Lloyd for linear systems of equations, which uses the phase register to invert a matrix spectrally, and which we apply to a discretized differential equation and to the resolvent $(z-\hat{H})^{-1}$ of a many-body Hamiltonian, the object behind every Green's function. The programs are in `BookPrograms/chapter6/`: a QFT built gate by gate on the simulator of Chapter 2 and checked against the Fourier matrix, a `PhaseEstimation` class, the model calculations, and an `HHL` class. Every number quoted below comes from them.

6.2 From the discrete Fourier transform to the quantum one

The classical discrete Fourier transform of a vector $x = (x_0, \ldots, x_{N-1})$ is

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j \, e^{2\pi i jk/N}, \qquad k = 0, \ldots, N-1, \tag{6.1}$$

that is, multiplication by the unitary matrix

$$\boldsymbol{F}_N = \frac{1}{\sqrt{N}} \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \cdots & \omega^{(N-1)^2} \end{pmatrix}, \qquad \omega = e^{2\pi i/N}. \tag{6.2}$$

The matrix is unitary, $\boldsymbol{F}_N^\dagger \boldsymbol{F}_N = \boldsymbol{I}$, by the orthogonality relation $\sum_j \omega^{j(k-k')} = N\delta_{kk'}$; it therefore preserves inner products and norms, which is exactly the condition for a matrix to be an admissible quantum gate. Its columns are an orthonormal basis, and $\boldsymbol{F}_N$ is the change of basis between them and the computational basis.

The *quantum* Fourier transform is the same matrix, applied to the amplitudes of a quantum state. For $N = 2^n$ it is the unitary defined on the computational basis $|x\rangle$ of n qubits by

$$\mathrm{QFT}_N |x\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i xk/N} |k\rangle, \qquad x = 0, \ldots, N-1, \tag{6.3}$$

and extended to a general state $|\psi\rangle = \sum_x c_x |x\rangle$ by linearity, $\mathrm{QFT}_N |\psi\rangle = \sum_x c_x \mathrm{QFT}_N |x\rangle$. Here and throughout the chapter the integer x and the bit string $x_1 x_2 \cdots x_n$ of the register are the same object, with qubit 0 of Chapters 2–4 holding the most significant bit x_1,

$$x = \sum_{j=1}^{n} x_j 2^{n-j}, \qquad x_j \in \{0, 1\}. \tag{6.4}$$

Two things should be said at once, because they are the source of both the power and the disappointment of the QFT. The power: the transform acts on $N = 2^n$ amplitudes using, as we shall see, only $\mathcal{O}(n^2)$ gates, against the $\mathcal{O}(N \log N)$ operations of the classical fast Fourier transform, exponentially fewer. The disappointment: the result sits in the *amplitudes* of a quantum state, which cannot be read out. Measuring gives one basis state drawn from $|y_k|^2$, not the vector y. The QFT is therefore useless as a subroutine for computing Fourier transforms of data, and enormously useful as a subroutine inside algorithms that need only one number out of the transform, a period, or, as here, a phase.

6.2.1 The product form

Everything about the QFT circuit follows from rewriting Eq. (6.3) as a product of one-qubit states. We do it twice: once in general, and once explicitly for two qubits, where every step can be followed by eye. Besides the binary expansion (6.4) we need the *binary fraction*

$$0.x_j x_{j+1} \ldots x_n = \frac{x_j}{2} + \frac{x_{j+1}}{2^2} + \cdots + \frac{x_n}{2^{n-j+1}}. \tag{6.5}$$

This notation is not a convenience: the phases of the QFT factor into exactly such fractions, and the same fractions reappear as the output of phase estimation in Section 6.3.

The general derivation. Start from Eq. (6.3) with $N = 2^n$ and write the *output* label in binary, $k = \sum_{m=1}^{n} k_m 2^{n-m}$. Then

$$\frac{xk}{2^n} = x \sum_{m=1}^{n} \frac{k_m}{2^m}, \qquad \text{so} \qquad e^{2\pi i x k/2^n} = \prod_{m=1}^{n} e^{2\pi i x k_m/2^m}. \tag{6.6}$$

The exponential has factorized into one term per output qubit, and each term depends on k only through the single bit k_m. A sum over all k is a sum over all the bits independently, so it factorizes too:

$$\mathrm{QFT}_N |x\rangle = \frac{1}{2^{n/2}} \bigotimes_{m=1}^{n} \left(|0\rangle + e^{2\pi i x/2^m} |1\rangle \right). \tag{6.7}$$

Finally, $e^{2\pi i x/2^m}$ depends only on the fractional part of $x/2^m$, and that fractional part is precisely the binary fraction $0.x_{n-m+1} x_{n-m+2} \ldots x_n$. Hence

$$\begin{aligned} \mathrm{QFT}_N |x_1 x_2 \ldots x_n\rangle = \frac{1}{2^{n/2}} \left(|0\rangle + e^{2\pi i 0.x_n} |1\rangle \right) \otimes \left(|0\rangle + e^{2\pi i 0.x_{n-1} x_n} |1\rangle \right) \\ \otimes \cdots \otimes \left(|0\rangle + e^{2\pi i 0.x_1 x_2 \ldots x_n} |1\rangle \right). \end{aligned} \tag{6.8}$$

Three consequences, all of them important. First, the output is a *product* state: the QFT of a computational basis state is completely unentangled, and each qubit carries a phase and nothing else. It is worth pausing on that, because the $\bigotimes$ in Eq. (6.8) is doing real work. A general state of n qubits needs 2^n amplitudes; a product state needs $2n$. The QFT of a basis state is therefore an exponentially compressible object, and that is precisely why it can be built with polynomially many gates. Second, the first output qubit depends only on the *last* input bit, the second on the last two, and so on: the output order is reversed relative to the input. Third, each factor can be produced by a Hadamard followed by a few controlled phase rotations, which is the circuit.

Two qubits, explicitly. For $n = 2$, write $|x\rangle = |x_1 x_2\rangle$ with $x = 2x_1 + x_2$, and $k = 2k_1 + k_2$. Then

$$\mathrm{QFT}_4|x_1x_2\rangle = \frac{1}{2}\sum_{k_1,k_2\in\{0,1\}} e^{2\pi i(2x_1+x_2)(2k_1+k_2)/4}|k_1k_2\rangle, \tag{6.9}$$

and the exponent expands as

$$\frac{(2x_1+x_2)(2k_1+k_2)}{4} = x_1k_1 + \frac{x_1k_2}{2} + \frac{x_2k_1}{2} + \frac{x_2k_2}{4}. \tag{6.10}$$

The first term is an *integer*, so $e^{2\pi i x_1k_1} = 1$ and it drops out. This is the essential simplification, and it is what makes the QFT efficient: the high-order products of input and output bits contribute nothing at all. Collecting what remains by output bit,

$$e^{2\pi i xk/4} = e^{2\pi i k_1\left(\frac{x_2}{2}\right)}\, e^{2\pi i k_2\left(\frac{x_1}{2}+\frac{x_2}{4}\right)} = e^{2\pi i k_1(0.x_2)}\, e^{2\pi i k_2(0.x_1x_2)}, \tag{6.11}$$

so the sum factorizes into

$$\mathrm{QFT}_4|x_1x_2\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle + e^{2\pi i 0.x_2}|1\rangle\right) \otimes \frac{1}{\sqrt{2}}\left(|0\rangle + e^{2\pi i 0.x_1x_2}|1\rangle\right), \tag{6.12}$$

which is Eq. (6.8) for $n = 2$. The first factor depends only on x_2 and the second on both bits: the least significant phase appears first, which is the reversal again, now visible in three lines of algebra.

6.2.2 The circuit

Two gates suffice, and both are in Chapter 2. The Hadamard $\mathrm{H} = (X+Z)/\sqrt{2}$ creates an equal superposition, and the phase rotation

$$R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix} \tag{6.13}$$

adds one binary digit to the phase. R_k is a rotation about the z axis up to a global phase, $R_k = e^{i\pi/2^k} R_z(2\pi/2^k)$, and $R_1 = Z$, $R_2 = S$, $R_3 = T$ are the standard names. A *controlled* R_k applies the rotation only on the subspace where the control qubit is $|1\rangle$; it is symmetric in its two qubits, since it multiplies the single amplitude of $|11\rangle$ by $e^{2\pi i/2^k}$ and leaves the other three alone.

Read Eq. (6.8) backwards to see the construction. The last output factor carries the phase $e^{2\pi i 0.x_1x_2\ldots x_n}$. A Hadamard on qubit 1 produces $|0\rangle + e^{2\pi i 0.x_1}|1\rangle$, since $\mathrm{H}|x_1\rangle = (|0\rangle + (-1)^{x_1}|1\rangle)/\sqrt{2}$ and $(-1)^{x_1} = e^{2\pi i 0.x_1}$, and controlled rotations from qubits $2,3,\ldots,n$ append the digits $x_2, x_3, \ldots, x_n$ one at a time: a controlled R_2 from qubit 2 multiplies the $|1\rangle$ component by $e^{2\pi i x_2/4}$, a controlled R_3 from qubit 3 by $e^{2\pi i x_3/8}$, and so on. The same pattern then repeats on qubit 2, which needs the digits $x_3, \ldots, x_n$, and so on down the register. At the end the qubit that was processed first holds the *last* factor of Eq. (6.8), so the register is reversed with $\lfloor n/2 \rfloor$ SWAP gates. Figures 6.1 and 6.2 show the result for two and three qubits, and for $n = 3$ the product form reads

$$\mathrm{QFT}_8|x_1x_2x_3\rangle = \frac{1}{\sqrt{8}}\left(|0\rangle + e^{2\pi i 0.x_3}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.x_2x_3}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.x_1x_2x_3}|1\rangle\right), \tag{6.14}$$

so the first qubit needs the phase contributions $x_2/2 + x_3/4$, which is exactly what the controlled R_2 and R_3 of Fig. 6.2 supply.

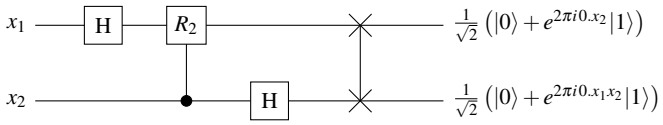


Fig. 6.1 The two-qubit QFT: a Hadamard on qubit 1, a controlled R_2 with $R_2 = \mathrm{diag}(1, e^{i\pi/2})$, a Hadamard on qubit 2, and a SWAP that undoes the reversal of Eq. (6.12).

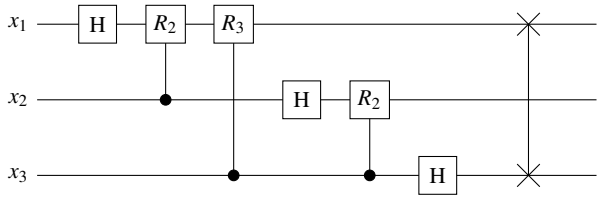


Fig. 6.2 The three-qubit QFT. Hadamard on x_1, then controlled R_2 and R_3 from x_2 and x_3; Hadamard on x_2, then a controlled R_2 from x_3; Hadamard on x_3; then the reversal, which for three qubits is a single SWAP of the outer pair. The nested triangular structure is the general pattern.

The general n-qubit circuit is therefore: for $j = 1, \ldots, n$, apply a Hadamard to qubit j and then, for each later qubit $k > j$, a controlled R_{k-j+1} with control k and target j; finally reverse the register. The function `qft_circuit` in `qft.py` builds it as a `Circuit` of Chapter 2, in the same dozen lines,

```
from qft import qft_circuit, qft_matrix
from statevector import StateVector
import numpy as np

circ = qft_circuit(3)                    # H, controlled R_k, SWAP, Fig. 6.3
print(circ.draw())
print(np.abs(circ.unitary() - qft_matrix(3)).max()) # the Fourier matrix, to 1e-15
out = circ.run(StateVector.from_bits("101"))
print(np.round(out.psi, 4))              # the eight amplitudes of QFT_8 |101>
```

and the script checks, for n up to five, that the unitary of the circuit is the matrix of Eq. (6.2), that its action on every basis state is the product state of Eq. (6.8) built with no circuit at all, that its action on a random state is `numpy.fft.ifft` with orthonormal scaling, and that the inverse circuit undoes it, all to 3.5×10^{-15}. The second check is the one that matters, since the product form is built directly from the binary-fraction factorization and therefore tests the derivation rather than the implementation's self-consistency.

6.2.3 A worked example

Take $|x\rangle = |101\rangle$, so $x_1 = 1$, $x_2 = 0$, $x_3 = 1$ and $x = 5$. The product form gives

$$\mathrm{QFT}_8|101\rangle = \frac{1}{\sqrt{8}}\left(|0\rangle + e^{2\pi i 0.1}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.01}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.101}|1\rangle\right). \tag{6.15}$$

Evaluating the binary fractions, $0.1 = \frac{1}{2}$, $0.01 = \frac{1}{4}$ and $0.101 = \frac{1}{2} + \frac{1}{8} = \frac{5}{8}$, so $e^{2\pi i 0.1} = -1$, $e^{2\pi i 0.01} = i$, $e^{2\pi i 0.101} = e^{5\pi i/4}$, and

$$\mathrm{QFT}_8|101\rangle = \frac{1}{\sqrt{8}}\left(|0\rangle - |1\rangle\right) \otimes \left(|0\rangle + i|1\rangle\right) \otimes \left(|0\rangle + e^{5\pi i/4}|1\rangle\right). \tag{6.16}$$

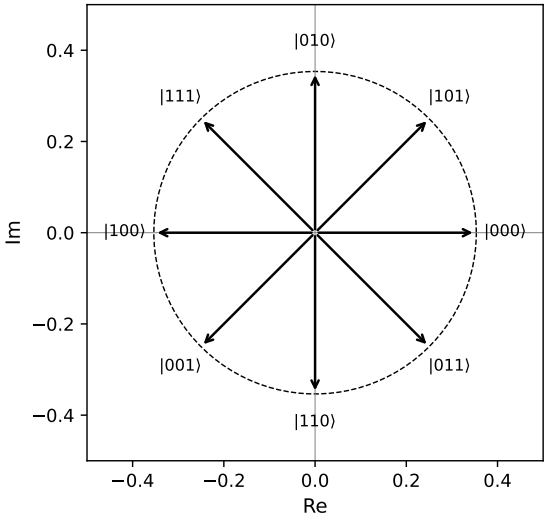


Fig. 6.3 The eight amplitudes of $\mathrm{QFT}_8|101\rangle$, drawn as vectors in the complex plane and labelled by the output basis state $|k\rangle$ they belong to. Every one of them has the same modulus $1/\sqrt{8} = 0.3536$, marked by the dashed circle, and the phase of $|k\rangle$ is $2\pi \times (5k/8 \bmod 1)$. Since 5 and 8 have no common factor the eight vectors occupy the eight eighth-roots of unity exactly once each, reached in the order $0,5,2,7,4,1,6,3$ as k runs from 0 to 7. Produced by `qft.py`.

Expanding the product gives eight amplitudes, all of modulus $1/\sqrt{8} = 0.353553$, with the phase of $|k\rangle$ equal to $2\pi \times (5k/8 \bmod 1)$, which is Eq. (6.3) with $x = 5$, as it must be. `qft.py` prints the eight amplitudes and confirms both statements, and Fig. 6.3 draws them.

Two things in Fig. 6.3 are worth dwelling on, and both are properties of the transform rather than of this particular input. The first is that all eight vectors have the same length. A measurement of the transformed register in the computational basis would return each of the eight outcomes with probability $1/8$, that is, it would return nothing at all: the entire content of $\mathrm{QFT}_8|101\rangle$ sits in the angles, and angles are exactly what a single measurement cannot see. This is the disappointment of Section 6.2 made visual, and it is why the QFT is never the last step of an algorithm. The second is the arrangement of the angles. Reading the labels anticlockwise from $|000\rangle$ gives $0,5,2,7,4,1,6,3$, which is the sequence $5k \bmod 8$: the input integer has been turned into a *rotation rate*, one eighth of a turn per unit of x per unit of k. Because 5 is coprime to 8 the rate is primitive and the eight vectors are all distinct, whereas for $x = 4$ the phase $4k/8$ takes only the two values 0 and $\frac{1}{2}$, and the eight vectors would pile up four deep on two positions. That collapse is periodicity, and reading it off is precisely what Shor's algorithm does; here we shall instead read off a single angle, which is phase estimation. The product form can be traced directly in the picture: expanding Eq. (6.16), the amplitude of $|k_1k_2k_3\rangle$ is $(-1)^{k_1}\, i^{k_2}\, e^{5\pi i k_3/4}/\sqrt{8}$, so the first output bit contributes a half-turn, the second a quarter-turn and the third five-eighths of a turn, independently of one another. Three angles, one per qubit, generate all eight, and that is the reason the circuit needs three Hadamards and three controlled rotations rather than an 8×8 matrix multiplication.

6.2.4 Cost, approximation and conventions

The inverse. The inverse QFT is obtained by reversing the order of the gates and replacing each R_k by $R_k^\dagger$; the SWAP network is its own inverse. It is therefore no more expensive than

Table 6.1 The cost of the QFT circuit against the cost of multiplying by the $2^n \times 2^n$ Fourier matrix. At $n = 100$ the quantum circuit needs about five thousand gates and the classical matrix multiplication needs more operations than there are atoms in the observable universe. Produced by `qft.py`.

n	H	controlled R_k	swaps	direct classical DFT
3	3	3	1	6.4×10^1
6	6	15	3	4.1×10^3
10	10	45	5	1.0×10^6
100	100	4950	50	1.6×10^{60}

Table 6.2 The approximate QFT on $n = 6$ qubits, applied to $|x\rangle$ with $x = 37$: the number of controlled rotations kept when only R_k with $k \le d$ are retained, and the largest error in any output amplitude. Produced by `qft.py`.

d	controlled rotations	largest amplitude error
1	0	2.50×10^{-1}
2	5	2.43×10^{-1}
3	9	1.29×10^{-1}
4	12	3.67×10^{-2}
5	14	1.23×10^{-2}
6	15	1.4×10^{-16} (exact)

the forward transform, and in practice it is the one algorithms use, phase estimation among them.

The gate count. Counting the circuit of Section 6.2.2: n Hadamards, $\sum_{j=1}^{n}(n-j) = n(n-1)/2$ controlled rotations, and $\lfloor n/2 \rfloor$ swaps, so

$$\text{gates} = \mathcal{O}(n^2). \tag{6.17}$$

Compare the classical cost of applying $\boldsymbol{F}_N$ directly to a 2^n-dimensional vector, which is $\mathcal{O}(N^2) = \mathcal{O}(4^n)$ operations, or $\mathcal{O}(N \log N)$ with the fast Fourier transform; Table 6.1 gives the numbers.

The approximate QFT. The rotation R_k turns the phase by $2\pi/2^k$, which for large k is tiny. Dropping every controlled rotation with $k - j + 1 > d$ gives the *approximate* QFT, with $\mathcal{O}(nd)$ gates instead of $\mathcal{O}(n^2)$; choosing $d \sim \log n$ keeps the error bounded and gives $\mathcal{O}(n \log n)$. Table 6.2 and Fig. 6.4 show the trade on six qubits, and the shape they reveal is not the one the asymptotic argument leads one to expect. Keeping the five largest rotations, every controlled R_2, each of which turns the phase by a quarter of a turn, lowers the largest amplitude error only from 2.50×10^{-1} to 2.43×10^{-1}, a gain of three per cent for a third of the circuit. The error then falls by roughly a factor of three at each further level, while the cost of a level shrinks: level d adds $n - d + 1$ rotations, so the step from $d = 5$ to $d = 6$ is a single controlled R_6, turning the phase by $2\pi/64 \approx 5.6^\circ$, and that one gate takes the error from 1.23×10^{-2} to zero. The marginal value of a controlled rotation therefore *increases* along the ladder, which is the opposite of the truncation's motivation. There is no contradiction, only a question of scale. The bound of Exercise 2, $2\pi n 2^{-d}$, equals 37.7×2^{-d} for $n = 6$ and does not drop below unity until $d = 6$: over the whole of Fig. 6.4 the estimate is vacuous, and what the figure shows is the pre-asymptotic regime in which every rotation still matters. The prescription $d \sim \log_2 n$ becomes a saving only when n is large enough that $\log_2 n$ is genuinely smaller than n; at $n = 100$, the last row of Table 6.1, it replaces the 4950 controlled rotations by some seven hundred, and there it earns its keep. The lesson for a small calculation is to keep the whole triangle, and the lesson for a large one is that the gates one may safely discard are the numerous cheap-looking ones at the far end of a *long* ladder, not the last few of a short one.

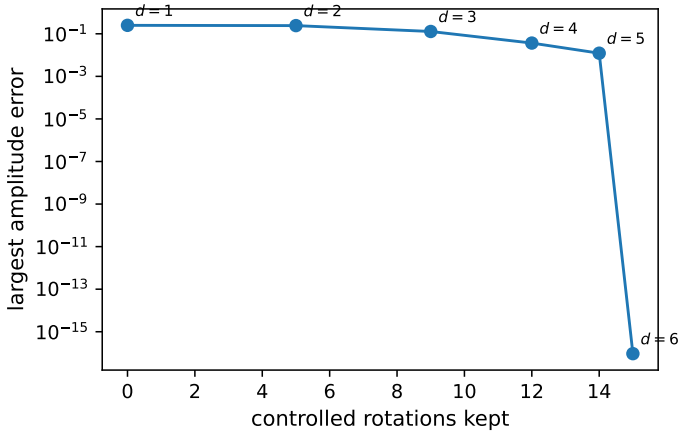


Fig. 6.4 The largest amplitude error of the approximate QFT_6 applied to $|x\rangle$ with $x = 37$, against the number of controlled rotations retained, for the six truncation levels $d = 1, \ldots, 6$ of Table 6.2; the exact transform is drawn on the 10^{-17} floor of the axis. The curve is nearly flat over its first third and then falls by about a factor of three per level, so that the last and cheapest rotations are the ones that carry the accuracy. Produced by `qft.py`.

Bit reversal: the commonest confusion. Equation (6.8) produces the qubits in reversed order, and the final SWAP network exists only to undo that. Many software libraries omit the swaps and document the reversed convention instead, since a relabelling costs nothing classically. The consequence is that a circuit copied from one library into another can be silently wrong. `qft.py` makes the statement precise: without the swaps the amplitude that should sit on $|k\rangle$ sits on the basis state whose bits are those of k in reverse order, so that for $\mathrm{QFT}_8|001\rangle$ the phase $1/8$ appears on $|100\rangle$, $2/8$ on $|010\rangle$ and $4/8$ on $|001\rangle$. When in doubt, transform $|0\cdots01\rangle$ and look at where the phase $e^{2\pi i/N}$ ends up.

6.3 Quantum phase estimation

Let U be unitary with an eigenstate $|\psi\rangle$,

$$U|\psi\rangle = e^{2\pi i\phi}|\psi\rangle, \qquad 0 \le \phi < 1. \tag{6.18}$$

The spectral theorem guarantees that such eigenstates exist and form a complete orthonormal set, and that

$$U = \sum_j e^{2\pi i\phi_j}|\psi_j\rangle\langle\psi_j|, \qquad U^k = \sum_j e^{2\pi ik\phi_j}|\psi_j\rangle\langle\psi_j|, \tag{6.19}$$

so that powers of U are obtained by raising the eigenvalues to that power and nothing else, the fact on which the whole algorithm turns. Because U is unitary its eigenvalues have unit modulus, so a single real number ϕ_j per eigenstate carries all the information. Quantum phase estimation determines it [**?**, **?**, **?**].

The algorithm uses two registers: a *control* register of m qubits prepared in $|0\rangle^{\otimes m}$, which will hold the answer, and a *system* register holding $|\psi\rangle$.

Step 1: superposition. Hadamards on every control qubit give

$$|0\rangle^{\otimes m}|\psi\rangle \;\longmapsto\; \frac{1}{2^{m/2}}\sum_{k=0}^{2^m-1}|k\rangle|\psi\rangle. \tag{6.20}$$

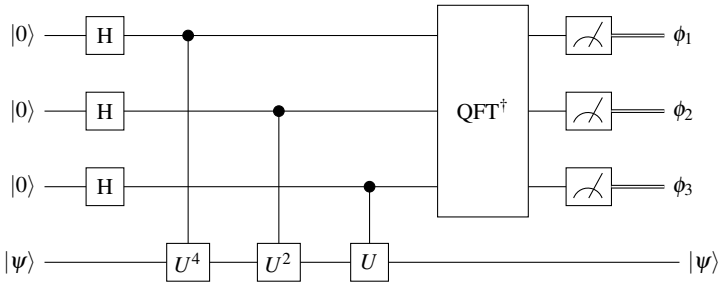


Fig. 6.5 Quantum phase estimation with a three-qubit control register. Hadamards, then the controlled powers U^4, U^2, U from the control qubits of weight 4, 2, 1, then the inverse QFT, then measurement of the control register, which returns the bits of $\phi = 0.\phi_1\phi_2\phi_3$. The system register emerges in the state it entered.

Step 2: controlled powers. Apply U^{2^j} controlled on the control qubit that holds the bit of weight 2^j, for $j = 0, 1, \ldots, m-1$. Since $|\psi\rangle$ is an eigenstate, $U^{2^j}|\psi\rangle = e^{2\pi i 2^j \phi}|\psi\rangle$, so if the control register is in $|k\rangle$ the total operation applied is U^k and

$$U^k|\psi\rangle = e^{2\pi i k\phi}|\psi\rangle. \tag{6.21}$$

The system register is therefore unchanged, factors out, and the control register is left in the *Fourier state*

$$\frac{1}{2^{m/2}} \sum_{k=0}^{2^m-1} e^{2\pi i k\phi}|k\rangle. \tag{6.22}$$

The eigenstate has acted as a catalyst: it is returned untouched, having written its phase into the control register. Nothing about the system register was measured, and nothing about it needs to be known beyond the fact that it is an eigenstate.

Step 3: the inverse QFT. Compare Eq. (6.22) with Eq. (6.3). If $\phi = k'/2^m$ for an integer k', the control register is *exactly* $\mathrm{QFT}|k'\rangle$. Applying the inverse QFT therefore returns $|k'\rangle$, and measuring gives the binary digits of ϕ with certainty:

$$\mathrm{QFT}^{-1}\left(\frac{1}{2^{m/2}}\sum_k e^{2\pi i k\phi}|k\rangle\right) = |\phi_1\phi_2\ldots\phi_m\rangle, \qquad \phi = 0.\phi_1\phi_2\ldots\phi_m. \tag{6.23}$$

This is why the inverse QFT is the right final step, and it is the whole of the algorithm: step 2 writes the phase into a Fourier state, step 3 inverts the Fourier transform. Figure 6.5 draws the circuit.

6.3.1 The bits, one at a time

It is worth seeing the same thing in the product form, because it shows the bits emerging individually. Writing $\phi = 0.\phi_1\phi_2\ldots$, the state (6.22) factorizes exactly as Eq. (6.7) did,

$$\frac{1}{2^{m/2}}\left(|0\rangle + e^{2\pi i 2^{m-1}\phi}|1\rangle\right)\otimes\left(|0\rangle + e^{2\pi i 2^{m-2}\phi}|1\rangle\right)\otimes\cdots\otimes\left(|0\rangle + e^{2\pi i\phi}|1\rangle\right). \tag{6.24}$$

Now $2^{m-1}\phi = \phi_1\phi_2\cdots\phi_{m-1}.\phi_m\phi_{m+1}\ldots$, and only the fractional part survives in the exponential, so the first factor is $|0\rangle + e^{2\pi i 0.\phi_m\phi_{m+1}\cdots}|1\rangle$ and so on down the register:

$$\frac{1}{2^{m/2}}\left(|0\rangle + e^{2\pi i 0.\phi_m}|1\rangle\right)\otimes\cdots\otimes\left(|0\rangle + e^{2\pi i 0.\phi_1\phi_2\ldots\phi_m}|1\rangle\right), \tag{6.25}$$

Table 6.3 Phase estimation of $\phi = 0.3$ with the eigenstate as input. The estimate is the modal outcome read as a binary fraction; the error falls by a factor of two per control qubit. The last column is the combined probability of the two most likely outcomes. Produced by `phase_estimation.py`.

m	outcome	probability	estimate	error	two best
3	$\|010\rangle$	0.5775	0.250000	5.0×10^{-2}	0.837
4	$\|0101\rangle$	0.8756	0.312500	1.3×10^{-2}	0.931
6	$\|010011\rangle$	0.8752	0.296875	3.1×10^{-3}	0.930
8	$\|01001101\rangle$	0.8751	0.300781	7.8×10^{-4}	0.930

which is precisely the QFT state (6.8) of the bit string $\phi_1\phi_2\ldots\phi_m$ when the expansion of ϕ terminates after m bits. Each control qubit carries one successive binary suffix of the phase; the inverse QFT decodes them all at once. The control qubit that was given $U^{2^{m-1}}$ carries the least significant bit, since multiplying ϕ by 2^{m-1} pushes all but the last digit into the integer part: the *highest* power of U resolves the *finest* digit, which is the reason the resolution improves exponentially with the number of controlled powers.

6.3.2 The exact and the inexact case

Exact. Let $\phi = 5/8 = 0.101$ and $m = 3$, with $U = \mathrm{diag}(1, e^{2\pi i\phi})$ on one qubit and the eigenstate $|1\rangle$. Then the output is $|101\rangle$ with probability one. `phase_estimation.py` runs the circuit of Fig. 6.5 on the simulator and confirms it: probability 1.000000, estimate 0.625000.

Inexact. If ϕ is not of the form $k'/2^m$, which for a Hamiltonian eigenvalue it never is, the control register is not a computational basis state and the measurement returns a *distribution*. Its shape is a short calculation. The amplitude of the outcome $|k'\rangle$ after the inverse QFT is the geometric sum

$$a_{k'} = \frac{1}{2^m}\sum_{k=0}^{2^m-1} e^{2\pi i k\left(\phi - k'/2^m\right)} = \frac{1}{2^m}\,\frac{1-e^{2\pi i 2^m\delta}}{1-e^{2\pi i\delta}}, \qquad \delta = \phi - \frac{k'}{2^m}, \tag{6.26}$$

and with $|1-e^{i\theta}| = 2|\sin(\theta/2)|$ the probability is the Dirichlet kernel

$$P(k') = |a_{k'}|^2 = \frac{\sin^2(\pi 2^m\delta)}{2^{2m}\sin^2(\pi\delta)}. \tag{6.27}$$

Its worst case is $|\delta| = 2^{-m-1}$, exactly halfway between two representable values, where $\sin^2(\pi 2^m\delta) = 1$ and $\sin^2(\pi\delta) \le (\pi\delta)^2 = (\pi 2^{-m-1})^2$, so that the nearest outcome is obtained with probability at least

$$P \;\ge\; \frac{1}{2^{2m}}\cdot\frac{1}{\left(\pi 2^{-m-1}\right)^2} = \frac{4}{\pi^2} \approx 0.405, \tag{6.28}$$

whatever ϕ may be, and the two nearest outcomes together carry at least $8/\pi^2 \approx 0.81$. `phase_estimation.py` scans ϕ across a grid interval and finds the minimum 0.4054 at the midpoint and the minimum for the two nearest outcomes 0.8107, both within a tenth of a per cent of the bounds. Table 6.3 takes $\phi = 0.3$, which has the non-terminating expansion $0.0100110011\ldots$, and increases the register.

Figure 6.6 shows the two extreme rows of Table 6.3 as distributions rather than as modal outcomes, and the comparison makes a point that the table cannot. Going from four control qubits to eight divides the grid spacing by sixteen and the error with it, from 1.3×10^{-2} to

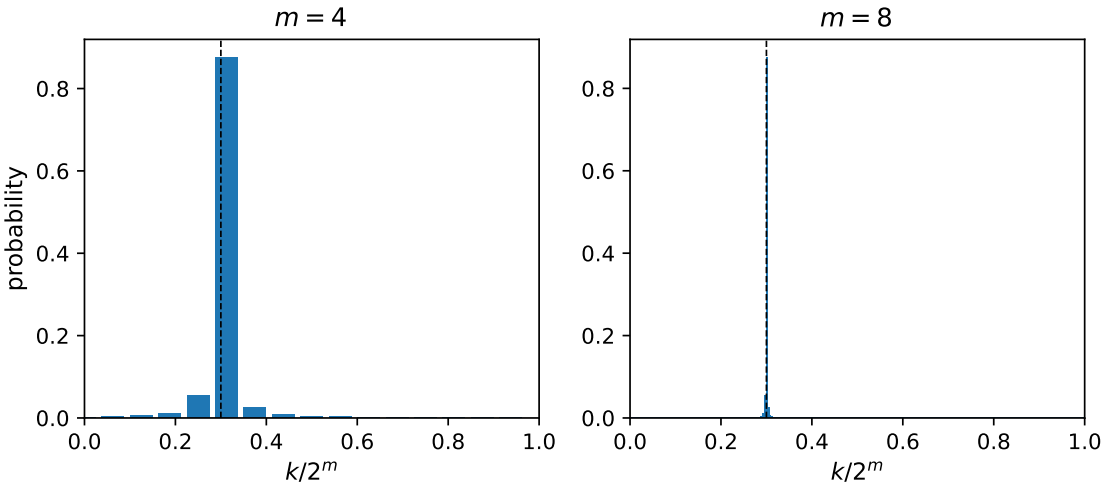


Fig. 6.6 The full output distribution of phase estimation applied to $\phi = 0.3$, with a control register of $m = 4$ qubits (left) and $m = 8$ qubits (right). The bars give the probability of each of the 2^m representable values $k/2^m$, plotted at that value; the dashed line marks the true phase. The peak carries probability 0.876 at $m = 4$ and 0.875 at $m = 8$: enlarging the register narrows the distribution by a factor of sixteen without making the modal outcome any more likely, since the height is fixed by the offset in Eq. (6.27) and not by m. Produced by `phase_estimation.py`.

7.8×10^{-4}, yet the peak barely moves in height: 0.8756 against 0.8751. The two numbers are equal for a reason that is arithmetic rather than accidental. The binary expansion of 0.3 has period four, so multiplying by 2^4 shifts it by a whole period and leaves the fractional part unchanged; the distance from ϕ to the nearest grid point, measured in units of the grid, is therefore $u = 2^m\delta = 0.2$ for $m = 4$ and for $m = 8$ alike, and the probability $\sin^2(\pi u)/(\pi u)^2 = 0.875$ comes out the same both times. It is the offset u, not the size of the register, that decides how likely the best answer is, which is exactly what the $4/\pi^2$ bound says, since $u = \frac{1}{2}$ is its worst case.

The shoulders are worth a look too, because they are the tails of the same kernel and they are not small. In the $m = 4$ panel the outcome one grid step below the peak carries 0.055 and the one step above 0.024; together with the peak they account for the 0.931 of Table 6.3. The tails fall only as the inverse square of the distance from the peak, so a run of phase estimation has a small but genuinely polynomial chance of landing far from the answer, and the histogram never becomes a delta function however large m is made. This is why the standard statement of the theorem bounds the probability of landing within one unit of the last bit rather than the probability of landing on it, why the practical recipe is to repeat the circuit and take the median of the estimates rather than to trust a single shot, and, as Section 6.7 will show, why an algorithm that uses the phase register *coherently*, without measuring it, has to worry about the tails in a way that an energy measurement does not.

Precision. With m control qubits the resolution is

$$\Delta\phi \sim \frac{1}{2^m}, \tag{6.29}$$

exponential in the register size, and to obtain ϕ to within 2^{-b} with probability at least $1 - \varepsilon$ one needs $m = b + \lceil \log_2(2 + 1/2\varepsilon) \rceil$ control qubits (Exercise 3). This is the property that makes QPE worth its cost: to gain one more binary digit one adds one qubit and doubles the number of applications of U, whereas a statistical estimate of the same phase from repeated measurements of an observable, the method of the variational algorithms of Chapter 7, needs

four times as many samples per digit. In the language of metrology the first scaling is the Heisenberg limit and the second the standard quantum limit, and phase estimation reaches the first because the 2^m applications of U are made coherently, in one circuit, rather than one at a time.

6.3.3 Phase estimation with one control qubit

The m control qubits of Fig. 6.5 interact only through the inverse QFT, and the inverse QFT can be dispensed with. Read Eq. (6.25) from the top: the control qubit that was given $U^{2^{m-1}}$ is in the state $\left(|0\rangle + e^{2\pi i 0.\phi_m}|1\rangle\right)/\sqrt{2}$, and $0.\phi_m$ is either 0 or $\frac{1}{2}$, so a Hadamard turns it into $|\phi_m\rangle$ outright: the least significant bit can be measured with a single control qubit and no Fourier transform at all. The next qubit is in the state with phase $0.\phi_{m-1}\phi_m$; now that ϕ_m is known, a rotation R_z by $-2\pi 0.0\phi_m$ removes it, and a Hadamard returns ϕ_{m-1}. Continuing upwards, the bit ϕ_j is obtained from one control qubit, one controlled $U^{2^{j-1}}$, one rotation by the phase $-2\pi 0.0\phi_{j+1}\cdots\phi_m$ built from the bits already measured, and one Hadamard. This is Kitaev's iterative phase estimation [**?**, **?**], and it is what the inverse QFT does anyway, since the controlled $R_k^\dagger$ of the inverse transform are exactly these corrections, applied coherently instead of classically. The saving is that a single control qubit is recycled m times, so the register of Fig. 6.5 shrinks to one ancilla plus the system, at the price of running the circuit m times and of a classical feedback loop between runs. When ϕ is not exactly representable each bit is obtained only with a probability, and the measurement of each bit is repeated and decided by majority. `phase_estimation.py` implements the procedure and recovers the bits 101 of $\phi = 5/8$ with three rounds and the estimate 0.300781 of $\phi = 0.3$ with eight rounds of eleven shots each, the same eight bits as the last row of Table 6.3.

The one-control-qubit form invites a more statistical reading. Each round with the controlled power U^{2^j} and a rotation by θ returns 0 with probability $\cos^2(\pi 2^j\phi + \theta/2)$, a sinusoidal signal in ϕ whose frequency is set by the power. Choosing the powers and rotation angles adaptively, and updating a probability distribution for ϕ after every shot with Bayes' rule, is *Bayesian* phase estimation [**?**]; it tolerates noise better than the digit-by-digit scheme, needs no long coherent circuit, and still reaches the Heisenberg scaling of Eq. (6.29) in the number of applications of U, and it is the form of phase estimation closest to what has actually been run on hardware. The rodeo algorithm of Chapter 8 takes the same one-ancilla circuit in yet another direction, using the measurement outcome to filter the system register rather than to read a bit.

Using the class. The class `PhaseEstimation` in `phase_estimation.py` runs the circuit of Fig. 6.5 on the simulator of Chapter 2, with the control register in qubits 0 to $m-1$, the most significant bit first, and the system register after it. It is built either from a unitary matrix, whose powers it obtains by repeated squaring, or from a Hermitian matrix and a time t, in which case $U = e^{-i\hat{H}t}$ and the powers come from the spectral theorem:

```
from phase_estimation import PhaseEstimation, dirichlet_probability
import numpy as np

phi = 0.3
U = np.diag([1, np.exp(2j * np.pi * phi)])          # eigenstate |1>, eigenphase phi
qpe = PhaseEstimation.from_unitary(U, m=4)
qpe.run([0, 1])                                     # the system state |1>
k, estimate, prob = qpe.estimate()
print(format(k, "04b"), estimate, round(prob, 4)) # 0101 0.3125 0.8756
p = qpe.distribution()                              # all sixteen outcomes
print(np.allclose(p, dirichlet_probability(phi, 4, np.arange(16))))
```

For a system of n qubits the controlled power is a $2^{n+1} \times 2^{n+1}$ matrix applied to the control qubit and the system register together, exactly as a quantum computer would apply it; for the eight-qubit Hubbard Hamiltonian of Section 6.6 this is slow, and the class offers a second method that rotates the system register into the eigenbasis of U once, where every controlled power is diagonal, and rotates back at the end. The two are the same unitary, since the rotation commutes with the controls, and the script checks that they agree to machine precision on a random two-qubit Hamiltonian.

6.4 From phases to energies

Everything so far concerned a unitary. Physics gives us a Hamiltonian, and the bridge is time evolution. If $\hat{H}|E\rangle = E|E\rangle$ then

$$U(t) = e^{-i\hat{H}t} \qquad \Longrightarrow \qquad U(t)|E\rangle = e^{-iEt}|E\rangle, \tag{6.30}$$

so comparing with Eq. (6.18),

$$\phi = -\frac{Et}{2\pi} \bmod 1. \tag{6.31}$$

An eigenstate of the Hamiltonian is an eigenstate of $U(t)$, and QPE measures its energy [**?**, **?**]. The whole of Hamiltonian simulation, Chapter 5, becomes a subroutine of a measurement.

6.4.1 The time step and the reading of the phase

The modulus in Eq. (6.31) is not a formality. Phases are defined only modulo one, so two energies differing by $2\pi/t$ are indistinguishable: aliasing, exactly as in classical signal processing. Since a Hamiltonian has eigenvalues of both signs, the natural convention is to read the register as a signed fraction, the two's complement of the computer scientists,

$$\tilde{\phi} = \begin{cases} k/2^m, & k < 2^{m-1}, \\ k/2^m - 1, & k \geq 2^{m-1}, \end{cases} \qquad E = -\frac{2\pi\tilde{\phi}}{t}, \tag{6.32}$$

so that the outcomes with the leading bit 0 are the negative energies and those with the leading bit 1 the positive ones, and the time step must satisfy $|E|t < \pi$ for every eigenvalue that can appear. This requires a bound on the spectrum. On real hardware the spectrum is precisely what one does not know, and the bound available from the Hamiltonian as written is the 1-norm of the Pauli coefficients of Section 4.2.1,

$$\Lambda = \sum_\alpha |h_\alpha| \;\geq\; \left\|\hat{H}\right\|, \qquad t = \frac{\pi s}{\Lambda}, \quad s < 1, \tag{6.33}$$

with a safety margin s, which we take as 0.9 throughout. The register then resolves energies on the grid

$$\Delta E = \frac{2\pi}{t\,2^m} = \frac{2\Lambda}{s\,2^m}, \tag{6.34}$$

and the estimate is never further than $\Delta E/2$ from an eigenvalue, with the probabilities of Section 6.3.2.

Two consequences follow from Eq. (6.33), and both are part of designing a calculation rather than details. The first is that the identity term of the Pauli sum should never be evolved.

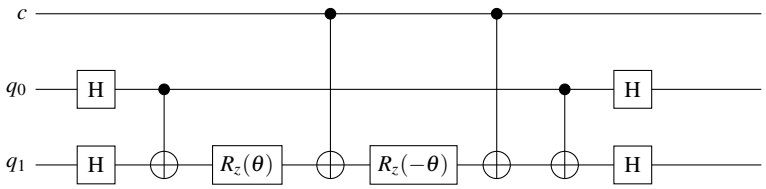


Fig. 6.7 The controlled rotation $e^{-i\theta X_0 X_1}$ with control qubit c: the gadget of Fig. 4.3, with the central $R_z(2\theta)$ replaced by the controlled rotation $R_z(\theta)$, CNOT, $R_z(-\theta)$, CNOT. When $c=|0\rangle$ the two rotations cancel and the rest of the circuit is the identity; when $c=|1\rangle$ the CNOTs flip the sign of the second rotation and the two add up to $R_z(2\theta)$.

It shifts every eigenvalue by the same amount, contributes only a global phase $e^{-ih_I t}$, and is invisible to any measurement; but it enters Λ and therefore shrinks t and coarsens the grid. The rule is: *subtract the identity, estimate the rest, add it back*. The second is that Λ is a loose bound, by a factor of one and a half or so for the models of Table 4.1, and that even the exact norm is loose when the spectrum is not symmetric about zero or when only one symmetry sector matters. Every factor of two that a tighter bound gains in t is one control qubit saved, and Section 6.6 puts numbers to this.

6.4.2 Building the controlled evolution

QPE needs the controlled powers $U^{2^j}=e^{-i\hat{H}t2^j}$, so it needs Hamiltonian simulation. For a Pauli sum $\hat{H}=\sum_\alpha h_\alpha P_\alpha$ the ingredients are those of Section 4.5: each factor $e^{-ih_\alpha P_\alpha \tau}$ is the rotation gadget of Eq. (4.19), a basis change into the Z eigenbasis of P_α, a ladder of CNOT gates collecting the parity, one $R_z(2h_\alpha\tau)$ and the ladder undone, and the strings are assembled by the Trotter formula of Eq. (3.47),

$$e^{-i\hat{H}t} \approx U_r = \left(\prod_\alpha e^{-ih_\alpha P_\alpha t/r}\right)^r , \tag{6.35}$$

with the first-order error $\mathcal{O}(t^2/r)$ governed by the commutators $[P_\alpha,P_\beta]$, or the symmetric splitting of Eq. (3.46) with error $\mathcal{O}(t^3/r^2)$. Strings that commute with one another can be exponentiated together with no splitting error at all, so the number of commuting groups, not the number of strings, is the quantity that sets the error, and it is tabulated for the models below.

What is new is the control. A controlled $e^{-i\theta P}$ is $e^{-i\theta\,|1\rangle\langle 1|\otimes P}$, and since $|1\rangle\langle 1|=(I-Z_c)/2$ on the control qubit c it is the product of two commuting rotations, $e^{-i\theta P/2}$ and $e^{+i\theta Z_c P/2}$. In the gadget of Eq. (4.19) only the central $R_z(2\theta)$ depends on θ, so the control can be attached there alone: replace $R_z(2\theta)$ on the last qubit of the support by a controlled $R_z(2\theta)$, which is $R_z(\theta)$, CNOT, $R_z(-\theta)$, CNOT with the control qubit as the control of both CNOTs, Fig. 6.7. The basis change and the parity ladder are applied unconditionally, which is harmless because they cancel when the control is $|0\rangle$. A controlled Pauli rotation therefore costs two CNOT gates and one rotation more than the uncontrolled one, whatever the weight of the string, and a controlled Trotter step costs $2K$ CNOT gates more than an ordinary one for a Hamiltonian of K strings. The powers U^{2^j} are obtained by repeating the step $2^j r$ times, so that the total number of controlled Pauli rotations in the circuit of Fig. 6.5 is $Kr(2^m-1)$: the price of the exponential precision is an exponential number of gates, and it is the number of *applications of* U, not of qubits, that limits phase estimation on real hardware.

Table 6.4 Phase estimation on the one-qubit pairing Hamiltonian of Eq. (6.36), prepared in the exact ground state, with the identity removed and added back and $t = 1.8850$. The exact answer is $E_- = -0.618034$; the grid spacing $\Delta E = 2\pi/(t2^m)$ halves with each control qubit. Produced by `qpe_models.py`.

m outcome	E_{QPE}	$\lvert E_{\mathrm{QPE}} - E_- \rvert$	probability
3 011	-0.750000	1.3×10^{-1}	0.714
4 0101	-0.541667	7.6×10^{-2}	0.630
5 01011	-0.645833	2.8×10^{-2}	0.787
6 010101	-0.593750	2.4×10^{-2}	0.461
7 0101011	-0.619792	1.8×10^{-3}	0.985
8 01010110	-0.619792	1.8×10^{-3}	0.941
10 0101010111	-0.616536	1.5×10^{-3}	0.471

What the algorithm is, physically. QPE measures the dynamical phase e^{-iEt} accumulated by an energy eigenstate, using the control register as an interferometer: each control qubit is a which-path degree of freedom, the controlled evolution puts a relative phase between the paths, and the inverse QFT is the recombining beam splitter. It is an interferometric measurement of an energy, carried out to exponential precision. Once the eigenvalue is known to ΔE, the algorithm has also, as Section 6.6.3 will show, prepared the eigenstate, which is why the same circuit is the starting point of the state-preparation methods of Chapter 8.

6.5 Two small Hamiltonians, worked through

Before the many-body models it is worth running the algorithm on two Hamiltonians small enough to hold in the head, both of which Chapter 4 left ready to use. Using the same operators again is the point: it is the only way to compare two algorithms without also comparing two problems, and both return in Chapter 7, where the variational quantum eigensolver is applied to exactly the same Pauli sums.

6.5.1 One qubit: the tapered pairing Hamiltonian

Section 4.4.6 reduced the two-level pairing model with one pair to the single-qubit Hamiltonian of Eq. (4.18),

$$\hat{H} = \frac{1}{2} I + Z - \frac{1}{2} X, \qquad E_\pm = \frac{1}{2} \pm \frac{\sqrt{5}}{2} = -0.618034,\ 1.618034, \tag{6.36}$$

whose ground state is $R_y(\theta)|0\rangle$ with $\tan\theta = -1/2$. The identity carries $h_I = \frac{1}{2}$; the traceless part $Z - \frac{1}{2}X$ has $\Lambda = 1.5$ against the exact norm $\sqrt{5}/2 = 1.118$, and Eq. (6.33) gives $t = 0.9\pi/1.5 = 1.8850$. The evolution operator $e^{-i(Z-X/2)t}$ is a single rotation of the qubit about the axis $(-\frac{1}{2}, 0, 1)$ by the angle $2 \cdot 1.118t$, and its controlled version is a controlled rotation about the same axis, which costs two CNOT gates and a few single-qubit gates. Table 6.4 shows the result of feeding the exact ground state to `PhaseEstimation` with increasing registers.

Three features of the table recur in every application and are worth naming at once. The leading bit is 0 in every row: the energy is negative, and Eq. (6.32) reads the register accordingly. The error is not monotone in m: it falls in steps, because the estimate is always the

Table 6.5 Phase estimation on the two-qubit Hamiltonian of Eq. (6.37), prepared in the exact one-particle ground state, with $t = 1.3000$. The exact energy is 0.916905. Produced by `qpe_models.py`.

m outcome	E_{QPE}	$\lvert E_{\mathrm{QPE}} - E_0 \rvert$	probability
3 001	1.020833	1.0×10^{-1}	0.908
4 0010	1.020833	1.0×10^{-1}	0.667
5 00101	0.869792	4.7×10^{-2}	0.718
6 001001	0.945313	2.8×10^{-2}	0.613
7 0010011	0.907552	9.4×10^{-3}	0.814
8 00100110	0.907552	9.4×10^{-3}	0.413
10 0010010110	0.916992	8.7×10^{-5}	0.999

nearest point of a grid of spacing ΔE, and a phase that happens to sit near a grid point is resolved well early, as at $m = 7$, and then not improved until the grid refines past it. And the success probability wanders between 0.46 and 0.99 without trend, as the $4/\pi^2$ bound allows; in practice one repeats and takes the mode, which costs a constant factor and removes the worry. The script also runs the case one should not: keeping the identity in the Hamiltonian and choosing t from the full 1-norm 2 gives $t = 1.4137$ and a grid 0.0087 instead of 0.0065 at $m = 8$, a third of the resolution thrown away on a constant that no measurement can see. Finally, the two-qubit form of the same model, Eq. (4.17) in the one-pair sector, returns *identical* outcomes and probabilities row for row, which the script checks: tapering changes the number of qubits and the depth of the circuit, and nothing about what is measured.

6.5.2 Two qubits: the two-mode Hamiltonian

The next case up is the two-mode Hamiltonian of Eq. (4.9) with $\varepsilon_0 = 1$, $\varepsilon_1 = 2$, $t_{01} = 0.3$ and $U = 0.5$,

$$\hat{H} = 1.625\,II - 0.625\,ZI - 1.125\,IZ + 0.125\,ZZ + 0.15\,(XX + YY), \tag{6.37}$$

six strings, with the spectrum 0, 0.916905, 2.083095, 3.5 of which the middle two form the one-particle block of Section 3.2.6. After the identity is removed $\Lambda = 2.175$ and $t = 1.3000$, and the five remaining strings fall into two commuting groups, $\{ZI, IZ\}$ and $\{ZZ, XX, YY\}$, so that a single Trotter step of U is two exponentials with one splitting error between them; the step costs 6 CNOT gates, and its controlled version $2 \cdot 5 = 10$ more, two per string. Table 6.5 gives the runs from the exact one-particle ground state.

At $m = 10$ the phase happens to be very nearly representable, the distribution collapses onto a single outcome with probability 0.999, and the error drops by two orders of magnitude at once. This is the behaviour the $4/\pi^2$ bound describes: when ϕ is exactly a dyadic rational the algorithm is deterministic, and near-exactness is nearly deterministic. It is also a warning. Nothing about the $m = 8$ run announces that its answer is no better than the $m = 7$ one; only knowing the exact answer reveals it. Phase estimation returns a number and a probability, not an error bar, and turning the one into the other requires the analysis of Section 6.3.2 rather than inspection of the output. The leading bit is now 1 in no row, because after the shift by $h_I = 1.625$ the one-particle energies are negative; the state $|11\rangle$ with energy $3.5 - 1.625 = 1.875$ would give the leading bit 1, and $1.875\,t = 2.44 < \pi$ confirms that no eigenvalue wraps.

The script ends this example by feeding the algorithm the reference determinant $|10\rangle$ of Section 4.5.4, the particle in the lower mode, instead of the exact eigenstate. Its overlaps with the two one-particle eigenstates are $|c_j|^2 = 0.9287$ and 0.0713, and the histogram of the $m = 8$ register shows two peaks, at 0.9169 and at 2.0831, carrying 0.795 and 0.066 within one grid

Table 6.6 The four models of Table 4.8 as inputs to phase estimation: qubits, Pauli strings, largest weight, number of groups of mutually commuting strings (identity excluded), the 1-norm Λ of the traceless part, its exact spectral norm, the time step $t = 0.9\pi/\Lambda$, and the exact ground-state energy of Chapter 3 in the relevant sector. Produced by `qpe_models.py`.

model	qubits	strings	weight	groups	Λ	$\|\hat{H}\|$	t	E_0
Lipkin, $N = 4$	4	16	2	3	6.000	4.213	0.4712	-4.212877
pairing, $L = 4$	4	17	2	4	8.500	5.779	0.3326	0.635548
Hubbard ring	8	29	4	3	20.000	12.000	0.1414	-2.102748
Heisenberg ring	4	12	2	2	3.000	2.000	0.9425	-2.000000

step of each eigenvalue: the weights of the two peaks are the two overlaps, reduced by the Dirichlet factor of Section 6.3.2. Nothing appears at the expectation value $\langle 10|\hat{H}|10\rangle = 1$. This is the first sight of a behaviour that Section 6.6.3 examines in detail: QPE does not average, it projects.

6.6 The model Hamiltonians on a quantum computer

Table 4.8 collected the four models of Chapter 3 in the qubit encodings that use their symmetries: the Lipkin model with $N = \Omega = 4$, $\varepsilon = 2$, $V = -1/3$, $W = -1/4$ on one qubit per doublet, Eq. (4.11); the pairing model with $L = 4$ levels, $\xi = g = 1$ and two pairs on one qubit per level, Eq. (4.16); the half-filled Hubbard ring with $U = 4t$ on eight qubits in the spin-major ordering of Section 4.4.4; and the four-site Heisenberg ring with $J = 1$, Eq. (3.34). We now feed them to phase estimation. The cost of the calculation is set by the numbers in Table 6.6: how many Pauli strings the Hamiltonian has and what their weight is, which fix the gate count of one Trotter step; into how many mutually commuting groups they fall, which fixes the number of splittings and hence the Trotter error; and the norm bound Λ, which fixes the time step and hence the energy resolution of a given register.

The table repays reading. The Lipkin and Heisenberg models have weight-two Hamiltonians with no Jordan–Wigner strings at all, for the reasons of Chapter 4: in the Lipkin model the quasispin operators of one doublet act on one qubit, and in the Heisenberg model the degrees of freedom are spins to begin with. The Hubbard ring pays the fermionic price, weight four for the bond that closes the ring, and its 1-norm $\Lambda = 20$ overestimates the norm by nearly a factor of two, which will cost it a control qubit. The Heisenberg ring has the fewest commuting groups, two, because the six strings of the bonds $(0,1)$ and $(2,3)$ act on disjoint pairs of qubits and the three strings XX, YY, ZZ of one bond commute among themselves, and the same for the bonds $(1,2)$ and $(3,0)$: the even bonds form one group and the odd bonds the other, the familiar even-odd splitting of a spin chain. The Lipkin model splits into its XX, YY and Z parts, and the pairing model into three sets of disjoint $XX+YY$ pairs and the Z part. The Heisenberg ring is the model on which the Trotter error is studied below.

6.6.1 Phase estimation on the models

Table 6.7 runs the algorithm. The system register is prepared in the exact ground state, the time step is that of Table 6.6, the evolution operator is exact, and the control register is varied. For the four-qubit models the controlled powers are applied to the register as the 32×32 matrices a quantum computer would apply; for the eight-qubit Hubbard ring, where

Table 6.7 Phase estimation of the ground-state energies of Table 6.6, with the exact ground state as input and the exact $e^{-i\hat{H}t}$. The error falls by roughly a factor of two per control qubit, plateauing when the modal outcome does not change, and is never larger than half the grid spacing ΔE of Eq. (6.34). Produced by `qpe_models.py`.

model	m	outcome	E_{QPE}	error	probability
Lipkin, $N=4$	4	0101	-4.166667	4.6×10^{-2}	0.990
	6	010100	-4.166667	4.6×10^{-2}	0.848
	8	01010001	-4.218750	5.9×10^{-3}	0.959
	10	0101000100	-4.218750	5.9×10^{-3}	0.486
pairing, $L=4$	4	0100	0.277778	3.6×10^{-1}	0.733
	6	001111	0.572917	6.3×10^{-2}	0.860
	8	00111011	0.646701	1.1×10^{-2}	0.927
	10	0011101101	0.628255	7.3×10^{-3}	0.581
Hubbard ring	4	0010	-1.555556	5.5×10^{-1}	0.879
	6	001001	-2.250000	1.5×10^{-1}	0.861
	8	00100011	-2.076389	2.6×10^{-2}	0.926
	10	0010001101	-2.119792	1.7×10^{-2}	0.585
Heisenberg ring	4	0101	-2.083333	8.3×10^{-2}	0.876
	6	010011	-1.979167	2.1×10^{-2}	0.875
	8	01001101	-2.005208	5.2×10^{-3}	0.875
	10	0100110011	-1.998698	1.3×10^{-3}	0.875

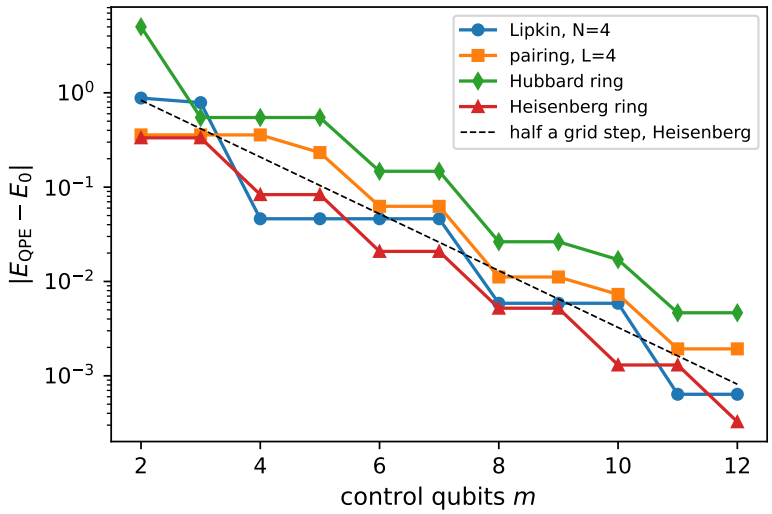


Fig. 6.8 The error of the phase-estimated ground-state energy against the number of control qubits for the four models of Table 6.6, with the exact ground state and the exact evolution operator; the dashed line is half the grid spacing of Eq. (6.34) for the Heisenberg ring. Every curve halves per qubit on average and none crosses its own half-grid line. Produced by `qpe_models.py`.

$m=10$ control qubits make an 18-qubit register, the spectral method of `PhaseEstimation` is used, and the script checks on the Lipkin model that the two give the same distribution.

The pattern is the expected one and it is worth stating plainly: the accuracy is controlled entirely by the size of the control register and the time step, not by the size or difficulty of the physical system. Figure 6.8 extends the table to $m=12$ and shows all four curves falling by a factor of two per qubit on average, each below its own half-grid line. What the difficulty

Table 6.8 Trotterized evolution inside phase estimation on the four-site Heisenberg ring, $m = 8$ control qubits, exact $E_0 = -2$: the energy the algorithm reports, its error, the probability of the modal outcome, the spectral norm of $U_r - e^{-i\hat{H}t}$, and the number of CNOT gates in U_r (the controlled version needs $24r$ more). The first-order error falls as $1/r$ and the second-order error as $1/r^2$; the energy error stops at the resolution of the register. Produced by `qpe_models.py`.

order	r	E_{QPE}	error	probability	$\|U_r - U\|$	CNOTs
1	1	−1.927083	7.3×10^{-2}	0.721	4.76×10^{-1}	24
1	2	−1.979167	2.1×10^{-2}	0.851	2.48×10^{-1}	48
1	4	−2.005208	5.2×10^{-3}	0.667	1.25×10^{-1}	96
1	8	−2.005208	5.2×10^{-3}	0.830	6.25×10^{-2}	192
1	16	−2.005208	5.2×10^{-3}	0.864	3.13×10^{-2}	384
1	32	−2.005208	5.2×10^{-3}	0.872	1.56×10^{-2}	768
2	1	−1.979167	2.1×10^{-2}	0.936	8.73×10^{-2}	46
2	2	−2.005208	5.2×10^{-3}	0.530	2.29×10^{-2}	92
2	4	−2.005208	5.2×10^{-3}	0.803	5.79×10^{-3}	184
2	8	−2.005208	5.2×10^{-3}	0.858	1.45×10^{-3}	368
2	16	−2.005208	5.2×10^{-3}	0.871	3.63×10^{-4}	736
2	32	−2.005208	5.2×10^{-3}	0.874	9.08×10^{-5}	1472

of the Hamiltonian controls is the *cost of each application of* U, through the strings and their weights, and, as Section 6.6.2 shows, the number of Trotter steps. The Heisenberg curve is the lowest not because the model is simplest but because its Λ is smallest, and the Hubbard curve the highest because its $\Lambda = 20$ is the loosest bound in the table: for the four-particle, $S_z = 0$ ground state one is estimating an eigenvalue of magnitude 2.1 on a grid designed for eigenvalues up to 20. The script quantifies what a better bound buys at $m = 10$. With t from the 1-norm the grid is 0.0434 and the error 1.7×10^{-2}; with t from the exact norm 12 the grid is 0.0260 and the error 9.0×10^{-3}; with t from the width of the four-particle sector alone, $\max|E - h_I| = 6.1$, the grid is 0.0132 and the error 2.6×10^{-3}, with the modal probability up from 0.585 to 0.875. The last choice is not available on hardware, since it uses the spectrum one is trying to compute, but a shift of the Hamiltonian by an estimate of the ground-state energy, from Hartree–Fock or from the variational methods of Chapter 7, recovers most of it: the grid one needs is set by the width of the spectrum that actually appears, and Exercise 5 works out the optimal shift. Choosing t and the offset well is part of designing a QPE calculation, not a detail.

6.6.2 The second error: Trotterization

On hardware $e^{-i\hat{H}t}$ is itself built from gates by Eq. (6.35), and this introduces an error with nothing to do with the control register. Table 6.8 and Fig. 6.9 separate the two on the Heisenberg ring, whose twelve strings fall into two commuting groups, using the circuits of `trotter_step` from Chapter 4, 84 gates and 24 CNOTs per first-order step at $t = 0.9425$, with $m = 8$ control qubits and the exact ground state as input. The Trotterized unitary U_r is built as a matrix from the circuit and handed to `PhaseEstimation.from_unitary`, which raises it to the powers 2^j by repeated squaring, exactly as the controlled powers of Fig. 6.5 would repeat the step circuit.

Two errors, one budget. The last column but one of Table 6.8 falls cleanly as $1/r$ for the first-order formula and $1/r^2$ for the symmetric one, the rates of Section 3.5.3. The reported energy follows it down and then stops, at $r = 4$ in first order and $r = 2$ in second, on the

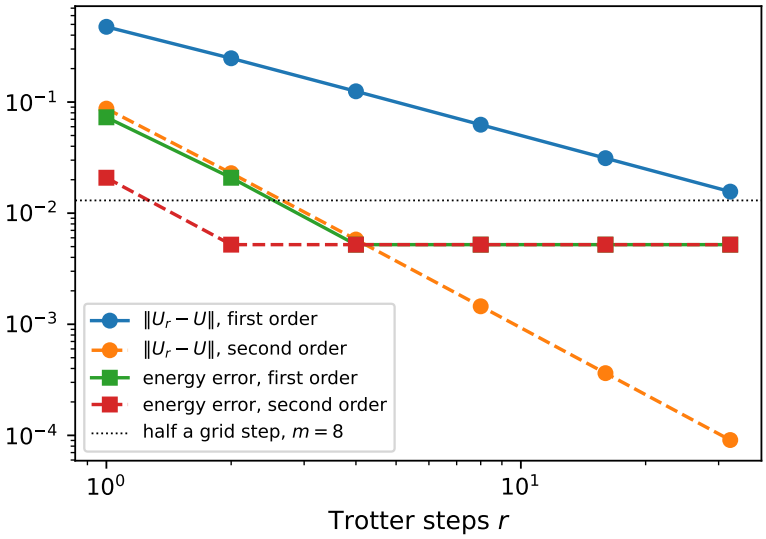


Fig. 6.9 The two error sources of a phase-estimation run on the four-site Heisenberg ring with $t = 0.9425$ and $m = 8$ control qubits, against the number of Trotter steps r on logarithmic axes: the spectral norm of $U_r - U$ (circles) and the error of the reported energy (squares), for the first-order splitting (full lines) and the symmetric one (dashed); the dotted line is half the grid spacing of the register. The operator error falls as $1/r$ and $1/r^2$; the energy error follows it down and then stops at the grid. Produced by `qpe_models.py`.

value -2.005208 that the exact evolution also gives at $m = 8$ (Table 6.7), because the eight-qubit register cannot resolve anything finer than its grid of 0.0104. Beyond that point every additional Trotter step is wasted work: the first-order calculation with $r = 32$ spends 768 CNOT gates per application of U, and 255 applications, to report exactly the same energy as the one with $r = 4$. The two errors must be balanced against each other, not minimized separately, and it is that balance which fixes the circuit depth of a real calculation; Exercise 6 works out the optimal pair (m, r) and shows that the symmetric splitting improves the total gate count from $\mathcal{O}(\varepsilon^{-1})$ to $\mathcal{O}(\varepsilon^{-1/2})$ in the target accuracy ε. Getting the balance wrong in either direction is the commonest way to waste a quantum computer.

The table also shows that a large error in the *operator* is not the same thing as a large error in the *eigenvalue* being measured. At $r = 1$ the first-order U_1 differs from U by 0.48 in spectral norm, which is enormous, and yet the ground-state energy it yields is off by 0.073, one and a half per cent of Λ. The operator norm is a worst case taken over the whole Hilbert space; phase estimation interrogates a single eigenphase, and the singlet ground state of the ring is displaced less than the rest of the spectrum by a splitting that respects the symmetries of the Hamiltonian. Where the crude splitting does show is in the sharpness of the peak, since the input is no longer an exact eigenstate of the operator being powered: the modal probability is 0.72 at $r = 1$ against 0.875 for exact evolution. There is no guarantee of that, however; a less symmetric state, a looser grouping of the strings or a longer evolution time displaces the peak instead of merely broadening it.

Why the Trotter error is fatal here and harmless in unitary coupled cluster. Section 3.6.4 met exactly the same splitting. The unitary coupled-cluster operator cannot be applied in one step either, and its Trotterized form has the same first-order error; yet for the pairing model every Trotter number from one upwards gave the same energy to ten digits once the amplitudes were re-optimized, and the four-parameter circuit of Section 4.7 reproduced the classical UCCD energies to eight digits with a single step. The reason is that the Trotterized ansatz is a *different parametrization of a similar family of states*, and the amplitudes are variational: the optimizer finds whichever values are best for the parametrization it is handed

Table 6.9 Simple trial states for the four models: the overlap $|c_0|^2$ with the exact ground state, the expected number of repetitions, and the probability of reading the bits of E_0 from an $m = 8$ register. Produced by `qpe_models.py`.

model	trial state	$\lvert c_0\rvert^2$	$1/\lvert c_0\rvert^2$	$P(E_0$ bits)
Lipkin, $N = 4$	reference $\lvert 0000\rangle$	0.9358	1.1	0.8973
pairing, $L = 4$	reference $\lvert 1100\rangle$	0.8676	1.2	0.8044
Hubbard ring	Néel determinant $\lvert 10100101\rangle$	0.2410	4.1	0.2368
Heisenberg ring	Néel state $\lvert 0101\rangle$	0.3333	3.0	0.2917

and absorbs the splitting error into them. Nothing of the kind is available to phase estimation. There is no parameter to re-optimize; the circuit is fixed, and what it measures is an eigenphase of whatever unitary was actually built. Trotterizing U does not perturb the answer around the right one, it changes the operator whose spectrum is being measured, and QPE reports the eigenvalue of the wrong operator to full precision. That is why r appears in Table 6.8 as a genuine, irreducible error source, and does not appear in the corresponding table of Chapter 3 at all. The contrast is the difference between the two algorithmic philosophies of this part of the book: a variational method has parameters that absorb the imperfections of its own circuit; a projective method like QPE has none, and every approximation made in constructing the circuit propagates straight into the answer. What one gains in exchange is the exponential precision of Eq. (6.29), which no variational method offers.

6.6.3 The state you start from

One assumption has been carried silently through the whole chapter: that the system register contains an eigenstate. It need not. The eigenstates are complete, so any state may be expanded in them,

$$|\psi_{\text{trial}}\rangle = \sum_j c_j |E_j\rangle, \qquad c_j = \langle E_j|\psi_{\text{trial}}\rangle, \tag{6.38}$$

and Eq. (6.19) then carries every term through the algorithm independently: the controlled powers act on each $|E_j\rangle$ with its own phase, and the control register ends *entangled* with the system register,

$$\sum_j c_j |\widetilde{\phi_j}\rangle |E_j\rangle, \tag{6.39}$$

with $|\widetilde{\phi_j}\rangle$ the peaked control state of Section 6.3.2. Measuring returns the bits of E_j with probability $|c_j|^2$, up to the Dirichlet factor, and collapses the system register onto $|E_j\rangle$. Phase estimation does not average the energies of a superposition, and it does not degrade gracefully: it returns *one* of the eigenvalues, chosen at random with probability $|c_j|^2$, and the run that returns E_0 returns it to the full precision of the register. The overlap does not control the accuracy; it controls how many repetitions are needed, about $1/|c_0|^2$ of them.

Table 6.9 gives the numbers for the cheapest trial states available, the computational basis states of Section 4.5.4. For the Lipkin and pairing models at these couplings the reference determinant has an overlap above 0.85 and a single run succeeds most of the time; for the Hubbard ring at $U = 4t$ the Néel determinant carries a quarter of the ground state, and the Heisenberg Néel state exactly one third, so that four and three runs respectively are needed on average. In every case the probability of reading the ground-state bits is the overlap times the Dirichlet factor of the exact-eigenstate run, to within a per cent. The overlap of a product state with the ground state of an interacting system decreases exponentially with the size of

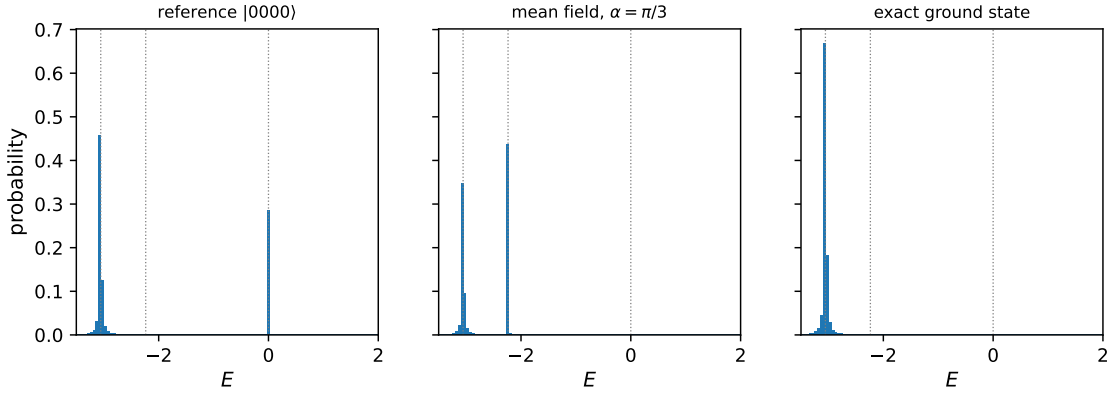


Fig. 6.10 Output histograms of phase estimation on the Lipkin model with $N = 4$, $\varepsilon = 1$, $V = -2/3$, $W = 0$ (coupling $\chi = 2$) and $m = 8$ control qubits, for three input states: the reference $|0000\rangle$, the symmetry-broken mean-field product state of Table 4.10 with $\alpha = \pi/3$, and the exact ground state. The horizontal axis is the energy of Eq. (6.32); the dotted lines mark the five eigenvalues of the $J = 2$ quasispin block, -3.0551, -2.2361, 0, 2.2361, 3.0551. The peaks never move: only their weights change, and no probability appears between them. Produced by `qpe_models.py`.

the system, which is the orthogonality catastrophe of many-body physics, and it is the reason a good approximate eigenstate is valuable even though QPE does not use it variationally: it is what keeps the number of repetitions bounded, and finding such states cheaply is the subject of Chapters 7 and 8.

Figure 6.10 makes the projection visible, on the Lipkin model at the coupling $\chi = 2$ where the mean field of Section 4.5.5 breaks the symmetry. Read the three panels from right to left. With the exact ground state the histogram has a single peak at $E_0 = -3.0551$ carrying 0.894, the rest in the Dirichlet tails. With the reference $|0000\rangle$, whose energy is -2, the histogram has peaks at E_0 and at the third eigenvalue 0 with weights 0.612 and 0.286, the overlaps 0.6845 and 0.2857 times the Dirichlet factor, and nothing at all at -2: the reference has the parity of the ground state and cannot see the odd levels. With the mean-field product state, whose energy -2.5 is lower and which the variational principle therefore prefers, the histogram has peaks at E_0 and at $E_1 = -2.2361$ with weights 0.466 and 0.442, from the overlaps 0.5208 and 0.4440: the deformed state has *less* overlap with the ground state than the reference, not more, because breaking the parity symmetry mixes in the nearly degenerate parity partner, the first excited state. A lower variational energy does not mean a better trial state for phase estimation, and QPE says so at once, by showing the two members of the doublet as two peaks of nearly equal height. This is also what makes the histogram more than a ground-state calculation: accumulated over many runs it is the spectrum weighted by the overlaps, spectroscopy rather than energy minimization, and three of the five $J = 2$ levels are read off the middle panel with no further effort.

Phase estimation as state preparation. The collapse that accompanies the measurement is not a nuisance but a resource. After the control register has been read as the bits of E_0, the system register is left in the state $|E_0\rangle$, up to the small admixture of other eigenstates that the Dirichlet tails allow; the script computes the fidelity of the collapsed register with the exact ground state for all three inputs of Fig. 6.10 and finds 1.0000 in each case. Phase estimation therefore does two things at once: it measures an eigenvalue and it *prepares* the corresponding eigenstate, from any input that overlaps with it, ready to be used for the measurement of other observables. This is the projective view of the algorithm, and Chapter 8 builds a cheaper version of it, the rodeo algorithm, that keeps the projection and drops the digits.

Table 6.10 The two algorithms. The natural workflow uses both: VQE as a state-preparation front end, QPE as the readout.

	VQE	QPE
Output	a variational upper bound	an eigenvalue, to 2^{-m}, and its eigenstate
Circuit depth	shallow	deep, coherent
Needs controlled $e^{-i\hat{H}t}$	no	yes
Cost per digit	$\mathcal{O}(\varepsilon^{-2})$ measurements	one more qubit, twice the applications of U
Limited by	the ansatz and the optimizer	hardware coherence
Trotter error	absorbed by the parameters	propagates into the answer
Era	near-term	fault-tolerant

Where this leads. That last observation is the reason the two great algorithms of quantum many-body computing fit together rather than compete. The variational quantum eigensolver of Chapter 7 prepares a state by minimizing $\langle\psi(\boldsymbol{\theta})|\hat{H}|\psi(\boldsymbol{\theta})\rangle$ over the parameters of a shallow circuit; it tolerates noise, needs no controlled evolution, and gives a variational upper bound, but its accuracy is limited by the ansatz and by the difficulty of the classical optimization. QPE needs deep coherent circuits and controlled $e^{-i\hat{H}t}$, which puts it out of reach of present hardware, and in exchange gives the eigenvalue itself to exponential precision, together with the eigenstate. Table 6.10 sets the two side by side. The combination is the standard long-term plan: use VQE, or any other method in this book, to prepare a state with substantial overlap with the target eigenstate; then use QPE to refine the energy and project onto the eigenstate. Whether this plan delivers an exponential advantage for ground-state energies is a question of the overlaps of Table 6.9 for systems far larger than the ones here, and the evidence is examined critically in [**?**].

6.7 An application: the linear-system algorithm of Harrow, Hassidim and Lloyd

Phase estimation writes an eigenvalue into a register. Everything so far has read that register out; the algorithm of this section leaves it unread and uses it, coherently, to *control a further operation whose strength depends on the eigenvalue*. The operation is a division, and the result is the solution of a linear system of equations. The construction is due to Harrow, Hassidim and Lloyd [**?**], it is the template of a whole class of quantum algorithms that apply a function of a matrix to a state, and it has a direct many-body reading: the inverse of $z-\hat{H}$ is a Green's function.

6.7.1 The problem and the spectral idea

Let A be a Hermitian and invertible $N\times N$ matrix and $\boldsymbol{b}$ a vector, and consider $A\boldsymbol{x}=\boldsymbol{b}$. Many problems of physics and numerical analysis have this form, finite-difference discretizations of differential equations, implicit time steps, and linear-response equations among them, and

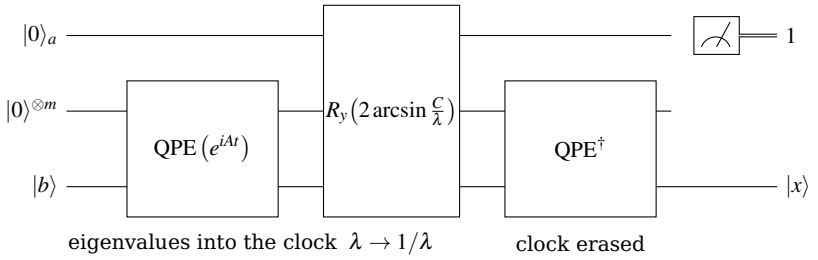


Fig. 6.11 The HHL circuit. Phase estimation of e^{iAt} labels every eigencomponent of $|b\rangle$ with its eigenvalue in the clock register; a rotation of the ancilla controlled by the clock puts the amplitude C/λ on its $|1\rangle$ branch; the inverse of the phase estimation returns the clock to $|0\rangle^{\otimes m}$; and post-selecting the ancilla on $|1\rangle$ leaves the system register in $|x\rangle \propto A^{-1}|b\rangle$.

the classical cost of solving it is $\mathcal{O}(N^3)$ for a dense matrix or $\mathcal{O}(N)$ times the number of iterations for a sparse one. The quantum version of the problem [**?**] encodes $\boldsymbol{b}$ in the amplitudes of an n-qubit register, $N = 2^n$,

$$|b\rangle = \frac{1}{\|\boldsymbol{b}\|}\sum_{i=0}^{N-1} b_i|i\rangle, \tag{6.40}$$

and asks for the state

$$|x\rangle = \frac{A^{-1}|b\rangle}{\|A^{-1}|b\rangle\|}. \tag{6.41}$$

The output is a quantum state, not a classical vector: its 2^n amplitudes are the solution, but they can no more be read out than those of the QFT. What can be extracted is an expectation value $\langle x|O|x\rangle$, an overlap with another state, or a sample of the amplitudes. This is the first entry of the fine print of Section 6.7.6, and it should be kept in mind from the start.

The idea is spectral. Write $A = \sum_j \lambda_j |u_j\rangle\langle u_j|$ and expand $|b\rangle = \sum_j \beta_j |u_j\rangle$ with $\beta_j = \langle u_j|b\rangle$. Then

$$A^{-1}|b\rangle = \sum_j \frac{\beta_j}{\lambda_j}|u_j\rangle, \tag{6.42}$$

so the algorithm has to multiply every eigencomponent of $|b\rangle$ by $1/\lambda_j$. The map $\lambda \mapsto 1/\lambda$ is not unitary, and no circuit implements it directly; but a circuit can implement it *conditionally*, on a branch of an ancilla whose amplitude is proportional to $1/\lambda_j$, provided the eigenvalue is available in a register to condition on. Phase estimation puts it there.

6.7.2 The circuit

The register has three parts: the system register in $|b\rangle$, a clock register of m qubits in $|0\rangle^{\otimes m}$, and one ancilla in $|0\rangle$. Figure 6.11 shows the four steps.

Step 1: phase estimation. Apply the circuit of Fig. 6.5 with $U = e^{iAt}$, whose eigenphases are $\phi_j = \lambda_j t/2\pi$, read with the signed convention of Eq. (6.32) so that negative eigenvalues are allowed. By the linearity of Section 6.6.3 every eigencomponent is labelled with its own eigenvalue,

$$|0\rangle_a|0\rangle^{\otimes m}\sum_j \beta_j|u_j\rangle \;\longmapsto\; |0\rangle_a\sum_j \beta_j|\tilde{\lambda}_j\rangle|u_j\rangle, \tag{6.43}$$

where $|\tilde{\lambda}_j\rangle$ is the clock state peaked at the m-bit approximation of λ_j. This is the moment at which the hidden spectral decomposition of $|b\rangle$ becomes explicit, and it is the only step that touches A. Nothing is measured.

Step 2: the controlled rotation. Choose a constant C no larger than the smallest $|\lambda_j|$ that occurs, and rotate the ancilla, for every value λ that the clock register can hold, by the angle $\theta_\lambda = 2\arcsin(C/\lambda)$,

$$|\tilde{\lambda}\rangle|0\rangle_a \longmapsto |\tilde{\lambda}\rangle\left(\sqrt{1-\frac{C^2}{\lambda^2}}\,|0\rangle_a + \frac{C}{\lambda}\,|1\rangle_a\right), \tag{6.44}$$

since $R_y(\theta)|0\rangle = \cos(\theta/2)|0\rangle + \sin(\theta/2)|1\rangle$. On the superposition (6.43) the rotation acts on every branch with its own eigenvalue, and regrouping by the ancilla,

$$|0\rangle_a \sum_j \beta_j \sqrt{1-\frac{C^2}{\lambda_j^2}}\,|\tilde{\lambda}_j\rangle|u_j\rangle + |1\rangle_a \sum_j \frac{C\beta_j}{\lambda_j}|\tilde{\lambda}_j\rangle|u_j\rangle. \tag{6.45}$$

The $|1\rangle_a$ branch carries exactly the inverse weights of Eq. (6.42). On a quantum computer the rotation is a sequence of R_y gates on the ancilla, each controlled by the bits of the clock register, and its angles are computed classically from the value that each bit pattern represents; $\mathcal{O}(m)$ controlled rotations suffice if $\arcsin(C/\lambda)$ is approximated bit by bit, and a reversible arithmetic circuit does it exactly.

Step 3: uncomputation. Apply the inverse of step 1. On every branch it maps $|\tilde{\lambda}_j\rangle|u_j\rangle$ back to $|0\rangle^{\otimes m}|u_j\rangle$, so the clock register is disentangled from the system,

$$|0\rangle^{\otimes m}\left[|0\rangle_a \sum_j \beta_j \sqrt{1-\frac{C^2}{\lambda_j^2}}\,|u_j\rangle + |1\rangle_a \sum_j \frac{C\beta_j}{\lambda_j}|u_j\rangle\right]. \tag{6.46}$$

This is why the uncomputation matters: without it the eigenvalue labels would remain entangled with the system register, and the superposition $\sum_j(\beta_j/\lambda_j)|u_j\rangle$ would be a mixture rather than a state.

Step 4: post-selection. Measure the ancilla. With probability

$$p_{\text{succ}} = C^2 \sum_j \frac{|\beta_j|^2}{\lambda_j^2} = C^2\left\|A^{-1}|b\rangle\right\|^2 \tag{6.47}$$

the outcome is 1, and the system register is then in $C\sum_j(\beta_j/\lambda_j)|u_j\rangle$ up to normalization, which is $|x\rangle$ of Eq. (6.41). Otherwise the run is discarded and repeated; amplitude amplification reduces the number of repetitions from $1/p_{\text{succ}}$ to $1/\sqrt{p_{\text{succ}}}$. The success probability also carries information: since C is known, p_{succ} gives the norm of the solution, and the classical vector with its scale is $\boldsymbol{x} = \|\boldsymbol{b}\|\sqrt{p_{\text{succ}}}\,|x\rangle/C$.

The condition number. If the eigenvalues lie in $[1/\kappa, 1]$ in suitable units, the largest admissible C is $1/\kappa$ and Eq. (6.47) gives $p_{\text{succ}} \geq 1/\kappa^2$, with equality when $|b\rangle$ is the eigenvector of the largest eigenvalue; `hhl.py` checks this for $\kappa = 2, 5, 10$. The condition number enters twice more: the clock must resolve $1/\kappa$, which costs $\log_2\kappa$ qubits, and the evolution time must be long enough for the smallest eigenvalue to accumulate a phase, $t \sim \kappa$ in the same units, which costs $\mathcal{O}(\kappa)$ applications of a Trotter step. The original algorithm therefore runs in $\mathcal{O}(\kappa^2 \log N/\varepsilon)$ operations for a sparse A and a target accuracy ε, later improved to $\mathcal{O}(\kappa\,\text{polylog}(\kappa/\varepsilon)\log N)$ [**?**], against the $\mathcal{O}(N)$ or worse of any classical method that writes down the solution. The $\log N$ is the exponential advantage; the κ is what usually takes it away.

6.7.3 A two-by-two example

Take

$$A = \begin{pmatrix} 1.5 & 0.5 \\ 0.5 & 1.5 \end{pmatrix}, \qquad \boldsymbol{b} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad \lambda_{1,2} = 1,\ 2, \qquad |u_{1,2}\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ \mp 1 \end{pmatrix}, \tag{6.48}$$

so that $\beta_1 = \beta_2 = 1/\sqrt{2}$, $\kappa = 2$, and the exact solution is $\boldsymbol{x} = (0.75, -0.25)$. Choose $t = \pi/4$, so that the eigenphases are $\phi_1 = 1/8$ and $\phi_2 = 1/4$, exactly representable with $m = 3$ clock qubits, and $C = 1$. `hhl.py` runs the circuit of Fig. 6.11 on the simulator, with the ancilla as qubit 0, the clock as qubits 1 to 3 and the system as qubit 4. After step 1 the clock distribution is 0.5 on $|001\rangle$ and 0.5 on $|010\rangle$, the two eigenvalues with the weights $|\beta_j|^2$ and nothing else. After steps 2 and 3 the ancilla reads 1 with probability $0.625 = C^2(\frac{1}{2}/1 + \frac{1}{2}/4)$, the clock has returned to $|000\rangle$ with probability 1.000000, and the system register is $(0.948683, -0.316228) = (3, -1)/\sqrt{10}$, the exact $|x\rangle$; with the scale from the success probability the classical vector $(0.75, -0.25)$ is recovered to machine precision. Every algebraic step of Section 6.7.2 has been executed as a gate, and the whole calculation is

```
from hhl import HHL
import numpy as np

A = np.array([[1.5, 0.5], [0.5, 1.5]])
hhl = HHL(A, m=3, t=np.pi / 4, C=1.0, window="uniform").run([1.0, 0.0])
print(np.round(hhl.clock_distribution, 6))      # [0. 0.5 0.5 0. 0. 0. 0. 0.]
print(round(hhl.success, 6), round(hhl.clock_reset, 6)) # 0.625 1.0
print(np.round(hhl.x.real, 4), np.round(hhl.solution().real, 4)) # [0.9487 −0.3162] [0.75 −0.25]
```

When the clock is inexact. The example was arranged so that the eigenvalues are exactly representable. For a generic t, here $t = 0.9\pi/2$ from Eq. (6.33), they are not, and two things happen that did not happen in phase estimation. First, the rotation constant must sit below the *grid points that carry weight*, not below $\lambda_{\min}$: the peak of $|\tilde{\lambda}_1\rangle$ straddles λ_1, and the branch at the grid point just below it would need $C/\lambda > 1$ and has to be discarded; `hhl.py` takes $C = 0.9\,\lambda_{\min}$. Second, and more seriously, the tails of the Dirichlet kernel are now multiplied by $C/\tilde{\lambda}$ and are then *not* undone by step 3, because the rotation applied to them was not the rotation the main peak received. Table 6.11 shows the consequence: with the uniform clock state of Fig. 6.5 the fidelity of $|x\rangle$ with the exact solution improves only slowly with m, from 10^{-4} at $m = 6$ to 10^{-6} at $m = 10$, and a few per cent of the successful runs come back with the clock not reset. The remedy, and the reason the original paper does not start the clock in the uniform superposition, is to prepare it instead in the *sine window*

$$|w\rangle = \sqrt{\frac{2}{2^m}} \sum_{k=0}^{2^m-1} \sin\frac{\pi(k+\frac{1}{2})}{2^m}\,|k\rangle, \tag{6.49}$$

whose Fourier transform has tails falling as $1/d^2$ instead of $1/d$ from the peak; the uncomputation then applies the inverse of the preparation. The lower half of Table 6.11 shows the effect: the fidelity error drops to 10^{-12} at $m = 10$ and the clock resets to one part in 10^6. The fat tails of Section 6.3.2 were harmless for reading an energy, where a median takes care of them; inside a coherent algorithm they are the dominant error, and shaping the clock is what removes them.

6.7.4 A differential equation

The advertised applications of the algorithm are the large sparse systems that come from discretized differential equations, and the smallest such system is instructive. The Poisson equation

$$-u''(x) = \sin(\pi x), \qquad x \in (0,1), \qquad u(0) = u(1) = 0, \qquad u(x) = \frac{\sin(\pi x)}{\pi^2}, \tag{6.50}$$

Table 6.11 HHL on the system of Eq. (6.48) with the generic time step $t = 0.9\pi/2$ and $C = 0.9\lambda_{\min}$, against the number of clock qubits, for the uniform clock state and for the sine window of Eq. (6.49): the success probability, the probability that the clock has returned to $|0\rangle^{\otimes m}$, the infidelity of the output state with the exact solution, and the classical solution recovered with its scale (exact $(0.75, -0.25)$). Produced by `hhl.py`.

clock	m	p_{succ}	clock reset	$1 - \lvert\langle x_{\text{exact}} \rvert x\rangle\rvert^2$	$\boldsymbol{x}$
uniform	3	0.408	0.6367	3.3×10^{-1}	$(0.4225, -0.5704)$
uniform	4	0.313	0.6415	4.1×10^{-2}	$(0.6178, -0.0725)$
uniform	6	0.487	0.9371	1.4×10^{-4}	$(0.7381, -0.2365)$
uniform	8	0.498	0.9821	4.8×10^{-5}	$(0.7452, -0.2427)$
uniform	10	0.505	0.9953	1.6×10^{-6}	$(0.7490, -0.2486)$
sine	3	0.360	0.5466	2.4×10^{-1}	$(0.4496, -0.4926)$
sine	4	0.299	0.5179	2.3×10^{-3}	$(0.5853, -0.1642)$
sine	6	0.508	0.9984	4.0×10^{-7}	$(0.7509, -0.2508)$
sine	8	0.506	0.9999	5.6×10^{-10}	$(0.7501, -0.2500)$
sine	10	0.506	1.0000	2.0×10^{-12}	$(0.7500, -0.2500)$

discretized on N interior points $x_i = ih$, $h = 1/(N+1)$, with the centred second difference, becomes $A\boldsymbol{u} = \boldsymbol{b}$ with

$$A = \frac{1}{h^2}\begin{pmatrix} 2 & -1 & & \\ -1 & 2 & \ddots & \\ & \ddots & \ddots & -1 \\ & & -1 & 2 \end{pmatrix}, \qquad b_i = \sin(\pi x_i), \qquad \lambda_k = \frac{4}{h^2}\sin^2\frac{k\pi}{2(N+1)}, \tag{6.51}$$

a symmetric positive-definite tridiagonal matrix with known spectrum and condition number $\kappa \approx 4(N+1)^2/\pi^2$, growing with the resolution of the grid. For $N = 4$ the system register is two qubits, the eigenvalues are 9.549, 34.549, 65.451, 90.451 and $\kappa = 9.47$; the finite-difference solution differs from $\sin(\pi x)/\pi^2$ by 3.2×10^{-3}, the discretization error of a four-point grid. Table 6.12 and Fig. 6.12 show HHL with the sine window and $t = 0.9\pi/\lambda_{\max}$ against the number of clock qubits. The fidelity of $|x\rangle$ with the exact solution is 1.000000 in every row, for a reason that says something about the problem rather than the algorithm: $\sin(\pi x)$ is very nearly the lowest eigenvector of A, so $|b\rangle$ and $|x\rangle$ are the same state and the only information in the answer is its *norm*, $1/\lambda_1$, which the success probability carries. The recovered solution therefore improves with m through p_{succ} alone, from an error of 3.5×10^{-2} against the finite-difference vector at $m = 4$ to 1.4×10^{-5} at $m = 10$, where it has long since reached the 3.2×10^{-3} floor of the discretization. A quantum linear solver does not remove the truncation error of the discretization it is handed; it only refines the solution of the discrete system, and the two errors add exactly as the two errors of Section 6.6.2 did.

6.7.5 Resolvents, Green's functions and linear response

For a many-body physicist the matrix to invert is not a discretized Laplacian but $z - \hat{H}$. The resolvent

$$G(z) = (z - \hat{H})^{-1} = \sum_j \frac{|E_j\rangle\langle E_j|}{z - E_j} \tag{6.52}$$

is the object behind every Green's function, and the quantity most often wanted is a diagonal matrix element in a state $|b\rangle = \hat{O}|\Psi_0\rangle$ made by acting with an excitation operator on the ground

Table 6.12 HHL for the discretized Poisson equation of Eq. (6.51) with $N = 4$, two system qubits, the sine window and $t = 0.9\pi/\lambda_{\max}$: the success probability, the largest deviation of the recovered solution from the finite-difference solution $\boldsymbol{u}_{\mathrm{FD}}$, and from the exact $u(x)$, against the number of clock qubits. The discretization error alone is 3.2×10^{-3}. Produced by `hhl.py`.

m	p_{succ}	$\max\lvert u_{\mathrm{HHL}} - u_{\mathrm{FD}}\rvert$	$\max\lvert u_{\mathrm{HHL}} - u\rvert$
4	0.345	3.5×10^{-2}	3.1×10^{-2}
5	0.244	4.5×10^{-2}	4.2×10^{-2}
6	0.726	5.3×10^{-3}	2.1×10^{-3}
7	0.751	3.7×10^{-3}	4.9×10^{-4}
8	0.810	2.6×10^{-5}	3.3×10^{-3}
9	0.811	4.7×10^{-5}	3.3×10^{-3}
10	0.810	1.4×10^{-5}	3.3×10^{-3}

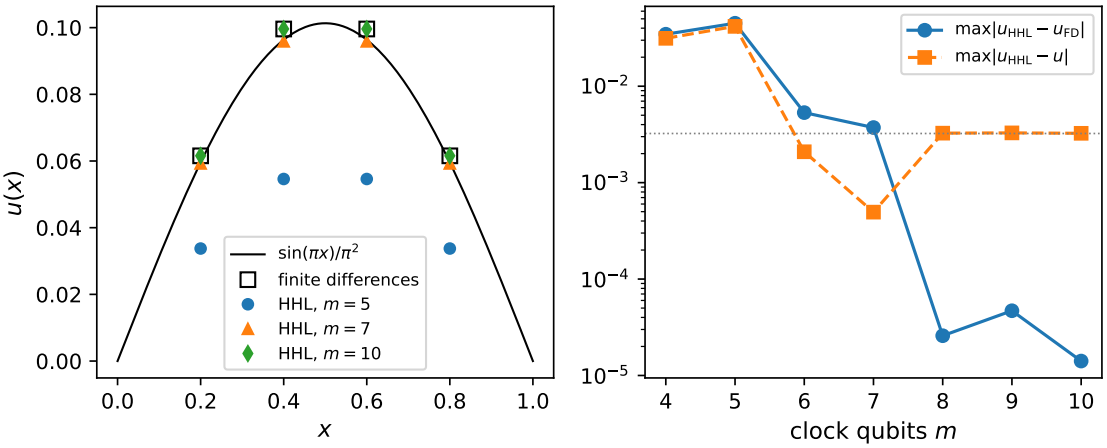


Fig. 6.12 Left: the exact solution $\sin(\pi x)/\pi^2$ of Eq. (6.50), the finite-difference solution on four interior points, and the HHL solution with $m = 5$, 7 and 10 clock qubits, rescaled with the success probability. Right: the deviation of the HHL solution from the finite-difference vector and from the exact function against m; the dotted line is the discretization error. Produced by `hhl.py`.

state,

$$G_b(z) = \langle b|(z - \hat{H})^{-1}|b\rangle = \sum_j \frac{|\langle E_j|b\rangle|^2}{z - E_j}, \tag{6.53}$$

whose poles are the excitation energies and whose residues are the transition strengths; with $z = \omega + i\eta$ its imaginary part is the spectral or strength function $S(\omega) = -\frac{1}{\pi}\mathrm{Im}\, G_b(\omega + i\eta)$ that response experiments measure. The linear response of a system to a weak external field, the random-phase approximation and the time-dependent Hartree–Fock equations all reduce, in the frequency domain, to a linear system $(\omega - \mathscr{L})\,\delta\rho = s$ with a source s and a linearized Hamiltonian or Liouvillian $\mathscr{L}$, that is, to the application of a resolvent to a source state. Solving $(z - \hat{H})|x\rangle = |b\rangle$ is therefore not an application of HHL borrowed from numerical analysis; it is the many-body problem in its own frequency-domain form, and the algorithm implements the spectral map $E_j \mapsto 1/(z - E_j)$ exactly as Eq. (6.42) implements $\lambda_j \mapsto 1/\lambda_j$. The matrix element itself comes out of the run: since the post-selected branch of Eq. (6.46) is $|\tilde{x}\rangle = C\sum_j (\beta_j/\lambda_j)|u_j\rangle$ before normalization,

$$G_b(z) = \frac{\langle b|\tilde{x}\rangle}{C} = \frac{\sqrt{p_{\mathrm{succ}}}\,\langle b|x\rangle}{C}, \tag{6.54}$$

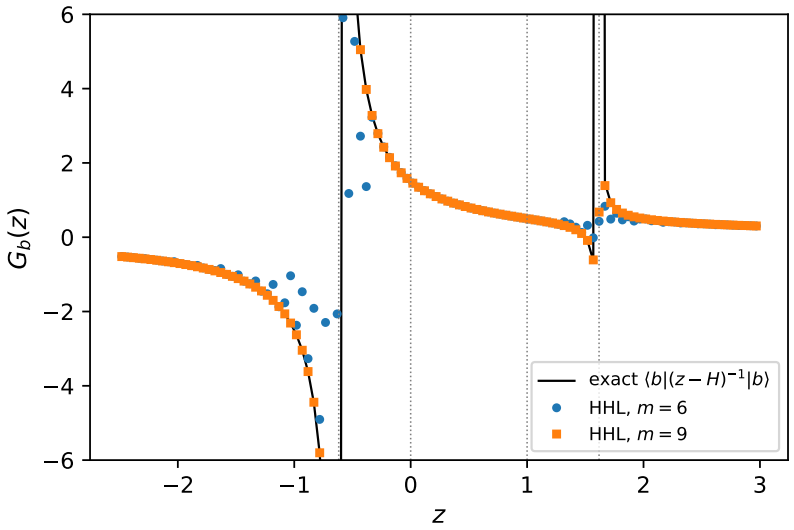


Fig. 6.13 The resolvent matrix element $G_b(z) = \langle b|(z-\hat{H})^{-1}|b\rangle$ of the two-level pairing Hamiltonian of Eq. (4.17) in the reference state $|b\rangle = |10\rangle$, exact and from HHL runs with $m = 6$ and $m = 9$ clock qubits at 110 values of z; the dotted lines are the four eigenvalues, of which the two in the one-pair sector, -0.618 and 1.618, are the poles. Produced by `hhl.py`.

a success probability and one overlap, which a Hadamard test provides on hardware.

Figure 6.13 does this for the two-level pairing Hamiltonian of Eq. (4.17) and the reference state $|b\rangle = |10\rangle$, one pair in the lower level, whose overlaps with the one-pair eigenstates are 0.9472 and 0.0528. For each of 110 values of z the matrix $A = z - \hat{H}$ is indefinite, which the signed clock of Eq. (6.32) handles without further ado, and the four eigenvalues of A differ from one z to the next, so every run has its own t and C. With $m = 9$ the curve is reproduced to better than one per cent everywhere except within a few grid steps of the poles, -1.0909 against the exact -1.0909 at $z = -1.5$ and 0.3636 at $z = 2.5$; with $m = 6$ the errors are of a few per cent. Near a pole the smallest eigenvalue of $z - \hat{H}$ goes to zero, κ diverges, the success probability with it, and the clock cannot resolve the eigenvalue: this is the regularization η of the Green's function reappearing as the resolution of the register, and it is the practical face of the κ-dependence of the cost. Away from the poles the success probability is between 0.1 and 0.9 and one run in a few suffices. The calculation is small, two system qubits and nine clock qubits, but it is the complete workflow of a frequency-domain response calculation on a quantum computer, and the same circuit with the four-qubit Hamiltonians of Table 6.6 and the excitation operators of Chapter 3 is Exercise 9.

6.7.6 The fine print

The exponential advantage of the algorithm is real, but it holds under conditions that are seldom all met at once, and the list of them is worth having [**?**]. The vector $\boldsymbol{b}$ must be loadable into a register in $\mathcal{O}(\log N)$ time, which is possible for structured inputs such as the smooth right-hand side of Eq. (6.50) or a state produced by a circuit, and impossible in general: loading N arbitrary numbers costs $\mathcal{O}(N)$ gates, which already cancels the advantage. The matrix A must be sparse, or more generally efficiently simulable, so that e^{iAt} is a short circuit; the Pauli sums of Chapter 4 qualify. The condition number κ must be small, since the cost is at least linear in it and the success probability quadratically small in it; discretized differential equa-

tions have κ growing with the number of grid points, which is exactly the regime in which one would want them solved. And the output is a state: reading all of $\boldsymbol{x}$ costs $\mathcal{O}(N)$ measurements, so only expectation values, overlaps and samples are available cheaply, as in Eq. (6.54). Finally, the comparison must be with the best classical algorithm for the same restricted task; when $\boldsymbol{b}$ is loadable in the sense above and only samples of $\boldsymbol{x}$ are wanted, classical randomized algorithms working in the same input model achieve polynomial rather than exponential separations for some matrix classes [**?**]. None of this makes the algorithm less important. It is the prototype of the *quantum singular value transformation* and of the block-encoding methods that Chapter 5 mentions, in which an arbitrary function of a Hamiltonian, not only $1/\lambda$ but $e^{-\beta\lambda}$, a step function or a filter, is applied to a state through an ancilla; the rodeo algorithm of Chapter 8 is another member of the family, with the ancilla measurement used to filter energies rather than to invert them. What the fine print says is that the many-body applications in which the input is a state one can prepare, the matrix is a Hamiltonian one can simulate, and the output is an expectation value, which is exactly the setting of Section 6.7.5, are the ones in which the advantage survives.

6.8 Summary

The quantum Fourier transform is the map $|x\rangle \mapsto N^{-1/2}\sum_k e^{2\pi i x k/N}|k\rangle$. Its whole theory follows from one rewriting: the phase factors into binary fractions, Eq. (6.8), so that the output is an unentangled product of one-qubit phase states. That factorization gives the circuit, Hadamards, controlled R_k and a final reversal, a gate count of $\mathcal{O}(n^2)$ against $\mathcal{O}(4^n)$ for the matrix it represents, and an approximate version at $\mathcal{O}(n\log n)$. What it does not give is a way to read the transform out: the answer is in the amplitudes.

Quantum phase estimation is what makes that acceptable, because it needs only one number. Controlled powers of U write the eigenphase into a control register as a Fourier state, Eq. (6.22); the inverse QFT decodes it into binary, exactly when the phase is a dyadic rational and with the Dirichlet-kernel probabilities of Eq. (6.27) otherwise. With m control qubits the resolution is 2^{-m}, one more digit per qubit, at the price of doubling the number of applications of U; a single recycled control qubit, with classical feedback, does the same job. For physics the unitary is $e^{-i\hat{H}t}$ and the phase is $-Et/2\pi$ read as a signed fraction. Two consequences follow. The period forces a choice of t from a bound on the spectrum, and a loose bound, or an identity term left in the Hamiltonian, wastes register. And $e^{-i\hat{H}t}$ must be built by Trotterization, whose error is independent of the register and must be balanced against it, as Table 6.8 shows; unlike the variational Trotter error of Chapter 3, nothing absorbs it. Applied to the models of Chapter 4, the algorithm recovers the exact ground-state energies to 10^{-3} with ten control qubits on a well-chosen grid, and the difficulty of the Hamiltonian shows up in the cost of each U, not in the achievable accuracy.

Phase estimation does not require the exact eigenstate, only overlap with it, and what the overlap buys is repetitions rather than precision: the histogram of outcomes is the spectrum weighted by the overlaps, and the register left behind after a successful readout is the eigenstate. That is the hinge between this chapter and the two that follow, the variational preparation of states with large overlap in Chapter 7 and the filtering of states by measurement in Chapter 8. Finally, the eigenvalue register need not be read at all. Used coherently to control a rotation, it applies $1/\lambda$ to every eigencomponent of a state, which is the HHL algorithm for linear systems and, with $A = z - \hat{H}$, a quantum calculation of a Green's function. Its cost is dominated by the condition number, its output is a state rather than a vector, and it inherits from phase estimation a sensitivity to the tails of the Dirichlet kernel that a shaped clock register removes.

6.9 Exercises

1. **The quantum Fourier transform.** (a) Show that $\boldsymbol{F}_N$ of Eq. (6.2) is unitary, using $\sum_j \omega^{j(k-k')} = N\delta_{kk'}$. (b) Carry out the derivation of Eq. (6.8) in full, justifying the interchange of sum and product and the step in which only the fractional part of $x/2^m$ survives. (c) Repeat the two-qubit calculation of Section 6.2.1 for three qubits, and identify explicitly which products $x_j k_m$ are integers and therefore drop out. (d) Show that the QFT of a computational basis state is always a product state, and that the QFT of a general state need not be. What does that say about where the entanglement in phase estimation comes from? (e) Show that QFT_N has order four, $\mathrm{QFT}_N^4 = \mathbb{1}$, and find its eigenvalues.
2. **Circuits, cost and conventions.** (a) Verify that the circuit of Fig. 6.1 implements Eq. (6.12), by following $|x_1 x_2\rangle$ through gate by gate for all four inputs. (b) Show that a controlled R_k is symmetric in its two qubits, and that the Hadamard is the $k = 1$ member of the family R_k combined with a basis change. (c) Count the gates in the general circuit and derive Eq. (6.17). (d) Show that reversing the gate order and conjugating each R_k gives the inverse transform. (e) Estimate the error of the approximate QFT when all rotations with $k - j + 1 > d$ are dropped, and show that it is bounded by $2\pi n 2^{-d}$; hence justify the choice $d \sim \log n$ and the $\mathcal{O}(n \log n)$ gate count. (f) Using `qft.py`, omit the swaps in the inverse QFT of a phase estimation and see what the measured phase becomes.
3. **Phase estimation.** (a) Derive Eq. (6.22), being explicit about why the system register factors out. (b) Show that Eq. (6.22) is exactly $\mathrm{QFT}|k'\rangle$ when $\phi = k'/2^m$. (c) Derive the Dirichlet kernel of Eq. (6.27) and the bound $4/\pi^2$ of Eq. (6.28), and show that the two nearest outcomes together carry at least $8/\pi^2$. (d) Show that to obtain ϕ to within 2^{-b} with probability at least $1 - \varepsilon$ one needs $m = b + \lceil \log_2(2 + 1/2\varepsilon) \rceil$ control qubits. (e) Show that the iterative scheme of Section 6.3.3 produces the same bits as the inverse QFT, by identifying the classical phase corrections with the controlled $R_k^\dagger$ of the inverse transform. (f) Why does a median over repeated runs convert the constant success probability into an exponentially small failure probability, and why would a mean not?
4. **Controlled evolution.** (a) Show that a controlled $e^{-i\theta P}$ equals $e^{-i\theta P/2} e^{i\theta Z_c P/2}$ and hence that Fig. 6.7 is correct. (b) Count the gates of a controlled first-order Trotter step of the two-level pairing Hamiltonian of Eq. (4.17) and of the Heisenberg ring, starting from Table 4.9. (c) The controlled powers U^{2^j} can be built by repeating the step circuit or by Trotterizing $e^{-i\hat{H}2^j t}$ afresh with $2^j r$ steps; are the two the same circuit? (d) For the one-qubit Hamiltonian of Eq. (6.36), write $e^{-i(Z-X/2)t}$ as a rotation about an axis of the Bloch sphere and give its controlled version as a circuit of two CNOT gates and single-qubit rotations. Check with the `Circuit` class.
5. **Energies, aliasing and the time step.** (a) Derive Eq. (6.31) and explain the modulus. (b) Two eigenvalues E and $E + 2\pi/t$ give the same phase. Show that choosing $t < \pi/\|\hat{H}\|$ with the signed reading of Eq. (6.32) removes the ambiguity, and that shifting $\hat{H} \to \hat{H} - \lambda\mathbb{1}$ lets one use the range more efficiently. What is the optimal λ if the spectrum lies in $[E_{\min}, E_{\max}]$, and how many control qubits does it save for the Hubbard ring of Table 6.6? (c) Show that $\Lambda = \sum_\alpha |h_\alpha| \geq \|\hat{H}\|$, and give an example in which the bound is loose by a factor growing with the system size. (d) Reproduce Table 6.4 with the identity kept in the Hamiltonian and t chosen from the full 1-norm, and explain row by row what was lost.
6. **Two errors, one budget.** (a) Starting from Eq. (3.44), show that the first-order Trotter error over a time t with r steps is bounded by $\frac{t^2}{2r} \sum_{\alpha<\beta} \|[H_\alpha, H_\beta]\|$, and hence that grouping commuting terms costs nothing. Evaluate the bound for the Heisenberg ring from Table 4.1 and compare with Table 6.8. (b) Given a target energy error ε, balance the register error $\Delta E/2$ against the Trotter error $\mathcal{O}(t^2/r)$ and find the optimal (m, r); repeat for the symmetric splitting and show that the total number of controlled Pauli rotations improves from $\mathcal{O}(\varepsilon^{-1})$ to $\mathcal{O}(\varepsilon^{-1/2})$. (c) In Table 6.8 the reported energy at $r = 1$ is far more accurate than $\|U_1 -$

$U\|$ suggests. Show that the first-order error of an *eigenvalue* of U_r is first order in the commutators projected onto the eigenstate, and explain why the singlet ground state of the ring is favoured. (d) Section 3.6.4 found that Trotterizing the unitary coupled-cluster operator costs nothing once the amplitudes are re-optimized. Construct a hybrid in which a variational parameter is inserted into the QPE circuit, and explain precisely what goes wrong.

7. **The models.** (a) Reproduce Table 6.7, then apply QPE to the first excited state of the Lipkin model and confirm that the algorithm is indifferent to which eigenstate it is given. (b) The pairing Hamiltonian of Eq. (4.16) on four qubits contains all pair numbers at once. Feed QPE the reference $|1100\rangle$ and explain why no outcome outside the two-pair sector ever appears. (c) For the Hubbard ring, replace the Néel determinant of Table 6.9 by the Hartree–Fock determinant of Section 3.5.5 and compare the overlaps. (d) Estimate the resources for a calculation one would actually want: the pairing model with ten levels and five pairs, to an accuracy of 10^{-3} in units of ξ. How many qubits, how many Trotter steps, how many controlled Pauli rotations? Compare with the size of the seniority-zero matrix of Section 3.3.2.
8. **Spectroscopy and state preparation.** (a) Reproduce Fig. 6.10 and extract the three visible eigenvalues of the $J = 2$ block from the middle panel, with their weights. (b) Show that the reference $|0000\rangle$ is an eigenstate of $Z_0Z_1Z_2Z_3$, the parity of the number of raised doublets, that the Lipkin Hamiltonian commutes with it, and hence that only every other level of the $J = 2$ block appears in the left panel. (c) Compute the overlap of the mean-field product state with the ground state as a function of χ and locate the coupling above which it falls below that of the reference. (d) After reading the bits of E_0, the collapsed system register has a fidelity with $|E_0\rangle$ that the script reports as 1.0000. Show that the admixture of $|E_j\rangle$ is controlled by the Dirichlet tail at the distance $|E_j - E_0|/\Delta E$, and find the smallest m for which the fidelity exceeds 0.99 when the input is the mean-field state.
9. **HHL.** (a) Verify the algebra of Section 6.7.2 on the example of Eq. (6.48) by hand: the eigenvector expansion of $|b\rangle$, the two rotation angles, the post-selected state and the success probability. (b) Show that $p_{\mathrm{succ}} \geq 1/\kappa^2$ when $C = \lambda_{\min}$ and the eigenvalues lie in $[\lambda_{\min}, \kappa\lambda_{\min}]$, with equality when $|b\rangle = |u_{\max}\rangle$. (c) Show that the Fourier transform of the sine window of Eq. (6.49) has tails falling as $1/d^2$, and explain with Eq. (6.45) why the uniform clock leaves a residual error that decreases only slowly with m. (d) Run the Poisson example with $N = 8$ and $N = 16$ interior points; how do κ, the required m and the success probability change? (e) Repeat the resolvent calculation of Fig. 6.13 for the four-qubit pairing Hamiltonian of Table 6.6 with the source state $\hat{P}_3^+\hat{P}_1^-|\Psi_0\rangle$, the pair excitation of Chapter 3 acting on the exact ground state, and identify the poles.
10. **Numerical explorations.** (a) Reproduce Table 6.2 and extend it to $n = 8$ and $n = 10$; at what d does the error fall below 10^{-3}, and how does that d grow with n? (b) Verify the $4/\pi^2$ bound numerically by scanning ϕ across a whole grid interval and recording the worst case. (c) Reproduce Table 6.8 with the Trotter steps replaced by the exact exponentials of the two commuting groups, and explain the result. (d) Implement Bayesian phase estimation for the one-qubit Hamiltonian of Eq. (6.36): a grid of prior probabilities for E, single-shot measurements with random powers and angles, Bayes' updates, and the posterior width against the number of shots.

Programs for this chapter

Every number and figure in this chapter comes from the programs in `BookPrograms/chapter6/`, which build on the shared modules of Chapters 2–4 (`statevector`, `circuit`,

`gates`, `pauli`, `models`) and on `mapped_models.py` and `pauli_circuits.py` of Chapter 4. All of them are imported by the notebook `Notebooks/chapter6/chapter6.ipynb`, and all run on `numpy` and `matplotlib` alone.

`qft.py` The Fourier matrix of Eq. (6.2), the circuit of Section 6.2.2 as a `Circuit`, with and without the swaps and with the rotation ladder truncated, the product form of Eq. (6.8) built without a circuit, the four verifications, the worked example of Fig. 6.3, Tables 6.1 and 6.2, Fig. 6.4 and the bit-reversal check. About one second.

`phase_estimation.py` The class `PhaseEstimation` running the circuit of Fig. 6.5 on the state-vector simulator, by controlled matrices or in the eigenbasis of U, the Dirichlet kernel of Eq. (6.27), the signed phase-to-energy conversion and the time step of Eqs. (6.32) and (6.33), iterative phase estimation with one control qubit, the exact example $\phi = 5/8$, Table 6.3, Fig. 6.6 and the numerical check of the $4/\pi^2$ and $8/\pi^2$ bounds. About one second.

`qpe_models.py` Phase estimation on the one- and two-qubit Hamiltonians of Section 6.5, Tables 6.4 and 6.5; the four models of Table 6.6, Table 6.7 and Fig. 6.8, with the comparison of time steps for the Hubbard ring; the Trotter budget of Table 6.8 and Fig. 6.9; the trial states of Table 6.9, the Lipkin histograms of Fig. 6.10 and the fidelity of the collapsed register. Every energy is checked against Chapters 3 and 4. About ten seconds.

`hhl.py` The class `HHL` implementing the circuit of Fig. 6.11 with a uniform or a sine-window clock, the example of Eq. (6.48) and Table 6.11, the $1/\kappa^2$ check, the Poisson equation of Table 6.12 and Fig. 6.12, and the resolvent of Fig. 6.13. About five seconds.

Chapter 7
Variational Quantum Algorithms for Many-Body Systems

Abstract In this chapter we explore variational quantum algorithms as a practical approach for near-term devices. We explain the Variational Quantum Eigensolver (VQE): preparing a parametrized quantum state (ansatz), measuring its energy, and using a classical optimizer to minimize the expectation value. Common ansätze (hardware-efficient circuits, or problem-inspired ones like unitary coupled-cluster) are discussed, as well as measurement strategies for obtaining Hamiltonian expectation values. Practical issues such as noise, barren plateaus, and error mitigation strategies are addressed. We also mention related approaches such as the Quantum Approximate Optimization Algorithm (QAOA) and strategies for targeting excited states using variational subspace methods.

7.1 Introduction

Phase estimation, the subject of Chapter 6, returns an eigenvalue to a precision that grows exponentially with the size of a register, and it needs for that a coherent circuit whose depth grows in the same proportion, together with a controlled version of $e^{-i\hat{H}t}$. On the hardware of the present decade neither is available, and the tables of that chapter were produced on a simulator whose state vector never decohered. This chapter is about what can be done when the circuit must be shallow, the gates are noisy, and the only operation that can be repeated a million times is a short preparation followed by a measurement. The answer is the oldest idea in many-body physics, the variational principle of Chapter 3, run with a quantum computer in the place where the trial state is evaluated: the *variational quantum eigensolver*, VQE [**?**, **?**].

The idea is simple enough to state in one sentence. A parametrized circuit prepares a state $|\psi(\boldsymbol{\theta})\rangle$; the quantum computer measures its energy $\langle\psi(\boldsymbol{\theta})|\hat{H}|\psi(\boldsymbol{\theta})\rangle$ one Pauli string at a time, exactly as in Section 4.6; a classical optimizer moves the parameters downhill; and the minimum is an upper bound on the ground-state energy. Everything difficult about the method is hidden in the words *circuit*, *measures* and *optimizer*, and the chapter takes them in that order. The circuit, the ansatz, decides what states are reachable at all, and the central lesson of the calculations below is that the quality of a variational result is a property of the ansatz rather than of the algorithm: a circuit that knows nothing about the Hamiltonian can converge smoothly to a wrong answer, and one that respects its symmetries can be exact with a handful of parameters. The measurement decides the cost, which is dominated not by the gates but by the number of repetitions needed to beat down the statistical noise of the estimate, and there is a way of reading the qubits that looks reasonable and produces a number that is not the energy of any state. The optimizer needs gradients, which the parameter-shift rule supplies exactly from two shifted energies when the gate generators have two eigenvalues,

and it must cope with a landscape that flattens exponentially with the number of qubits when the circuit is too generic.

The main business of the chapter is Sections 7.6 and 7.7, where the algorithm is run on the same Hamiltonians that Chapter 3 treated with configuration interaction, Hartree–Fock and coupled cluster and Chapter 6 with phase estimation: the tapered one-qubit and the two-mode Hamiltonians, then the Lipkin model, the pairing model, the Hubbard ring and the Heisenberg ring of Table 6.6. The results are set against the phase-estimation energies of Table 6.7, and the two algorithms are weighed against each other. Section 7.8 then lets the algorithm choose its own circuit, one operator at a time from a pool, which is the adaptive VQE, and finds the exact ground state of the Heisenberg ring with a single operator once the right reference and the right pool are supplied. Section 7.9 closes the chapter with a second variational algorithm that is not about Hamiltonians of physics at all: the quantum approximate optimization algorithm, QAOA, which encodes a combinatorial problem in a diagonal Hamiltonian and alternates it with a mixing term. It belongs here because it is a VQE with a very particular ansatz, because its ansatz is the Trotterized adiabatic evolution of Chapter 5, and because it is the point of contact between the methods of this book and quantum machine learning, of which we say a few words and no more. The programs are in `BookPrograms/chapter7/`: the ansatz classes and the VQE loop, the measurement and gradient checks, the model calculations, an `AdaptVQE` class and a `QAOA` class. Every number quoted below is printed by one of them.

7.2 The variational principle on a circuit

The foundation is the inequality used by every approximate method of Chapter 3. For a Hamiltonian $\hat{H}$ with ground-state energy E_0 and any normalized state $|\psi\rangle$,

$$E[\psi] = \langle\psi|\hat{H}|\psi\rangle \geq E_0, \tag{7.1}$$

with equality if and only if $|\psi\rangle$ is a ground state. The proof is one line in the eigenbasis: with $|\psi\rangle = \sum_j c_j|\phi_j\rangle$ and $\hat{H}|\phi_j\rangle = E_j|\phi_j\rangle$,

$$\langle\psi|\hat{H}|\psi\rangle = \sum_j |c_j|^2 E_j \geq E_0 \sum_j |c_j|^2 = E_0, \tag{7.2}$$

since every $E_j \geq E_0$ and the weights sum to one. Nothing about this is quantum-computational; it is a statement about weighted averages. What changes on a quantum computer is which states are available. Restricting $|\psi\rangle$ to a family $|\psi(\boldsymbol{\theta})\rangle$ labelled by real parameters turns Eq. (7.1) into a minimization,

$$E_{\mathrm{VQE}} = \min_{\boldsymbol{\theta}} \langle\psi(\boldsymbol{\theta})|\hat{H}|\psi(\boldsymbol{\theta})\rangle \geq E_0, \tag{7.3}$$

and that is the whole algorithm. The family is generated by a parametrized circuit acting on a reference state that is easy to prepare,

$$|\psi(\boldsymbol{\theta})\rangle = U(\boldsymbol{\theta})\,|\Phi_0\rangle, \tag{7.4}$$

so that the trial states are exactly those a quantum computer can prepare in a few layers of gates. This is the point of the exercise. A classical variational calculation is limited to states whose amplitudes can be written down and summed over: the Hartree–Fock determinant of Section 3.5 is a product state in a rotated basis, and the coupled-cluster wave function of Section 3.6 is an exponential whose evaluation had to be organized into the amplitude

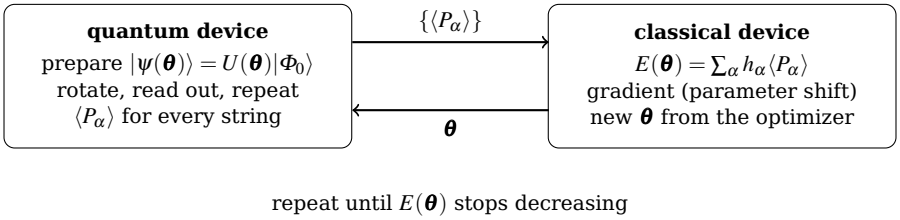


Fig. 7.1 The hybrid loop. The quantum device does one thing, the preparation and measurement of $|\psi(\boldsymbol{\theta})\rangle$, and does it many times per iteration; the classical device assembles the energy from the string expectation values and proposes the next parameters.

equations of Section 3.6.2 precisely because the state itself cannot be stored. A circuit of depth d on n qubits prepares a state with 2^n amplitudes, correlated in ways that no compact classical parametrization reproduces, using $\mathcal{O}(nd)$ parameters, and its energy is obtained without ever writing the amplitudes down. The hope of VQE rests on the belief that such states are useful trial functions for physical Hamiltonians. The belief is plausible, and largely unproven.

Two remarks about Eq. (7.3) are worth making at once, because the whole chapter turns on them. The first is what the bound is worth. It says that the answer lies above E_0; it does not say that it lies close. If the family contains nothing near the ground state, the optimizer converges beautifully to a number that is simply too high, and its convergence tells us nothing about the quality of the answer. This is the difficulty of every variational method in this book, since a Hartree–Fock calculation also converges and also converges to the wrong energy, and the remedies are the same: enlarge the family and watch whether the number moves, compare families of different structure, and check against exact results wherever the system is small enough to allow it. The second is what the bound gives for free. Two variational numbers can be compared, and the lower one is the better one, with no knowledge of the exact answer. Phase estimation offers nothing of the kind: Table 6.5 contains an estimate *below* the true ground-state energy, which is a perfectly normal QPE result and one that no variational calculation could ever produce.

The loop. As executed, the algorithm is the following.

1. Choose an ansatz $U(\boldsymbol{\theta})$, a reference $|\Phi_0\rangle$ and initial parameters $\boldsymbol{\theta}^{(0)}$.
2. On the quantum device, prepare $|\psi(\boldsymbol{\theta})\rangle$ and measure enough Pauli strings, enough times, to reconstruct $E(\boldsymbol{\theta}) = \langle\psi(\boldsymbol{\theta})|\hat{H}|\psi(\boldsymbol{\theta})\rangle$.
3. On the classical device, use E and, if available, its gradient to propose $\boldsymbol{\theta}^{(k+1)}$.
4. Repeat until the energy stops decreasing.

Steps 2 and 3 alternate hundreds or thousands of times, and Fig. 7.1 draws the division of labour. Step 2 is where the quantum computer earns its place and where almost all of the cost lies; the classical side does what any optimizer does. Most of this chapter is about how much step 2 costs, how it can go wrong, and how the ansatz of step 1 decides whether the loop is worth running at all.

The reader has met this loop twice already. Section 2.10 minimized the energy of a two-level system over the Bloch angles of a one-qubit state, Eq. (2.91), and Section 4.7 minimized the seventeen-string pairing Hamiltonian over the four angles of the pair-excitation circuit of Eq. (4.25), reproducing the unitary coupled-cluster energies of Table 3.15 digit for digit. Both were VQE calculations with the expectation value taken exactly on the simulator. What this chapter adds is the general machinery, the ansätze that are not tied to one model, the statistics of the readout and the gradients, and the systematic application to the models.

7.3 The ansatz

The circuit $U(\boldsymbol{\theta})$ is built from parametrized rotations and fixed entangling gates. The rotations are the exponentials of the Pauli matrices of Eq. (2.81),

$$R_x(\theta) = e^{-i\theta X/2}, \qquad R_y(\theta) = e^{-i\theta Y/2}, \qquad R_z(\theta) = e^{-i\theta Z/2}, \tag{7.5}$$

and since $X^2 = Y^2 = Z^2 = I$ each closes on two terms, $R_a(\theta) = \cos(\theta/2)\, I - i \sin(\theta/2)\, \sigma_a$, Eq. (2.80). The same is true of the Pauli rotations $e^{-i\theta P/2}$ of Eq. (4.19) for any string P. This innocuous identity is responsible for the parameter-shift rule of Section 7.5: because the exponential closes on two terms, the energy is a trigonometric polynomial of degree one in each angle, and such a polynomial can be differentiated exactly by evaluating it at two shifted points.

7.3.1 One qubit is a sphere

For a single qubit the general normalized state, up to a phase, is the Bloch-sphere state of Eq. (2.35), $\cos(\theta/2)|0\rangle + e^{i\phi}\sin(\theta/2)|1\rangle$, two angles for two real degrees of freedom. A product of two rotations,

$$|\psi(\theta_1, \theta_2)\rangle = R_z(\theta_2) R_y(\theta_1) |0\rangle, \tag{7.6}$$

reaches every one-qubit state, and for a Hamiltonian with real matrix elements the single rotation $R_y(\theta)|0\rangle$ already suffices, since the ground state of a real symmetric matrix can be chosen real. For a one-qubit Hamiltonian the ansatz is therefore *exact*: the variational minimum is the true ground state, and any error in the result is the optimizer's fault and nobody else's. This is why Section 7.6 begins there. It is also the last case in which anything of the kind is true.

7.3.2 The hardware-efficient ansatz

The workhorse of the near-term literature [**?**] places a pair of rotations on every qubit and follows them with a fixed layer of entangling gates, repeating the pattern L times,

$$U(\boldsymbol{\theta}) = \left[\prod_{q=0}^{n-1} R_z(\theta^z_{Lq}) R_y(\theta^y_{Lq}) \right] \prod_{\ell=1}^{L-1} \left[U_{\text{ent}} \prod_{q=0}^{n-1} R_z(\theta^z_{\ell q}) R_y(\theta^y_{\ell q}) \right], \tag{7.7}$$

applied to $|0\cdots0\rangle$ and read from right to left. With a ladder of CNOT gates as U_{ent}, control q and target $q+1$ for $q = 0, \ldots, n-2$, this uses $2nL$ parameters and $(n-1)(L-1)$ two-qubit gates; Fig. 7.2 shows two layers on three qubits. It is called hardware-efficient because it is built from whatever gates the machine executes natively and respects whatever connectivity it has, and the name is honest about its virtue and silent about its defect: *nothing in it knows anything about the Hamiltonian.* It does not conserve particle number, does not respect spin or any other symmetry, and is not guaranteed to contain any state resembling the ground state. It is a generic family that happens to be cheap.

Two consequences follow, and both appear in the calculations of Section 7.7. The optimization landscape has many local minima, so that restarts from several random initializations are not a refinement but a necessity, and all the hardware-efficient results below are the best of four. And as n and L grow the landscape flattens: for a sufficiently random circuit the gradient of the energy has zero mean and a variance that decays exponentially with the

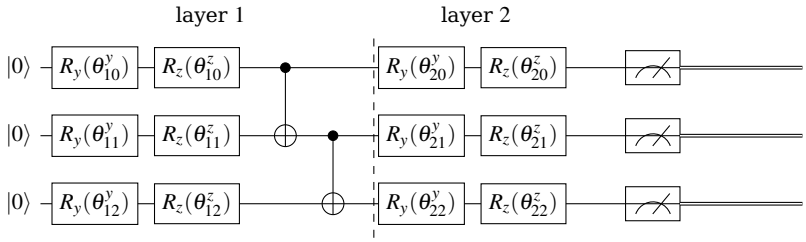


Fig. 7.2 The hardware-efficient ansatz of Eq. (7.7) with $L = 2$ layers on three qubits: R_y and R_z on every qubit, a CNOT ladder, a second layer of rotations, and the readout. Twelve parameters and two CNOT gates; the last layer carries no ladder, since entangling gates just before a measurement of the energy could be absorbed into the measurement.

number of qubits. This is the *barren plateau* of Section 7.5.4. A third consequence is specific to many-body physics and is the one the tables below make most vividly. The Hamiltonians of Chapter 4 act on a register that contains every particle number at once, and the physical ground state is the lowest state of one sector. A circuit that does not conserve the particle number is free to leave the sector, and if the register has a lower eigenvalue elsewhere, as the pairing and Hubbard Hamiltonians of Table 6.6 do, the optimizer will find it, quite correctly, and report an energy that is not the energy of the system one wanted.

7.3.3 Ansätze that know the physics

The alternative is to build the circuit from the Hamiltonian. The unitary coupled-cluster ansatz of Section 3.6.3,

$$|\psi(\boldsymbol{t})\rangle = e^{\hat{T}(\boldsymbol{t}) - \hat{T}^\dagger(\boldsymbol{t})} |\Phi_0\rangle, \tag{7.8}$$

starts from the reference determinant and applies particle-hole excitations, so it conserves the particle number by construction, has parameters that are recognizable amplitudes, and reduces to the classical coupled-cluster result in the appropriate limit. Its circuit form is the Trotterized product of Eq. (3.58), one Pauli rotation per string of each excitation, and Section 4.7 showed for the pairing model that the one-step product with re-optimized amplitudes is exactly as good an ansatz as the full exponential. Every circuit of this kind has the form

$$|\psi(\boldsymbol{\theta})\rangle = \prod_{k=K}^{1} e^{-i\theta_k G_k} \, |\Phi_0\rangle, \tag{7.9}$$

with Hermitian generators G_k that are sums of Pauli strings and a reference that is a computational basis state or another easily prepared state, the first generator acting first. The pair excitation of Eq. (4.23) is $G = -\frac{1}{2}(Y_i X_a - X_i Y_a)$; the fermionic single excitation of Eq. (4.20) is the same with the Jordan–Wigner string in between; a double excitation $a_r^\dagger a_t^\dagger a_q a_p - \text{h.c.}$ is eight strings. The class `PauliRotationAnsatz` in `vqe.py` implements Eq. (7.9) for any list of generators, applying each exponential through the spectral decomposition of its matrix, which is exact and fast at the sizes of this book, and it can also emit the circuit of Section 4.5.1 for generators whose strings commute. The script checks that it reproduces the pair-excitation circuit of Fig. 4.5 to machine precision.

A second physically motivated family uses the Hamiltonian itself as the generator. The *Hamiltonian-variational ansatz* [**?**] splits $\hat{H} = \sum_g \hat{H}_g$ into groups of mutually commuting strings, the same groups that Chapter 5 exponentiated in a Trotter step, and alternates them with one angle per group and layer,

$$|\psi(\boldsymbol{\theta})\rangle = \prod_{\ell=1}^{L} \left[\prod_{g} e^{-i\theta_{\ell g}\hat{H}_g} \right] |\Phi_0\rangle. \tag{7.10}$$

With all angles equal to Δt this is the Trotterized time evolution of Section 3.5.3, so the family contains the dynamics of the system, and with a slowly varying schedule of angles it contains the adiabatic preparation of Section 7.9.2; the optimizer is free to find something better than either. The generators commute with every symmetry of $\hat{H}$, so the ansatz stays in the symmetry sector of the reference, and it has very few parameters: the Heisenberg ring has two groups, the even and the odd bonds, and the Hubbard ring the on-site and the hopping terms, so one layer is two angles. Where the two families differ is in what they assume. Unitary coupled cluster assumes a reference determinant that is a reasonable starting point and a hierarchy of excitations from it; the Hamiltonian-variational ansatz assumes a reference that shares the symmetries of the target and a Hamiltonian whose pieces, applied in alternation, connect the two. Section 7.7 will show both at work.

Between the generic and the physical families lie the adaptive schemes, which grow a circuit of the form of Eq. (7.9) one generator at a time, choosing at each step, from a pool of candidates, the one with the largest energy gradient. They produce compact, problem-specific circuits at the price of many extra measurements during the construction, and they are the subject of Section 7.8. Symmetry is the thread through all of this. An ansatz that conserves the quantities the Hamiltonian conserves searches a smaller space and searches it better, and it cannot fall into the wrong sector; an ansatz that breaks a symmetry the Hamiltonian respects, as the CNOT ladder breaks the permutation symmetry of the Lipkin model, must reconstruct the symmetric state by a conspiracy among its parameters that the optimizer has no particular reason to find.

7.4 Measuring the energy

Everything so far has been written as though $\langle\psi|\hat{H}|\psi\rangle$ were simply available. It is not. A quantum computer performs one kind of measurement, it reads each qubit in the computational basis and returns a bit, and the state is destroyed in the process, so that the whole preparation must be repeated for every sample. Section 4.6 set out how the expectation value of a Pauli sum is assembled from such readouts, and we recall the result before adding what the variational loop needs from it. With $\hat{H} = \sum_\alpha h_\alpha P_\alpha$,

$$E(\boldsymbol{\theta}) = \sum_{\alpha} h_\alpha \, \langle\psi(\boldsymbol{\theta})|P_\alpha|\psi(\boldsymbol{\theta})\rangle, \tag{7.11}$$

so the problem reduces to one number per string, and the identity string needs no measurement at all. A string of Z's is read directly: the eigenvalue on the observed bit string is the parity $(-1)^{\sum_q b_q}$ over the qubits the string acts on, and its average over the shots estimates $\langle P\rangle$. A qubit carrying X is preceded by a Hadamard, since $HZH = X$, and a qubit carrying Y by $S^\dagger$ and then H, since $HS^\dagger Y SH = Z$; Fig. 7.3 shows the three circuits, which applied qubit by qubit suffice for every string and hence for every Hamiltonian. Strings that commute qubit by qubit share the rotation and the shots, so the number of distinct circuits is the number of such groups, Table 4.12, and not the number of strings. Table 7.1 checks the recipe on a random two-qubit state, with the outcome distribution taken exactly rather than sampled: the rotated-and-averaged parity agrees with the matrix element to machine precision for every string tried.

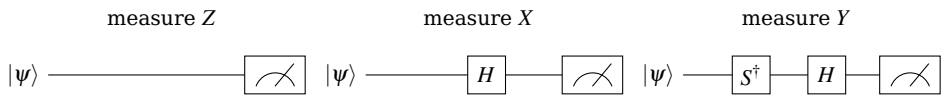


Fig. 7.3 Measuring Z, X and Y on one qubit. The meter always reads the computational basis; the observable is selected by what precedes it. For a string, the circuits are applied qubit by qubit and the parity of the outcome bits on the support of the string is averaged.

Table 7.1 The rotation recipe checked against direct evaluation on a random two-qubit state. The second column is what hardware does, the rotations of Fig. 7.3 followed by a readout of both qubits and an average of the parity, here over the exact outcome distribution; the third is the matrix element. Produced by `measurement.py`.

string	rotation and readout	$\langle\psi\|P\|\psi\rangle$	difference
ZI	-0.300445	-0.300445	1×10^{-16}
XI	-0.020521	-0.020521	9×10^{-17}
YI	-0.414294	-0.414294	2×10^{-16}
XX	0.577375	0.577375	2×10^{-16}
YY	0.919859	0.919859	4×10^{-16}
ZZ	-0.573748	-0.573748	1×10^{-16}
XZ	-0.599067	-0.599067	1×10^{-16}
YX	-0.078774	-0.078774	8×10^{-17}

7.4.1 Shot noise, and what it costs the loop

Each shot returns ±1, so the estimator of $\langle P\rangle$ from N shots is a sample mean of bounded variables, unbiased, with variance

$$\mathrm{Var}\Big[\widehat{\langle P\rangle}\Big] = \frac{1-\langle P\rangle^2}{N}, \tag{7.12}$$

and a standard error at most $1/\sqrt{N}$. When the strings are measured independently the errors add in quadrature,

$$\mathrm{Var}\big[\hat{E}\big] = \sum_{\alpha} \frac{h_\alpha^2\,\big(1-\langle P_\alpha\rangle^2\big)}{N_\alpha} \le \frac{1}{N}\Big(\sum_{\alpha\neq I}|h_\alpha|\Big)^2, \tag{7.13}$$

where the bound assumes a total budget N divided optimally among the strings in the manner of Section 4.6, and $\Lambda=\sum_{\alpha\neq I}|h_\alpha|$ is the Pauli 1-norm of the traceless part, the same quantity that set the time step of phase estimation in Eq. (6.33). The scaling is the point. An accuracy ε needs

$$N \sim \frac{\Lambda^2}{\varepsilon^2} \tag{7.14}$$

shots, so that every additional decimal digit costs a factor of one hundred. Table 7.2 and Fig. 7.4 show this for a fixed trial state of the two-mode Hamiltonian of Eq. (4.9), a two-layer hardware-efficient state with energy 0.330283 measured in its three settings $\{II, ZI, IZ, ZZ\}$, $\{XX\}$ and $\{YY\}$: the standard error times $\sqrt{N}$ is constant to within the statistical uncertainty of a standard deviation estimated from 200 repetitions, about 5%, and the bias shows no trend. The dashed line of the figure is the $N^{-1/2}$ law drawn through the first point with nothing fitted, and the measured points follow it across three decades. There is no regime at the right of the figure in which the curve steepens, no threshold beyond which further shots become more productive, and no reason to expect one: the central limit theorem fixes the exponent once and for all. Everything that can be done, better grouping, the optimal allocation of shots to strings of Table 4.11, the classical-shadow estimators of the recent literature, moves the

Table 7.2 Shot noise in the energy of a fixed two-qubit trial state of the two-mode Hamiltonian ($E = 0.330283$, $\Lambda = 2.175$, three measurement settings), over 200 independent repetitions of the whole measurement. The third column is the second scaled by $\sqrt{N}$ and is constant within its own uncertainty; the estimator is unbiased, and the last column shows no trend. Produced by `measurement.py`.

shots per setting N	standard error	$\times\sqrt{N}$	bias
10	0.2649	0.84	−0.0031
100	0.0948	0.95	+0.0150
1000	0.0276	0.87	+0.0015
10000	0.0102	1.02	−0.0008

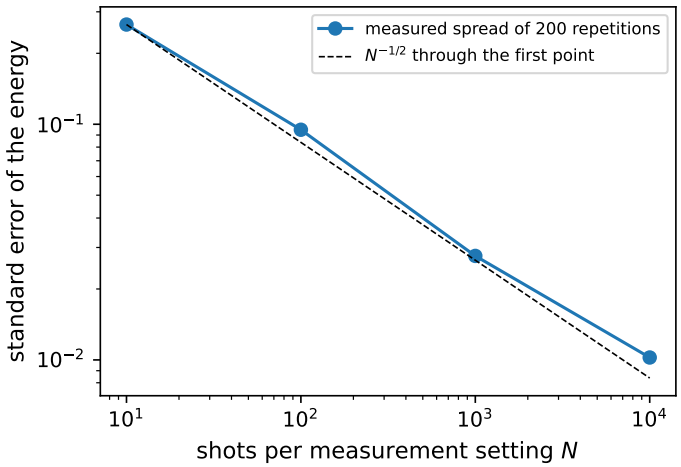


Fig. 7.4 The standard error of the sampled energy of Table 7.2 against the number of shots per measurement setting, on doubly logarithmic axes. The dashed line is the $N^{-1/2}$ law of Eq. (7.12) through the leftmost point, with no fitted parameter. Produced by `measurement.py`.

line down by reducing the prefactor, and none of it tilts the line. That distinction, between a prefactor one may attack and an exponent one may not, is the whole of the economics of VQE, and it is the one line of the comparison in Section 7.7.3 on which phase estimation is unambiguously the better algorithm.

Two remarks follow. Chemical accuracy, 1.6 mHartree, on a molecular Hamiltonian with $\Lambda \sim 10^3$ Hartree needs of order 10^{11} shots for a single energy, and an optimization needs thousands of energies; at kilohertz repetition rates this is centuries, and it is why the reduction of the measurement count is one of the most active areas of the field [**?**]. And the noise enters the optimization as well as the answer. The classical optimizer of step 3 is minimizing a function it can only evaluate stochastically, which rules out the finite-difference gradients and the line searches that a classical minimizer takes for granted, and which is the reason Section 7.5 is needed at all.

7.4.2 Why measuring one qubit at a time does not work

Here is a shortcut that looks reasonable and is wrong. Reading all n qubits gives, in particular, the one-qubit statistics $\langle Z_0\rangle, \ldots, \langle Z_{n-1}\rangle$, and it is tempting to build the energy out of those, treating a two-qubit term as a product,

$$\langle Z_0 Z_1 \rangle \stackrel{?}{=} \langle Z_0 \rangle \langle Z_1 \rangle. \tag{7.15}$$

Some early treatments of VQE do exactly this. It is false, and the way in which it is false deserves care, because the resulting error is not noisy or unstable but quiet and confident.

Theorem 7.1 (Marginals do not determine correlators). *There exist states ρ and ρ' on two qubits with identical single-qubit reduced density matrices, and an observable $\hat{H}$ with $\mathrm{Tr}(\rho\hat{H}) \neq \mathrm{Tr}(\rho'\hat{H})$. Consequently the expectation value of $\hat{H}$ is not a function of the single-qubit measurement statistics.*

Proof. Take $\hat{H} = Z \otimes Z$ and the two Bell states of Eq. (2.59), $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ and $|\Psi^+\rangle = (|01\rangle + |10\rangle)/\sqrt{2}$. Both have the maximally mixed reduced density matrices $\rho_0 = \rho_1 = I/2$ of Section 2.6.3, so *every* single-qubit observable, X, Y and Z on either qubit, has the same expectation value, zero, in the two states, and hence so does any function of them. Yet $\langle \Phi^+ | Z \otimes Z | \Phi^+ \rangle = +1$ and $\langle \Psi^+ | Z \otimes Z | \Psi^+ \rangle = -1$.

The mechanism is not an artefact of a contrived example. The single-qubit statistics of a state are precisely the content of its reduced density matrices, and a reduced density matrix is obtained by tracing out the other qubit, Eq. (2.54), an operation whose entire purpose is to discard correlations. Asking for $\langle Z_0 Z_1 \rangle$ from $\langle Z_0 \rangle$ and $\langle Z_1 \rangle$ is asking to recover what was deliberately thrown away. For a product state the factorization holds; for anything entangled it does not, and entanglement is the reason for using a quantum computer at all.

What it does to a VQE calculation. Take $\hat{H} = Z \otimes Z$ with the one-parameter family

$$|\psi(\theta)\rangle = \cos\theta\,|00\rangle + \sin\theta\,|11\rangle. \tag{7.16}$$

Every state in it has $\langle Z_0 Z_1 \rangle = 1$, since both basis states present have even parity, so the true landscape is flat, $E_{\mathrm{true}}(\theta) = 1$. The marginals are $\langle Z_0 \rangle = \langle Z_1 \rangle = \cos^2\theta - \sin^2\theta = \cos 2\theta$, so the reconstruction of Eq. (7.15) reports

$$E_{\mathrm{naive}}(\theta) = \cos^2 2\theta, \tag{7.17}$$

an oscillating function with minima at $\theta = \pi/4$ and $3\pi/4$ where it reports 0 for a quantity whose value is 1. Figure 7.5 draws the two. The error of the reconstruction, $1 - \cos^2 2\theta = \sin^2 2\theta$, is exactly the square of the concurrence $|\sin 2\theta|$ of the state, so the two curves touch only at the three product states of the family and separate most widely at the Bell states, which sit at the bottom of the fictitious wells; and the slope of the dashed curve points, on every descending branch, towards the maximally entangled state. An optimizer handed this landscape is not merely misinformed. It is driven towards the configuration on which the readout is at its worst, and the more successfully it optimizes the larger the error it accumulates, while every diagnostic the classical half of the loop can see says that the run is going well.

Table 7.3 runs the same failure inside a full VQE on the two-level pairing Hamiltonian of Eq. (4.17), whose ground state -0.618034 is an entangled superposition of $|10\rangle$ and $|01\rangle$, with a two-layer hardware-efficient ansatz from the same starting parameters. The joint readout finds the exact energy. The marginal readout converges smoothly to a state lying 0.097 above the ground state and reports for it a number that is not the energy of any state, and in this case the reported number lies *above* the true energy of the state prepared. It need not: a quantity that is not $\langle \psi | \hat{H} | \psi \rangle$ for any $|\psi\rangle$ is under no obligation to respect the variational principle and can equally well come out below E_0, in which case nothing whatever identifies it as wrong. The final irony is that the shortcut buys nothing. A single shot returns *both* bits, and the same data that give $\langle Z_0 \rangle$ and $\langle Z_1 \rangle$ give $\langle Z_0 Z_1 \rangle$ at no additional cost, by multiplying the two outcomes before averaging instead of averaging first and multiplying afterwards. The rule is therefore easy to state and should never be broken: measure all qubits, form the parity

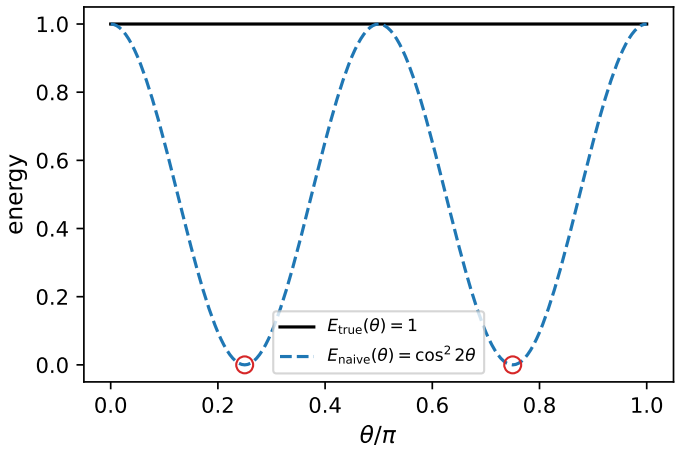


Fig. 7.5 The two energy landscapes of $Z \otimes Z$ over the family of Eq. (7.16): the true energy, which is flat, and the marginal reconstruction of Eq. (7.17). The circles mark the spurious minima at the Bell states. Produced by `measurement.py`.

Table 7.3 VQE on the two-level pairing Hamiltonian of Eq. (4.17) with joint and with marginal readout, hardware-efficient ansatz with two layers, identical starting parameters. The marginal run converges, and the number it reports is not the energy of any state. Produced by `measurement.py`.

	energy
exact ground state	-0.618034
VQE, joint readout	-0.618034
VQE, marginal reconstruction (reported)	-0.497409
true energy of the state it prepared	-0.521024

of each string, then average. The order of the two operations is the whole of the difference, $\mathbb{E}[ab] \neq \mathbb{E}[a]\mathbb{E}[b]$ unless a and b are independent, and a state that is worth preparing is one in which they are not.

7.5 Gradients on a quantum computer

The classical half of the loop wants derivatives. On a classical computer one differentiates the code; here the energy is accessible only through measurement, and the obvious substitute, the finite difference

$$\frac{\partial E}{\partial \theta_k} \approx \frac{E(\theta_k + h) - E(\theta_k - h)}{2h}, \tag{7.18}$$

is a poor one for a reason that has nothing to do with quantum mechanics. It has a truncation error $\mathcal{O}(h^2)$ that wants h small, and it subtracts two noisy numbers whose difference is $\mathcal{O}(h)$ while their errors are not, so that with shot noise σ on each energy the error of the estimate is σ/h, minimized at $h \sim \sigma^{1/3}$ and then far larger than σ itself. The parameter-shift rule [**?**, **?**] replaces this with something better in every respect: exact, with a shift of order one rather than infinitesimal, at the cost of two energy evaluations of exactly the kind already being performed.

7.5.1 The parameter-shift rule

The rule is a statement about the *generators of the gates*, not about the Hamiltonian, which enters only through the requirement that $E(\boldsymbol{\theta})$ be measurable at all. This is the usual source of confusion and is worth stating first.

Theorem 7.2 (Two-term parameter-shift rule). *Let $E(\theta) = \langle\psi|U^\dagger(\theta)\hat{H}U(\theta)|\psi\rangle$ with $U(\theta) = e^{-i\theta G}$, $\hat{H}$ and G Hermitian, and let G have exactly two distinct eigenvalues $\pm r$, $r > 0$. Then*

$$\frac{dE}{d\theta} = r\left[E\left(\theta + \frac{\pi}{4r}\right) - E\left(\theta - \frac{\pi}{4r}\right)\right]. \tag{7.19}$$

Proof. Because G has only the eigenvalues $\pm r$ one has $G^2 = r^2 I$, and the projectors onto its two eigenspaces are $\Pi_\pm = \frac{1}{2}(I \pm G/r)$, with $\Pi_\pm^2 = \Pi_\pm$, $\Pi_+\Pi_- = 0$, $\Pi_+ + \Pi_- = I$ and $G = r(\Pi_+ - \Pi_-)$. The exponential therefore terminates,

$$U(\theta) = e^{-i\theta r}\,\Pi_+ + e^{i\theta r}\,\Pi_-, \tag{7.20}$$

the same closing on two terms as in Eq. (2.80). Substituting into $E(\theta)$ gives four terms, two of which carry no θ and two of which carry $e^{\pm 2ir\theta}$; collecting them, and using that E is real,

$$E(\theta) = A + B\cos(2r\theta) + C\sin(2r\theta), \tag{7.21}$$

with $A = \langle\psi|\Pi_+\hat{H}\Pi_+ + \Pi_-\hat{H}\Pi_-|\psi\rangle$, $B = 2\,\mathrm{Re}\langle\psi|\Pi_+\hat{H}\Pi_-|\psi\rangle$ and $C = 2\,\mathrm{Im}\langle\psi|\Pi_+\hat{H}\Pi_-|\psi\rangle$. The energy is a pure sinusoid of frequency $2r$, three unknowns and no more, so that $E'(\theta) = -2rB\sin(2r\theta) + 2rC\cos(2r\theta)$. Evaluating Eq. (7.21) at $\theta \pm s$ with $s = \pi/(4r)$, so that $2rs = \pi/2$ and the shifted cosines and sines exchange places, $E(\theta+s) - E(\theta-s) = 2[-B\sin(2r\theta) + C\cos(2r\theta)]$, which is $E'(\theta)/r$.

The standard case is $G = P/2$ with P a Pauli string, the rotations of Eq. (7.5) and the Pauli rotations of Eq. (4.19), so that $r = 1/2$, $s = \pi/2$ and

$$\frac{\partial E}{\partial \theta_k} = \frac{1}{2}\left[E\left(\theta_k + \frac{\pi}{2}\right) - E\left(\theta_k - \frac{\pi}{2}\right)\right], \tag{7.22}$$

all other parameters held fixed. Equation (7.22) looks like Eq. (7.18) with $h = \pi/2$, and it is essential to see why it is a different kind of object. The shift is not small and there is no truncation error to trade against noise: the identity is exact for every θ. What makes it work is that Eq. (7.21) has three unknown constants, so that three evaluations determine the function completely and two determine its derivative; the rule is interpolation of an exactly known form, not approximation of an unknown one. On hardware this is worth a great deal, because the two terms are of order one and well separated and their difference does not amplify the noise. A gradient with P parameters costs $2P$ energies, each of them the shots of Section 7.4.1, and this is where the expense of VQE lives. Two checks in `gradients.py` make the theorem concrete. Fitting A, B, C to three energies of a two-qubit, two-layer hardware-efficient state as a function of one R_y angle and predicting the energy at 25 other angles reproduces it to 9×10^{-16}, which confirms the functional form and not merely the derivative; and the shift rule agrees with a converged finite difference to 1×10^{-11}, the residual being the finite difference's own error.

7.5.2 When two terms are not enough

The hypothesis of Theorem 7.2 is not decorative. Consider

$$G = \frac{1}{2}(Z_0 + Z_1), \tag{7.23}$$

with eigenvalues $-1, 0, +1$. What the energy sees are the *differences* of eigenvalues, now $0, \pm 1, \pm 2$, so that Eq. (7.21) becomes a two-frequency polynomial

$$E(\theta) = A + B_1 \cos\theta + C_1 \sin\theta + B_2 \cos 2\theta + C_2 \sin 2\theta, \tag{7.24}$$

with five unknowns, and two evaluations cannot determine five constants. The failure is not graceful. For the generator of Eq. (7.23) acting on $|{+}{+}\rangle$ with $\hat{H} = 0.7\,XI + 0.4\,IX + 0.5\,XX + 0.3\,YZ$ at $\theta = 0.6$ the true derivative is -1.087126 and the two-term rule returns -0.621107, a number of the right sign and the wrong size, which an optimizer would follow without complaint. The remedy is the generalized rule for R equidistant frequencies [**?**],

$$\frac{dE}{d\theta} = \sum_{\mu=1}^{2R} \frac{(-1)^{\mu-1}}{4R \sin^2(x_\mu/2)} E(\theta + x_\mu), \qquad x_\mu = \frac{(2\mu-1)\pi}{2R}, \tag{7.25}$$

which reduces to Eq. (7.22) at $R = 1$ and at $R = 2$ recovers -1.087126 to 10^{-8}. The cost is $2R$ evaluations per parameter. This is not an academic example: the pair-excitation generator $-\frac{1}{2}(Y_i X_a - X_i Y_a)$ of Eq. (4.23), and every fermionic single and double excitation, has the spectrum $\{-1, 0, +1\}$, since the anti-Hermitian excitation operator squares to minus a projector. Applied to the four pair excitations of the circuit of Eq. (4.25) the two-term rule is wrong by 1.3×10^{-2} while the four-term rule agrees with the finite difference to 3×10^{-10}, which is why the class `VQE` asks every ansatz, parameter by parameter, which rule applies: two terms for the rotations of the hardware-efficient family, four for the excitation generators, and a finite difference as the fallback for a generator with a spectrum it does not recognize. The alternative, and the usual practice, is to Trotterize each excitation into its individual Pauli strings and give every string its own angle, at which point the two-term rule applies again at the price of more parameters.

7.5.3 Optimizers

With gradients available the classical half of the loop is ordinary numerical optimization, and the programs of this chapter use three methods. Plain gradient descent, $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla E$, is robust, slow, and easy to reason about, and it is what the convergence histories below show. The BFGS quasi-Newton method of `common/optimize.py`, already used for the coupled-cluster amplitudes of Chapter 3, converges in a small fraction of the iterations when the energies are exact, which is the case on a simulator and is what every table of Section 7.7 uses; on hardware its line searches are fragile because they compare energies that differ by less than the shot noise. On the two-mode Hamiltonian from the same start, gradient descent needs 5101 energy evaluations to reach the minimum, BFGS 204.

The third is not gradient descent at all. Simultaneous perturbation stochastic approximation, SPSA [**?**], estimates the whole gradient from *two* energies whatever the number of parameters, by perturbing all of them at once along a random direction $\boldsymbol{\Delta}$ with entries ± 1,

$$\hat{g}_k = \frac{E(\boldsymbol{\theta} + c_k\boldsymbol{\Delta}) - E(\boldsymbol{\theta} - c_k\boldsymbol{\Delta})}{2c_k}\,\boldsymbol{\Delta}, \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - a_k \hat{g}_k, \tag{7.26}$$

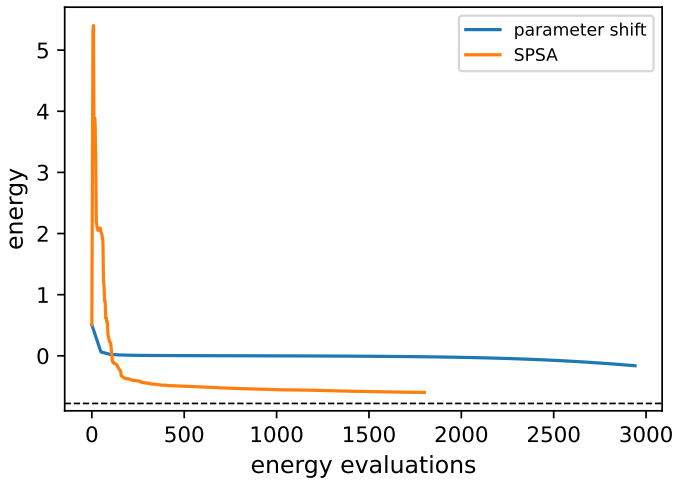


Fig. 7.6 Gradient descent with parameter-shift gradients against SPSA on the four-qubit pairing Hamiltonian with a three-layer hardware-efficient ansatz (24 parameters), from the same starting point, against the number of energy evaluations. The dashed line is the lowest eigenvalue of the register. Produced by `gradients.py`.

with step sizes a_k and c_k that decay slowly with the iteration count. The estimate is unbiased for the gradient up to $\mathcal{O}(c_k^2)$ and very noisy, and the noise averages out over the iterations in the way that stochastic gradient descent tolerates. Against the $2P$ energies of a parameter-shift gradient this is a large saving when P is large, and the right currency for the comparison is the number of energy evaluations rather than the number of iterations, since each evaluation is the shots of Section 7.4.1. Figure 7.6 makes the comparison on the four-qubit pairing Hamiltonian of Eq. (4.16) with a three-layer hardware-efficient ansatz, 24 parameters, both methods from the same start: after 2940 evaluations the parameter-shift descent has reached -0.162, and SPSA after 1800 has reached -0.599, on a landscape whose lowest point is the register minimum -0.779. The comparison is not universal, since it depends on the learning rates, the number of parameters and the noise, but on hardware, where every evaluation is dear, SPSA or a relative of it is very often what is used.

7.5.4 Barren plateaus

The second feature has no classical analogue. For a sufficiently expressive random circuit the energy landscape flattens exponentially [**?**]: the gradient has zero mean and a variance

$$\mathrm{Var}\left[\frac{\partial E}{\partial \theta_k}\right] \sim 2^{-cn} \tag{7.27}$$

for some constant c of order one, so that on 50 qubits the gradient is of order 10^{-15} and cannot be distinguished from zero by any feasible number of shots. This is the *barren plateau*, and the argument behind it is a concentration result: as the circuit approaches a unitary 2-design, the expectation values it produces concentrate about their averages over the Haar measure, and the derivatives of a concentrated quantity are small. Figure 7.7 shows it on the hardware-efficient ansatz with a ring of CNOT gates and $L = n$ layers, for the observable $Z_0 Z_1$, sampling 300 random parameter vectors at each n and computing one gradient component by the shift rule: the variance falls from 0.19 at two qubits to 0.0019 at eight, and the slope of its logarithm for $n \geq 4$ is -0.99 bits per qubit. The trouble is structural, since expressiveness is exactly

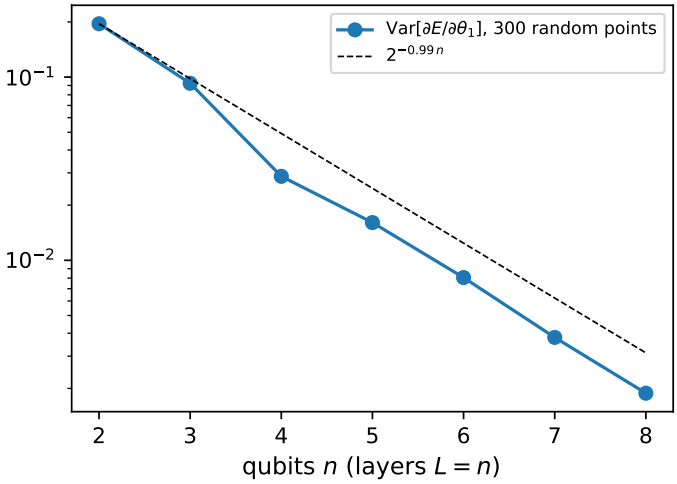


Fig. 7.7 The variance of one gradient component of the hardware-efficient ansatz with a CNOT ring and $L = n$ layers, for the observable $Z_0 Z_1$, over 300 random parameter vectors, against the number of qubits. The dashed line is the exponential fitted for $n \geq 4$. Produced by `gradients.py`.

what causes it, and the escapes proposed all amount to being less generic: shallow circuits, ansätze that respect the symmetries and locality of the Hamiltonian, layerwise training, and initializations near a known good state such as the Hartree–Fock determinant. The physically motivated ansätze of Section 7.3.3 are, among other things, the natural remedy, and the review [**?**] sets out what is known. It is the most serious theoretical obstacle to VQE at scale, and it has no counterpart in phase estimation, which optimizes nothing.

7.6 Two small Hamiltonians, worked through

We return to the two systems of Section 6.5, now solved variationally. Reusing them is deliberate: the comparison of Section 7.7.3 is between two algorithms on one problem, not between two problems.

7.6.1 One qubit: the tapered pairing Hamiltonian

The tapered one-qubit form of the two-level pairing model, Eq. (4.18), $\hat{H} = \frac{1}{2}I + Z - \frac{1}{2}X$, has the ground state $E_- = \frac{1}{2} - \frac{\sqrt{5}}{2} = -0.618034$. The matrix is real, so the single rotation $R_y(\theta)|0\rangle = \cos(\theta/2)|0\rangle + \sin(\theta/2)|1\rangle$ suffices, and its energy is

$$E(\theta) = \frac{1}{2} + \cos\theta - \frac{1}{2}\sin\theta, \tag{7.28}$$

minimized where $\tan\theta = -\frac{1}{2}$ on the branch with $\cos\theta < 0$. The program finds -0.6180339887 at $\theta = 2.6779$, that is $\tan\theta = -0.5000$, with an error of 2×10^{-16}: the ansatz is exact, so the residual measures how far the optimizer ran and nothing else. Compare Table 6.4, where phase estimation with ten control qubits reached 1.5×10^{-3}, limited by the grid $2\pi/(t2^m)$ of its energy register. VQE is not quantizing the answer at all. On a single qubit, with an exact ansatz and

Table 7.4 VQE on the two-mode Hamiltonian of Eq. (4.9), whose register spectrum is $\{0, 0.916905, 2.083095, 3.5\}$ and whose one-particle ground state is 0.916905. The hardware-efficient ansatz leaves the sector and finds the vacuum; the number-conserving single excitation finds the physical ground state. Produced by `vqe_models.py`.

ansatz	parameters	E_{VQE}	$\langle \hat{N} \rangle$
hardware-efficient, 1 layer	4	0.000000	0
hardware-efficient, 2 layers	8	0.000000	0
single excitation from $\lvert 10 \rangle$	1	0.916905	1

no shot noise, the variational method simply wins, and neither of the two conditions in that sentence survives contact with a larger problem.

7.6.2 Two qubits: the two-mode Hamiltonian

The two-mode Hamiltonian of Eq. (4.9) is where the first real lesson appears. Its register holds the spectrum $\{0,\ 0.916905,\ 2.083095,\ 3.5\}$, and the physical problem of Section 3.2.6 is the one-particle sector, whose ground state is 0.916905; the register minimum, zero, is the vacuum $\lvert 00 \rangle$ with no particle at all. Table 7.4 shows what the two kinds of ansatz make of this. The hardware-efficient circuit, with one layer or two, converges without difficulty to 0.000000 and $\langle \hat{N} \rangle = 0$: it has found the ground state of the register, which is the correct answer to the question it was asked and the wrong answer to the question we meant. The number-conserving ansatz, the single fermionic rotation of Eq. (4.20) applied to $\lvert 10 \rangle$ with one parameter, cannot leave the sector and finds 0.916905 at $\theta = -0.2702$, the value of Section 4.5.5, to machine precision. The point generalizes to every fermionic Hamiltonian in an orbital encoding, since the register contains every particle number and the ground state of the full Fock space is rarely the state one wants; the remedies are an ansatz that conserves the number, a penalty term $\mu(\hat{N} - N)^2$ added to the Hamiltonian, or a symmetry-adapted encoding that removes the other sectors, as the tapered form of Section 4.4.6 does. The ground state of the one-particle sector is entangled, a superposition of $\lvert 10 \rangle$ and $\lvert 01 \rangle$, and the single excitation gate reaches it with four CNOT gates and one angle; and the variational value lies above the exact one, as Eq. (7.1) requires, in contrast to the $m = 5$, 7 and 8 rows of Table 6.5, which lie below it.

7.7 The model Hamiltonians

We now run VQE on the four systems that Section 6.6 fed to phase estimation, with the Hamiltonians, couplings and exact energies of Table 6.6: the Lipkin model with $N = 4$ in the doublet encoding of Eq. (4.11), $E_0 = -4.212877$; the pairing model with four levels and two pairs in the compact encoding of Eq. (4.16), $E_0 = 0.635548$; the four-site Hubbard ring at $U = 4t$ with four particles and $S_z = 0$ in the spin-major Jordan–Wigner encoding of Section 4.4.4, eight qubits, $E_0 = -2.102748$; and the four-site Heisenberg ring, $E_0 = -2$. The programs take the Hamiltonians from `qpe_models.py` of Chapter 6, so that the two chapters agree by construction on what is being solved. The energies are evaluated exactly on the simulator, and every result is the best of four random restarts of BFGS with parameter-shift gradients; the question of shot noise is separated out, since its effect is the one already quantified in Section 7.4.1 and its interaction with the optimizer is taken up in the exercises.

Table 7.5 The hardware-efficient ansatz on the four models, best of four random restarts of BFGS. Parameters and CNOT counts, the variational energy, the lowest eigenvalue of the register, the exact energy of the physical sector, and the overlap of the optimized state with the sector ground state. Produced by `vqe_models.py`.

model	L	param.	CNOT	E_{VQE}	register min.	sector E_0	$\lvert\langle E_0\lvert\psi\rangle\rvert^2$
Lipkin, $N=4$	1	8	0	-4.000000	-4.212877	-4.212877	0.9358
	2	16	3	-4.105370	-4.212877	-4.212877	0.9726
	3	24	6	-4.211395	-4.212877	-4.212877	0.9998
pairing, $L=4$	1	8	0	-0.500000	-0.779164	0.635548	0.0000
	2	16	3	-0.622012	-0.779164	0.635548	0.0000
	3	24	6	-0.779164	-0.779164	0.635548	0.0000
Hubbard ring	1	16	0	-1.527864	-3.418551	-2.102748	0.0067
	2	32	7	-2.000000	-3.418551	-2.102748	0.0000
Heisenberg ring	1	8	0	-1.000000	-2.000000	-2.000000	0.3333
	2	16	3	-1.619494	-2.000000	-2.000000	0.7994
	3	24	6	-1.914214	-2.000000	-2.000000	0.9714

7.7.1 The hardware-efficient ansatz

Table 7.5 is the generic ansatz applied without thought to all four models, at one, two and three layers. It is read against two reference values: the lowest eigenvalue of the whole register, which is what an unconstrained minimization is entitled to find, and the exact ground-state energy of the physical sector, which is what we want. The last column is the overlap of the optimized state with the exact sector ground state, the quantity that Table 6.9 showed to govern the success of a subsequent phase-estimation run.

The table is a catalogue of what can go wrong, and each row is instructive. The one-layer rows have no entangling gate at all, so they are mean-field calculations, product states optimized over their angles: for the Lipkin model the result, -4, is the Hartree–Fock energy of Section 3.5.4 with an overlap 0.9358 that is exactly the overlap of the reference state in Table 6.9, because at this coupling the mean field is the reference; for the Heisenberg ring it is the Néel energy -1 with the Néel overlap $1/3$. Adding layers helps the Lipkin and Heisenberg models, and at three layers both are within 10^{-3} and 10^{-1} of the exact energy with overlaps of 0.9998 and 0.97, good enough to make either an excellent input for phase estimation. Neither is exact, and the Lipkin error is the characteristic one: the exact ground state is symmetric under every permutation of the four qubits, the CNOT ladder singles out a direction along the chain, and the symmetric state can only be reached by a conspiracy among 24 parameters. The pairing rows are the failure of Section 7.6.2 at full size. The physical ground state, 0.635548, is the lowest state with two pairs; the register also contains the sectors with zero, one, three and four pairs, and the lowest eigenvalue of all of them is -0.779164 in the one-pair sector. The ansatz has no reason to stay at two pairs, so with three layers it finds -0.779164 exactly, with zero overlap with the state we wanted, and the "error" of -1.4 in the comparison table below is not an error of the optimizer at all. The Hubbard rows fail in the same way and more slowly: the register minimum -3.418551 lies in another particle-number sector, the two-layer circuit with 32 parameters and 7 CNOT gates reaches -2.000000 with no overlap with the four-particle ground state, and more layers would only take it further down. The lesson is the one stated in Section 7.3.3. The optimizer converged in every row, and its convergence carries no information about whether the answer is right; only symmetry, or the exact column that is unavailable for any problem worth solving on a quantum computer, tells the difference.

Table 7.6 Ansätze built from the physics of each model, best of four random restarts (two for the eight-qubit Hubbard ring): the number of parameters, the variational energy, its error, and the overlap of the optimized state with the exact ground state. HVA is the Hamiltonian-variational ansatz of Eq. (7.10). Produced by `vqe_models.py`.

model	ansatz	param.	E_{VQE}	error	$\lvert\langle E_0\lvert\psi\rangle\rvert^2$
Lipkin, $N=4$	mean field, one R_y angle	1	-4.00000000	2.1×10^{-1}	0.935763
	HVA, 1 layer	3	-4.21238352	4.9×10^{-4}	0.999939
	HVA, 2 layers	6	-4.21287670	$<10^{-14}$	1.000000
pairing, $L=4$	four pair excitations (UCCD)	4	0.63698698	1.4×10^{-3}	0.999822
	all six pair transfers	6	0.63554847	$<10^{-14}$	1.000000
Hubbard ring	HVA, 1 layer, from $U\to0$	2	-1.97671868	1.3×10^{-1}	0.988473
	HVA, 2 layers	4	-2.10274848	$<10^{-14}$	1.000000
Heisenberg ring	HVA, 1 layer, from singlets	2	-2.00000000	$<10^{-15}$	1.000000

7.7.2 Ansätze that know the physics

Table 7.6 repeats the calculation with the circuits of Section 7.3.3, chosen for each model from what Chapters 3 and 4 know about it.

For the Lipkin model the Hamiltonian of Eq. (4.11) has three commuting groups, its Z, XX and YY parts, so a layer of the Hamiltonian-variational ansatz is three angles; the generators are built from the collective quasispin operators and commute with the permutations of the qubits, so the ansatz lives in the symmetric subspace from the start, and two layers, six parameters, give the exact energy where 24 hardware-efficient parameters gave 10^{-3}. One layer already has an overlap of 0.99994. For the pairing model the four pair excitations from the reference $|1100\rangle$ are the one-step unitary coupled-cluster circuit of Eq. (4.25), and the energy 0.63698698 is the UCCD value of Table 4.14, reproduced here by the general `PauliRotationAnsatz` rather than by the hand-built circuit of Chapter 4; it misses the exact answer by 1.4×10^{-3} because the one-step product of four excitations from a single reference does not span the whole two-pair space. Adding the two remaining pair transfers, between the two occupied and between the two empty levels, which are not excitations of the reference but rotations within it once the first four have acted, gives a circuit of six angles that is exact. The Hubbard ring illustrates the other requirement of the Hamiltonian-variational ansatz, a reference with the right symmetries. The Néel determinant of Table 6.9 has an overlap of 0.24 and, being a product state, is not an eigenstate of the hopping term; the natural alternative is the ground state of the non-interacting ring, but at $U=0$ and half filling that state is six-fold degenerate and an arbitrary member of the manifold has an overlap of 0.007 with the interacting ground state. The program takes instead the ground state at $U=0.01t$, the member of the free manifold that the interaction selects, with an overlap of 0.8509, and alternates the on-site and the hopping terms: one layer gives -1.9767, two layers with four angles give the exact -2.10274848. The Heisenberg ring is the most striking row. The reference is the product of two singlets on the bonds $(0,1)$ and $(2,3)$, the ground state of the even-bond half of the Hamiltonian, and one layer of the odd and the even bond groups, two angles, gives the exact ground state -2; Section 7.8 will show that a single well-chosen operator suffices. Figure 7.8 shows the gradient-descent histories of the deepest circuit of each model from a random start, and it is worth noticing that the Lipkin curve stalls at 5×10^{-4}, the value of the one-layer ansatz: plain descent from this start has found a local minimum that BFGS with restarts escapes. Restarts are a necessity for physical ansätze too.

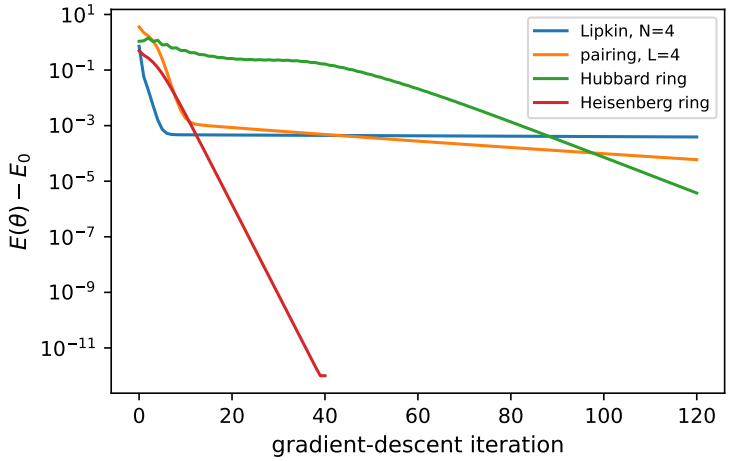


Fig. 7.8 Gradient-descent histories, with parameter-shift gradients, of the deepest physically motivated ansatz of each model in Table 7.6, from a random start; the error is plotted on a logarithmic scale. The Lipkin run stalls in a local minimum at the energy of the one-layer ansatz. Produced by `vqe_models.py`.

Table 7.7 The four models: the error of phase estimation with $m = 10$ control qubits and exact evolution (Table 6.7), the signed error of the best hardware-efficient energy of Table 7.5, and the error of the best physically motivated ansatz of Table 7.6. The negative hardware-efficient entry for the pairing model is a minimum in the wrong particle-number sector, not a violation of the variational principle. Produced by `vqe_models.py`.

model	QPE, $m = 10$	VQE, hardware-efficient	VQE, physics
Lipkin, $N = 4$	5.9×10^{-3}	$+1.5 \times 10^{-3}$	$< 10^{-14}$
pairing, $L = 4$	7.3×10^{-3}	-1.4 (wrong sector)	$< 10^{-14}$
Hubbard ring	1.7×10^{-2}	$+1.0 \times 10^{-1}$	$< 10^{-14}$
Heisenberg ring	1.3×10^{-3}	$+8.6 \times 10^{-2}$	0

7.7.3 Phase estimation and VQE weighed against each other

Table 7.7 puts the results of this chapter and the last side by side: the error of phase estimation with ten control qubits and the exact evolution operator, from Table 6.7, against the best hardware-efficient and the best physically motivated variational energies. Two algorithms, the same Hamiltonians, and almost nothing in common; the literature tends to present the choice as one between a rigorous method that cannot be run and a practical method that works, and both halves of that summary are misleading.

Phase estimation solves the eigenvalue problem. Given an eigenstate and enough control qubits it returns the eigenvalue to any precision, with a probability bounded below by $4/\pi^2$ per run, and its resources grow as $\log(1/\varepsilon)$ in qubits and $1/\varepsilon$ in depth; the guarantee is a theorem. What the theorem is silent about is where the eigenstate came from. The algorithm needs $|E_0\rangle$ as input, in practice a trial state that succeeds with probability $|\langle E_0|\psi_{\text{trial}}\rangle|^2$, and for a strongly correlated system that overlap decays exponentially with the particle number, at which point the guarantee, while still true, is vacuous. VQE promises less and delivers it more reliably. It returns an upper bound and nothing more; if the ansatz is good the bound is tight, if not it is not, and no feature of the output distinguishes the two cases. Against this, the state is produced rather than assumed, so the state-preparation problem does not appear separately: it has been absorbed into the ansatz, which is where the difficulty now lives. Neither algorithm has made the many-body problem easy. Phase estimation concentrates the difficulty in state preparation and circuit depth; VQE concentrates it in ansatz design and

measurement count. The difficulty is conserved, and recognizing where each method has hidden it is the beginning of using either one sensibly.

The two error scalings are the crux. Phase estimation buys one bit per control qubit, exponential accuracy in the register size, and the Lipkin and Heisenberg columns of the table would fall by another factor of a thousand with ten more qubits; VQE buys accuracy at $\varepsilon \sim N^{-1/2}$ in the shot count, a hundredfold per digit, and on hardware the physics column of the table, exact on the simulator, would be limited by Eq. (7.14) at whatever accuracy the budget allows. Asymptotically this is no contest, and the crossover is not asymptotic: at $\varepsilon = 10^{-3}$ with $\Lambda \sim 10^3$ VQE needs of order 10^{12} measurements, which is out of reach but not absurd, while phase estimation needs a coherent circuit of 10^6 to 10^9 gates, which requires error correction and a machine that does not yet exist. VQE is not better; it is runnable, because its errors are of a kind that can be averaged away rather than a kind that destroys the computation. The two also fail differently, and the difference matters more than the scalings. Phase estimation fails *loudly*: an insufficient overlap is a histogram with no dominant peak, a small register is a coarse answer whose resolution is known in advance, an unremoved identity term is an obviously aliased number, and decoherence flattens the output. VQE fails *quietly*: a poor ansatz gives a smooth convergence curve and a wrong number, a wrong sector gives a lower one, a barren plateau gives a gradient indistinguishable from noise that the optimizer reads as convergence, and the marginal readout of Section 7.4.2 gives a confident number that is not an energy. In every one of these the algorithm reports success, and the only defence is external: symmetry checks, comparison across families, and exact results at small size. Building those in is not decoration; it is the part of a variational calculation that does the epistemic work.

The two are usually presented as successive eras, VQE now and phase estimation after error correction, and that understates how much they need each other. Phase estimation needs a trial state with a large overlap, and the last column of Table 7.5 and of Table 7.6 is exactly that: the three-layer Lipkin state with overlap 0.9998 would make a phase-estimation run succeed on the first attempt, and the exact physics-ansatz states would make it deterministic. VQE needs a way to certify its answer, and a phase-estimation run on the state it produced, Eq. (6.39), both measures the energy and projects onto the eigenstate. The combination, the shallow circuit to prepare and the deep one to measure, is the standard long-term plan, Table 6.10, and whether it delivers an exponential advantage for ground-state energies is the question of the overlaps at sizes far beyond these, examined critically in [**?**].

Excited states. The variational principle is a statement about the ground state, but the same loop reaches excited states with a modification of the cost function. Given the optimized ground state $|\psi_0\rangle$, minimizing $\langle\psi|\hat{H}|\psi\rangle + \mu|\langle\psi_0|\psi\rangle|^2$ with a penalty μ larger than the gap deflates the ground state out of the search and returns the first excited state, the overlap being measurable as the probability of reading $|0\cdots0\rangle$ after the inverse of the ground-state circuit; the quantum subspace expansion instead diagonalizes $\hat{H}$ in the space spanned by $|\psi_0\rangle$ and a set of excitations of it, whose matrix elements are measured once, and returns several levels at a time. The Lipkin spectroscopy of Fig. 6.10 did the same with phase estimation, by reading off several peaks of one histogram, and Chapter 8 does it by filtering. Exercise 8 runs the deflation on the Lipkin model.

7.8 Letting the algorithm choose the circuit: ADAPT-VQE

Every ansatz of Section 7.7 was chosen by hand from what we knew about the model, and the tables show how much that choice matters. The adaptive variational quantum eigensolver of Grimsley, Economou, Barnes and Mayhall [**?**], ADAPT-VQE, makes the choice part of the

algorithm. One supplies a reference state and a *pool* of Hermitian generators $\{G_j\}$, and the circuit of Eq. (7.9) is grown one generator at a time.

The algorithm. Start from $|\psi\rangle = |\Phi_0\rangle$ with an empty circuit. At every step, for every generator in the pool, measure the derivative of the energy with respect to a new angle inserted at the end of the current circuit and evaluated at zero,

$$\frac{\partial}{\partial\theta}\langle\psi|e^{i\theta G_j}\hat{H}e^{-i\theta G_j}|\psi\rangle\Big|_{\theta=0} = i\,\langle\psi|[G_j,\hat{H}]|\psi\rangle = 2\,\mathrm{Im}\,\langle\hat{H}\psi|G_j\psi\rangle, \tag{7.29}$$

where $|\psi\rangle$ is the current optimized state. If the largest |gradient| is below a threshold, stop. Otherwise append $e^{-i\theta G_j}$ for the generator with the largest gradient, with $\theta = 0$ as its starting value, re-optimize *all* the angles of the circuit by the VQE loop, and repeat. On hardware the commutator of Eq. (7.29) is a Pauli sum whose expectation value is measured like any other, so each step costs one energy measurement per pool operator on top of the optimization, which is the price of letting the algorithm decide. What one buys is a circuit that is as short as the problem allows and contains only operators the energy actually responds to, and a natural convergence criterion, the vanishing of every gradient, that the fixed ansätze lack. The class `AdaptVQE` in `adapt_vqe.py` does all of this on the simulator,

```
import numpy as np
from qpe_models import book_models
from vqe_models import pair_generator
from adapt_vqe import AdaptVQE

H = {name: H for name, H, E0, psi, sector in book_models()}["pairing, $L=4$"]
pool = [pair_generator(4, i, a) for i in range(4) for a in range(i + 1, 4)] # six pair transfers
labels = [f"P{i}{a}" for i in range(4) for a in range(i + 1, 4)]
adapt = AdaptVQE(H, "1100", pool, labels)
theta, E = adapt.run(max_ops=8, tol=1e-6, verbose=True)
print(adapt.chosen, E)                    # five operators, E = 0.63554847
```

and the run prints, step by step, the operator chosen, its gradient and the re-optimized energy.

Pools. The pool is where the physics enters, and three kinds are used below. For a fermionic system the natural pool is the set of spin-conserving singles and doubles, $i(a_q^\dagger a_p - \text{h.c.})$ and $i(a_r^\dagger a_t^\dagger a_q a_p - \text{h.c.})$, in the Jordan–Wigner encoding, which is the pool of the original fermionic ADAPT-VQE and is what unitary coupled cluster would use all at once; for the pairing model in the compact encoding it is the six pair transfers $-\frac{1}{2}(Y_iX_a - X_iY_a)$ between all pairs of levels. For a spin model one can use the *qubit pool* of [**?**], the individual Pauli strings of low weight, which are cheaper to exponentiate than the fermionic operators and give up the particle-number conservation; or operators built from the symmetries of the model, the collective quasispin operators of the Lipkin model or, for the Heisenberg ring, the bond spin currents $X_iY_j - Y_iX_j$ and the scalar chiralities $\boldsymbol{S}_i\cdot(\boldsymbol{S}_j\times\boldsymbol{S}_k)$. One condition is common to all of them and easy to get wrong. The Hamiltonians of this book are real symmetric matrices and their ground states can be chosen real, and a real reference state has, for any real generator G, a gradient that vanishes identically, since $\langle\psi|[G,\hat{H}]|\psi\rangle$ is the difference of a real number and its transpose. A pool of real generators, strings with an even number of Y's, therefore never moves a real reference at all, and the algorithm stops at step zero reporting convergence. Every generator below is *imaginary*, a string with an odd number of Y's or a real combination of such strings, so that $e^{-i\theta G}$ is a real orthogonal matrix and the state stays real while the gradients do not vanish. The same reality is what made the single excitation of Eq. (4.20) a combination of YX and XY strings and the pair excitation of Eq. (4.23) likewise.

Results. Table 7.8 and Fig. 7.9 collect the runs. For the pairing model the algorithm chooses, in order, the four excitations from the occupied levels 0 and 1 to the empty levels 2 and 3, with gradients of order one, and after these four it is at 0.63698698, which is exactly the

Table 7.8 ADAPT-VQE on the four models: the pool, its size, the number of operators in the final circuit, the energy, its error and the overlap with the exact ground state. The gradient threshold is 10^{-6}; the Hubbard run is stopped at twelve operators. Produced by `adapt_vqe.py`.

model	reference, pool	pool	ops	E_{ADAPT}	error	$\lvert\langle E_0\rvert\psi\rangle\rvert^2$
pairing, $L=4$	$\lvert 1100\rangle$, six pair transfers	6	5	0.63554847	$<10^{-14}$	1.000000
Lipkin, $N=4$	$\lvert 0000\rangle$, qubit pool	28	7	−4.21287670	$<10^{-13}$	1.000000
	$\lvert 0000\rangle$, collective (2)	2	1	−4.21240735	4.7×10^{-4}	0.999942
	$\lvert 0000\rangle$, collective (3)	3	2	−4.21287670	$<10^{-14}$	1.000000
Heisenberg ring	singlet covering, currents and chiralities	8	1	−2.00000000	$<10^{-15}$	1.000000
	Néel $\lvert 0101\rangle$, same pool	8	3	−2.00000000	$<10^{-15}$	1.000000
Hubbard ring	$U\to 0$ ground state, singles and doubles	162	12	−2.09135394	1.1×10^{-2}	0.997668

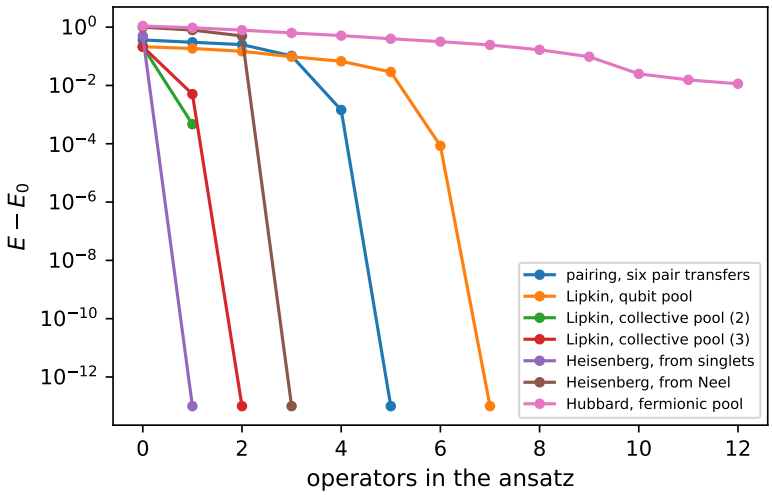


Fig. 7.9 The energy error of ADAPT-VQE against the number of operators in the circuit, for the runs of Table 7.8. Errors below 10^{-13} are drawn at that value. Produced by `adapt_vqe.py`.

UCCD energy of Table 7.6: it has rebuilt the unitary coupled-cluster circuit by itself. The fifth operator, the transfer between the empty levels 2 and 3, has a gradient of 1.1×10^{-2}, small but not zero, and it takes the energy to the exact value; the sixth, between the occupied levels, is then never needed, since every remaining gradient is below 10^{-6}. For the Lipkin model the qubit pool of the 28 one- and two-qubit strings with one Y needs seven operators, all of them X_pY_q or Y_pX_q type, and it is exact, so that the number-breaking strings have not hurt here; the collective pool with three operators needs two, the third-order $i(J_+^2J_z-J_zJ_-^2)$ first with a gradient of 16 and then $i(J_+^2-J_-^2)$, and is exact. The collective pool with two operators is the instructive failure. It contains $i(J_+-J_-)=2J_y$, which changes the quasispin projection by one, and $i(J_+^2-J_-^2)$, which changes it by two; the Hamiltonian of Eq. (4.11) conserves the parity of m, so that the first operator has a gradient that vanishes by symmetry in every state the second one can reach from the reference, and the second, once optimized, has a gradient that vanishes because it is at its own minimum. After one operator every gradient in the pool is zero, the algorithm reports convergence, and the energy is 4.7×10^{-4} too high, the exact ground state in the three-dimensional even-parity subspace needing two real angles that one operator cannot supply. This is the *gradient trap*: a zero gradient means only that no operator of the pool, applied once, lowers the energy to first order, and a pool that is too small, or whose operators are forbidden by a symmetry from connecting the current state to the target, produces a confident and wrong convergence. It is the adaptive version of the quiet failure of

Section 7.7.3, and the remedy is the same, a pool that is complete under the symmetries of the problem.

The Heisenberg ring is the success of the section. From the Néel state the algorithm takes the two bond currents on $(0,1)$ and $(2,3)$, which rotate the two Néel bonds into singlets, and then the scalar chirality on the sites $0,1,2$, three operators, and is exact. From the singlet covering it needs only the chirality: a single operator with a single angle takes the product of two singlets to the exact ground state of the ring, -2, to machine precision. The ground state of the four-site Heisenberg ring is a specific superposition of the two singlet coverings, and the chirality generator, which is the Hermitian combination of the spin currents that rotates one covering into the other, is the operator that connects them; no hardware-efficient circuit of Table 7.5 came within 8×10^{-2} of what one well-chosen generator does exactly. The Hubbard ring is the honest case. From the weak-coupling reference with the full pool of 162 spin-conserving singles and doubles, the algorithm chooses twelve doubles, each with a gradient between 0.2 and 1, and after twelve it is 1.1×10^{-2} above the exact energy with an overlap of 0.9977, still improving, on a circuit that in the Jordan–Wigner encoding would be some hundreds of CNOT gates deep. The two-layer Hamiltonian-variational ansatz of Table 7.6 was exact with four parameters, because its generators, the whole hopping term and the whole interaction, are the right collective operators for this problem in the way that the chirality was for the Heisenberg ring; a pool of individual excitations rebuilds them one piece at a time. That is the general trade-off of the adaptive method, and it is worth stating plainly: ADAPT-VQE removes the need to guess an ansatz, and replaces it with the need to guess a pool.

7.9 A second variational algorithm: QAOA

The variational loop of Fig. 7.1 does not care where the Hamiltonian came from. This last section applies it to a Hamiltonian that is diagonal in the computational basis and encodes a combinatorial optimization problem, so that its ground state is a bit string and the "energy" is the cost of a candidate solution. The algorithm that does this, the quantum approximate optimization algorithm of Farhi, Goldstone and Gutmann [**?**], is a VQE with a fixed and very particular ansatz, alternating exponentials of the cost Hamiltonian and of a transverse field, and it is included here for three reasons. It is the form in which variational quantum computing is most often encountered outside physics; its ansatz is exactly the Trotterized adiabatic evolution of Chapter 5, so that the material of this book explains it completely; and it is the bridge to quantum machine learning, which is not a subject of this book but deserves to be located on the map.

7.9.1 From a combinatorial problem to a Hamiltonian

A large class of combinatorial problems can be written as *quadratic unconstrained binary optimization*, QUBO: minimize

$$C(\boldsymbol{x}) = \boldsymbol{x}^T \boldsymbol{Q}\boldsymbol{x} = \sum_i Q_{ii}x_i + \sum_{i\neq j} Q_{ij}x_i x_j, \qquad x_i \in \{0,1\}, \tag{7.30}$$

over the 2^n bit strings, with a real matrix $\boldsymbol{Q}$ that holds the problem; since $x_i^2 = x_i$ the diagonal carries the linear terms. Constraints are added as penalties, and the reduction of a problem to this form is a craft of its own. The substitution $x_i = (1-s_i)/2$ with $s_i = \pm1$ turns Eq. (7.30)

Table 7.9 The QUBO with $Q_{00}=Q_{11}=1$, $Q_{01}=Q_{10}=-2$ and its Ising form $\hat{H}_C=-Z_0Z_1+\frac{1}{2}(Z_0+Z_1)$, evaluated on every bit string. Produced by `qaoa.py`.

$\boldsymbol{x}$	(s_0,s_1)	$\boldsymbol{x}^T\boldsymbol{Q}\boldsymbol{x}$	$\langle\boldsymbol{x}\vert\hat{H}_C\vert\boldsymbol{x}\rangle$
00	$(+1,+1)$	0	0
01	$(+1,-1)$	1	1
10	$(-1,+1)$	1	1
11	$(-1,-1)$	-2	-2

into a classical Ising energy,

$$C=\sum_{i<j}J_{ij}s_is_j+\sum_i h_is_i+c,\qquad J_{ij}=\frac{Q_{ij}+Q_{ji}}{4},\quad h_i=-\frac{Q_{ii}}{2}-\sum_{j\neq i}\frac{Q_{ij}+Q_{ji}}{4},\quad c=\sum_i\frac{Q_{ii}}{2}+\sum_{i<j}\frac{Q_{ij}+Q_{ji}}{4},\tag{7.31}$$

and replacing s_i by the Pauli matrix Z_i, whose eigenvalue on $|x_i\rangle$ is $(-1)^{x_i}=s_i$, gives the *cost Hamiltonian*

$$\hat{H}_C=\sum_{i<j}J_{ij}Z_iZ_j+\sum_i h_iZ_i,\tag{7.32}$$

a Pauli sum of weight at most two, diagonal in the computational basis, with $\hat{H}_C|\boldsymbol{x}\rangle=(C(\boldsymbol{x})-c)|\boldsymbol{x}\rangle$. The ground state of $\hat{H}_C$ is the optimal bit string, and the many-body problem has become a spin glass. Table 7.9 carries the two-variable example $\boldsymbol{Q}=\begin{pmatrix}1&-2\\-2&1\end{pmatrix}$ through: $J_{01}=-1$, $h_0=h_1=\frac{1}{2}$, $c=0$, so that $\hat{H}_C=-Z_0Z_1+\frac{1}{2}(Z_0+Z_1)$, and its diagonal reproduces $\boldsymbol{x}^T\boldsymbol{Q}\boldsymbol{x}$ bit string by bit string, with the minimum -2 at $\boldsymbol{x}=(1,1)$.

The standard example, used for the rest of the section, is *MaxCut*: given a graph with vertices V and edges E, partition the vertices into two sets so that as many edges as possible cross the partition. With $z_i=\pm1$ marking the side of vertex i, an edge (i,j) is cut when $z_i\neq z_j$, that is when $(1-z_iz_j)/2=1$, so that the number of cut edges is the eigenvalue of

$$\hat{H}_C=\sum_{(i,j)\in E}\frac{I-Z_iZ_j}{2}\tag{7.33}$$

on $|\boldsymbol{z}\rangle$, and MaxCut is the search for the *largest* eigenvalue of $\hat{H}_C$, a maximization; the two conventions differ by a sign and we keep the maximization, as the literature does. MaxCut is NP-hard, and the best classical algorithm with a guarantee, the semidefinite relaxation of Goemans and Williamson, reaches 0.878 of the optimum on every graph [**?**].

7.9.2 The ansatz and where it comes from

The QAOA state of depth p is

$$|\boldsymbol{\gamma},\boldsymbol{\beta}\rangle=\prod_{l=p}^{1}\left[e^{-i\beta_l\hat{H}_M}\,e^{-i\gamma_l\hat{H}_C}\right]|+\rangle^{\otimes n},\qquad \hat{H}_M=\sum_{q=0}^{n-1}X_q,\tag{7.34}$$

with $2p$ real parameters, and the algorithm maximizes $F_p(\boldsymbol{\gamma},\boldsymbol{\beta})=\langle\boldsymbol{\gamma},\boldsymbol{\beta}|\hat{H}_C|\boldsymbol{\gamma},\boldsymbol{\beta}\rangle$ by the loop of Fig. 7.1, then samples the optimized state in the computational basis and keeps the best bit string seen. The two exponentials are the *phase separator* and the *mixer*. The first is diagonal, $e^{-i\gamma\hat{H}_C}|\boldsymbol{z}\rangle=e^{-i\gamma C(\boldsymbol{z})}|\boldsymbol{z}\rangle$, and imprints on every bit string a phase proportional to its cost; for

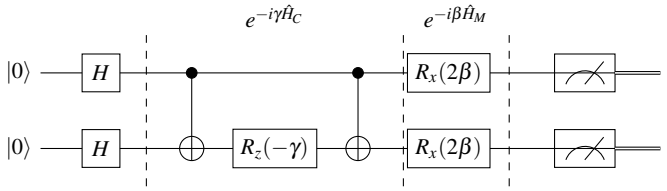


Fig. 7.10 One QAOA layer for MaxCut on a single edge: the uniform superposition, the phase separator $e^{-i\gamma(I-Z_0Z_1)/2}$ as a CNOT, $R_z(-\gamma)$, CNOT (up to a global phase), the mixer $R_x(2\beta)$ on both qubits, and the readout. For a graph the phase separator is one such gadget per edge; for depth p the two blocks are repeated with their own angles.

MaxCut it is one $e^{i\gamma Z_iZ_j/2}$ per edge, a CNOT, an $R_z(-\gamma)$ and a CNOT, the gadget of Fig. 4.3, and all the edges commute so that the order is immaterial and the depth is the largest vertex degree. The second is a product of single-qubit rotations $e^{-i\beta X_q} = R_x(2\beta)$ that turn the phases into amplitudes; without it nothing would ever change the outcome probabilities, which are uniform in $|+\rangle^{\otimes n}$. Figure 7.10 shows one layer for a single edge. The state is periodic in γ with period 2π when the costs are integers and in β with period π, and the function F_p inherits the symmetries of the graph, so that the search space is a torus of modest size.

Where does the ansatz come from? From the adiabatic theorem of Chapter 5 and the adiabatic algorithm of [**?**]. The ground state of $-\hat{H}_M$ is $|+\rangle^{\otimes n}$, easy to prepare; the ground state of $-\hat{H}_C$ is the optimal cut, which is what we want; and the interpolation

$$\hat{H}(s) = -(1-s)\hat{H}_M - s\hat{H}_C, \qquad s: 0 \to 1, \tag{7.35}$$

run slowly enough carries the one into the other. Discretizing the evolution under $\hat{H}(s)$ into p steps of length Δt and splitting each step by the Trotter formula of Eq. (3.45) gives exactly the product of Eq. (7.34) with $\gamma_l = s_l\Delta t$ and $\beta_l = (1-s_l)\Delta t$ up to signs: QAOA is Trotterized adiabatic evolution with the schedule handed to the optimizer. It follows that F_p increases with p, since a deeper circuit contains the shallower one, and that $F_p \to C_{\max}$ as $p \to \infty$, since the adiabatic limit is contained in the family; it also follows that the Trotter error, which is controlled by the commutator

$$[\hat{H}_C, \hat{H}_M] = -i \sum_{(i,j)\in E} (Y_iZ_j + Z_iY_j) \tag{7.36}$$

for MaxCut, is not an error at all in the variational setting, because the angles are re-optimized rather than fixed by a schedule, which is the observation of Section 3.6.4 in another guise. The QAOA ansatz is, in the language of Section 7.3.3, the Hamiltonian-variational ansatz of Eq. (7.10) for the Hamiltonian $-\hat{H}_C - \hat{H}_M$ with $|+\rangle^{\otimes n}$ as the reference; the mixer plays the role of the hopping term and the cost that of the interaction.

7.9.3 What can be computed by hand, and what the program finds

At $p = 1$ the expectation value of one edge term can be computed in closed form, and the calculation is short enough to be worth doing. For a single edge the operator Z_0Z_1 is carried through the mixer in the Heisenberg picture, $e^{i\beta X}Ze^{-i\beta X} = \cos 2\beta\, Z + \sin 2\beta\, Y$ on each qubit, so that $Z_0Z_1 \to \cos^2 2\beta\, Z_0Z_1 + \sin 2\beta \cos 2\beta\, (Z_0Y_1 + Y_0Z_1) + \sin^2 2\beta\, Y_0Y_1$; then through the phase separator, where Z_0Z_1 and Y_0Y_1 commute with Z_0Z_1 and have zero expectation in $|{+}{+}\rangle$, while $e^{-i\gamma Z_0Z_1/2}$ conjugating Z_0Y_1 gives $\cos\gamma\, Z_0Y_1 - \sin\gamma\, X_1$, whose expectation in $|{+}{+}\rangle$ is $-\sin\gamma$, and the same for Y_0Z_1. Together,

Table 7.10 QAOA for MaxCut on the four-vertex ring ($C_{\max} = 4$) and on the six-vertex prism graph, two triangles joined by three edges ($C_{\max} = 7$): the optimized expectation value F_p, the approximation ratio $F_p/C_{\max}$, and the probability that a sample of the optimized state is an optimal cut, for depths $p = 1, 2, 3$, best of ten random starts. Produced by `qaoa.py`.

graph	p	F_p	$F_p/C_{\max}$	P(optimal cut)
ring C_4	1	3.0000	0.7500	0.5312
	2	4.0000	1.0000	1.0000
	3	4.0000	1.0000	1.0000
prism, 6 vertices	1	5.9392	0.8485	0.4008
	2	6.6029	0.9433	0.7209
	3	6.8780	0.9826	0.9233

$$\langle Z_0 Z_1 \rangle_{\gamma,\beta} = -\sin 4\beta \, \sin \gamma, \qquad F_1 = \frac{1 + \sin 4\beta \sin \gamma}{2}, \tag{7.37}$$

which the program verifies against the simulator on a grid of 13×11 angles to 7×10^{-16}, and whose maximum, $F_1 = 1$ at $\gamma = \pi/2$, $\beta = \pi/8$, solves the one-edge problem exactly. On a graph the same calculation gives, for an edge (u, v) that belongs to no triangle and whose end points have degrees d_u and d_v, $\langle C_{uv} \rangle = \frac{1}{2} + \frac{1}{4} \sin 4\beta \sin \gamma (\cos^{d_u - 1} \gamma + \cos^{d_v - 1} \gamma)$, because every other edge at u contributes a factor $\cos \gamma$ when the phase separator is carried through; for the ring, $d = 2$ at every vertex,

$$F_1(\gamma, \beta) = \frac{n}{2} \left(1 + \frac{1}{2} \sin 4\beta \sin 2\gamma \right), \tag{7.38}$$

with maximum $3n/4$, which the program also verifies. The ratio $3/4$ of the optimum for the ring at $p = 1$ is the number Farhi, Goldstone and Gutmann computed in the original paper, and it is below the Goemans–Williamson 0.878; a depth of one is not competitive, and the question of the field is how fast $F_p/C_{\max}$ rises with p.

Table 7.10 and Figs. 7.11 and 7.12 show what the program finds. On the ring, $p = 1$ gives $3/4$ as the formula says, and the landscape of Fig. 7.11 is the function of Eq. (7.38), smooth and periodic with its maxima where $\sin 4\beta \sin 2\gamma = 1$; $p = 2$ is exact, $F_2 = 4$, and the optimized state is a superposition of the two optimal cuts and nothing else. The four-vertex ring is a bipartite graph, and a depth of two suffices for it in the same way that two layers of the Hamiltonian-variational ansatz sufficed for the Hubbard ring. The six-vertex prism graph, which has triangles and is not bipartite, behaves as a hard instance does: the ratio climbs from 0.85 at $p = 1$ through 0.94 to 0.98 at $p = 3$, and the probability of sampling one of the six optimal cuts from 0.40 to 0.92, so that after three layers a single sample is almost certainly the answer; Fig. 7.12 shows the six optimal strings rising out of the distribution as p grows. What is being bought with depth is concentration, and Fig. 7.12 is the picture to keep in mind: the algorithm does not compute the cut, it prepares a distribution from which the cut is likely to be drawn. Everything said about VQE applies. The energy is measured from shots, here a single setting since $\hat{H}_C$ is diagonal; the gradients follow from the parameter-shift rule, with the caveat that γ multiplies a sum of commuting strings with several eigenvalue gaps, so that the generalized rule of Eq. (7.25) or a per-string shift is needed; the landscape has many local maxima and the program uses ten random starts; and at large n and p the landscape flattens into a barren plateau. What is specific to QAOA is that the classical competition is strong, since the semidefinite relaxation and the heuristics of the operations-research literature solve instances of thousands of variables, and that no instance is known on which a shallow QAOA beats them. The algorithm is valuable as a laboratory for variational circuits and as the simplest setting in which their behaviour can be worked out by hand, and it is the natural entry point for what follows.

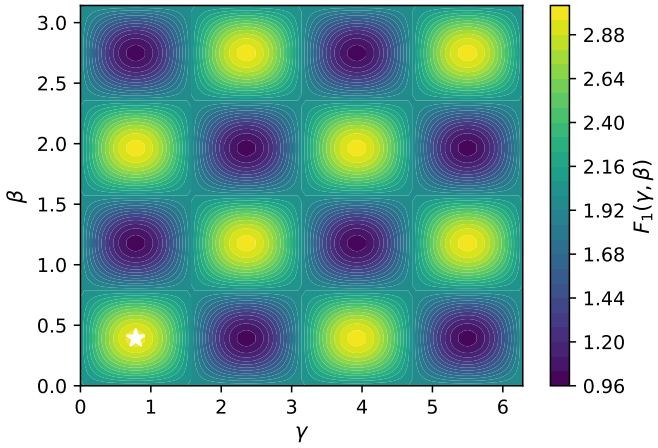


Fig. 7.11 The $p = 1$ landscape $F_1(\gamma, \beta)$ of QAOA for MaxCut on the four-vertex ring, Eq. (7.38), over one period in each angle; the star is the optimum found by the program, $F_1 = 3$. Produced by `qaoa.py`.

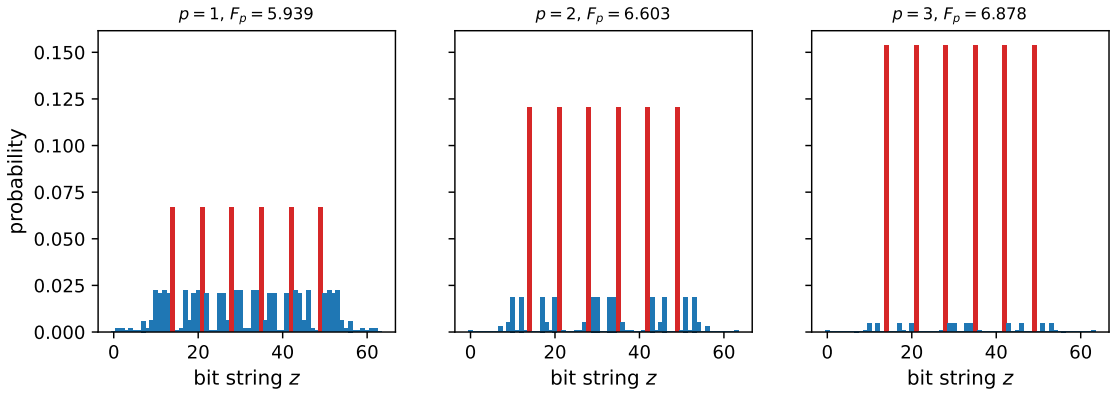


Fig. 7.12 Output distributions of the optimized QAOA state for MaxCut on the six-vertex prism graph at depths $p = 1$, 2 and 3, over the 64 bit strings; the six optimal cuts are drawn in red. Produced by `qaoa.py`.

7.9.4 Perspectives: variational circuits as machine-learning models

QAOA is where the methods of this book touch quantum machine learning, and it is worth seeing why, even though the subject is not treated here. Read Eq. (7.34) once more. The matrix $\boldsymbol{Q}$ of the QUBO is *data*: it could be the adjacency matrix of a graph, a covariance matrix of a portfolio, the couplings of a Hopfield network, or a distance matrix between samples, and the circuit is a parametrized function of that data whose parameters are trained by minimizing a cost. That is a machine-learning model. The general form, of which QAOA and VQE are both instances, is a parametrized circuit $U(\boldsymbol{x}, \boldsymbol{\theta})$ in which some gates carry the input $\boldsymbol{x}$ and others the trainable parameters, and an output $f(\boldsymbol{x}; \boldsymbol{\theta}) = \langle 0|U^\dagger \hat{O} U|0\rangle$ for some observable $\hat{O}$; the training loop is the loop of Fig. 7.1 with a loss function in the place of the energy, the gradients come from the same parameter-shift rule, and the difficulties are the same shot noise and the same barren plateaus, which in this setting are made worse by the need to evaluate the circuit on every sample of a training set. The data can also enter the way it enters QAOA, through the Hamiltonian whose exponential is the first layer, in which case the circuit's output is a feature map $\boldsymbol{x} \mapsto |\phi(\boldsymbol{x})\rangle$, and the overlap $|\langle\phi(\boldsymbol{x})|\phi(\boldsymbol{x}')\rangle|^2$, measurable by the same method as the deflation overlap of Section 7.7.3, is a kernel that can be handed to

a classical support-vector machine [**?**, **?**]. Finally, the optimized QAOA state sampled in the computational basis is a generative model over bit strings, a Born machine, whose training adjusts the angles so that the samples follow a target distribution. The reviews [**?**, **?**] set out the field. Its questions, which models can be trained without barren plateaus, which kernels cannot be evaluated classically, and whether any of it has an advantage over classical learning on classical data, are open; the questions of this book, whether a variational circuit can prepare the ground state of a many-body Hamiltonian that no classical method can reach, share their tools and not their answers, and the reader who has followed this chapter has what is needed to read either literature.

7.10 Summary

The variational quantum eigensolver minimizes $\langle\psi(\boldsymbol{\theta})|\hat{H}|\psi(\boldsymbol{\theta})\rangle$ over a family of states prepared by a parametrized circuit, using the quantum computer to evaluate the energy and a classical one to move the parameters, and what it returns is a variational upper bound with the same standing as any other in this book. The energy is obtained by decomposing $\hat{H}$ into Pauli strings, rotating each into the computational basis with nothing, H or $S^\dagger H$ on each qubit, and averaging parities over shots; the statistical error falls as $1/\sqrt{N}$ and this, not the circuit, is the dominant cost. Correlators must be measured jointly: reconstructing $\langle Z_0 Z_1\rangle$ from $\langle Z_0\rangle\langle Z_1\rangle$ is wrong, costs nothing to avoid and fails silently. Gradients come from the parameter-shift rule, exact when the gate generators have two eigenvalues and interpolation rather than approximation; generators with the three eigenvalues of the excitation operators need the four-term rule. SPSA estimates a gradient from two energies whatever the number of parameters, and for a generic circuit the gradient's variance decays exponentially with the number of qubits.

Applied to the models of Chapters 4 and 6, the hardware-efficient ansatz reached the exact energies of none of them, came within 10^{-3} for the Lipkin model at three layers, and for the pairing and Hubbard models left the particle-number sector and found lower states of the register that are not the states we wanted. Ansätze built from the physics were exact with six, six, four and two parameters: the Hamiltonian-variational ansatz for the Lipkin model, all six pair transfers for the pairing model, two layers of hopping and interaction from the weak-coupling reference for the Hubbard ring, one layer of the two bond groups from the singlet covering for the Heisenberg ring. The quality of a variational result is a property of the ansatz, and symmetry is the most reliable guide to a good one. ADAPT-VQE grows the circuit from a pool by the largest gradient $i\langle[G,\hat{H}]\rangle$, rebuilt the unitary coupled-cluster circuit for the pairing model by itself, found the Heisenberg ground state with a single scalar-chirality operator, and stopped in a gradient trap when the pool was incomplete under a symmetry. Against phase estimation, VQE trades an exponential precision guarantee for shallow circuits and noise tolerance; QPE fails loudly and needs a machine that does not exist, VQE fails quietly and runs today, and the overlaps of Tables 7.5 and 7.6 are the reason the most plausible use of the two is together. QAOA is the same loop with the Trotterized adiabatic evolution as its ansatz and a combinatorial problem as its Hamiltonian, solvable by hand at $p = 1$, exact at $p = 2$ for the four-vertex ring, and the meeting point of variational quantum algorithms with machine learning.

7.11 Exercises

1. **The variational principle on a circuit.** (a) Prove Eq. (7.1) and show that equality holds if and only if $|\psi\rangle$ lies in the ground-state eigenspace. (b) Show that the ansatz of Eq. (7.6) reaches every pure state of one qubit up to a global phase, and that R_y alone reaches every real state. Derive Eq. (7.28) and its minimum. (c) The hardware-efficient ansatz with L layers on n qubits has $2nL$ parameters, while a general state has $2^{n+1}-2$ real degrees of freedom. At what L do the counts match, and why does matching them not imply that the ansatz reaches every state? (d) Show that if a noisy device prepares a mixed state ρ instead of $|\psi\rangle\langle\psi|$, then $\mathrm{Tr}(\rho\hat{H}) \geq E_0$ still holds. Which property of ρ does the work? (e) Explain why the two-layer hardware-efficient result of Table 7.4 is *correct* and nevertheless useless, and propose two ways of obtaining 0.916905 with the same ansatz, one that changes the Hamiltonian and one that changes the reference state. Test both with `vqe.py`.
2. **Measurement.** (a) Verify $HZH = X$ and $HS^{\dagger}YSH = Z$ by direct multiplication. (b) Write down the measurement circuit for the string $X \otimes Y \otimes Z \otimes I$ and the classical post-processing that turns an outcome bit string into ± 1. (c) Derive Eq. (7.12) and show that the variance vanishes when $|\psi\rangle$ is an eigenstate of P. (d) Partition the strings of the Lipkin Hamiltonian of Eq. (4.11) into qubit-wise commuting groups by hand and compare with Table 4.12. (e) Reproduce Table 7.2 with `measurement.py`, extend it to 10^5 shots, fit the standard error to aN^{-b} and check $b = 1/2$. Then repeat the whole VQE optimization of Table 7.5 for the Heisenberg ring with the energies drawn from 10^2, 10^3 and 10^4 shots per setting (the argument `shots` of `VQE`), using SPSA, and plot the converged energy against the shot count. At what shot count does the ansatz error dominate the statistical error?
3. **The marginal trap.** (a) Compute the reduced density matrices of $|\Phi^+\rangle$ and $|\Psi^+\rangle$ and confirm the claim in the proof of Theorem 7.1. (b) Derive Eq. (7.17) and show that the error of the reconstruction equals the squared concurrence of the state. (c) Show that $\langle Z_0 Z_1\rangle = \langle Z_0\rangle\langle Z_1\rangle$ for every product state, and construct an entangled state for which it holds accidentally. What does the existence of such states imply about detecting the error empirically? (d) Run the marginal readout of `measurement.py` on the Lipkin Hamiltonian with the three-layer hardware-efficient ansatz and report both the energy the optimizer believes and the true energy of the state it prepared. Can the reported number lie below E_0?
4. **The parameter-shift rule.** (a) Fill in the algebra between Eq. (7.20) and Eq. (7.21), obtaining the coefficients A, B, C. (b) Show that three evaluations of $E(\theta)$ at distinct points determine the three coefficients, and hence that the rule is exact interpolation. (c) For $G = \frac{1}{2}(Z_0 + Z_1)$ derive Eq. (7.24) and work out exactly which frequency components the two-term rule of Eq. (7.22) sees and which it loses. (d) Verify that Eq. (7.25) reduces to Eq. (7.22) at $R = 1$, and derive it at $R = 2$ by differentiating Eq. (7.24) and solving for the four shifted energies. (e) Show that the pair-excitation generator $-\frac{1}{2}(Y_i X_a - X_i Y_a)$ has the eigenvalues $\{-1, 0, 0, +1\}$ and explain, from the structure of Eq. (4.24), why it must. (f) Estimate the number of shots for one gradient of a 20-qubit, five-layer hardware-efficient ansatz on a Hamiltonian with 500 strings in 30 groups at 10^4 shots per group, and for the same gradient with SPSA.
5. **Barren plateaus.** (a) Reproduce Fig. 7.7 with `gradients.py` and extend it to $n = 10$. (b) Repeat with a fixed number of layers, $L = 2$, and with the Heisenberg ring Hamiltonian in place of $Z_0 Z_1$; which of the two changes the slope? (c) Repeat with the Hamiltonian-variational ansatz of the Heisenberg ring at $L = n$ from the singlet covering, sampling the angles from $[-\pi, \pi]$ and then from $[-0.1, 0.1]$, and explain the difference. (d) Argue from Eq. (7.12) how many shots are needed to resolve a gradient of variance 2^{-n}, and at what n this exceeds 10^{12}.

6. **Ansätze that know the physics.** (a) Show that the three commuting groups of the Lipkin Hamiltonian of Eq. (4.11) are permutation symmetric and hence that the Hamiltonian-variational ansatz from $|0000\rangle$ stays in the $J = 2$ subspace; how many real parameters does the ground state of that subspace need, and why is one layer of the ansatz not exact? (b) Show that the six pair transfers of the compact pairing encoding generate the full seniority-zero space of two pairs in four levels from $|1100\rangle$, and that the four excitations from the reference do not, by counting the states each circuit can reach at first order. (c) For the Hubbard ring, write down the ground state of the non-interacting Hamiltonian in the four-particle, $S_z = 0$ sector and show that it is six-fold degenerate. Which combination does the interaction select at first order in U? Compare with the overlap 0.8509 of Table 7.6. (d) Show that the product of two singlets on the bonds $(0,1)$ and $(2,3)$ is the ground state of the even-bond part of the Heisenberg ring, and that the exact ground state of the ring is a linear combination of the two singlet coverings. Find the coefficients.
7. **ADAPT-VQE.** (a) Derive Eq. (7.29) and show that $i[G,\hat{H}]$ is Hermitian, so that the gradient is a measurable expectation value. (b) Prove that for a real reference state, a real symmetric Hamiltonian and a real symmetric generator the gradient vanishes identically, and that for an imaginary generator (an odd number of Y's) $e^{-i\theta G}$ is a real orthogonal matrix. (c) Explain the gradient trap of the two-operator collective pool in Table 7.8 from the parity of the quasispin projection, and show that the three-operator pool is complete in the even subspace. (d) Show that the scalar chirality $\boldsymbol{S}_0 \cdot (\boldsymbol{S}_1 \times \boldsymbol{S}_2)$ is an imaginary Hermitian operator, write it as a sum of six Pauli strings, and verify with `adapt_vqe.py` that one angle takes the singlet covering to the exact ground state of the ring. (e) Run the Hubbard case of `adapt_vqe.py` to 24 operators and plot the error against the number of operators and against the number of CNOT gates of the corresponding circuit (each double excitation is eight strings; use the gate counts of Chapter 4). Compare with the four-parameter Hamiltonian-variational ansatz.
8. **Excited states.** Implement the deflation of Section 7.7.3: after optimizing the ground state $|\psi_0\rangle$ of the Lipkin model with the two-layer Hamiltonian-variational ansatz, minimize $\langle\psi|\hat{H}|\psi\rangle + \mu|\langle\psi_0|\psi\rangle|^2$ over the same ansatz, and show that for μ larger than the gap the result is the first excited state of the $J = 2$ multiplet. Compare with the levels read off Fig. 6.10. Explain how the overlap is measured on hardware, and why the method becomes expensive for the k-th excited state.
9. **QAOA.** (a) Derive Eq. (7.31) and check it on the example of Table 7.9. (b) Show that $e^{-i\gamma(I-Z_0Z_1)/2}$ is, up to a global phase, the CNOT–$R_z(-\gamma)$–CNOT gadget of Fig. 7.10, and that $e^{-i\beta X} = R_x(2\beta)$. (c) Carry out the Heisenberg-picture calculation of Eq. (7.37) in full, and extend it to an edge of a triangle-free graph with degrees d_u and d_v. (d) Derive Eq. (7.35)–(7.36), and show that $F_{p+1} \geq F_p$. (e) Encode the graph-colouring or the number-partitioning problem as a QUBO, build its cost Hamiltonian with `qaoa.py` and run QAOA at $p = 1,2,3$. (f) Show that the QAOA state of Eq. (7.34) is the Hamiltonian-variational ansatz of Eq. (7.10) for $-\hat{H}_C - \hat{H}_M$, and run the `PauliRotationAnsatz` of `vqe.py` on the ring MaxCut Hamiltonian to confirm that it reproduces Table 7.10.
10. **Comparing the two algorithms.** (a) For a target accuracy ε, estimate the number of measurements VQE needs and the circuit depth phase estimation needs, both as functions of Λ and ε, and find where the two cross for $\Lambda = 10$ and $\Lambda = 10^3$. (b) Table 6.5 contains phase-estimation results below the exact energy. Explain why this is not an error and why no variational run can produce such a number. (c) Take the three-layer hardware-efficient state of the Lipkin model from Table 7.5 as the trial state of the `PhaseEstimation` class of Chapter 6 with $m = 8$ control qubits, and compare the probability of reading the ground-state bits with the entries of Table 6.9. Repeat with the two-layer pairing state, and explain the result. (d) Both algorithms need something they cannot supply themselves, a state with a large overlap and an expressive ansatz. Argue that these are the same difficulty in

different clothes, and identify one respect in which they are not. (e) Write a paragraph, of the kind that would appear in a paper, stating what has and has not been demonstrated by the calculations of Chapters 6 and 7 on the four models.

Programs for this chapter

Every number and figure in this chapter comes from the programs in `BookPrograms/chapter7/`, which build on the shared modules of Chapters 2–4 (`statevector`, `circuit`, `gates`, `pauli`, `models`, `optimize`, `jordanwigner`), on `mapped_models.py` and `pauli_circuits.py` of Chapter 4, and on `qpe_models.py` of Chapter 6 for the model Hamiltonians. All of them are imported by the notebook `Notebooks/chapter7/chapter7.ipynb`, and all run on `numpy` and `matplotlib` alone.

`vqe.py` The ansatz classes `HardwareEfficient`, Eq. (7.7), `ProductState` and `PauliRotationAnsatz`, Eq. (7.9); the class `VQE` with exact or sampled energies, the parameter-shift gradients of Eqs. (7.22) and (7.25), gradient descent, BFGS and the SPSA of Eq. (7.26); and `run_restarts`. As a script it checks the ansätze against the circuits of Chapter 4, the shift rules against finite differences, and the three optimizers on the two-mode Hamiltonian. A few seconds.

`measurement.py` The rotation recipe of Table 7.1, the shot-noise experiment of Table 7.2 and Fig. 7.4, the Bell-state counterexample of Theorem 7.1, the landscapes of Fig. 7.5 and the marginal VQE run of Table 7.3. About ten seconds.

`gradients.py` The trigonometric form of Eq. (7.21) fitted and tested, the two-term rule against finite differences, the three-eigenvalue generator of Eq. (7.23) and the four-term rule `general_shift`, the barren plateau of Fig. 7.7 and the SPSA comparison of Fig. 7.6. About fifteen seconds.

`vqe_models.py` The generators `pair_generator`, `single_generator` and `hva_generators`, the overlap with the exact sector ground state, and the calculations of Tables 7.4, 7.5, 7.6 and 7.7 and Fig. 7.8. About one minute, the eight-qubit Hubbard runs dominating.

`adapt_vqe.py` The class `AdaptVQE` with the pool gradients of Eq. (7.29), the pools `lipkin_pools`, `heisenberg_pool` and `hubbard_pool`, and the runs of Table 7.8 and Fig. 7.9. About fifteen seconds.

`qaoa.py` `qubo_to_ising`, `ising_hamiltonian` and `maxcut_hamiltonian`, Eqs. (7.31)–(7.33); the class `QAOA` implementing Eq. (7.34) with the optimization over $(\boldsymbol{\gamma}, \boldsymbol{\beta})$; the checks of Table 7.9 and of Eqs. (7.37) and (7.38); Table 7.10, Fig. 7.11 and Fig. 7.12. About five seconds.

Chapter 8
Advanced Eigenstate Algorithms: The Rodeo Algorithm and Filtering, Ryan and Dean

Abstract This chapter presents algorithms that prepare and extract specific eigenstates of a Hamiltonian by filtering, together with a way of supplying them with the input they need. We first introduce resolution refinement, which lifts an eigenstate of a coarse model space into a finer one with a prolongation operator and follows a gapped adiabatic path between the two Hamiltonians, delivering a state with a large overlap with the target. We then derive the rodeo algorithm, a measurement-driven stochastic method in which one ancilla qubit, controlled time evolutions of random duration and post-selection suppress all components outside a target energy window exponentially fast, and which also serves as a spectrometer. The two are combined end to end on the pairing model and compared, as state-preparation methods, with the variational eigensolver and with quantum phase estimation in terms of resource requirements and precision. The same pipeline is then applied to the Lipkin, Heisenberg and Hubbard Hamiltonians. The chapter closes with a gate-level compilation of every stage, including Trotterized evolution, and a depth budget that is held against present and forthcoming hardware as an overview of the bottlenecks of quantum computing for many-body problems. Other spectral filtering techniques are mentioned as alternative approaches.

8.1 Introduction

The last two chapters ended on the same sentence, read from opposite sides. Phase estimation, Chapter 6, returns an eigenvalue to any precision *provided* it is handed a state with a large overlap with the eigenstate it is asked about, and is silent about where that state comes from. The variational eigensolver, Chapter 7, produces a state rather than assuming one, but returns an upper bound whose quality is a property of the ansatz, and no feature of the output says how good it is. Section 7.7.3 concluded that the difficulty of the many-body problem is conserved: one method hides it in state preparation and circuit depth, the other in ansatz design and measurement count. This chapter is about a third family of algorithms, the *projective spectral filters*, and about a way of supplying them with the input they need.

A filter is an operation that multiplies the component of a state along the eigenvector $|E_j\rangle$ by a factor $F(E_j)$ that is close to one in a window around a chosen energy and small outside it. Applied to $|\psi\rangle = \sum_j c_j |E_j\rangle$ and followed by normalization, it returns a state that is closer to the eigenstate in the window. Phase estimation is a filter of this kind, as Section 6.6.3 showed when it collapsed the system register by reading the control register; filters built from polynomials of $\hat{H}$ by quantum signal processing [**?**, **?**] and projection by measurement [**?**] are others. The one this chapter develops in detail is the *rodeo algorithm* [**?**, **?**], which uses a single ancilla qubit, measured and reused, and controlled time evolutions of random duration. Its filter factor is a product of cosines, off-resonant components are suppressed by a factor

of two per cycle on average, and the error of the prepared state therefore falls *exponentially* with the number of cycles. The name is apt: every cycle gives the state a shake, and only the eigenstate at the chosen energy stays on.

All filters share one limitation, and it is the one we met for phase estimation. A filter cannot create weight that is not there. If the input has squared overlap $p = |c_0|^2$ with the target, the filter succeeds with probability p, the expected cost grows as $1/p$, and for an uninformed input p falls exponentially with the number of particles [**?**]. The exponential convergence is real, and it begins only once somebody has supplied a good input. The first half of the chapter is therefore not about filtering at all. It introduces *resolution refinement* [**?**], a state-preparation method that bootstraps an eigenstate from a coarse model space, where it is cheap to prepare, to a fine one, by lifting it with a *prolongation* operator and following an adiabatic path between the two Hamiltonians. Because coarse and fine Hamiltonians describe the same low-energy physics, the path stays gapped, and the output has an overlap close to one. It also has a slow tail: the last percent of overlap costs as much adiabatic time as the first ninety-nine. That is exactly the profile a filter wants, and the two methods are complementary in a precise sense. Refinement supplies the large p that the rodeo algorithm requires; the rodeo algorithm removes, exponentially fast, the tail that refinement leaves behind.

The order of the chapter follows this logic. Section 8.2 develops resolution refinement and runs it on the pairing model of Section 3.3.2, refined from two single-particle levels to four. Section 8.3 derives the rodeo algorithm from its one-ancilla circuit, works through a one-qubit example by hand, and establishes the convergence criterion. Section 8.4 puts the two together, end to end, on the pairing Hamiltonian with four particles on four levels, eight qubits in the Jordan–Wigner encoding of Section 4.4.3, where every stage can be checked against exact diagonalization. Section 8.5 sets the result against the variational calculations of Chapter 7 and the phase-estimation results of Chapter 6 for the same Hamiltonian, comparing the three as state-preparation methods at equal cost. Section 8.6 carries the pipeline over to the other three Hamiltonians of the book, the Lipkin model, the Heisenberg ring and the Hubbard ring, each with the resolution hierarchy that suits it, and to a genuine two-stage lattice refinement. The chapter closes, in Section 8.7, with what is perhaps its most broadly useful part: every stage is compiled to rotations and CNOT gates, the product-formula errors are put back in, the gates are counted, and the count is held against what quantum hardware can execute. The resulting budget is an overview, on one fully controlled example, of the bottlenecks of quantum computing for many-body physics: overlap, coherent depth, Trotter error, encoding, optimization, and noise. The programs are in `BookPrograms/chapter8/`, and every number quoted below is printed by one of them.

8.2 Resolution refinement

8.2.1 The idea

Every calculation in this book has a resolution. For the pairing model it is the number k of single-particle levels; for a lattice model it is the lattice spacing; for a nucleus it is the size of the harmonic oscillator basis; for a molecule the size of the orbital basis. The resolution sets the number of qubits, $2k$ for k doubly degenerate levels in the Jordan–Wigner encoding, and with it the cost of every gate sequence. It does *not* set the physics. A well-constructed low-resolution Hamiltonian $\hat{H}_{\mathrm{low}}$ has the same low-lying states as the high-resolution $\hat{H}_{\mathrm{high}}$, only described more coarsely; this is the content of effective field theory and of the renormalization group, and classical numerical analysis has exploited it for half a century in multigrid methods, where the solution of a differential equation on a coarse grid is interpolated to a

fine grid and used there as the starting point of a short relaxation. Resolution refinement [**?**] is the quantum version of the multigrid idea, and it has three steps:

1. *prepare* an eigenstate $|\psi_{\text{low}}\rangle$ of $\hat{H}_{\text{low}}$ on a small register, by any method; the register is small, so a variational circuit, phase estimation, or even a classical calculation followed by state loading are all affordable;
2. *prolong* it into the large register, $|\Phi_0\rangle = \hat{P}|\psi_{\text{low}}\rangle$, with an isometry $\hat{P}$ that is cheap to implement;
3. *refine* it by evolving adiabatically from a Hamiltonian of which $|\Phi_0\rangle$ is an eigenstate to $\hat{H}_{\text{high}}$.

The third step is adiabatic state preparation, which Section 7.9.2 met as the ancestor of QAOA, and the reader may ask what has been gained, since adiabatic preparation has a well-known cost: the evolution time grows as the inverse square of the smallest gap along the path [**?**], and nothing prevents the gap between an arbitrary starting Hamiltonian and the target from closing. What has been gained is the path. Coarse and fine Hamiltonians share their low-energy spectrum, so the interpolation between them does not pass through a rearrangement of the ground state; the gap stays open, of the order of the physical gap, and the required time grows only as its inverse. The steps can also be chained, from k to $2k$ to $4k$, each stage starting from an overlap that is already large.

8.2.2 The prolongation operator

The prolongation $\hat{P}$ maps the coarse N-particle space into the fine one. It is an isometry, $\hat{P}^\dagger\hat{P} = \mathbb{1}$, while $\hat{P}\hat{P}^\dagger$ is the projector onto the image of the coarse space inside the fine space, and $\hat{P}^\dagger$ annihilates everything outside that image. The most general form we need acts on the creation operators,

$$\hat{P}: \quad a_c^\dagger \longmapsto \sum_f W_{fc}\, a_f^\dagger, \qquad \sum_f W_{fc}^* W_{fc'} = \delta_{cc'}, \tag{8.1}$$

where c labels coarse and f fine single-particle states. A coarse Slater determinant with occupied states $C = \{c_1 < \cdots < c_N\}$ is then mapped, by expanding the product of the N sums and using the anticommutation relations, to

$$\hat{P}\, a_{c_1}^\dagger \cdots a_{c_N}^\dagger |0\rangle = \sum_{F=\{f_1<\cdots<f_N\}} \big|\big(W_{FC}\big)\, a_{f_1}^\dagger \cdots a_{f_N}^\dagger |0\rangle, \tag{8.2}$$

with W_{FC} the $N \times N$ submatrix of W with rows F and columns C. This is the formula behind the method `orbital` of `Prolongation` in `refinement.py`. Two special cases cover everything in this chapter.

Basis refinement. When the fine basis contains the coarse one, as for the pairing model with $k_{\text{low}} < k_{\text{high}}$ levels, W is a rectangular identity: every coarse level is the fine level with the same label and the new levels are left empty. With the mode ordering of Chapter 3, mode $2p+\sigma$ for level p and spin σ, the prolonged register is the coarse register followed by $2(k_{\text{high}} - k_{\text{low}})$ qubits in $|0\rangle$,

$$\hat{P}|\psi_{\text{low}}\rangle = |\psi_{\text{low}}\rangle \otimes |0\cdots 0\rangle, \tag{8.3}$$

and the Jordan–Wigner strings of the old modes do not reach the new qubits. The prolongation costs no gates at all. Since $\hat{H}_{\text{high}}$ restricted to states with the new levels empty *is* $\hat{H}_{\text{low}}$, the prolonged state has $\langle\Phi_0|\hat{H}_{\text{high}}|\Phi_0\rangle = E_{\text{low}}$ exactly.

Lattice refinement. When a lattice is refined by halving its spacing, a particle on coarse site c is distributed over the two fine sites $2c$ and $2c+1$ that replace it, $W_{2c,c} = W_{2c+1,c} = 1/\sqrt{2}$.

On qubits this is a single-excitation rotation of the kind met in Section 4.5.5 between two neighbouring qubits, a constant number of gates per site, so that the prolongation costs $\mathcal{O}(N_{\text{sites}})$ gates per spatial direction [**?**]. The energy scale changes with the lattice spacing, so $\langle\Phi_0|\hat{H}_{\text{high}}|\Phi_0\rangle$ is no longer E_{low}; Section 8.6 returns to this.

8.2.3 The refinement path

The prolonged state is refined by following the ground state of the interpolated, energy-shifted Hamiltonian

$$\hat{H}(s) = \cos^2\Big(\frac{\pi s}{2}\Big)\,\hat{P}\big(\hat{H}_{\text{low}} - \mu\big)\hat{P}^\dagger + \sin^2\Big(\frac{\pi s}{2}\Big)\,\big(\hat{H}_{\text{high}} - \mu\big), \qquad s = \frac{t}{T}, \tag{8.4}$$

from $s = 0$ to $s = 1$ in a total time T. Three features of this expression deserve a comment.

The first is the shift μ. At $s = 0$ the Hamiltonian is $\hat{P}(\hat{H}_{\text{low}} - \mu)\hat{P}^\dagger$. On the image of the coarse space it has the eigenvalues $E^{\text{low}}_j - \mu$; on everything else, a space that is exponentially larger, it vanishes. If the coarse ground state is to be the ground state of $\hat{H}(0)$, and separated from that enormous null space by a gap, it must have a negative eigenvalue, which requires $\mu > E_{\text{low}}$. We write

$$\mu = E_{\text{low}} + \mu_{\text{buf}}, \tag{8.5}$$

with a *buffer* $\mu_{\text{buf}} > 0$ that is the gap at $s = 0$. If the coarse space has more than one state there is also a ceiling: a coarse excited state pushed below zero as well is harmless, but the ordering of the coarse levels must be the ordering one wants to track. The energy E_{low} is the expectation value of $\hat{H}_{\text{low}}$ in the state that was actually prepared, which is the only energy a device knows.

The second is the schedule. The functions $\cos^2$ and $\sin^2$ add to one, and both have zero slope at the two ends of the path, so that the Hamiltonian changes slowly where the adiabatic evolution begins and ends. The adiabatic theorem explains why this matters. For a state $|n(s)\rangle$ that follows the instantaneous ground state, the amplitude for a transition to the instantaneous level m is, to first order in $1/T$,

$$c_m(1) \simeq -\int_0^1 ds\,\frac{\langle m(s)|\partial_s\hat{H}|n(s)\rangle}{E_m(s) - E_n(s)}\,\exp\Big(iT\int_0^s \big[E_m(s') - E_n(s')\big]ds'\Big). \tag{8.6}$$

The phase oscillates with frequency $T\Delta E$, and an integration by parts turns the integral into boundary terms of order $\|\partial_s\hat{H}\|/(T\,\Delta E^2)$ plus corrections of higher order in $1/T$. With $\partial_s\hat{H} = 0$ at both ends the leading boundary terms vanish. In general, keeping the transition probability small requires $T \gg \|\partial_s\hat{H}\|/\Delta E^2_{\text{min}}$; when the matrix element in Eq. (8.6) is itself of the order of the gap, as it is when the two Hamiltonians differ only in how they resolve the same physics, this becomes $T \gtrsim 1/\Delta E_{\text{min}}$. It is this first-order dependence on the inverse gap that refinement delivers, and Section 8.2.5 verifies it numerically.

The third is the cost of one time step. On a quantum computer the evolution is a product of short-time propagators $e^{-i\hat{H}(s_j)\Delta t}$, each split by a product formula as in Eq. (3.45). The second term of Eq. (8.4) is the fine Hamiltonian with a rescaled time step. The first term is the coarse Hamiltonian acting on the old qubits, multiplied by the projector onto the new qubits being empty, $\hat{P}\hat{P}^\dagger = \prod_{q\,\text{new}}(1 - n_q)$; in the simplest case below it reduces to a multiply controlled phase. We count its gates in Section 8.7. Until then the slices $e^{-i\hat{H}(s_j)\Delta t}$ are applied exactly, so that the properties of the *path* can be separated from the errors of the *product formula*.

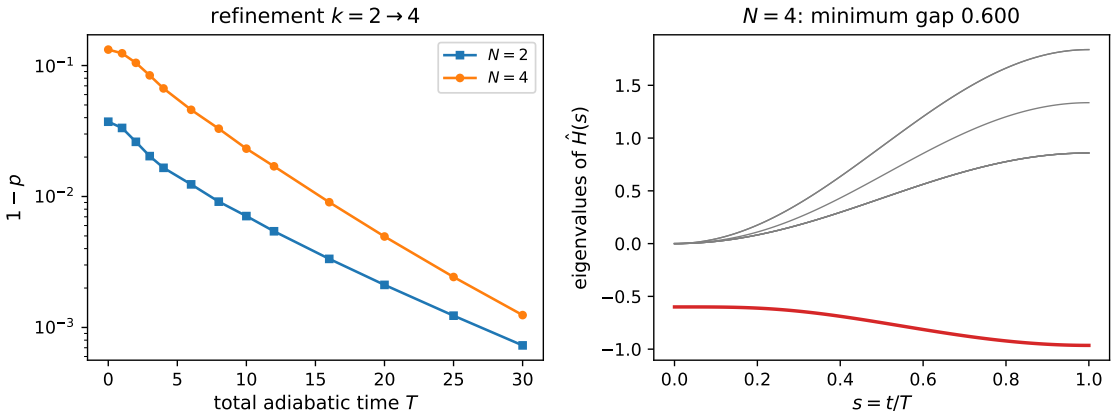


Fig. 8.1 Resolution refinement of the pairing model from $k = 2$ to $k = 4$ levels at $g = 1$, $\mu_{\text{buf}} = 0.6$. Left: the missing overlap $1 - p$ of the evolved state with the exact ground state against the total adiabatic time T, for two and four particles. Right: the lowest eigenvalues of $\hat{H}(s)$ of Eq. (8.4) for $N = 4$; the tracked level (thick) starts at $-\mu_{\text{buf}}$ below the null space of $\hat{P}^\dagger$ and stays separated from it along the whole path. Produced by `refinement.py`.

8.2.4 A first example: the pairing model from two levels to four

The pairing Hamiltonian of Eq. (3.28) has k doubly degenerate levels with spacing $\xi = 1$ and a pairing force of strength g that scatters time-reversed pairs between the levels,

$$\hat{H} = \xi \sum_{p=0}^{k-1} p \sum_{\sigma=\pm} a^\dagger_{p\sigma} a_{p\sigma} - \frac{g}{2} \sum_{p,q=0}^{k-1} a^\dagger_{p+} a^\dagger_{p-} a_{q-} a_{q+}. \tag{8.7}$$

where we label the levels from zero, so that level p and spin σ is the mode, and the qubit, $2p + \sigma$. It is the natural laboratory for this chapter. It conserves the number of broken pairs, the seniority, so that the ground state of an even number of particles lives in the small space of pair configurations; it is exactly solvable by the Richardson equations [**?**, **?**]; and "resolution" has an obvious meaning, the number of levels k at fixed particle number N. With $N = 4$ the N-particle sector has dimension $\binom{2k}{4}$: one state at $k = 2$, where the two lowest levels are simply full, 15 at $k = 3$, 70 at $k = 4$ and 1820 at $k = 8$, on $2k$ qubits. The Hartree–Fock determinant fills the two lowest levels and has the energy $2\xi - g = 1$ at $g = 1$, whatever k.

For $N = 4$ and $k_{\text{low}} = 2$ the coarse space is that single determinant. The "coarse solver" has nothing to do, the prolonged state is the Hartree–Fock determinant of the larger space, and $\hat{P}(\hat{H}_{\text{low}} - \mu)\hat{P}^\dagger = -\mu_{\text{buf}} |\Phi_0\rangle\langle\Phi_0|$ is a projector. This is the simplest possible instance, and a good one to start from because it isolates the path; a coarse stage that does real work follows in Section 8.4.2. For $N = 2$ the coarse space has six states and the coarse ground state is already correlated. In the programs the whole calculation is three lines:

```
from refinement import pairing_refinement

rr = pairing_refinement(k_low=2, k_high=4, N=4, g=1.0, buffer=0.6)
print(rr.overlap(rr.phi0), rr.energy(rr.phi0)) # 0.8676 1.0 (= E_low)
psi = rr.evolve(T=30, n_steps=160)          # exact slices of Eq. (8.path)
print(rr.overlap(psi), rr.energy(psi), rr.E_high, rr.min_gap())
```

which prints the overlap 0.9988, the energy 0.6397 against the exact $E_0 = 0.635548$, and the minimum gap 0.600.

Figure 8.1 shows what happens. The prolonged state already has an overlap of 0.963 with the $k = 4$ ground state for two particles and 0.868 for four; this is the multigrid effect, the

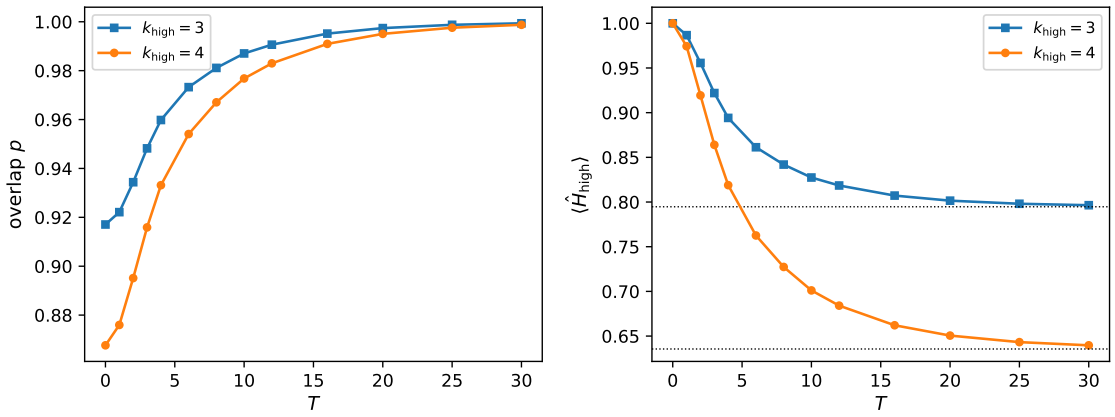


Fig. 8.2 Four particles refined from $k = 2$ to $k_{\text{high}} = 3$ and 4: overlap (left) and energy $\langle \hat{H}_{\text{high}} \rangle$ (right) against T. The energy approaches the exact value (dotted) from above, as it must for any state, while the overlap approaches one. Produced by `refinement.py`.

coarse solution is a good first guess. Under the evolution the missing overlap falls roughly exponentially in T, by a factor of a hundred between $T = 0$ and $T = 30$ for $N = 4$, where $p = 0.9988$. The right panel shows why this works: the tracked level starts at $-\mu_{\text{buf}} = -0.6$, all other levels start at zero and rise, and the gap never falls below its initial value. Figure 8.2 adds the energy and the intermediate resolution $k = 3$: the prolonged overlaps are 0.917 and 0.868, the overlaps at $T = 30$ are 0.9994 and 0.9988, and the energies 0.7964 and 0.6397 sit above the exact 0.794697 and 0.635548. The larger space starts lower and converges more slowly, the system-size dependence one expects. The time to reach an overlap of 0.99, which we call T^*, is 12 for $k = 3$ and 16 for $k = 4$.

Note what the figure also shows: the curve is a straight line on a logarithmic scale and it is not steep. Going from $p = 0.99$ to $p = 0.999$ costs as much time again as reaching 0.99 in the first place, and 0.9999 would cost as much once more. Adiabatic evolution is a slow way to buy the last digits. We shall buy them with the rodeo algorithm instead.

8.2.5 What sets the time: the gap, and the state one starts from

The claim that the refinement time grows as the inverse gap, and not as its square, can be tested directly, because in the example above the gap is under our control: it is the buffer. The right panel of Fig. 8.3 varies μ_{buf} from 0.3 to 1.5 at $g = 1$; the minimum gap along the path equals the buffer throughout, and T^* falls from 42 to 5, on the straight line $T^* \simeq 13.9/\Delta E_{\text{min}} - 5.7$. The dependence is linear in the inverse gap. The scan is also the sensitivity study for the one free parameter of the method: $\mu_{\text{buf}} = 0.6$ is a conservative choice and a larger buffer *shortens* the refinement, until it reaches the ceiling set by the rest of the spectrum, which here is the gap of $\hat{H}_{\text{high}}$ itself.

The left panel varies the physics instead. From $g = 0.3$ to $g = 4$ at a fixed buffer the minimum gap hardly moves, from 0.600 to 0.568, but T^* grows from 1 to 51. What changes is the overlap of the prolonged state, which falls from 0.99 to 0.38 as the pairing correlations spread the ground state over more and more levels: at a fixed gap, a poorer starting state has more rearranging to do through the same bottleneck. This identifies the practical limiter of the method at strong coupling. It is not a closing gap but a degrading prolongation, and the

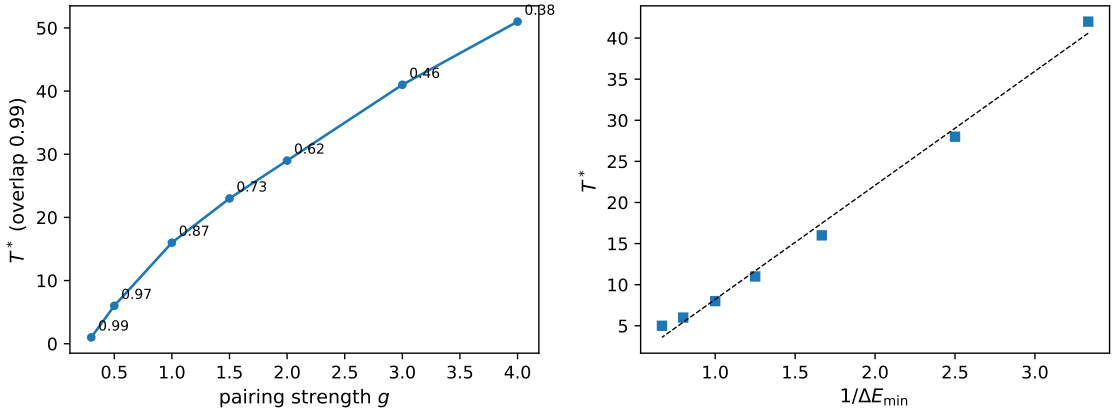


Fig. 8.3 Left: the adiabatic time T^* needed for an overlap of 0.99 in the refinement $k = 2 \to 4$ ($N = 4$, $\mu_{\text{buf}} = 0.6$) against the pairing strength g; the labels are the overlaps of the prolonged state. The minimum gap stays close to 0.6 throughout, and T^* grows because the starting overlap falls. Right: T^* against the inverse minimum gap, varied through the buffer at $g = 1$, with a linear fit. Produced by `pairing_pipeline.py`.

cure is to start the chain higher in the resolution hierarchy, where the coarse state already contains the correlations; Section 8.4.2 does so.

A remark on fairness. Resolution refinement is not the only gapped adiabatic path to the pairing ground state. One can stay at $k = 4$ throughout and ramp the coupling, $\hat{H}(s) = \hat{H}(g = 0) + s\hat{V}$, starting from the Hartree–Fock determinant, and for this model the ramp is excellent: `refinement.py` finds $p = 0.9994$ at $T = 10$, where the refinement with $\mu_{\text{buf}} = 0.6$ has 0.977. The reason is that the uncorrelated pairing model has a gap of 2ξ, larger than any buffer we used, and no level crossing on the way to $g = 1$. The example is a reminder that an adiabatic method is as good as its path and that paths should be compared. A coupling ramp needs a solvable, gapped starting point in the *full* space, which schematic models provide and realistic ones often do not: a nucleus or a lattice gauge theory has no small parameter to switch off, and a mean-field starting point may be gapless or of the wrong symmetry. Refinement asks only for a smaller version of the same problem. Exercise 3 explores the comparison.

8.3 The rodeo algorithm

8.3.1 One cycle

Recall the one-control-qubit form of phase estimation, Section 6.3.3: a Hadamard gate on the ancilla, a controlled power of $U = e^{-i\hat{H}t}$, a phase rotation, a second Hadamard gate, and a measurement whose outcome is read as one bit of the phase. The rodeo algorithm [**?**] uses the same circuit, Fig. 8.4, and asks a different question of it. It does not try to read the energy from the ancilla. It *chooses* an energy E, encodes it in the phase rotation, and uses the measurement outcome to decide whether the system register is kept.

Follow the register through the circuit. After the first Hadamard gate and the controlled evolution the state is $\big(|0\rangle|\psi\rangle + |1\rangle e^{-i\hat{H}t}|\psi\rangle\big)/\sqrt{2}$. The phase gate multiplies the second term by e^{iEt}, and the second Hadamard gate gives

$$|0\rangle \otimes \frac{1}{2}\Big(\mathbb{1} + e^{iEt}e^{-i\hat{H}t}\Big)|\psi\rangle + |1\rangle \otimes \frac{1}{2}\Big(\mathbb{1} - e^{iEt}e^{-i\hat{H}t}\Big)|\psi\rangle. \tag{8.8}$$

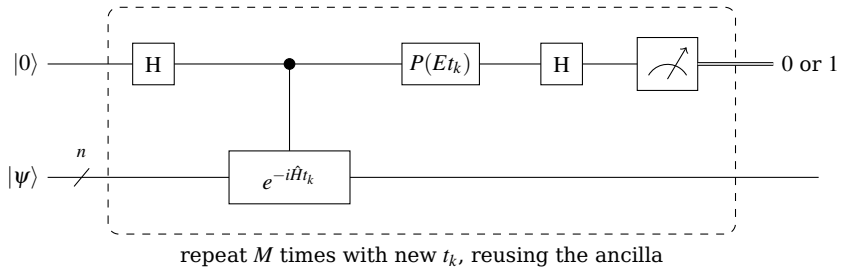


Fig. 8.4 One cycle of the rodeo algorithm. The ancilla is put in superposition, controls the time evolution of the n-qubit system register for a time t_k, receives the phase $P(Et_k) = \mathrm{diag}(1, e^{iEt_k})$ that encodes the target energy E, and is measured after a second Hadamard gate. The ancilla is then reset and reused. A run is accepted if all M outcomes are 0.

A measurement of the ancilla with outcome 0 therefore applies to the system register the non-unitary operator

$$\hat{F}(t,E) = \frac{1}{2}\Big(\mathbb{1} + e^{-i(\hat{H}-E)t}\Big), \tag{8.9}$$

which is the reason for the phase gate: it shifts the Hamiltonian by the target energy. On an eigenstate,

$$\hat{F}(t,E)|E_j\rangle = e^{-i(E_j-E)t/2}\cos\Big[\frac{(E_j-E)t}{2}\Big]|E_j\rangle, \tag{8.10}$$

so that the outcome 0 occurs with probability

$$P_0 = \cos^2\Big[\frac{(E_j-E)t}{2}\Big] \tag{8.11}$$

and leaves the eigenstate unchanged. If $E = E_j$ the outcome is 0 with certainty, for every t. If $E \neq E_j$ it is 0 or 1 with probabilities that oscillate in t. For a general input $|\psi\rangle = \sum_j c_j|E_j\rangle$ the outcome 0 has the probability $\sum_j |c_j|^2 \cos^2[(E_j-E)t/2]$ and rescales every amplitude by its cosine: components at the target energy pass untouched, the others are reduced.

8.3.2 Many cycles and random times

A single cosine is a poor filter, since it returns to one periodically. The remedy is to repeat the cycle M times with *different* times $t_1, \ldots, t_M$. The filters of Eq. (8.9) commute with one another, being functions of $\hat{H}$, so that the probability of the all-zero record for an eigenstate input is the product

$$P_{0^M}\big(E\,|\,t_1 \ldots t_M\big) = \prod_{k=1}^{M} \cos^2\Big[\frac{(E_j-E)t_k}{2}\Big], \tag{8.12}$$

and the product of cosines with incommensurate periods is small everywhere except at $E = E_j$. Choi *et al.* [**?**] draw the times from a Gaussian distribution of zero mean and width σ. Averaging one factor with $\cos^2 x = (1+\cos 2x)/2$ and $\langle\cos(\omega t)\rangle = e^{-\omega^2\sigma^2/2}$ gives

$$\Big\langle \cos^2\Big[\frac{(E_j-E)t}{2}\Big]\Big\rangle = \frac{1}{2}\Big(1 + e^{-(E_j-E)^2\sigma^2/2}\Big) \equiv f(E_j-E), \tag{8.13}$$

and since the times are independent,

$$\big\langle P_{0^M}(E)\big\rangle = \sum_j |c_j|^2 f(E_j-E)^M. \tag{8.14}$$

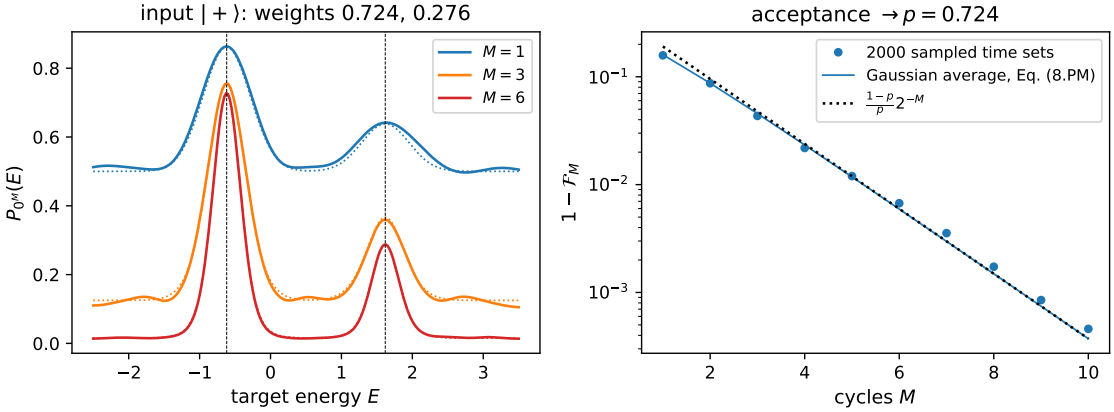


Fig. 8.5 The rodeo algorithm on the one-qubit Hamiltonian $\frac{1}{2}\mathbb{1}+Z-\frac{1}{2}X$ with the input $|+\rangle$ and $\sigma=3$. Left: the all-zero probability against the target energy for $M=1,3,6$ cycles, averaged over 300 sets of Gaussian times (full lines), and Eq. (8.14) (dotted); the vertical lines are the exact eigenvalues. Right: the infidelity of the accepted runs at $E=E_0$ against M, from 2000 sampled time sets, from the exact average of Eq. (8.16), and the estimate of Eq. (8.17). Produced by `rodeo.py`.

This is the central formula of the algorithm, and everything can be read off from it. As a function of the target energy E, the acceptance probability has a peak at every eigenvalue present in the input. The peak at E_j has the width $\sim 1/(\sigma\sqrt{M})$, which is the energy resolution, and, once the peaks are resolved, the height $|c_j|^2$, which is the spectral weight of the input state. Far from all eigenvalues $f\to\frac{1}{2}$ and the background is 2^{-M}: every cycle halves it, because the mean of a squared cosine is one half. Scanning E locates eigenvalues and measures overlaps; fixing E at an eigenvalue turns the circuit into a state-preparation device. The cost of resolving the energy to ε is a time $\sigma\sim 1/\varepsilon$ per cycle, the same Heisenberg scaling as phase estimation, since both are interferometers and no interferometer can resolve a phase faster.

8.3.3 One qubit, worked through

The smallest example is the one that Chapters 6 and 7 also used: the two-level pairing Hamiltonian with one pair, tapered to a single qubit in Eq. (4.18), $\hat{H}=\frac{1}{2}\mathbb{1}+Z-\frac{1}{2}X$, with the eigenvalues $E_{0,1}=(1\mp\sqrt{5})/2=-0.618034$ and 1.618034. The controlled evolution is a controlled single-qubit rotation and the whole rodeo circuit has two qubits. Take the input $|+\rangle$, whose weights on the two eigenstates are $p=|c_0|^2=0.7236$ and 0.2764, and $\sigma=3$. The gap is $\sqrt{5}$, so $e^{-5\sigma^2/2}\approx 10^{-10}$ and $f(E_1-E_0)=\frac{1}{2}$ to all digits: every cycle at $E=E_0$ halves the excited component on average. Equation (8.14) predicts the peak heights

$$\big\langle P_{0^M}(E_0)\big\rangle = 0.7236+0.2764\times 2^{-M}, \qquad \big\langle P_{0^M}(E_1)\big\rangle = 0.2764+0.7236\times 2^{-M}, \tag{8.15}$$

that is 0.862 and 0.638 for $M=1$, 0.758 and 0.367 for $M=3$, 0.728 and 0.288 for $M=6$, over a background of 0.5, 0.125 and 0.016. The left panel of Fig. 8.5 shows the scan, averaged over three hundred sets of random times, on top of the formula. With one cycle the "spectrum" is two bumps on a pedestal of one half; with six it is two clean peaks whose heights are the weights of the input state.

It is instructive to run the algorithm as a device would, sampling each ancilla outcome with its Born probability and carrying the collapsed register to the next cycle. With $E=E_0$ and $M=6$, the first eight runs of `rodeo.py` produce six records 000000, with fidelities

between 0.991 and 1.000000 with the ground state, and two records containing a 1, which are discarded; over four thousand runs the accepted fraction is 0.730, against the prediction 0.728. An accepted run is *heralded*: the experimenter knows from the measurement record, without any tomography, that the register now holds the eigenstate to the accuracy that M cycles give. A rejected run costs the circuit executed so far and nothing else. The program also verifies the two facts on which everything rests: that the circuit of Fig. 8.4, simulated gate by gate on the joint register, reproduces the filter of Eq. (8.9) to 10^{-16} for a random sixteen-dimensional Hamiltonian, and that the sampled Gaussian average agrees with Eq. (8.13).

8.3.4 Eigenstate preparation and the convergence criterion

Set $E = E_0$. Equation (8.14) becomes the acceptance probability after M cycles,

$$P_M = p + \sum_{j \neq 0} |c_j|^2 f(E_j - E_0)^M, \qquad p = |c_0|^2, \tag{8.16}$$

in which the target weight is preserved at every cycle while all other weights decay geometrically. The accepted ensemble has the fidelity $\mathscr{F}_M = p/P_M$ with the target. If σ is large compared with the inverse of the gap between E_0 and the nearest level that carries weight, every f is one half and

$$\mathscr{F}_M = \left[1 + \frac{1-p}{p}\, 2^{-M}\right]^{-1}, \qquad 1 - \mathscr{F}_M \leq \frac{1-p}{p}\, 2^{-M}. \tag{8.17}$$

In general 2^{-M} is replaced by $f_{\max}^M$ with $f_{\max} = \tfrac{1}{2}(1 + e^{-\Delta^2\sigma^2/2})$ and Δ that gap, which is how σ is chosen in practice: a few times $1/\Delta$. The right panel of Fig. 8.5 shows the three versions for the one-qubit example, where $1 - \mathscr{F}_M = 0.160, 0.046, 5.9\times10^{-3}, 3.7\times10^{-4}$ for $M = 1, 3, 6, 10$.

Three consequences follow. A fidelity $1-\delta$ requires

$$M \geq \log_2 \frac{1-p}{p\,\delta} \tag{8.18}$$

cycles, logarithmic in the accuracy, so that ten more cycles buy three more digits. The acceptance probability tends to p, so that the expected number of attempts per accepted state is $1/p$, and the total time of controlled evolution per accepted state is of the order of $M\sigma/p \sim |\log\delta|/(p\,\varepsilon)$ at an energy resolution ε [**?**]. And the whole construction is efficient precisely when p is not small. With $p = 0.9988$ the algorithm succeeds on almost every attempt and starts from an infidelity that is already 10^{-3}; with $p = 10^{-6}$ it would need a million attempts and twenty additional cycles. The filter is only as good as the state it is given, which is why this chapter began with refinement.

The Gaussian distribution is a convenient choice rather than an optimal one. Equation (8.10) shows that a cycle of duration $t = \pi/(E_j - E)$ removes the level E_j *exactly*, so that when the levels carrying weight are few and their energies are known, from a preliminary scan for instance, a handful of chosen times does better than many random ones. Deterministic and geometric sequences of times have been optimized for the general case [**?**, **?**]. We shall see the effect in Section 8.4.3.

Using the class. `RodeoFilter` in `rodeo.py` takes a Hermitian matrix and the width σ; its methods are the circuit (`cycle_ancilla`), the filter (`cycle`), the scan, the preparation statistics and a sampled run:

```
import numpy as np
from rodeo import RodeoFilter
```

```
H = np.array([[1.5, −0.5], [−0.5, −0.5]])    # I/2 + Z − X/2
rf = RodeoFilter(H, sigma=3.0, rng=np.random.default_rng(8))
plus = np.array([1.0, 1.0]) / np.sqrt(2)
E0 = rf.w[0]                                  # exact eigenvalues are kept in rf.w
print(rf.scan(plus, [E0, 0.5, rf.w[1]], M=6, n_sets=300))
infidelity, acceptance = rf.expected(plus, E0, M=10) # Eq. (8.PM)
record, state = rf.run(plus, E0, M=6)         # one run with sampled outcomes
print(record, abs(rf.V[:, 0] @ state) ** 2)
```

8.4 Refinement and rodeo together: the pairing model end to end

We now assemble the pipeline on the pairing Hamiltonian of Eq. (8.7) with $N = 4$ particles, $g = 1$ and the target resolution $k = 4$: eight system qubits in the Jordan–Wigner encoding with one qubit per spin orbital, Eq. (4.13), and a ground-state energy $E_0 = 0.635548$. The stages are a coarse solver, the prolongation, the refinement and the filter. Each is validated against exact diagonalization in the four-particle sector, which has 70 states. The compact encoding of Eq. (4.16), one qubit per level, which Chapters 6 and 7 used for the same Hamiltonian, would halve the register; we keep the full encoding here because it is the one that generalizes to Hamiltonians without a seniority symmetry, and return to the compact one in Sections 8.5 and 8.7, where the difference between the two turns out to be a factor of ten in gates.

8.4.1 The coarse solver

Any method that prepares a low-resolution eigenstate will do for the first stage. We use the variational eigensolver of Chapter 7 with the unitary coupled-cluster ansatz of Eq. (7.8), restricted to single and double excitations of the Hartree–Fock determinant and written as a product,

$$|\Psi(\boldsymbol{\theta})\rangle = \prod_{\mu} e^{\theta_\mu(\hat{\tau}_\mu - \hat{\tau}_\mu^\dagger)} |\Phi_{\mathrm{HF}}\rangle, \tag{8.19}$$

with the singles first. Every factor is a circuit of Pauli rotations, Section 4.7, and every generator $\hat{A} = \hat{\tau} - \hat{\tau}^\dagger$ obeys $\hat{A}^3 = -\hat{A}$, so that $e^{\theta\hat{A}} = \mathbb{1} + \sin\theta\,\hat{A} + (1-\cos\theta)\hat{A}^2$ can be applied to a vector of the N-particle sector without forming a matrix. The spectrum of $i\hat{A}$ is $\{-1, 0, 1\}$, which is the case in which the two-term parameter-shift rule fails, Section 7.5.2; `uccsd_sector.py` computes the exact gradient from one forward and one backward sweep through the product instead. This lets one program run from $k = 2$ to $k = 8$, sixteen qubits, in a few seconds.

Table 8.1 and Fig. 8.6 show what the solver delivers. At $k = 3$ it is exact to machine precision: there are two pair configurations beyond the reference and the ansatz reaches both. From $k = 4$ on it is a variational upper bound whose error grows slowly, from 1.4×10^{-3} at $k = 4$ to 1.2×10^{-2} at $k = 8$, while the non-variational CCD of Chapter 3 overbinds by an amount three to four times larger. The singles amplitudes vanish to machine precision at every k. The pairing force cannot change the seniority, so that a single excitation of the Hartree–Fock state has no matrix element with anything the Hamiltonian connects to the reference, and the solver is in effect a doubles theory, like its classical counterpart. The last column is the price, and it rises steeply: 272 CNOT gates at $k = 3$ and 1312 at $k = 4$. Section 8.7.1 shows where these numbers come from. The point for now is that the cheap resolutions are the low ones, which is the premise of refinement.

Table 8.1 Ground-state energies of the pairing model with $N = 4$, $g = 1$ against the number of levels k: exact diagonalization in the N-particle sector (FCI), coupled-cluster doubles of Section 3.6.2 (CCD), and the product-form UCCSD of Eq. (8.19) optimized from $\boldsymbol{\theta} = 0$, with its number of parameters and the CNOT count of the state-preparation circuit in the Jordan–Wigner encoding. Produced by `uccsd_sector.py` and `gate_budget.py`.

k	qubits	dim	FCI	CCD	UCCSD	parameters	CNOTs
2	4	1	1.000000	1.000000	1.000000	0	0
3	6	15	0.794697	0.794697	0.794697	8	272
4	8	70	0.635548	0.630443	0.636987	26	1312
5	10	210	0.497768	0.484977	0.501278	54	
6	12	495	0.372664	0.350058	0.378719	92	
7	14	1001	0.256052	0.221608	0.265052	140	
8	16	1820	0.145559	0.097263	0.157856	198	

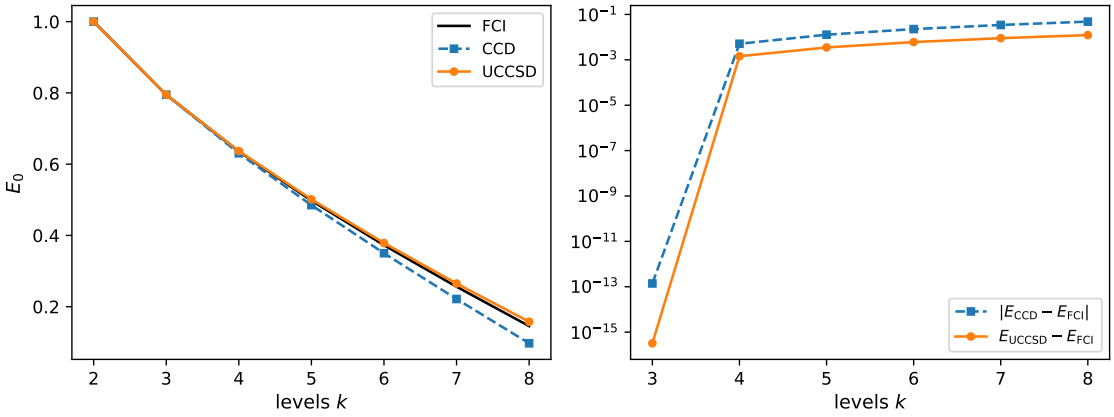


Fig. 8.6 The coarse solvers against the number of levels ($N = 4$, $g = 1$). Left: ground-state energies. Right: deviations from exact diagonalization; the UCCSD energy lies above the exact one and the CCD energy below. Produced by `uccsd_sector.py`.

8.4.2 Two chains to the same state

With $N = 4$ the chain $k = 2 \to 4$ of Section 8.2.4 has a coarse stage that does no work. To exercise the pipeline properly we also start from $k = 3$: the 272-CNOT variational state of Table 8.1 is prolonged into the $k = 4$ register and refined along the same path, with the shift of Eq. (8.5) referenced to the *variational* coarse energy. Figure 8.7 compares the two chains. The better coarse start raises the prolonged overlap from 0.868 to 0.964 and halves the refinement time, $T^* = 8$ instead of 16. In Section 8.7.3 one unit of refinement time will cost about 1500 CNOT gates, so that the eight units saved are worth some 10^4 gates, against the 272 spent on the coarse circuit. Starting higher in the hierarchy is not merely allowed; it is the economical choice, and it is the cure for the degrading prolongation of Fig. 8.3. The optimum is a trade-off between a coarse solver whose cost rises steeply with k and a refinement whose cost falls with the quality of its input, and it is quantified entirely in two-qubit gates.

8.4.3 Filtering the refined state

The state handed to the rodeo algorithm is the output of the chain $2 \to 4$ at $T = 30$, with $p = 0.9988$. For contrast we also filter the merely prolonged state, $p = 0.8676$. The spectral content

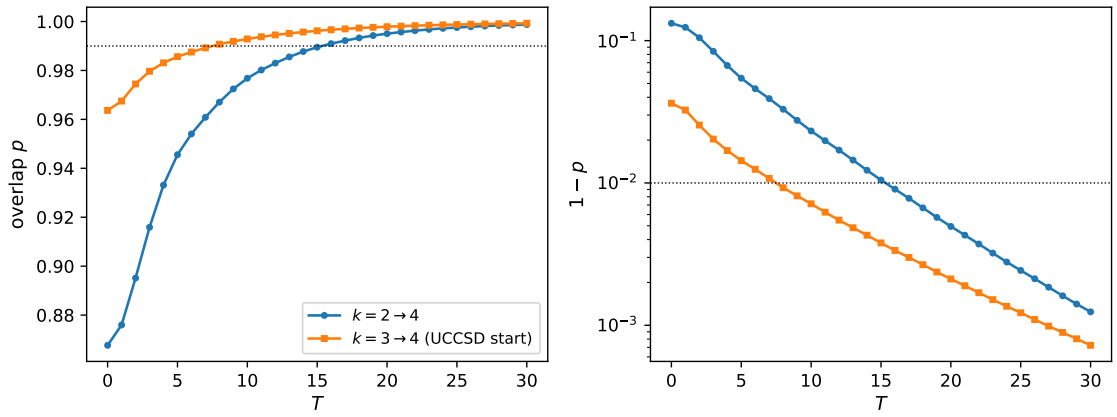


Fig. 8.7 Two refinement chains to the $k = 4$ ground state ($N = 4$, $g = 1$, $\mu_{\text{buf}} = 0.6$): from the gate-free $k = 2$ determinant and from the UCCSD state at $k = 3$. Left: overlap against the adiabatic time; right: the missing overlap on a logarithmic scale. The dotted line marks $p = 0.99$. Produced by `pairing_pipeline.py`.

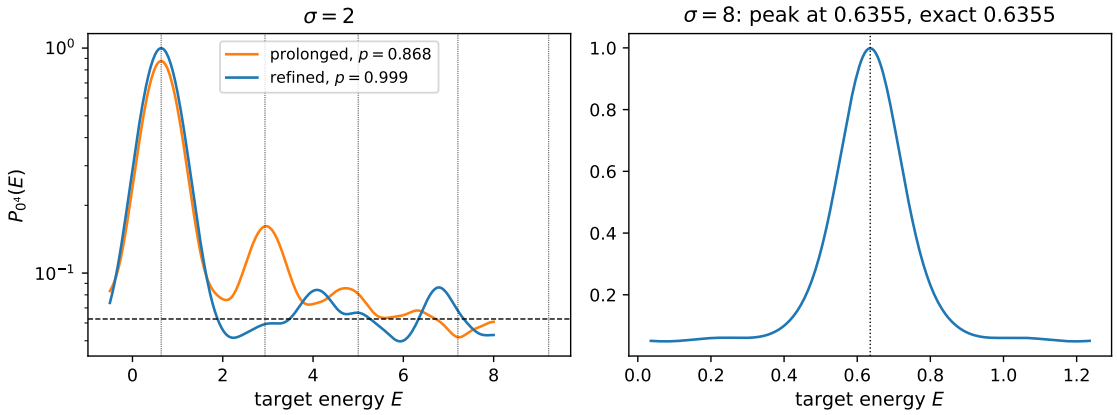


Fig. 8.8 Rodeo energy scans of the $k = 4$ pairing model with $M = 4$ cycles. Left: a coarse scan ($\sigma = 2$) of the prolonged and of the refined state; the dotted lines are the exact eigenvalues that carry weight in the prolonged state and the dashed line is the background 2^{-M}. Right: a fine scan ($\sigma = 8$) of the refined state around the ground-state peak. Produced by `pairing_pipeline.py`.

of the latter is instructive: of the seventy eigenstates of the sector only five carry weight, at the energies 0.636, 2.935, 5.000, 7.209 and 9.220 with the weights 0.868, 0.106, 0.024, 0.0015 and 0.0007. These are seniority-zero states, as they must be: the input has no broken pairs and neither the refinement nor the filter can break one. The dynamics of the whole chapter takes place in the six-dimensional space of pair configurations, even though the register has 2^8 states.

Figure 8.8 scans the target energy. With $\sigma = 2$ and four cycles the scan of the prolonged state is a spectrum: a peak of height 0.87 at the ground state, smaller ones at the excited pair states, each with the height of its weight, over the background 2^{-4}. The scan of the refined state shows one peak; the rest is background, since the refinement has already removed the excited components to the level of 10^{-3}. A finer scan with $\sigma = 8$ locates the peak at $E = 0.6355$, against the exact 0.635548. The scan is how the algorithm would be used when E_0 is not known: coarse first, to find the peak, then fine, to fix the energy at which the filter is run.

Table 8.2 and Fig. 8.9 fix $E = E_0$ and count cycles. Both inputs lose a factor of two in infidelity per cycle, exactly as Eq. (8.17) states, and the acceptance settles at p. The difference between them is where they start and what they cost. The refined input starts at 1.2×10^{-3},

Table 8.2 Rodeo preparation of the $k = 4$ pairing ground state at $E = E_0$ with Gaussian times, $\sigma = 4$: infidelity of the accepted runs and acceptance probability from Eq. (8.16), for the refined input ($p = 0.9988$) and the prolonged input ($p = 0.8676$). Produced by `pairing_pipeline.py`.

	refined input		prolonged input	
M	$1-\mathcal{F}_M$	P_M	$1-\mathcal{F}_M$	P_M
0	1.2×10^{-3}	1	1.3×10^{-1}	1
1	6.2×10^{-4}	0.9994	7.1×10^{-2}	0.9338
2	3.1×10^{-4}	0.9991	3.7×10^{-2}	0.9007
4	7.8×10^{-5}	0.9988	9.4×10^{-3}	0.8759
6	1.9×10^{-5}	0.9988	2.4×10^{-3}	0.8697
8	4.9×10^{-6}	0.9988	6.0×10^{-4}	0.8682

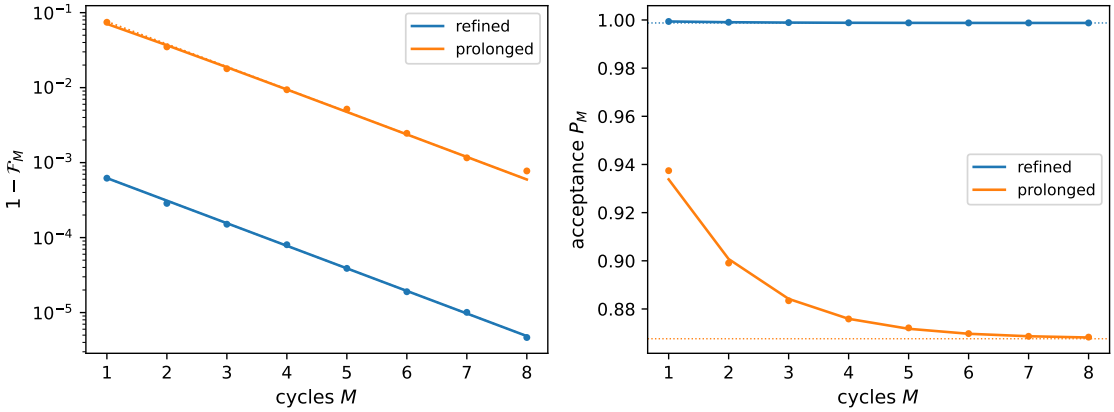


Fig. 8.9 Rodeo preparation at $E = E_0$, $\sigma = 4$. Left: infidelity of the accepted runs against the number of cycles for the two inputs; full lines are the exact Gaussian average of Eq. (8.16), points are 300 sampled sets of times, dotted lines the estimate $\frac{1-p}{p}2^{-M}$ of Eq. (8.17). Right: the acceptance probability converges to the input overlap p (dotted). Produced by `pairing_pipeline.py`.

reaches 4.9×10^{-6} in eight cycles and is accepted 999 times in a thousand. The prolonged input reaches 6.0×10^{-4} in the same eight cycles, a worse state than the refined input *before* any filtering, and is rejected in one attempt out of seven. To see what the filter has bought, extrapolate the straight line of Fig. 8.1: refinement alone would need $T \approx 70$ for an infidelity of 5×10^{-6}, where the eight cycles have a total mean duration of $8\sigma\sqrt{2/\pi} \approx 26$ in the same units, and eight more would square the accuracy.

Random times are not the best one can do here. Only a few levels carry weight, the coarse scan has located them, and a cycle with $t_j = \pi/(E_j - E_0)$ removes the level E_j exactly. Three such cycles, aimed at the levels 2.935, 5.000 and 7.209 with $t = 1.366$, 0.720 and 0.478, take the refined input to an infidelity of 5.3×10^{-6} and the prolonged one to 1.4×10^{-4}, with the acceptances 0.9988 and 0.868: the accuracy of eight Gaussian cycles in three, and in a tenth of the evolution time. The general lesson is that the time distribution is a resource to be optimized [**?**, **?**], and that prior knowledge of the spectrum, which the scan supplies, is worth a great deal.

Table 8.3 Phase estimation and the rodeo algorithm as state preparation for the $k = 4$ pairing ground state from the prolonged input ($p = 0.8676$): total evolution time, infidelity of the accepted runs and acceptance probability. Phase estimation accepts the modal outcome of an m-qubit control register; the rodeo algorithm uses Gaussian times with $\sigma = 4$ and its evolution time is the mean. The last column is the error of the phase-estimation energy. Produced by `filters_compared.py`.

	phase estimation				rodeo			
m	T_{tot}	$1-\mathscr{F}$	accept.	$\lvert\Delta E\rvert$	M	T_{tot}	$1-\mathscr{F}$	accept.
4	9.9	1.0×10^{-3}	0.760	1.2×10^{-1}	3	9.6	1.9×10^{-2}	0.884
5	20.4	3.7×10^{-4}	0.497	1.2×10^{-1}	6	19.1	2.4×10^{-3}	0.870
6	41.5	6.9×10^{-5}	0.759	3.0×10^{-2}	12	38.3	3.7×10^{-5}	0.868
8	168	4.6×10^{-6}	0.759	7.5×10^{-3}				
10	674	4.6×10^{-8}	0.759	1.9×10^{-3}				

8.5 The pairing model three ways: rodeo, phase estimation and VQE

The pairing model with four levels and two pairs has now been treated by three quantum algorithms: phase estimation in Section 6.6.1, the variational eigensolver in Section 7.7.2 and in Table 8.1, and the refinement–rodeo pipeline. Section 7.7.3 weighed the first two against each other as ways of obtaining an *energy*. Here we compare all three as ways of obtaining a *state*, which is what a calculation of any observable other than the energy needs.

8.5.1 Phase estimation as a filter

Phase estimation with m control qubits also filters. When the control register is read and returns the integer y, the component $|E_j\rangle$ of the system register is multiplied by the amplitude of Eq. (6.26),

$$a_y(\phi_j) = \frac{1}{2^m}\sum_{x=0}^{2^m-1} e^{2\pi i x(\phi_j - y/2^m)}, \qquad \phi_j = -\frac{E_j t}{2\pi} \bmod 1, \tag{8.20}$$

whose modulus squared is the Dirichlet kernel of Eq. (6.27). Accepting the runs that return the grid point nearest to ϕ_0 prepares the ground state with the fidelity $p|a_y(\phi_0)|^2/\sum_j |c_j|^2|a_y(\phi_j)|^2$. The kernel has a central peak of width 2^{-m} and tails that fall as the inverse square of the distance, so the infidelity falls as 4^{-m}. The total evolution time is $t(2^m-1)$, and in terms of it the infidelity of phase estimation falls as $1/T_{\mathrm{tot}}^2$, a power law. The rodeo filter falls as 2^{-M} with $T_{\mathrm{tot}} \propto M$: exponentially. `filters_compared.py` makes the comparison for the two inputs of the last section, with the time step $t = 0.9\pi/\|\hat{H} - \bar{E}\| = 0.659$ chosen as in Eq. (6.33) from the width of the four-particle spectrum, and the result is Table 8.3 and Fig. 8.10.

The table corrects two impressions that the asymptotic statements might leave. The first is that the exponential wins from the start. It does not: at short times phase estimation is the better filter for this spectrum, by more than a factor of ten at $T_{\mathrm{tot}} \approx 10$, because the few levels that carry weight are far from the ground state and already sit in the tails of the kernel. The exponential overtakes the power law at $T_{\mathrm{tot}} \approx 40$ for the prolonged input, and from there the gap widens quickly: the infidelity 4.6×10^{-6} costs phase estimation an evolution time of 168 and the rodeo algorithm about 50. The second is that the two algorithms need the same hardware. Phase estimation needs either m ancilla qubits and an inverse Fourier transform or, in its iterative form, one ancilla and a feedback of the measured bits into the rotation angles, and its longest single controlled evolution is half of the total, $t\,2^{m-1}$; the rodeo cycles

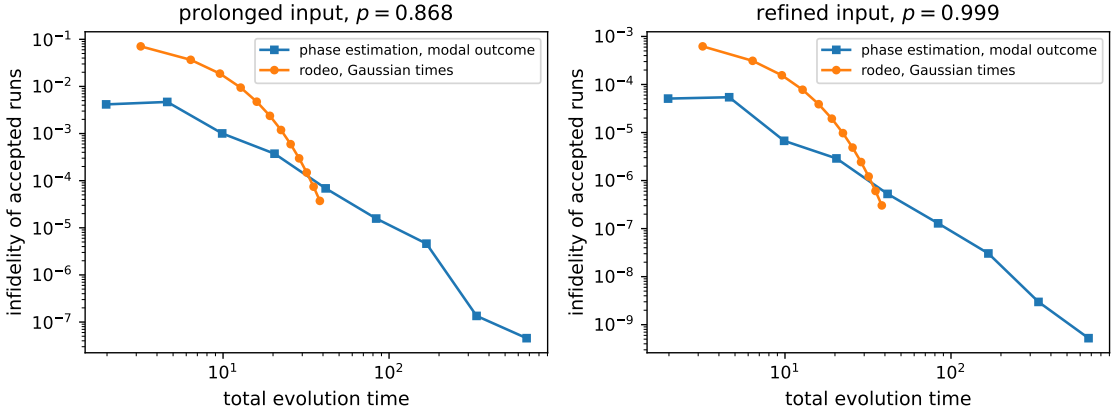


Fig. 8.10 Infidelity of the prepared $k = 4$ pairing ground state against the total time of controlled evolution, for phase estimation (modal outcome, $m = 2$ to 10 control qubits) and for the rodeo algorithm (Gaussian times, $\sigma = 4$, $M = 1$ to 12), from the prolonged input (left) and the refined input (right). The power law of the Dirichlet kernel and the exponential of the rodeo filter cross at an evolution time of about 40 for both inputs. Produced by `filters_compared.py`.

are short, independent and need no feedback. The acceptance columns differ as well. The rodeo acceptance tends to p. The phase-estimation acceptance is p times the height of the kernel at the nearest grid point, which oscillates between $4/\pi^2$ and one as m changes; here it is 0.50 or 0.76. What phase estimation gives in return is the last column, an energy with a guaranteed error bar from a single accepted run, where the rodeo algorithm needs a scan. For energies, phase estimation; for states, above some accuracy, the rodeo algorithm.

8.5.2 The variational state

The variational route prepares the $k = 4$ ground state directly with the circuit of Eq. (8.19): 26 parameters, 1312 CNOT gates, no ancilla and no post-selection. Table 8.1 gave its energy error, 1.4×10^{-3}. Its infidelity, which `filters_compared.py` computes, is 1.8×10^{-4}; the error of a variational energy is of the order of the infidelity times the excitation energy of the admixed states, which here is large. The comparison with the pipeline is therefore more even than the energies suggest, and it must be made like with like. The refined state at $T = 30$ has the infidelity 1.2×10^{-3} and the energy error 4.1×10^{-3}: as it leaves the adiabatic stage it is a *worse* state than the variational one, by a factor of seven in infidelity. Two to three rodeo cycles with exact evolution bring it level, and eight put it a factor of forty ahead.

What separates the two routes is not the accuracy at this size but what stands behind it. The variational number is a floor. It is the best the ansatz can do; no number of shots or optimizer iterations lowers it, and doing better means a larger ansatz, a longer circuit and a harder optimization, with the costs of Section 7.5. The pipeline has no floor of this kind, since each additional cycle halves the error, and it has no parameters: nothing is optimized at full resolution, so the measurement overhead of Section 7.4.1, the local minima and the barren plateaus of Section 7.5.4 do not arise. It pays for this in coherent depth, as Section 8.7 will show, fifty times more gates per run than the variational circuit. The floor is, moreover, a property of this ansatz and not of the variational method. In the compact encoding the pair-transfer ansatz of Table 7.6 is exact for this Hamiltonian with six parameters, because the space it has to cover is six-dimensional. That is a statement about small models: the number

Table 8.4 Three algorithms for the ground state of the same Hamiltonian, as state-preparation methods.

	VQE	phase estimation	refinement + rodeo
input needed	an ansatz	a state with overlap p	a coarse eigenstate
error of the state	floor set by the ansatz	$\sim 1/T_{\mathrm{tot}}^2$	$\sim 2^{-M}$, no floor
success probability	one	$p \times [4/\pi^2, 1]$	$\to p$, with $p \approx 1$
classical loop	optimizer, many shots	none (feedback if iterative)	none
ancilla qubits	none	m, or one with feedback	one, reset and reused
coherent depth	shallow	deep, one long evolution	deep, many short evolutions
energy	upper bound	error bar $\pi/(t2^m)$	from a scan, or from the state
fails	quietly	loudly	loudly (low acceptance)

of parameters of an exact ansatz grows with the dimension of the space, and at the sizes where a quantum computer is needed no ansatz that can be optimized is exact.

The two routes are also not exclusive, and the most economical state preparation for this problem uses both. The rodeo filter accepts any input. Fed with the $k = 4$ variational state, $p = 0.99982$, it returns the infidelities 8.9×10^{-5}, 4.4×10^{-5} and 2.8×10^{-6} after one, two and six cycles, with an acceptance of 0.9998, and the energy error of the filtered state falls from 1.4×10^{-3} to 3.6×10^{-4} after two cycles and 2.2×10^{-5} after six. The filter breaks the variational floor. Seen this way the adiabatic refinement and the variational circuit are alternatives for the same job, producing the large overlap, and the filter is what follows either. Which of them is preferable depends on how their costs scale. The variational circuit and its optimization become expensive as the resolution grows; the refinement becomes expensive as the gap closes or the prolonged overlap falls. Table 8.4 collects the comparison.

The last row continues a theme of Section 7.7.3. A rodeo run that goes wrong, because the target energy is off, because the input has no weight there, or, as Section 8.7.2 will show, because the compiled evolution is not the evolution of $\hat{H}$, announces itself through a low acceptance rate. The measurement record is a diagnostic as well as a herald.

8.6 The other three Hamiltonians

Nothing in Sections 8.2 and 8.3 is specific to pairing. The rodeo filter needs a controlled time evolution and nothing else. Refinement needs an answer to one question, what a coarser version of the problem is, and the answer depends on the model. This section gives it for the Lipkin model, the Heisenberg ring and the Hubbard ring, with the parameters of Table 6.6 as targets, so that the results can be set beside the phase-estimation energies of Table 6.7 and the variational energies of Tables 7.5 and 7.6.

One generalization of the path is needed first. For a basis refinement the prolonged state has the energy E_{low} in the fine Hamiltonian and one shift serves both terms of Eq. (8.4). When the prolongation changes the energy scale, as it does on a lattice or when particles are added, each term gets its own shift,

$$\hat{H}(s) = \cos^2\Big(\frac{\pi s}{2}\Big)\,\hat{P}\big(\hat{H}_{\mathrm{low}} - E_{\mathrm{low}} - \mu_{\mathrm{buf}}\big)\hat{P}^\dagger + \sin^2\Big(\frac{\pi s}{2}\Big)\,\big(\hat{H}_{\mathrm{high}} - \langle\hat{H}_{\mathrm{high}}\rangle_{\Phi_0} - \mu_{\mathrm{buf}}\big), \tag{8.21}$$

Table 8.5 Resolution refinement for the Lipkin, Heisenberg and Hubbard models with $\mu_{\text{buf}} = 1$ and the path of Eq. (8.21): dimension of the fine space, exact energy, energy and overlap of the prolonged state, minimum gap along the path, the time T^* for an overlap of 0.99, and the overlap at $T = 20$. The first, third and fifth rows are the Hamiltonians of Table 6.6; "Lipkin, $V = -1$" is a stronger interaction, beyond the mean-field transition of Section 3.5.4. Produced by `other_models.py`.

refinement	dim	E_0	$\langle \hat{H}_{\text{high}} \rangle_{\Phi_0}$	$p(0)$	gap	T^*	$p(20)$
Lipkin, $N = 2 \to 4$	16	-4.21288	-4.02759	0.9458	1.000	6	0.9998
Lipkin, $N = 2 \to 4$, $V = -1$	16	-5.53585	-4.23607	0.7506	1.000	14	0.9981
Heisenberg, $2 \to 4$ sites	6	-2.00000	-1.50000	0.7500	0.908	10	0.9999
Heisenberg, $4 \to 8$ sites	70	-3.65109	-3.00000	0.5043	0.522	14	0.9981
Hubbard, $2 \to 4$ sites	70	-2.10275	0.00000	0.4249	0.296	18	0.9955

so that the tracked state starts at $-\mu_{\text{buf}}$ and the fine Hamiltonian is measured from the energy of the prolonged state, which a device can measure. For a basis refinement this is Eq. (8.4) again. We use $\mu_{\text{buf}} = 1$ throughout this section.

The Lipkin model: more particles. In the encoding of Eq. (4.11) every doublet of the Lipkin model is one qubit, $|0\rangle$ for the particle in the lower level, and the Hamiltonian couples the qubits in pairs. A coarser problem is the same Hamiltonian with fewer particles. We take $N = 2$, solve it on two qubits, where the ground state is a rotation in the space of $|00\rangle$ and $|11\rangle$, and prolong by adding two doublets in their lower level, $\hat{P}|\psi\rangle = |\psi\rangle \otimes |00\rangle$, again without gates. The coarse term of the path is the two-qubit Hamiltonian multiplied by the projector on $|00\rangle$ of the new qubits.

The Heisenberg ring: magnons on a lattice, and a sign. For spins, think of the down spins as particles, magnons, hopping on a lattice of up spins. The $S^z = 0$ sector of a ring with n sites has $n/2$ magnons. A coarse ring with $n/2$ sites and the same number of magnons is completely full, a single state, exactly as the four-particle pairing model at $k = 2$. The lattice prolongation spreads every magnon over the two fine sites of its block, and here something new enters. With the plus sign of Section 8.2.2, each block becomes the triplet state $(|{\downarrow\uparrow}\rangle + |{\uparrow\downarrow}\rangle)/\sqrt{2}$ and the prolonged state has an overlap of 0.083 with the ground state of the four-site ring. With a minus sign each block becomes a singlet, the prolonged state is the singlet covering that Section 7.7.2 used as a variational reference, and the overlap is 0.750. The sign is Marshall's: the ground state of an antiferromagnet on a bipartite lattice has amplitudes whose signs alternate with the number of down spins on one sublattice, and a prolongation that ignores this lands in the wrong symmetry sector. The lesson generalizes. The prolongation must respect the symmetries and the sign structure of the fine ground state, because the adiabatic evolution conserves every symmetry that the path Hamiltonian has, and an input in the wrong sector stays there.

The Hubbard ring: bonding orbitals. For the half-filled ring with four sites, four particles and $U = 4t$, the coarse lattice has two sites and is again full. Every coarse orbital becomes the bonding combination of its two fine sites, $W_{2c,c} = W_{2c+1,c} = 1/\sqrt{2}$ for both spins, and Eq. (8.2) turns the coarse determinant into a product of two doubly occupied bond orbitals, a state of decoupled dimers with $\langle \hat{H}_{\text{high}} \rangle = 0$ and an overlap of 0.425 with the correlated ground state at $E_0 = -2.102748$.

Table 8.5 and the left panel of Fig. 8.11 show the refinements. The pattern of the pairing model repeats, with the models ordered by how much the prolonged state already knows. The weakly coupled Lipkin model starts at $p = 0.946$ and is refined to 0.99 by $T = 6$; with the stronger interaction $V = -1$ the start drops to 0.75 and the time more than doubles at the same gap, the effect of the left panel of Fig. 8.3. The Heisenberg ring starts from the singlet covering at 0.75 for four sites and 0.50 for eight, where the covering breaks the translation symmetry that the ground state has, and both reach 0.998 or better by $T = 20$. The Hubbard

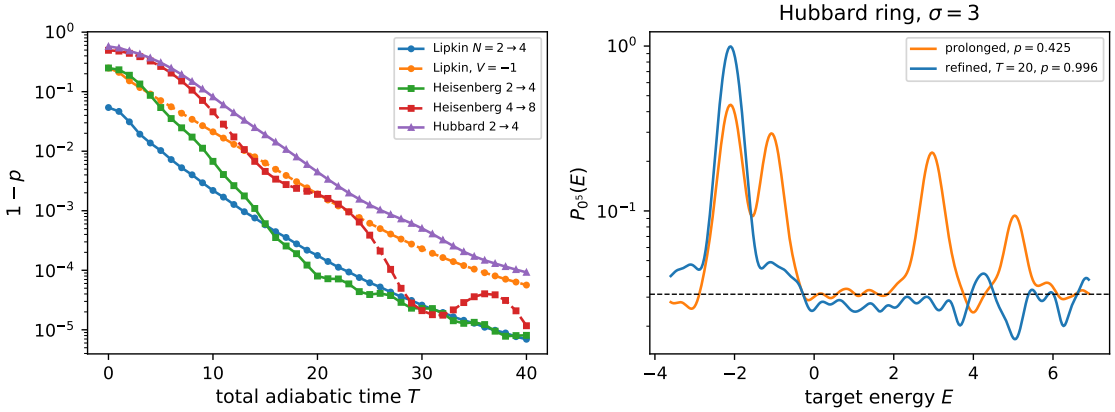


Fig. 8.11 Left: the missing overlap against the adiabatic time for the five refinements of Table 8.5. Right: rodeo scan ($\sigma = 3$, $M = 5$) of the Hubbard ring from the prolonged dimer state and from the state refined to $T = 20$; the dashed line is the background 2^{-5}. Produced by `other_models.py`.

Table 8.6 The rodeo filter ($\sigma = 4$, Gaussian times, $E = E_0$) on the prolonged states and on the states refined to $T = 20$ of Table 8.5: overlap of the input, infidelity of the accepted runs after M cycles from Eq. (8.16), and acceptance after eight cycles. Produced by `other_models.py`.

model	input	p	$M=2$	$M=4$	$M=8$	P_8
Lipkin	prolonged	0.9458	1.4×10^{-2}	3.6×10^{-3}	2.2×10^{-4}	0.946
	refined	0.9998	4.4×10^{-5}	1.1×10^{-5}	6.9×10^{-7}	1.000
Lipkin, $V=-1$	prolonged	0.7506	7.7×10^{-2}	2.0×10^{-2}	1.3×10^{-3}	0.752
	refined	0.9981	4.8×10^{-4}	1.2×10^{-4}	7.5×10^{-6}	0.998
Heisenberg, 4 sites	prolonged	0.7500	7.7×10^{-2}	2.0×10^{-2}	1.3×10^{-3}	0.751
	refined	0.9999	2.0×10^{-5}	5.0×10^{-6}	3.1×10^{-7}	1.000
Heisenberg, 8 sites	prolonged	0.5043	2.0×10^{-1}	5.8×10^{-2}	3.8×10^{-3}	0.506
	refined	0.9981	4.7×10^{-4}	1.2×10^{-4}	7.4×10^{-6}	0.998
Hubbard ring	prolonged	0.4249	2.5×10^{-1}	7.8×10^{-2}	5.3×10^{-3}	0.427
	refined	0.9955	1.1×10^{-3}	2.8×10^{-4}	1.8×10^{-5}	0.996

ring is the hardest case in the table: the prolonged overlap is 0.42, and the gap of the fine Hamiltonian, 0.296 to the lowest triplet, is below the buffer and sets the minimum along the path. The triplet is not coupled to the tracked singlet by a spin-symmetric path, so the quoted gap is pessimistic, and the refinement reaches $p = 0.9955$ at $T = 20$. In no case does the gap close, and in every case the curve is the same straight line on a logarithmic scale: fast to 0.99, slow after.

Table 8.6 applies the filter, and its rows come in pairs that tell the story of the chapter in numbers. The filter works on every input; the infidelity falls by a factor of four for every two cycles in all ten rows. But the prolonged Hubbard state is accepted in 43 runs out of a hundred and is, after eight cycles, where the refined state was after one, while the refined state is accepted in 996 out of a thousand and reaches 1.8×10^{-5}. The right panel of Fig. 8.11 shows the two scans. The prolonged dimer state is a broad spectrum: besides the ground state, with weight 0.425, it contains levels at -1.068, 2.965, 5.035 and 10.103 with the weights 0.276, 0.205, 0.071 and 0.018, and the scan shows them all. The refined state is a single line. Compare with the earlier chapters. Phase estimation with ten control qubits gave this energy to 1.7×10^{-2}, Table 6.7, starting from the exact ground state; the hardware-efficient ansatz missed it by 0.10 and the Hamiltonian-variational ansatz found it exactly, Table 7.7, after an optimization. The pipeline prepares the state to 10^{-5} from a product of dimers, with no optimization and no

prior knowledge of the state, at the price, as always, of coherent evolution: $T = 20$ in units of $1/t$, plus the cycles.

A chain with two stages. In all the lattice examples so far the coarse space was a single state, because a half-filled lattice with half as many sites is full. At low filling the coarse space is not trivial, and this is the setting of the original proposal [**?**]: particles in a continuum, discretized on a lattice whose spacing a is halved at every stage. `other_models.py` puts one spin-up and one spin-down particle on an open chain of 2, then 4, then 8 sites, sixteen qubits at the end, with the hopping $t \propto 1/a^2$ multiplied by four and the contact interaction $U \propto 1/a$ by two at every stage, and the on-site energy $2t$ of the discretized kinetic energy. The refined state of the first stage, not the exact one, is prolonged into the second. With a buffer and an evolution time that follow the energy scale, $\mu_{\text{buf}} = 4$ and $T = 2$ in units of the coarse hopping, the prolonged overlaps are 0.909 and 0.942, the refined overlaps 0.995 and 0.985, and four rodeo cycles at the end of each stage reduce the infidelity to 3×10^{-4} and 1×10^{-3}. The prolonged overlap does not deteriorate from one stage to the next. It improves, because the finer the lattice the smaller the change that one more halving makes to a smooth wave function. This is the property that makes refinement scale: the number of stages grows only logarithmically with the resolution, and each of them starts close to its target.

8.7 Circuits, depth and the bottlenecks of quantum computing

Up to this point the evolution operators were exact matrix exponentials, which is how one studies an algorithm. A device executes gates. In this section every stage of the pairing pipeline is compiled to single-qubit rotations and CNOT gates, the product-formula error is put back, the gates are counted, and the count is compared with what hardware can run. We do this in detail for one example because the exercise is the most direct way we know to see, in one place and in numbers, what limits quantum computing for many-body problems. The currency throughout is the CNOT gate: two-qubit gates are slower than single-qubit gates and have error rates that are typically an order of magnitude larger, so that their number is a fair first estimate of both the run time and the accumulated error.

8.7.1 Building blocks

Everything is built from the Pauli rotation $e^{-i\phi P/2}$ of Eq. (4.19): rotate every qubit on which P acts into the Z basis, collect the parity on the last qubit with a ladder of CNOT gates, apply one $R_z(\phi)$, and undo the ladder and the basis change. A string of weight w costs

$$N_{\text{CNOT}}(w) = 2(w-1) \tag{8.22}$$

CNOT gates and one parametrized rotation, Fig. 8.12; `gate_budget.py` checks the circuit against the exact exponential for the string $XXXY$, to 10^{-15}. A rotation *controlled* on an ancilla, which is what the rodeo cycle needs, costs two CNOT gates more, whatever the weight, because only the central R_z has to be controlled, Fig. 6.7.

Excitation operators. In the Jordan–Wigner encoding a single excitation $a_a^\dagger a_i - \text{h.c.}$ is a sum of two strings, X and Y on the two qubits with a string of Z between them, and its cost grows with the distance: 8 CNOT gates between neighbouring levels and 16 for the excitation from mode 0 to mode 4. A double excitation is a sum of eight strings, of weight four plus whatever Z strings lie between the two occupied and between the two empty modes; the double $(0,3) \to (4,5)$ has weight six and costs 80. The pair-hopping doubles that carry all the

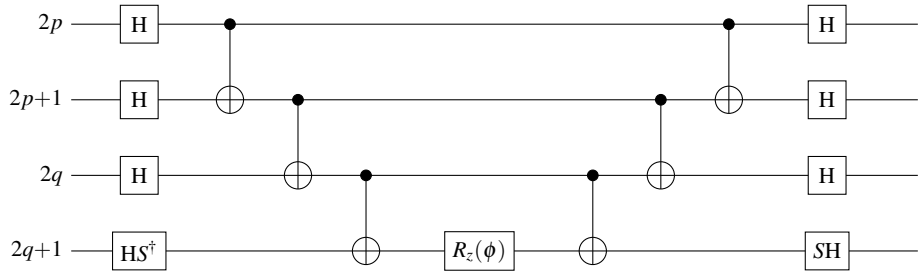


Fig. 8.12 The rotation $e^{-i\phi XXXY/2}$ on the four qubits of two pairing levels p and q: basis changes (the product $HS^\dagger$ means $S^\dagger$ first), a ladder of three CNOT gates, the rotation, and the reverse. Six CNOT gates. Eight such blocks make one pair-hopping term, and 160 rotations of this kind, of weights four to eight, make the UCCSD preparation at $k=4$.

amplitude in the pairing problem are the fortunate case. A time-reversed pair occupies the adjacent modes $2p$ and $2p+1$, the Z strings of its two operators cancel, and all eight strings have weight four: 48 CNOT gates per pair-hopping term, *whatever the distance between the levels*. The full ansatz of Eq. (8.19) has 8 operators, 40 rotations and 272 CNOT gates at $k=3$, and 26 operators, 160 rotations and 1312 CNOT gates at $k=4$, the numbers of Table 8.1.

One Trotter step. The Hamiltonian is split the way its circuit is,

$$\hat{H} = \hat{D} + \sum_{p<q} \hat{h}_{pq}, \qquad \hat{h}_{pq} = -\frac{g}{2}\left(\hat{P}_p^+ \hat{P}_q^- + \hat{P}_q^+ \hat{P}_p^-\right), \tag{8.23}$$

with the pair operators $\hat{P}_p^+ = a_{p+}^\dagger a_{p-}^\dagger$ of Eq. (3.29), which are not to be confused with the prolongation, and a diagonal part $\hat{D}$ that contains the single-particle energies and the $p=q$ term of the interaction: phases on single qubits and one ZZ rotation per level. Each $\hat{h}_{pq}$ is eight weight-four strings, 48 CNOT gates, and a first-order step costs

$$N_{\mathrm{CNOT}}^{\mathrm{step}}(k) = 24k(k-1) + 2k, \tag{8.24}$$

which is 296 at $k=4$ and 1360 at $k=8$; the program obtains the same numbers by mapping the Hamiltonian with the `JordanWigner` class and adding up Eq. (8.22) over its 60 strings. With the controls, two CNOT gates for each of the 60 rotations, the step costs 416. A symmetric second-order step has each hopping term twice, except the middle one, and merges its diagonal half-steps with those of its neighbours: 536 CNOT gates plain and 736 controlled. Finally the coarse term of the path, the projector on the four new qubits being empty, is a sum of 15 strings of Z operators, 34 CNOT gates per step.

The encoding. The same counts in the compact encoding of Eq. (4.16), one qubit per level and $\hat{h}_{pq} = -\frac{g}{4}(X_pX_q + Y_pY_q)$, are 4 CNOT gates per hopping term instead of 48, no two-qubit gates in $\hat{D}$, and 24 per first-order step at $k=4$ instead of 296, on half the qubits. The dynamics is the same, since, as we saw, it never leaves the seniority-zero space, and so are the splitting and its Trotter error. The factor of twelve is the price of carrying, through every gate, a part of Hilbert space that the calculation never visits, and using a symmetry to avoid it is the cheapest error mitigation there is. We keep both columns in the budget below: the compact one is what one would run for this model, the Jordan–Wigner one is what a Hamiltonian without a conserved seniority, a nuclear shell-model interaction for instance, would cost.

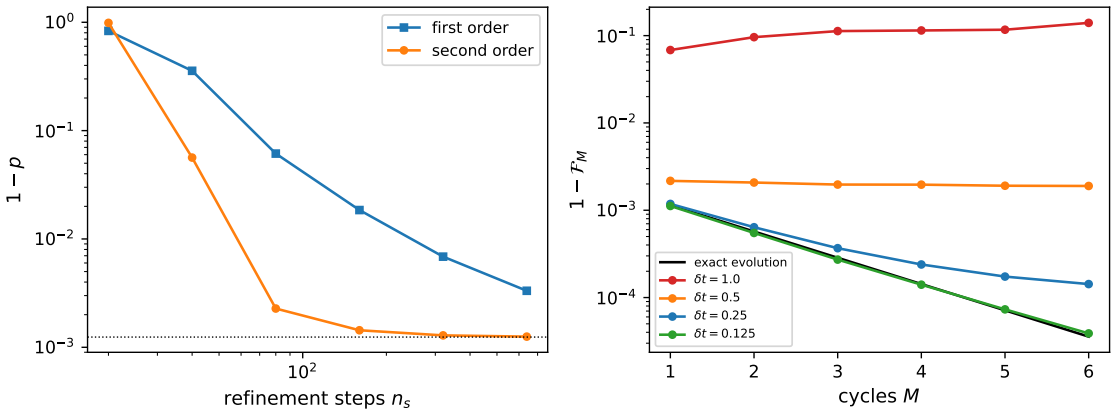


Fig. 8.13 The pipeline with product formulas ($k = 4$, $N = 4$, $g = 1$). Left: missing overlap of the Trotterized refinement $k = 2 \to 4$ with $T = 30$ against the number of steps, for the first- and second-order formulas; the dotted line is the result with exact slices. Right: infidelity of the rodeo filter against the number of cycles when the controlled evolution is a second-order product formula with a fixed step δt, for the input refined with $n_s = 80$ steps ($p = 0.9977$). Too coarse a step is rejected by the filter; $\delta t = 0.125$ is indistinguishable from the exact evolution. Produced by `trotter_pipeline.py`.

8.7.2 The pipeline with product formulas

The eight strings of one $\hat{h}_{pq}$ commute, so that $e^{-i\hat{h}_{pq}\delta t}$ is implemented *exactly* by its eight rotations, and the only error of the product formula is the non-commutativity between the terms of Eq. (8.23). At $k = 4$ and $t = 1$ the symmetric formula has a spectral-norm error of 3.4×10^{-2} with four steps and 8.3×10^{-3} with eight, falling by four when the number of steps doubles; the first-order formula has 0.31 and 0.15.

Refinement. The left panel of Fig. 8.13 replaces the exact slices of the path by product formulas. With the symmetric formula, $n_s = 80$ steps over $T = 30$ give $p = 0.9977$, against 0.9988 with exact slices, and 160 steps give 0.9986; with 40 steps the step is too long and the overlap drops to 0.94. The first-order formula needs more than 640 steps for the same quality. The adiabatic evolution is forgiving of Trotter error, for a reason worth noting: a product formula with a small step evolves *exactly* under a slightly different Hamiltonian, and an adiabatic passage under a slightly different Hamiltonian still ends near the ground state.

The filter. For the rodeo stage the honest prescription fixes the step *length* δt, so that a cycle of duration t_k uses $\lceil |t_k|/\delta t \rceil$ steps, thirteen on average for $\sigma = 4$ and $\delta t = 0.25$. The right panel of Fig. 8.13 and Table 8.7 show three regimes. With $\delta t = 1$ the compiled evolution has little to do with $e^{-i\hat{H}t}$, and the filter *rejects* it: the acceptance collapses to 0.08 and the accepted states are poor. With $\delta t = 0.5$ the filter converges, to a floor of 2×10^{-3}. With $\delta t = 0.25$ it follows the exact curve down to 1.4×10^{-4} at six cycles, with an acceptance of 0.986, and with $\delta t = 0.125$ it is indistinguishable from the exact evolution. The floor has a clean explanation. The rodeo algorithm filters the eigenstates of the operator it is given, which is the effective Hamiltonian of the product formula, and converges to *its* ground state. That state differs from the true one at order δt^2, and the last column of the table, computed by diagonalizing one Trotter step, reproduces the floors: 3.1×10^{-3}, 1.5×10^{-4} and 8.7×10^{-6}. No number of cycles goes below this; a smaller step does, by a factor of sixteen per halving. This is the rodeo version of the lesson of Section 6.6.2: an interferometer measures the eigenphases of the circuit it is given, not those of the Hamiltonian the circuit was meant to represent.

Table 8.7 The rodeo filter with a Trotterized controlled evolution (second order, step δt, $\sigma = 4$) on the circuit-level refined state, $p = 0.9977$: infidelity of the accepted runs after two and six cycles, acceptance after six, and the infidelity between the ground state of the effective Hamiltonian of one Trotter step and the true one. Produced by `trotter_pipeline.py`.

δt	$1-\mathscr{F}_2$	$1-\mathscr{F}_6$	P_6	floor
1	9.6×10^{-2}	1.4×10^{-1}	0.078	3.1×10^{-1}
0.5	2.1×10^{-3}	1.9×10^{-3}	0.814	3.1×10^{-3}
0.25	6.4×10^{-4}	1.4×10^{-4}	0.986	1.5×10^{-4}
0.125	5.5×10^{-4}	3.9×10^{-5}	0.997	8.7×10^{-6}
exact	5.7×10^{-4}	3.6×10^{-5}	0.998	

Table 8.8 CNOT budget for preparing the $k = 4$ pairing ground state ($N = 4$, $g = 1$): the variational circuit of Table 8.1 and the refinement–rodeo pipeline at the operating point of Fig. 8.13, in the Jordan–Wigner encoding ($8+1$ qubits) and in the compact pair encoding ($4+1$ qubits). Infidelities are those of Table 8.7 and of Section 8.5.2; they do not depend on the encoding. Produced by `gate_budget.py`.

		CNOT gates		
circuit	params	JW	compact	infidelity
UCCSD preparation, per run	26	1312		1.8×10^{-4} (floor)
coarse state and prolongation	0	0	0	1.3×10^{-1}
refinement, 80 steps	0	45 600	3680	2.3×10^{-3}
one rodeo cycle (mean)	0	9750	1270	
pipeline with $M = 2$	0	65 100	6220	6.4×10^{-4}
pipeline with $M = 6$	0	104 100	11 300	1.4×10^{-4}

8.7.3 The depth budget

Table 8.8 adds everything up at the operating point of Fig. 8.13: 80 symmetric refinement steps, and rodeo cycles with $\sigma = 4$ and $\delta t = 0.25$, thirteen controlled steps each on average. The comparison is meant to be honest in both directions.

The variational circuit is short. Once optimized, every preparation replays 1312 CNOT gates, and the infidelity is 1.8×10^{-4}. Its cost lies elsewhere: in the optimization, which executes the circuit once for every Pauli group of every energy and gradient evaluation, averaged over the shots of Eq. (7.14), and in the growth of the circuit with resolution, a factor of five from $k = 3$ to $k = 4$. The pipeline is long. One run costs 6.5×10^4 CNOT gates with two cycles, fifty times the variational circuit, and with an acceptance of 0.99 that is also the cost per accepted state. Seventy percent of it is the refinement. There is no free lunch in coherent depth. What the depth buys is the absence of everything else: no parameters, no optimizer, no measurement overhead during preparation, a heralded output, and an accuracy that is set by the number of cycles and the Trotter step, not by an ansatz. At this operating point the pipeline with six cycles matches the variational infidelity and the smaller step of Table 8.7 passes it; the real difference is that the pipeline can continue. The dominant cost can be attacked in three ways that we have met: a larger buffer, which shortens T^* linearly, Fig. 8.3; a start higher in the hierarchy, which halved it, Fig. 8.7; and the encoding, which divides every entry by ten. In the compact encoding the entire pipeline with two cycles, 6220 CNOT gates on five qubits, is five times the *Jordan–Wigner* variational circuit.

Table 8.9 Probability $e^{-N_2\varepsilon}$ that a circuit with N_2 CNOT gates runs without a two-qubit fault, for error rates per gate from 10^{-2} to 10^{-6}. Produced by `gate_budget.py`.

circuit	N_2	$\varepsilon = 10^{-2}$	10^{-3}	10^{-4}	10^{-6}
UCCSD, $k = 3$	272	0.066	0.76	0.97	1.00
UCCSD, $k = 4$	1312	2×10^{-6}	0.27	0.88	1.00
pipeline, compact, $M = 2$	6220	10^{-27}	2×10^{-3}	0.54	0.99
pipeline, Jordan–Wigner, $M = 2$	65 100	10^{-283}	5×10^{-29}	1.5×10^{-3}	0.94

8.7.4 Shorter variational circuits

The count of 1312 is not irreducible either, and the comparison should not be left unfair to the variational side. The adaptive VQE of Section 7.8 [**?**] grows the ansatz one operator at a time, keeping those with the largest energy gradient. For the pairing model the selection can be predicted. The Hamiltonian conserves seniority and the reference has seniority zero, so that the gradients of all singles and of all pair-breaking doubles vanish identically, and only the $\frac{N}{2}(k-\frac{N}{2})$ pair-hopping doubles survive: four operators at $k = 4$, or $4 \times 48 = 192$ CNOT gates instead of 1312, a sevenfold compression that discards only amplitudes that vanish at the optimum anyway. Drawing the pool from individual Pauli strings [**?**] removes the eightfold overhead per fermionic double as well. These developments complement the pipeline. The pipeline does not care how its coarse state is made, so that an adaptive solver slots in as the first stage; and no variational compression supplies by itself the exponential, optimizer-free convergence of the filter once the overlap is large.

8.7.5 The budget against the hardware

The requirements of the pipeline on a device are now explicit: $2k+1$ qubits, nine at $k = 4$ and seventeen at $k = 8$, or $k+1$ in the compact encoding; mid-circuit measurement of one ancilla with reset and reuse; and a coherent sequence of 10^4 to 10^5 two-qubit gates. The first two are available today. The third is the bottleneck, and a one-line estimate shows how severe it is. If every two-qubit gate fails independently with probability ε, a circuit of N_2 such gates runs without a fault with probability

$$P_{\text{clean}} \simeq (1-\varepsilon)^{N_2} \simeq e^{-N_2\varepsilon}, \tag{8.25}$$

and $N_2\varepsilon$ is the expected number of faults. Table 8.9 evaluates it for the circuits of this chapter.

At the time of writing, the best superconducting and trapped-ion processors have between one and two hundred qubits and two-qubit error rates at or somewhat below 10^{-3}, with mid-circuit measurement and feed-forward available on both; superconducting roadmaps speak of circuits with 5×10^3 to 10^4 reliable two-qubit gates with error mitigation [**?**], trapped-ion machines of the QCCD type [**?**] trade speed for fidelity and all-to-all connectivity, and the rodeo algorithm itself has been run on both kinds of device in one- and two-qubit form [**?**, **?**]. The column $\varepsilon = 10^{-3}$ is thus the present. It says that the $k = 3$ variational circuit runs cleanly three times out of four, that the $k = 4$ one does so one time in four, which error mitigation can work with, and that the full pipeline cannot be run: with $N_2\varepsilon \approx 65$ expected faults no post-processing recovers the signal. A truncated demonstration is within reach, the $k = 3$ chain with a short refinement and one or two cycles at a coarse step, some 10^3 to 10^4 two-qubit gates on seven qubits, or better its compact-encoding version on four; and all-to-all

connectivity matters for it, because on a planar lattice of qubits the long strings of Fig. 8.12 must be routed with SWAP gates that the counts above do not include. There the acceptance rate of the filter, which collapses when the compiled evolution is wrong, doubles as a sensitive diagnostic of the hardware.

The column $\varepsilon = 10^{-6}$ is the regime of error-corrected logical qubits, the subject of Chapter 9. Error correction below threshold has been demonstrated [**?**], neutral-atom processors have executed algorithms on tens of logical qubits with mid-circuit readout [**?**, **?**], and the first machines specified in logical rather than physical qubits, a few dozen of them, are being built. For such machines the full $k = 4$ pipeline, nine to seventeen logical qubits, 10^5 gates and the repeated measurement of one logical ancilla, is a natural early workload, small in width and demanding in exactly the quantity, depth, that logical qubits exist to provide. It would be a mistake to read the last column as free, however: a logical gate costs many physical operations, the non-Clifford rotations $R_z(\phi)$ with arbitrary angles are the expensive part of a fault-tolerant circuit, and their number, 60 per Trotter step here, becomes the currency in place of the CNOT count.

The bottlenecks, collected. The example has touched every limitation that the earlier chapters met one at a time, and it is worth listing them with the numbers attached.

1. *Overlap.* Every projective method, phase estimation and the rodeo algorithm alike, costs $1/p$, and p decays with system size for any input that is cheap to prepare. The prolonged states of this chapter had p from 0.95 down to 0.38 on four to sixteen qubits. State preparation is the problem; refinement, variational circuits and adiabatic ramps are attempts at it, and none comes with a guarantee.
2. *Coherent depth.* Adiabatic time and interferometric resolution are paid in gates executed without interruption: $T^* \sim 1/\Delta E_{\min}$ for the refinement and $\sigma \sim 1/\varepsilon$ per cycle for the filter, at 10^2 to 10^3 CNOT gates per unit of time. Depth, not the number of qubits, is what today's devices lack; nine qubits were enough to need 10^5 gates.
3. *Trotter error.* A product formula evolves under a slightly wrong Hamiltonian. The refinement tolerates this; the filter converges to the wrong eigenstate, with a floor $\propto \delta t^4$ in infidelity for the symmetric formula, and halving the step doubles the gates.
4. *Encoding.* The same physics cost 296 or 24 CNOT gates per step depending on whether the seniority symmetry was built into the qubits. Locality matters equally: the Jordan–Wigner strings that cancelled for pairs do not cancel in general, and the weight of a double excitation grows with the distance between its orbitals.
5. *Optimization and measurement.* Variational circuits are shallow because they move the cost to a classical loop: shot noise, $\varepsilon \sim N_{\text{shots}}^{-1/2}$, local minima, barren plateaus, and an accuracy floor set by the ansatz, 1.4×10^{-3} in energy here. The loop is absent from the pipeline and present, potentially, in its coarse stage.
6. *Noise.* With error rates of 10^{-3} a thousand two-qubit gates is roughly what runs; the algorithms with guarantees need 10^5 and more, which is the case for error correction in one sentence.

None of these is special to the pairing model. They are coupled, in that relieving one usually loads another, and the art of designing a quantum calculation is the art of choosing where to pay. The pairing model, exactly solvable at every resolution, is a good place to learn the prices.

8.8 Summary

A projective filter multiplies every eigencomponent of a state by a function of its energy and keeps the result when a measurement says so. The rodeo algorithm does this with one ancilla and controlled evolutions of random duration: an eigenstate at the target energy always passes, any other passes a cycle with probability $\cos^2[(E_j-E)t/2]$, one half on average, and after M cycles the error of the accepted state is $\frac{1-p}{p}2^{-M}$, exponentially small in the number of cycles, while the acceptance is the overlap p of the input. Scanned in energy the same circuit is a spectrometer whose peak heights are spectral weights. Compared with phase estimation it converges exponentially rather than as a power of the evolution time, overtaking it, for the pairing model, at an infidelity somewhat below 10^{-4}; it needs no Fourier transform and no feedback; and it gives up the certified energy of a single run. Compared with the variational eigensolver it has no ansatz, no optimizer and no accuracy floor, and pays with a circuit fifty times deeper. It can take the output of either as its input.

Its cost is $1/p$, and resolution refinement addresses exactly that. A coarse eigenstate is lifted by a prolongation, gate-free for a basis refinement and a layer of two-qubit rotations for a lattice, and followed along a shifted interpolation between the coarse and the fine Hamiltonian. The path stays gapped because the two describe the same low-energy physics; the time grows as the inverse gap, which we checked by tuning the gap through the shift; and the practical limit at strong coupling is the quality of the prolonged state, which is cured by starting higher in the hierarchy. The same construction, with the hierarchy chosen to suit the model and the prolongation made to respect the symmetry and sign structure of the target, prepared the ground states of the Lipkin, Heisenberg and Hubbard Hamiltonians from overlaps as low as 0.42. Refinement converges quickly to $p \approx 0.99$ and slowly afterwards; the filter takes over where it slows down.

Compiled to gates, the pairing pipeline at four levels is nine qubits and 10^5 CNOT gates, or five qubits and 10^4 when the seniority symmetry is built into the encoding. The Trotterized filter converges to the ground state of the effective Hamiltonian of its product formula and rejects an evolution that is too coarse. Held against hardware, the variational circuit runs today and the pipeline needs logical qubits, which is the subject of the next chapter.

8.9 Exercises

1. **The rodeo circuit.** (a) Derive Eq. (8.8) by following the register through Fig. 8.4, and show that the two branches have the probabilities $\cos^2$ and $\sin^2$ of $(E_j-E)t/2$ for an eigenstate. (b) Show that the filters $\hat{F}(t_k,E)$ commute, so that the order of the cycles is immaterial, and that $\hat{F}(t,E)^\dagger\hat{F}(t,E)=\cos^2[(\hat{H}-E)t/2]$. (c) Derive Eq. (8.13) from the characteristic function of the Gaussian. What replaces it for times drawn uniformly from $[0,\tau]$, and why is the Gaussian preferable? (d) What state does the register hold after the outcome 1? Show that a run with the record $1,0,0,\ldots$ at $E=E_0$ has no weight on the ground state. Is it useful for anything?
2. **Prolongation.** (a) Prove Eq. (8.2). (b) Show that $\hat{P}$ is an isometry if the columns of W are orthonormal, and that $\hat{P}\hat{P}^\dagger$ is a projector. (c) For the basis refinement of the pairing model, show that $\hat{P}^\dagger\hat{H}_{\text{high}}\hat{P}=\hat{H}_{\text{low}}$, and hence that the prolonged state has the energy E_{low}. Why does this fail for the lattice prolongation? (d) Write the circuit that prolongs one coarse site into two fine sites, $|1\rangle\to(|10\rangle+|01\rangle)/\sqrt{2}$ and $|0\rangle\to|00\rangle$, with one controlled rotation and one CNOT gate, and its version with the minus sign of Section 8.6.
3. **Paths compared.** Using `refinement.py`, compare the refinement $k=2\to4$ of the pairing model with the coupling ramp from $g=0$ at $k=4$, for $g=1$ and $g=4$ and buffers 0.6 and

1.5. (a) Plot the gap along each path. (b) Explain why the ramp wins at $g = 1$. (c) Add a term $-\xi' \sum_{p\sigma} p\, n_{p\sigma}$ that makes the single-particle levels degenerate, $\xi' = \xi$, and repeat. Which path survives, and why?

4. **The adiabatic estimate.** Starting from Eq. (8.6), integrate by parts once and show that the boundary terms vanish when $\partial_s \hat{H} = 0$ at both ends. Test the consequence numerically by replacing the schedule of Eq. (8.4) by the linear one, $(1 - s)$ and s, in `ResolutionRefinement`, and compare the overlap at $T = 30$.
5. **Choosing the times.** (a) For the prolonged $k = 4$ pairing state, reproduce the three-cycle result of Section 8.4.3. (b) Suppose the excited energies are known only to ± 0.05 from a coarse scan. How much of the advantage survives? (c) Implement the geometric sequence $t_k = t_{\max}/2^{k-1}$ and compare with Gaussian times at equal total evolution time. (d) Relate the geometric sequence to the controlled powers U^{2^j} of phase estimation.
6. **Excited states.** The scan of Fig. 8.8 shows a peak at $E = 2.935$ with weight 0.106 in the prolonged state. (a) Use the filter at that energy to prepare the first excited seniority-zero state, and report the infidelity and the acceptance against M. (b) Why is the refined state a poor input for this task? (c) Propose a refinement that tracks the excited state, and state the condition on μ that it requires.
7. **Phase estimation as a filter.** (a) Derive the fidelity formula of Section 8.5.1 and show that the infidelity falls as 4^{-m}. (b) Reproduce Table 8.3 and find the crossing point for the refined input. (c) Replace the uniform superposition of the control register by the sine window used for the HHL clock in Section 6.7.2. How do the tails, and the comparison, change?
8. **Counting gates.** (a) Derive Eq. (8.24). (b) Show that the Z strings of a pair-hopping double cancel, and find the CNOT count of the general double excitation $(i, j) \to (a, b)$ as a function of the four mode indices. (c) Derive the 34 CNOT gates of the projector on four empty qubits from its expansion in Z strings, and find a cheaper circuit using one ancilla and Toffoli gates. (d) Count the R_z rotations of the pipeline of Table 8.8.
9. **The Trotter floor.** (a) For the symmetric formula show that the effective Hamiltonian differs from $\hat{H}$ at order δt^2, that its ground state differs at the same order, and hence that the infidelity floor scales as δt^4; check against Table 8.7. (b) The filter was run at the exact E_0 although the effective Hamiltonian has a shifted ground-state energy. Scan E with the Trotterized evolution at $\delta t = 0.5$ and find where the acceptance peaks. Does targeting that energy lower the floor?
10. **Another model.** Carry the pipeline of Section 8.6 through for the Hubbard ring with $U = 8t$. (a) How do the prolonged overlap and T^* change? (b) Replace the bonding-orbital prolongation by the antibonding one and explain the result. (c) Estimate the CNOT count of the refinement from the 56 CNOT gates per first-order step that `gate_budget.py` reports for this Hamiltonian, and place it in Table 8.9.

Programs for this chapter

Every number and figure in this chapter comes from the programs in `BookPrograms/chapter8/`, which build on the shared modules of Chapters 2–4 (`gates`, `circuit`, `fockspace`, `models`, `pauli`, `jordanwigner`, `optimize`), on `coupledcluster.py` of Chapter 3, `mapped_models.py` and `pauli_circuits.py` of Chapter 4, and `phase_estimation.py` and `qpe_models.py` of Chapter 6. All of them are imported by the notebook `Notebooks/chapter8/chapter8.ipynb`, and all run on `numpy` and `matplotlib` alone.

`refinement.py` The classes `Prolongation` (embedding and orbital prolongation, Eq. (8.2)) and `ResolutionRefinement` (the paths of Eqs. (8.4) and (8.21), their spectrum and min-

imum gap, evolution by exact slices or by a supplied product formula, T^*); the pairing refinements of Figs. 8.1 and 8.2 and the comparison with the coupling ramp. About ten seconds.

`rodeo.py` The class `RodeoFilter`: the circuit of Fig. 8.4 on the joint register, the filter of Eq. (8.9), energy scans, the exact Gaussian averages of Eq. (8.16), sampled preparation statistics and sampled runs; the checks and the one-qubit example of Section 8.3.3, Fig. 8.5. About ten seconds.

`uccsd_sector.py` The class `UCCSDSector`, the product-form UCCSD ansatz in a number sector with exact gradients, and Table 8.1 with Fig. 8.6 from $k = 2$ to $k = 8$. About five seconds.

`pairing_pipeline.py` The two refinement chains of Fig. 8.7, the coupling and buffer scans of Fig. 8.3, the rodeo scans of Fig. 8.8, Table 8.2 with Fig. 8.9, and the three cycles with chosen times. About thirty seconds.

`filters_compared.py` Phase estimation as a filter, Eq. (8.20), against the rodeo algorithm at equal evolution time, Table 8.3 and Fig. 8.10; the infidelity of the UCCSD state and the rodeo cycles applied to it, Section 8.5.2. About two seconds.

`other_models.py` The hierarchies of Section 8.6 for the Lipkin, Heisenberg and Hubbard models, Tables 8.5 and 8.6, Fig. 8.11, the role of the Marshall sign, and the two-stage lattice refinement of two particles on $2 \to 4 \to 8$ sites. About ten seconds.

`trotter_pipeline.py` The class `PairingSplit` (the splitting of Eq. (8.23) and its first- and second-order steps), the Trotterized refinement and the Trotterized rodeo filter of Fig. 8.13 and Table 8.7, and the ground state of the effective Hamiltonian. About fifteen seconds.

`gate_budget.py` The Pauli-rotation check, the Jordan–Wigner strings and CNOT counts of the excitation operators and of the UCCSD preparation, the Trotter-step counts of Eq. (8.24) in both encodings, plain and controlled, the projector of the path, Table 8.8, the step counts of the other models and Table 8.9. About one second.

Chapter 9
Quantum Error Correction and Fault Tolerance, Ryan

Abstract Here we address the critical issue of errors and decoherence in quantum computing. Starting from simple error models (bit-flip, phase-flip) we introduce quantum error-correcting codes. We discuss basic codes such as the three-qubit bit-flip code and other, explaining how logical qubits are protected by redundancy. The concept of syndrome measurement and error correction is described. We also cover more scalable approaches like surface codes and topological codes, and explain the threshold theorem for fault tolerance. We emphasize how error correction (and error mitigation in the short term) is essential for reliable simulation of many-body systems.

Chapter 10
A Case Study: Toward Simulating Lattice Gauge Theories, Zohreh

Abstract This chapter first motivates what questions quantum simulation/computation aims to address in gauge theories, with a focus on nuclear and high-energy physics questions. We then illustrates simulation of a simple gauge field theory on a quantum computer as a case study. We introduce the (1+1)-dimensional Schwinger model (quantum electrodynamics), discretized on a lattice using the Kogut–Susskind Hamiltonian formulation. The mapping of fermionic matter fields and bosonic gauge fields to qubits is shown, and we construct quantum circuits for time evolution and observable measurements (such as charge density and electric-field correlators). Example results, using both numerical methods based in exact diagonalization and quantum-computed results using quantum circuits, are shown for small system sizes to demonstrate the method. The chapter concludes with implications for more complex gauge theories and connections to ongoing physics applications.

Chapter 11
Conclusion and Outlook, all

Abstract In the final chapter we summarize the key concepts covered in the book and reflect on future directions. We discuss the current status of quantum hardware (superconducting qubits, trapped ions, Rydberg atoms) and their applicability to many-body physics. Emerging topics such as analog quantum simulators, adiabatic quantum computing, and hybrid quantum-classical models are briefly mentioned. We outline open challenges (scaling to larger systems, improving algorithms, mitigating noise) and encourage further research. The chapter concludes by outlining future research directions in quantum simulation and many-body theory.

11.1 What the book has shown

The book began with a wall and ends with a map of the ways around it. The wall, the exponential growth of the many-body Hilbert space, was stated in Chapter 1 and quantified in Chapter 3, where the four models that run through the book, Lipkin, pairing, Hubbard and Heisenberg, were solved exactly at small size and approximately by Hartree–Fock and coupled cluster, and where the unitary coupled-cluster ansatz appeared as the first object whose evaluation is natural on a quantum computer and unnatural on a classical one. Chapter 4 turned the second-quantized Hamiltonians into sums of Pauli strings and showed that the cost of a model on a quantum computer is measured by three numbers, qubits, strings and weight, each of which a symmetry can reduce; the Lipkin, pairing and Heisenberg models all became four-qubit problems of two-qubit strings, and the eight-qubit Hubbard ring kept the parity strings that are the price of fermions. From the Pauli sum three circuits followed mechanically, the Pauli rotation, the Trotter step and the measurement of an energy, and everything in the later chapters was built from them.

The algorithms then divided along a line that the book has drawn many times. On one side stand the methods that need a deep coherent circuit and give a guarantee: Hamiltonian simulation in Chapter 5, whose Trotter error is controlled and whose cost is polynomial in the system size and the evolution time; phase estimation in Chapter 6, which converts that simulated evolution into an eigenvalue with a precision that doubles with every control qubit, and whose application to the four models showed the accuracy governed entirely by the register and the time step and not by the difficulty of the system; and the rodeo algorithm and its relatives in Chapter 8, which use the same controlled evolutions to project an eigenstate out of a trial state with one reusable ancilla, an error that falls exponentially with the number of cycles, and a probabilistic, heralded outcome whose success rate is the overlap of the trial state, which resolution refinement was introduced to make large; compiled to gates, the pairing calculation of that chapter came to 10^4 to 10^5 two-qubit gates on five to nine

qubits. On the other side stand the variational methods of Chapter 7, which need only shallow circuits and many repetitions, give an upper bound and nothing more, and whose quality was seen to be a property of the ansatz rather than of the algorithm: a hardware-efficient circuit converged smoothly to the wrong particle-number sector of the pairing and Hubbard Hamiltonians, while circuits built from the physics of each model were exact with a handful of parameters, and the adaptive version, given a pool complete under the symmetries of the problem, found those circuits by itself. The two sides need each other, since the variational state with its overlap of 0.9998 is what makes a phase-estimation run succeed on the first attempt, and the phase-estimation run is what certifies the variational state; that combination is the standard plan for the fault-tolerant era, and the small calculations of this book are its rehearsal.

Chapter 9 explained why the fault-tolerant era is needed at all and how it will be reached: the error models, the codes that detect and correct them by measuring syndromes rather than states, the surface code whose threshold has now been crossed on hardware, and the mitigation techniques that, in the meantime, trade repetitions for accuracy on the variational side. Chapter 10 took the whole apparatus to the problem that motivates much of the effort in nuclear and particle physics, a lattice gauge theory whose real-time and finite-density physics is closed to classical Monte Carlo by the sign problem, and showed on the Schwinger model that the mapping, the circuits and the measurements of the earlier chapters extend to gauge fields, and at what cost. What the book has not shown is a single result beyond the reach of a classical computer. Every table in it was produced on a simulator of at most eight qubits, checked against exact diagonalization, and chosen to be checkable; that was the point, and it is also the honest state of the field for the problems of many-body physics, where the calculations that have been run on hardware are, with a few exceptions in dynamics, still ones a laptop can verify.

11.2 Where the difficulty went

A theme has run through the book and deserves to be stated once more as the conclusion it is. Quantum computers do not remove the difficulty of the many-body problem; they move it, and the different algorithms move it to different places. In phase estimation the difficulty lives in state preparation, since the algorithm returns the energy of whatever eigenstate the trial state overlaps with, and in circuit depth, since every bit of precision doubles the number of applications of the evolution operator. In the variational methods it lives in the design of the ansatz, which decides whether the answer is close, and in the measurement count, which grows as the square of the inverse accuracy and of the Pauli norm of the Hamiltonian, and which no proposal so far reduces below that scaling. In Hamiltonian simulation it lives in the Trotter error, or in the ancilla overhead of the methods that replace Trotterization, and in the extraction of observables from the evolved state. In every case there is a quantity, the overlap, the measurement budget, the depth, that is polynomial for the small systems of this book and whose behaviour at large size is either unknown or known to be unfavourable in the worst case. The question of whether quantum computers offer an exponential advantage for ground-state problems reduces, as Chapter 6 discussed, to whether the overlap of an efficiently preparable state with the ground state decays polynomially or exponentially with the system size for the systems of interest, and the evidence, reviewed critically in [**?**], is that for the molecules and materials studied so far the answer is not the one the field had hoped for. For nuclei, whose ground states are often well approximated by a small number of configurations in a suitably chosen basis, and for real-time dynamics, where the advantage

rests on Lloyd's theorem and not on an overlap, the prospects are better, and it is no accident that the case study of Chapter 10 is a problem in dynamics.

11.3 The hardware

Every algorithm in the book was run on a simulator, and it is worth closing with a sober account of the machines it was written for. Four platforms carry most of the effort, and they differ in the resources that matter for many-body simulation, the number of qubits, the fidelity and speed of the two-qubit gates, the connectivity of the qubits and the coherence time, in ways that make each the natural home of a different class of problem [**?**, **?**]. Superconducting circuits, the platform of the largest commercial efforts, have hundreds of qubits on a chip with nearest-neighbour connectivity on a planar lattice, gate times of tens of nanoseconds, two-qubit gate errors that have fallen to a few parts in a thousand, and coherence times of a few hundred microseconds; the utility-scale dynamics experiments and the below-threshold surface-code experiments cited in Chapter 1 were performed on them, and their planar connectivity is a direct match to the surface code and to lattice models in two dimensions, while the parity strings of a fermionic mapping must be routed through swaps. Trapped ions have fewer qubits per trap, tens rather than hundreds, but all-to-all connectivity within a trap, the highest two-qubit gate fidelities demonstrated on any platform, and coherence times of seconds; their gates are slower by orders of magnitude, and their natural strength is in problems with long-range interactions and in the deep circuits with few qubits that phase estimation on small systems requires. Neutral atoms in optical tweezers combine hundreds to thousands of qubits, reconfigurable connectivity by moving the atoms, and, through the Rydberg blockade, entangling gates that have improved rapidly; the first logical-qubit experiments at scale were performed on them [**?**], and the same platform, operated in an analog rather than a digital mode, is a quantum simulator of spin models in its own right. Photonic and spin-qubit platforms are less mature for the problems of this book and are not discussed here.

Two remarks apply to all of them. The first is that the numbers that matter for a many-body calculation are the product of the two-qubit gate error and the number of two-qubit gates in the circuit. A Trotter step of the eight-qubit Hubbard ring of Chapter 4 costs a few dozen CNOT gates, a phase-estimation run with ten control qubits costs thousands of Trotter steps, and a gate error of 10^{-3} gives an error budget of a few hundred gates before the output is noise; the arithmetic is the reason the variational methods of Chapter 7 exist and the reason error correction is not optional. The second is that the demonstrations of error correction below threshold, on superconducting and on neutral-atom platforms, change the character of the projection. A logical qubit whose error falls as the code distance grows means that the depth budget is no longer fixed by the physics of the qubit but by the number of physical qubits one is willing to spend per logical one, of the order of a thousand for the surface code at useful error rates; the resource estimates for chemically relevant molecules and for nuclei in realistic model spaces, which run to millions of physical qubits and days of runtime, are the targets of the next decade of engineering, and they are being revised downwards by better algorithms as fast as the hardware improves.

11.4 Beyond the gate model

This book has been about the gate model, in which a many-body problem is compiled into a sequence of discrete unitaries on qubits, because the gate model is universal, because its

errors can be corrected, and because its algorithms have guarantees that can be stated as theorems. It is not the only way to use a quantum system to study another. The oldest is the *analog quantum simulator*, closest to Feynman's original proposal, in which a controllable system, ultracold atoms in an optical lattice, trapped ions with tunable spin-spin couplings, Rydberg atoms in tweezer arrays, superconducting circuits operated as coupled oscillators, is engineered to have the Hamiltonian one wants and then simply observed [**?**, **?**]. The Hubbard model of Chapter 3 has been realized this way with fermionic atoms at temperatures now approaching the scale of the antiferromagnetic exchange, the Heisenberg and transverse-field Ising models with ions and Rydberg atoms at hundreds of spins, and lattice gauge theories of the kind studied in Chapter 10 in the first small instances. Analog simulators have no error correction and no guarantee of accuracy, and their observables are limited to what the apparatus can measure; what they have is size and speed, and for the equilibrium and dynamical properties of lattice models in two dimensions they are ahead of any gate-model calculation and likely to remain so for some time. The relation between the two approaches is the relation between an experiment and a computation, and the field needs both.

Adiabatic quantum computation and *quantum annealing* occupy the ground between. The adiabatic theorem that motivated the QAOA ansatz of Chapter 7 can be run directly: prepare the ground state of a simple Hamiltonian, deform it slowly into the one of interest, and read out the result, with a running time set by the inverse square of the minimal gap along the path. As a model of computation it is equivalent to the gate model; as a practice, on the special-purpose annealers that implement transverse-field Ising Hamiltonians on thousands of qubits with limited coherence, it is a heuristic optimizer whose advantage over classical methods on any problem remains unestablished, and whose relevance to many-body physics lies mainly in its use as a sampler of classical spin-glass and, with some care, quantum spin-model configurations. The *hybrid* methods are the most recent and, for the near term, the most promising. Every variational algorithm of Chapter 7 is already hybrid, a quantum circuit inside a classical optimizer, and the pattern generalizes: a quantum computer can serve as the impurity solver inside a dynamical mean-field or embedding calculation, as the evaluator of a few hard matrix elements inside a classical configuration-interaction or coupled-cluster calculation, as the generator of samples for a classical tensor-network or neural-network state, or as the provider of a trial state that a classical method refines. In each of these the quantum resource is spent on the part of the problem where classical methods fail and the classical resource on everything else, which is where the exponential lives and where it does not, respectively. The quantum-machine-learning models glanced at in Chapter 7 belong here too, as parametrized circuits trained on data, and the question of whether they offer anything that a classical model does not is open in the same way, and for the same reasons, as the question for variational eigensolvers.

11.5 Open problems

The problems that remain are of three kinds, and it is useful to keep them apart, since they call for different people. The first kind is hardware: more qubits, better gates, longer coherence, faster readout, and the engineering of error correction from a demonstration into a platform. This is not a problem many-body theorists will solve, but it is one they should watch, because the algorithms that will run first on error-corrected machines are those with the smallest constant factors, and those factors are set by choices, of encoding, of Trotter ordering, of symmetry sector, that are theirs to make. The second kind is algorithmic, and here the many-body theorist is the expert. State preparation is the largest of them: the overlap problem of Chapter 6 is a question about the structure of physical ground states, about

which many-body theory knows a great deal and quantum information theory little, and the design of trial states with provably large overlap, from mean-field theory, from coupled cluster, from tensor networks, from adiabatic paths, is where the two fields most need each other. Measurement is the second: the ε^{-2} scaling of the variational methods is a statement about sample means, but the prefactor, the Pauli norm of the Hamiltonian, depends on the basis and the mapping and has been reduced by orders of magnitude for molecules by better choices, and the corresponding work for nuclear and lattice Hamiltonians is in its infancy. The third is the ansatz, whether hand-designed, adaptive or learned, and the barren-plateau obstruction that limits all of them: the escape routes, symmetry-preserving circuits, physically motivated initializations and problem-specific pools, are exactly the places where physical insight enters, and where the failures documented in Chapter 7 turn into a research programme. Beyond these, the extraction of observables other than the energy, response functions, transition rates, scattering amplitudes and the real-time correlators of Chapter 10, is where the advantage of Hamiltonian simulation should first become visible, and where the methods, including the resolvent techniques of Chapter 6, are least developed.

The third kind of problem is one of judgement. Classical many-body methods have not stood still: tensor networks in two dimensions, neural-network quantum states, and the steady improvement of coupled-cluster, Monte Carlo and configuration-interaction methods have moved the boundary of what a classical computer can do every year, and every claim of quantum advantage is a claim about that moving boundary. The discipline the field needs is the one this book has tried to practise: to compare against the best classical method on the same problem, to state the error and its source, to distinguish a theorem from a heuristic and a demonstration from a result, and to say when something is not known. The problems of many-body physics are hard enough that no tool will be wasted, and the reader who has reached this page has the two toolkits, the classical one of Chapter 3 and the quantum one of the chapters that followed, in the same hands. That is, in the end, what the book was for. The interesting work is in using both.

Appendix A
Quantum Computing Tools and Tutorials, all

Abstract This appendix provides practical resources for implementing the quantum algorithms discussed in the book. We give an overview of software frameworks such as Qiskit, Cirq, and Pennylane, including how to set up quantum circuits and run simulations. Example tutorials and Jupyter/Colab notebooks are referenced to demonstrate typical workflows (for example running a VQE for a simple Hamiltonian). We also discuss how to access cloud-based quantum processors and mention libraries for tensor-network and classical simulation. Links to community repositories and additional learning materials are provided for hands-on exploration.

Appendix B
Mathematical Background and Reference, all

Abstract This appendix collects the mathematical background needed throughout the book. We review linear algebra concepts (vector spaces, matrices, eigenproblems, tensor products) and provide relevant identities (e.g. commutators, matrix exponentials). Basics of group theory and representation theory used in many-body physics are summarized. The formalism of second quantization and occupation-number notation is restated for reference. We also list useful mathematical resources and key formulae (for example, Baker–Campbell–Hausdorff expansions and Trotter error bounds) for quick reference.

Index

ADAPT-VQE, 145
adiabatic quantum computing, 191
adiabatic state preparation, 160
adiabatic theorem, 160
aliasing, 105
amplitude damping, 21
analog quantum simulation, 191
annihilation operator, 38
ansatz
  adaptive, 145
  Hamiltonian-variational, 130
  hardware-efficient, 130
  unitary coupled cluster, 130
antisymmetrized matrix element, 40
approximate quantum Fourier transform, 98
avoided crossing, 33

Baker–Campbell–Hausdorff expansion, 54
barren plateau, 139
Bell measurement, 29
Bell states, 22
BFGS
  for VQE, 138
binary fraction, 95
bit reversal, 98
Bloch sphere, 14
Bloch vector, 14
Born rule, 16
bosons, 43
  qubit encoding, 79
Bravyi–Kitaev mapping, 72
Brillouin's theorem, 53

circuit depth, 30, 176
CNOT gate, 28
coherence, 19
commuting groups, 109
completeness relation, 11
computational basis, 10
configuration interaction
  truncated, 51
contraction, 41
controlled gate, 27
controlled Pauli rotation, 106
controlled rotation, 96
coupled-cluster theory, 58
creation operator, 38

decoherence, 20
density matrix, 11, 18
  one-body, 52
dephasing, 21
Dirac notation, 9
Dirichlet kernel, 102
discrete Fourier transform, 94

entanglement, 22
entanglement entropy, 25
exact diagonalization, 48
expectation value
  from shots, 84
exponential wall, 1, 50

Fermi vacuum, 41
Fock matrix, 42
Fock space, 38
Fourier state, 100
full configuration interaction, 48

gate, 8
gate count, 176
gate-based quantum computer, 7
gradient
  on a quantum computer, 136
Green's function, 119

Hadamard gate, 12
Hamiltonian simulation, 105
hard-core bosons, 43
Hartree–Fock theory, 52
Heisenberg model, 46
  phase estimation, 109
  resolution refinement, 173
Hermitian matrix, 10
HHL algorithm, 115
Hubbard model, 46
  phase estimation, 109
  qubit encoding, 77
  resolution refinement, 173

input/output bottleneck, 8

Ising model
  from QUBO, 148
iterative phase estimation, 104

Jordan–Wigner transformation, 69

Kitaev's algorithm, 104
Kraus operators, 20

Lanczos method, 50
Larmor precession, 26
linear response, 119
Lipkin model, 44
  Hartree–Fock, 55
  phase estimation, 109
  qubit encodings, 74
  resolution refinement, 173

many-body problem, 1
MaxCut, 148
maximally entangled state, 24
maximally mixed state, 19
mean field, 52
measurement, 8
  marginal reconstruction, 134
  of a Pauli sum, 132

noisy intermediate-scale quantum computers, 180
normal matrix, 10
normal ordering, 41

occupation-number representation, 39
outer product, 9
overlap, 113

pairing model, 45
  phase estimation, 109
  qubit encodings, 75
  resolution refinement, 161
  rodeo algorithm, 167
parameter-shift rule, 136
parity mapping, 71
parity sign, 39
parity string, 70
partial trace, 20
particle-hole formalism, 41
particle-hole interaction, 45
Pauli matrices, 11
Pauli rotation, 80
  gate count, 176
Pauli string, 68
Pauli sum, 68
phase gate, 12
Poisson equation, 118
post-selection, 166
product state, 22
programs
  chapter B, 35, 65, 89, 124, 156, 183
projection operator, 11
prolongation operator, 159
pure state, 18
purity, 19

QAOA, 148
quantum approximate optimization algorithm, 148
quantum channel, 20
quantum Fourier transform, 94
  circuit, 96
  gate count, 98
quantum hardware, 180, 191
quantum linear-system problem, 115
quantum machine learning, 152
quantum phase estimation, 100
  applications, 109
  as a filter, 171
  compared with VQE, 144
  energies, 105
  overlap requirement, 113
  worked examples, 107
quantum simulation
  history, 2
quantum wire, 29
quasispin, 44
qubit, 7
qubit encoding, 73
qubit tapering, 78
QUBO, 148

reduced density matrix, 20
resolution refinement, 158
resolvent, 119
rodeo algorithm, 163
  compared with QPE, 171
  compared with VQE, 171
  convergence, 166
  Gaussian times, 164
rotation gates, 27

Schmidt decomposition, 23
second quantization, 38
self-consistent field, 53
seniority, 45
separable state, 22
Shannon entropy, 24
shot noise, 17, 132
shots, 17
single-excitation gate, 83
size consistency, 51
Slater determinant, 38
Slater–Condon rules, 40
spectral decomposition, 13
SPSA, 138
state preparation, 113
superposition, 11
SWAP gate, 29
symmetry
  in Pauli form, 73
symmetry breaking, 55

tensor product, 12
Thouless' theorem, 54
transverse-field Ising model, 79
Trotter error
  in phase estimation, 111

in the rodeo algorithm, 178
Trotter step
circuit, 81
Trotter–Suzuki formula, 106
Trotter–Suzuki splitting, 54

unitary coupled cluster, 61
as a coarse solver, 167
circuit, 86
contrasted with QPE, 112
Trotterized, 62
unitary matrix, 10
universal gate set, 29
unrestricted Hartree–Fock, 58

variational principle
on a quantum computer, 128
variational quantum eigensolver, 128
applications, 141
compared with QPE, 144
von Neumann entropy, 24

wave function collapse, 16
Wick’s theorem, 41